Hashtag Treinamentos

CURSO COMPLETO

SQL

IMPRESSIONADOR

PostgreSQL

SUMÁRIO

MÓDULO 01	INTRODUÇÃO E INSTALAÇÃO DO POSTGRESQL	004
MÓDULO 02	CRIANDO QUERIES BÁSICAS NO POSTGRESQL	039
MÓDULO 03	FILTROS NO POSTGRESQL	065
MÓDULO 04	FUNÇÕES DE AGREGAÇÃO E AGRUPAMENTOS	107
MÓDULO 05	JOINS NO POSTGRESQL	135
MÓDULO 06	VIEWS NO POSTGRESQL	196
MÓDULO 07	OPERAÇÕES CRUD NO POSTGRESQL	233

SUMÁRIO

MÓDULO 08	FUNÇÕES DE NÚMERO, TEXTO E DATA	295
MÓDULO 09	SUBQUERIES	338
MÓDULO 10	VARIÁVEIS E BLOCOS ANÔNIMOS	395
MÓDULO 11	FUNCTIONS	430
MÓDULO 12	TRANSACTIONS E PROCEDURES	481

MÓDULO 1

INTRODUÇÃO E INSTALAÇÃO

INTRODUÇÃO E INSTALAÇÃO

INTRODUÇÃO E INSTALAÇÃO

PostgreSQL

O QUE VEREMOS NESTE MÓDULO

MÓDULO 1 INTRODUÇÃO E INSTALAÇÃO DO POSTGRESQL

01	Introdução: O que é um Banco de Dados?	006
02	Instalação do PostgreSQL	010
03	Abrindo o PostgreSQL / pgAdmin 4	023
04	Criando as tabelas do BD Northwind	027

Introdução: O que é um Banco de Dados?

Um banco de dados é um conjunto de dados organizados dentro de uma ou mais tabelas, que terão alguma relação entre si.

Partimos de uma informação isolada, na sua forma mais simples, e a partir de um conjunto de diversas informações isoladas, conseguimos organizar essas informações para criar tabelas e bancos de dados.

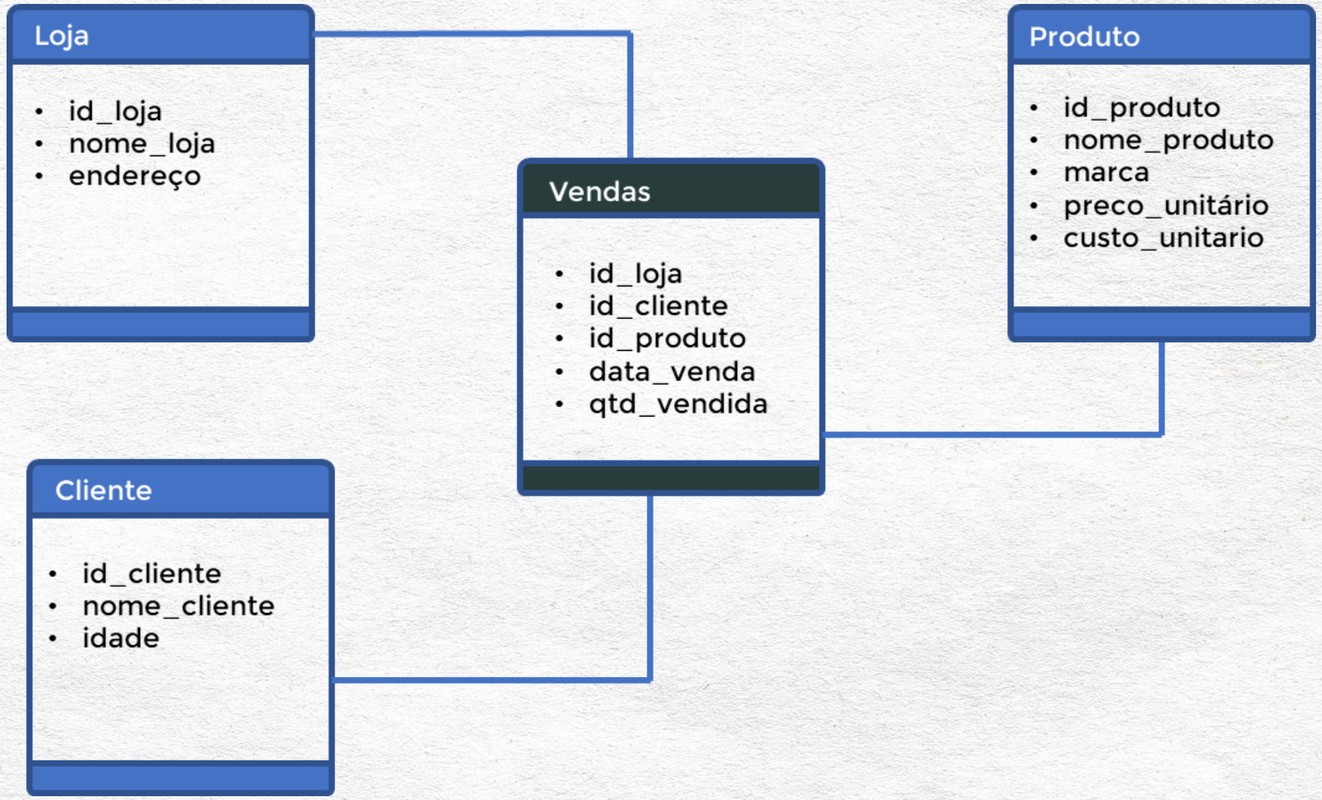
Introdução: O que é um Banco de Dados?

O desenho esquemático de um banco de dados é algo como o mostrado ao lado. Diversas tabelas, com diferentes informações sobre um negócio, e que possuem algum tipo de relação.

A esse banco de dados damos o nome de **RELACIONAL**.

Bancos de dados relacionais são o foco do nosso curso, por serem o tipo de bancos de dados mais comumente encontrado no mercado.

O SQL (**Structured Query Language)** é a linguagem padrão para trabalhar com tais bancos de dados denominados relacionais. Será através do SQL que seremos capazes de consultar e manipular os dados dos nossos bancos de dados.



PostgreSQL

Introdução: O que é um Banco de Dados?

Agora que já sabemos o que é o SQL, surge a seguinte dúvida: o que é um Sistema de Gerenciamento de Banco de Dados, ou SGBD?

Um SGBD permite ao desenvolvedor trabalhar com diferentes tabelas de um banco de dados através de uma interface. Essa interface seria basicamente um programa que nos permite fazer a leitura de tabelas de um banco de dados e utilizar o SQL para manipular esses dados, tudo de uma maneira bem visual e *user-friendly*.

Um SGBD é composto essencialmente por 2 partes:

Um **servidor**, onde vamos conseguir armazenar os nossos bancos de dados.

Uma **interface** amigável que nos permite escrever os códigos em SQL para acessar os bancos de dados.

PostgreSQL

Introdução: O que é um Banco de Dados?

Existem alguns SGBDs para Bancos de Dados Relacionais que são muito utilizados por grandes empresas. Abaixo, temos os 4 principais programas para SGBDs.

É importante que fique claro que **todos esses SGBDs** utilizam o SQL como linguagem de programação.

MySQL

É um SGBD relacional de código aberto, usado na maioria das aplicações gratuitas para gerir suas bases de dados.

A interface de código utilizada é o MySQL Workbench.

Oracle

A Oracle é uma das maiores empresas de tecnologia do mundo.

O SGBD da Oracle é focado em empresas de médio e grande porte.

A interface de código utilizada é o SQL Developer.

SQL Server

O SQL Server é o SGBD criado pela Microsoft, também para bancos de dados relacionais.

A interface de código utilizada é o SSMS.

PostgreSQL

O PostgreSQL é um SGBD relacional criado em 1989 e ainda um dos mais utilizados no mundo.

A interface de código utilizada é o pgAdmin.

PostgreSQL

Índice Geral

Índice deste Módulo

Instalação do PostgreSQL

Agora vamos instalar o programa pelo qual conseguiremos interagir com o nosso banco de dados.

O programa (SGBD) escolhido será o **PostgreSQL**.

O SGBD, conforme explicado anteriormente, será composto por um **servidor** e uma **interface**.

Para o caso do PostgreSQL, teremos que instalar o **PostgreSQL Server** e o **pgAdmin 4**, respectivamente.

PostgreSQL Server

SERVIDOR

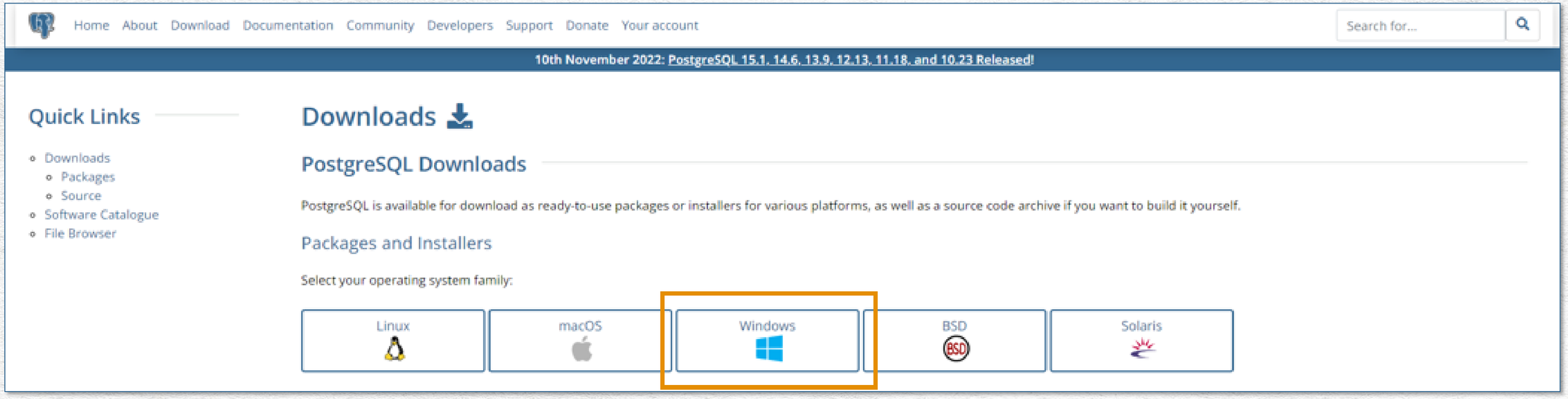
pgAdmin 4

INTERFACE

Instalação do PostgreSQL

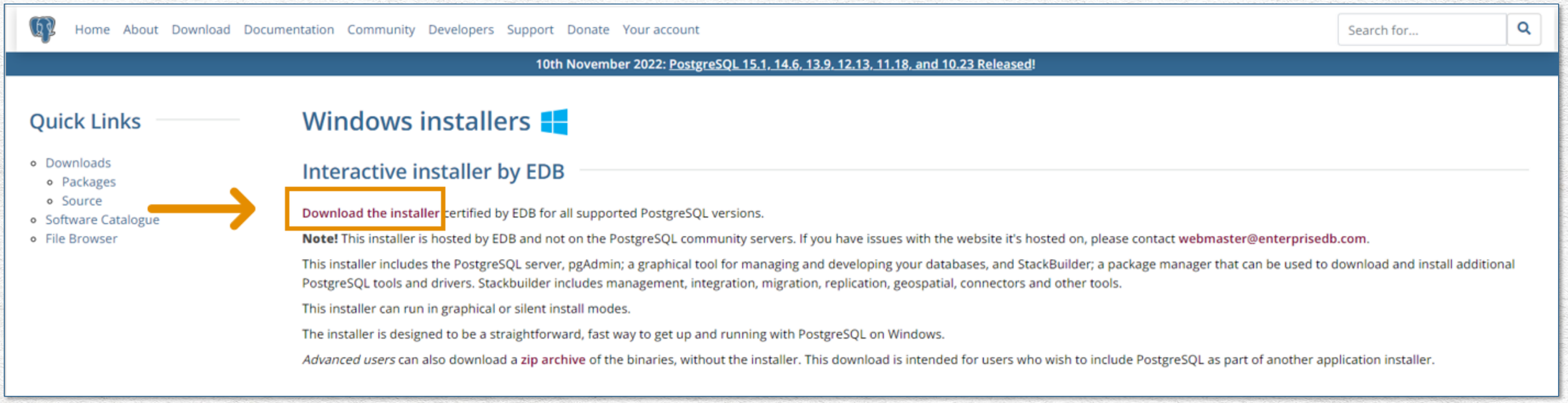
Para instalar, acesse o site: https://www.postgresql.org/download/

Na página inicial, selecione o sistema operacional do seu computador. Utilizaremos o Windows como exemplo:



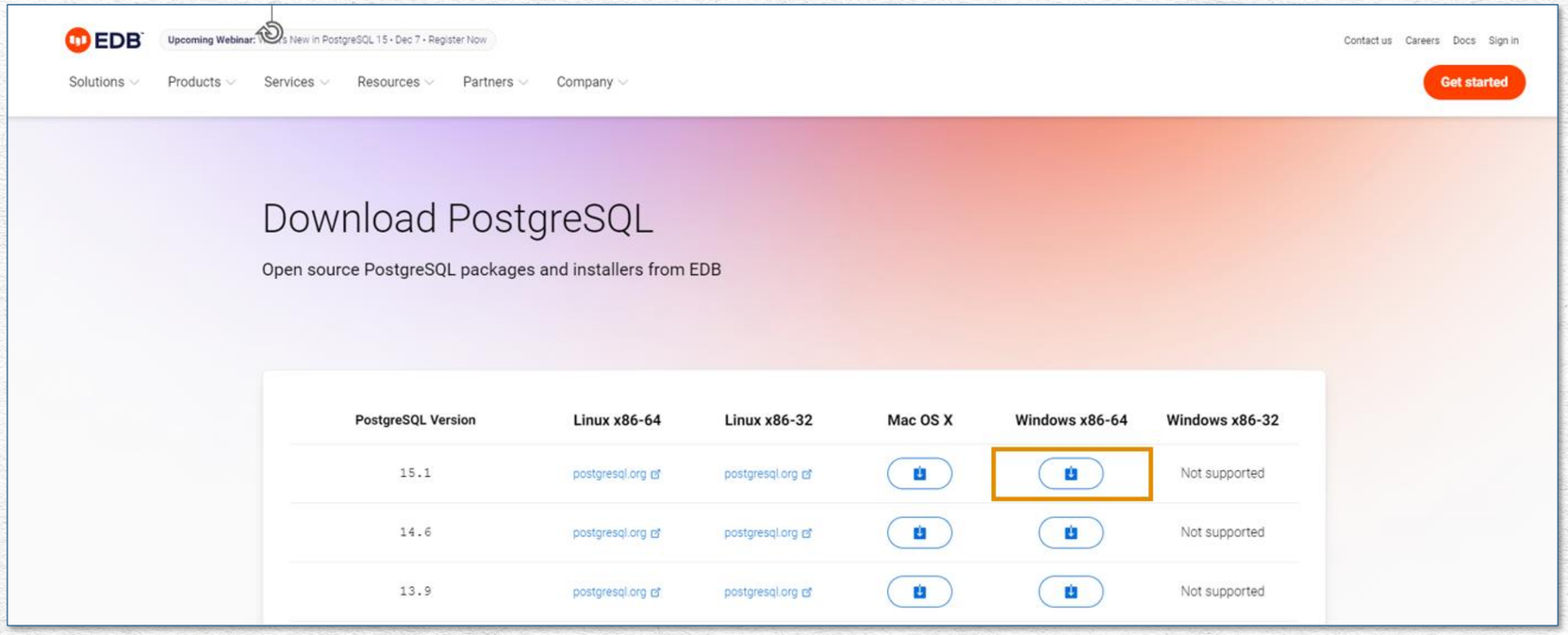
Instalação do PostgreSQL

Na página seguinte, clique em “Download the installer”:



Instalação do PostgreSQL

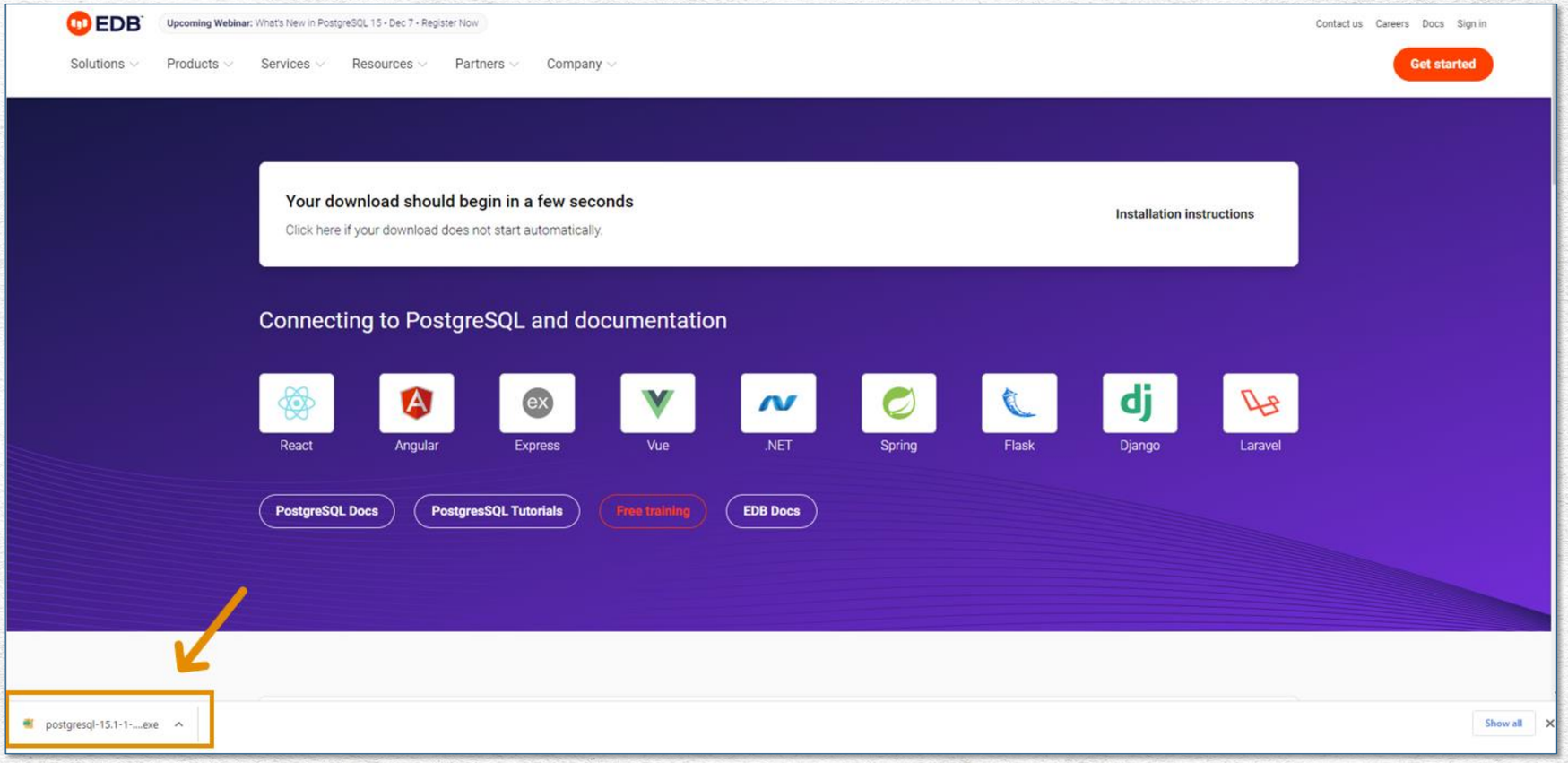
Selecione a versão suportada pelo seu sistema operacional. No nosso exemplo, instalaremos a versão mais atual suportada pelo Windows 64 bits:



Instalação do PostgreSQL

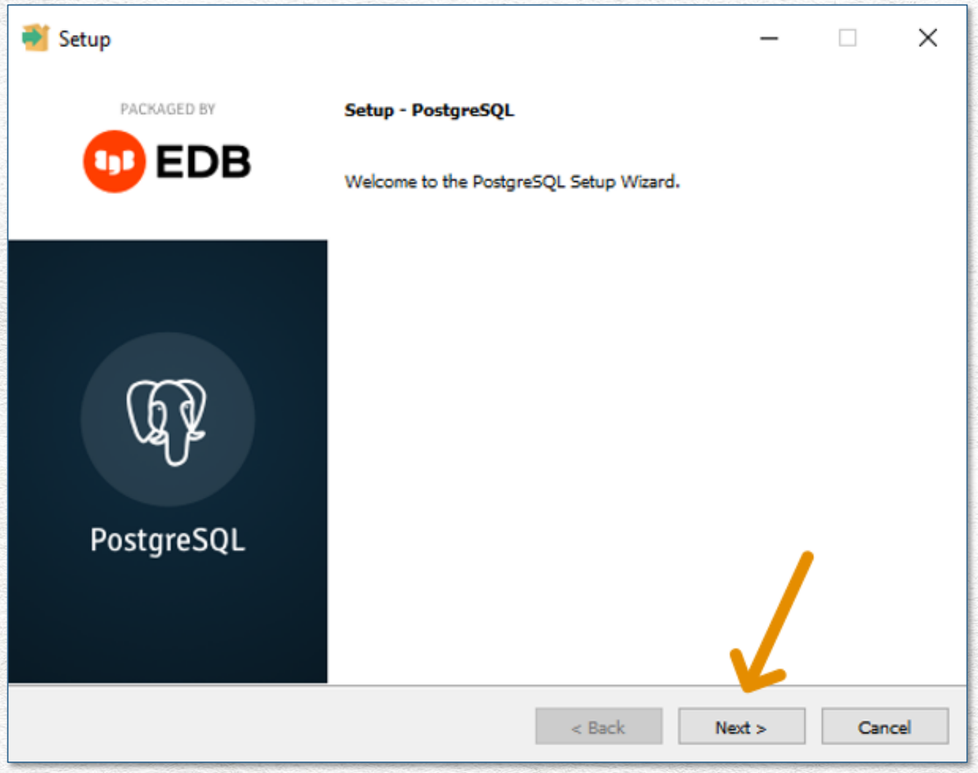
O download será iniciado automaticamente.

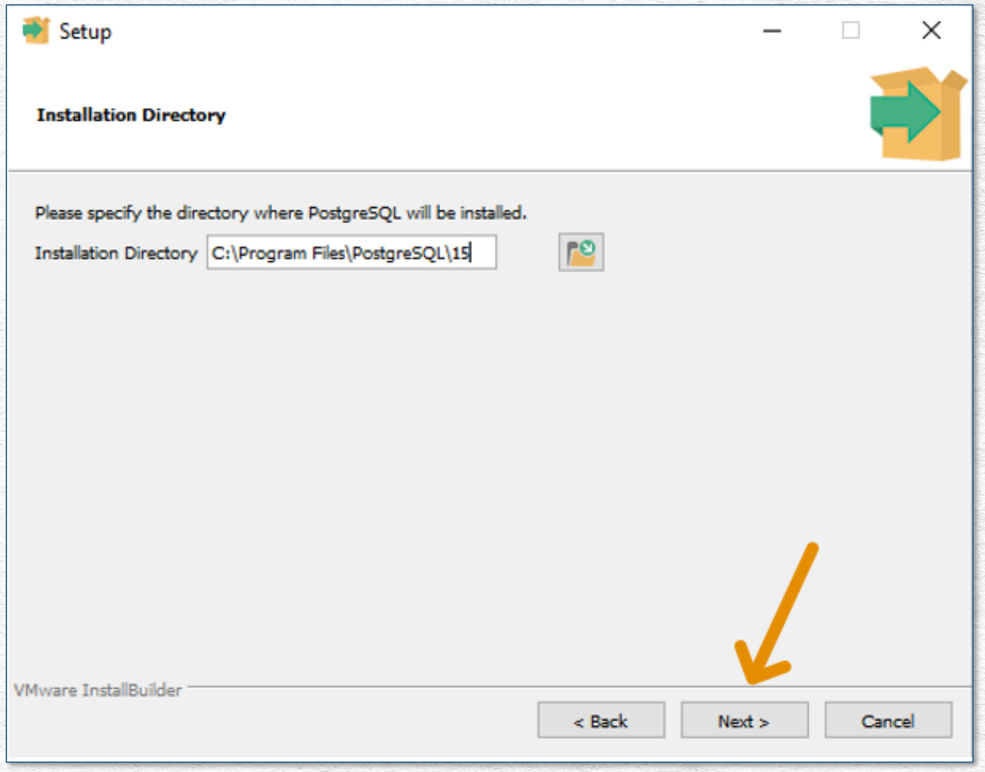
Finalizado o download, clique sobre o executável para iniciarmos a instalação.



Instalação do PostgreSQL

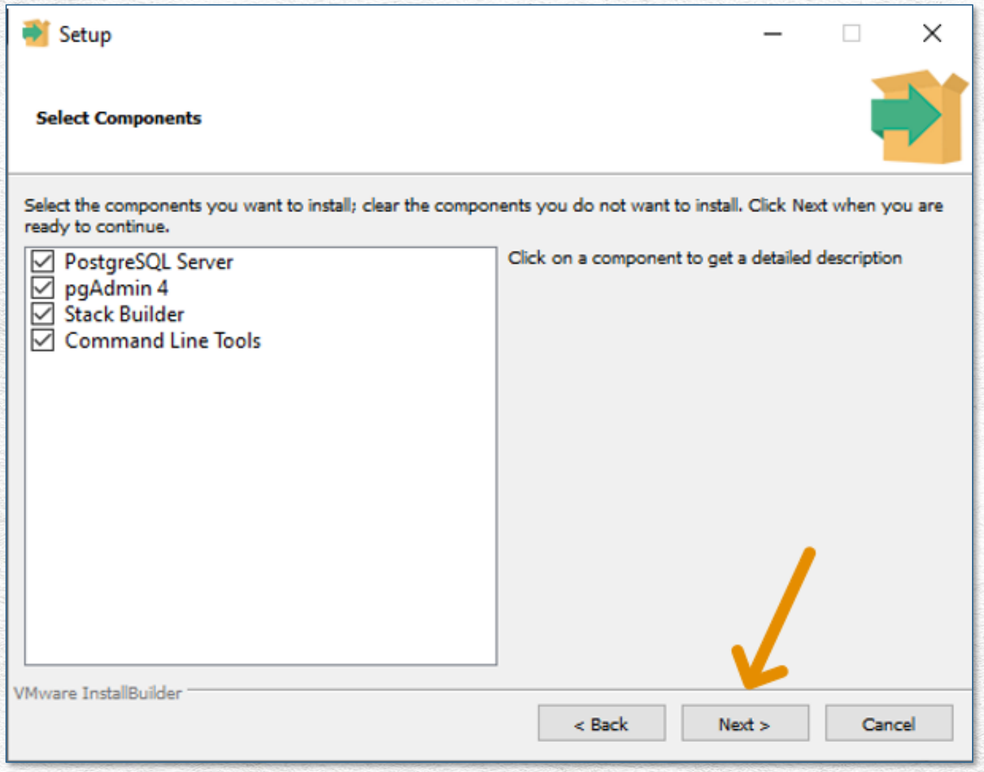
Nas janelas seguintes, clique em “Next”:

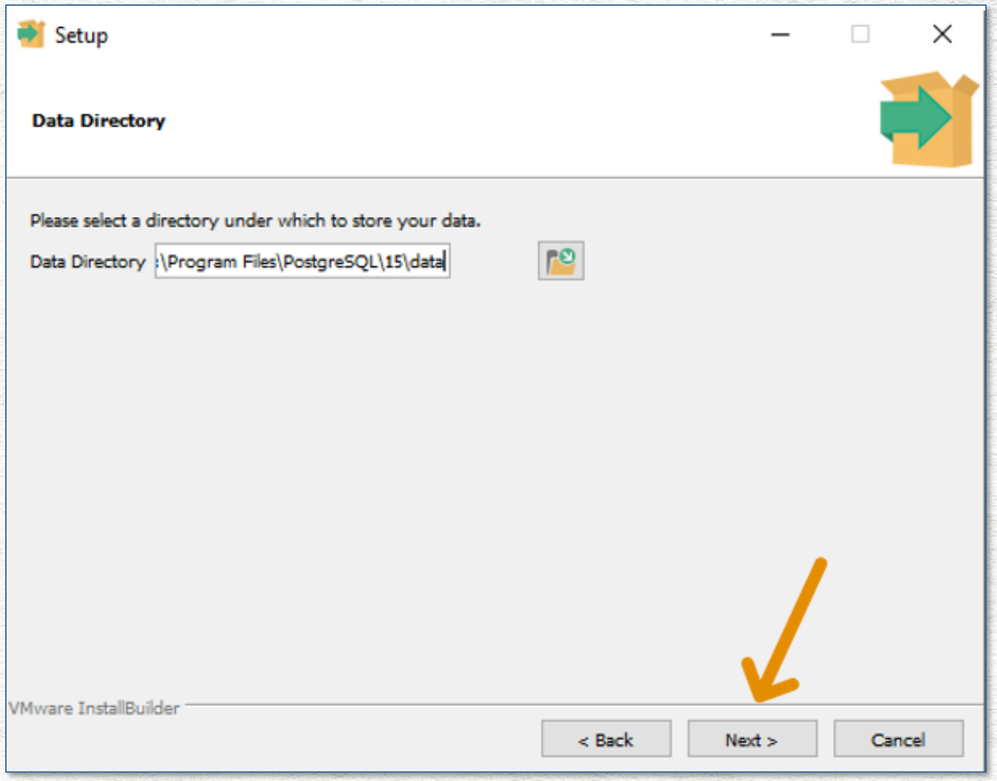




Instalação do PostgreSQL

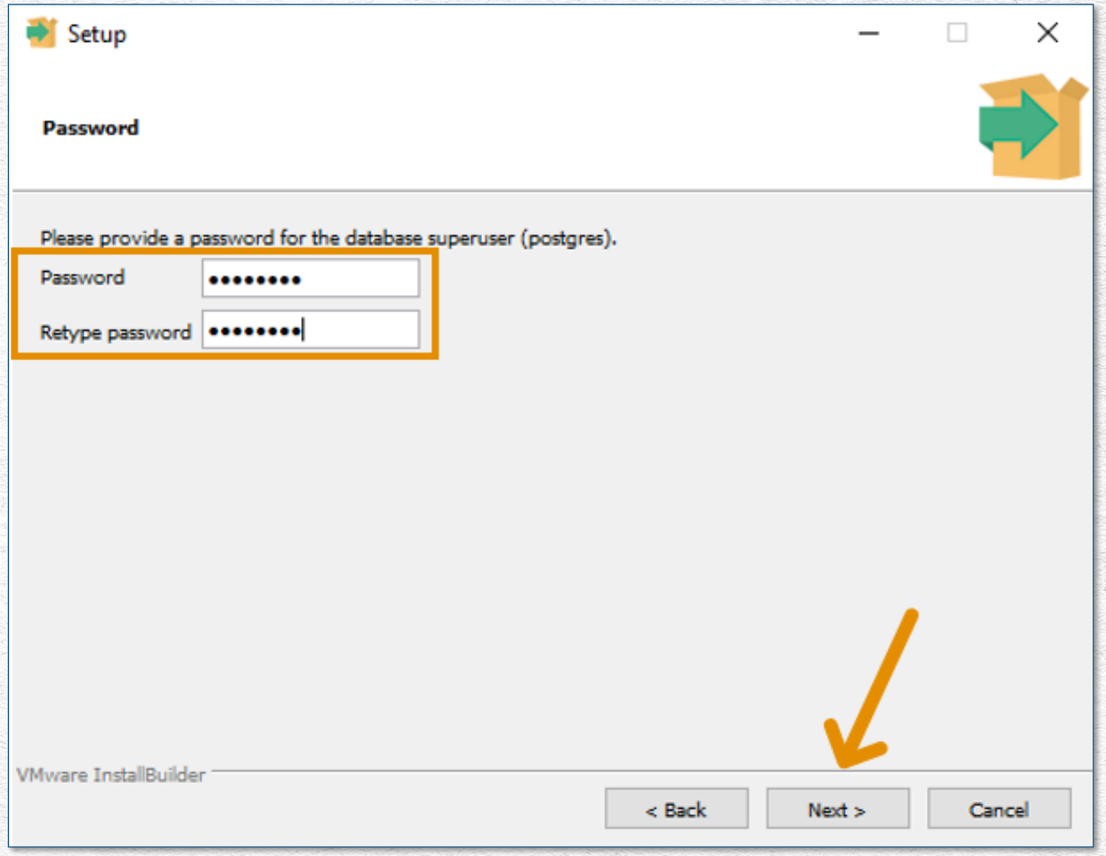
Nas janelas seguintes, clique em “Next”:





Instalação do PostgreSQL

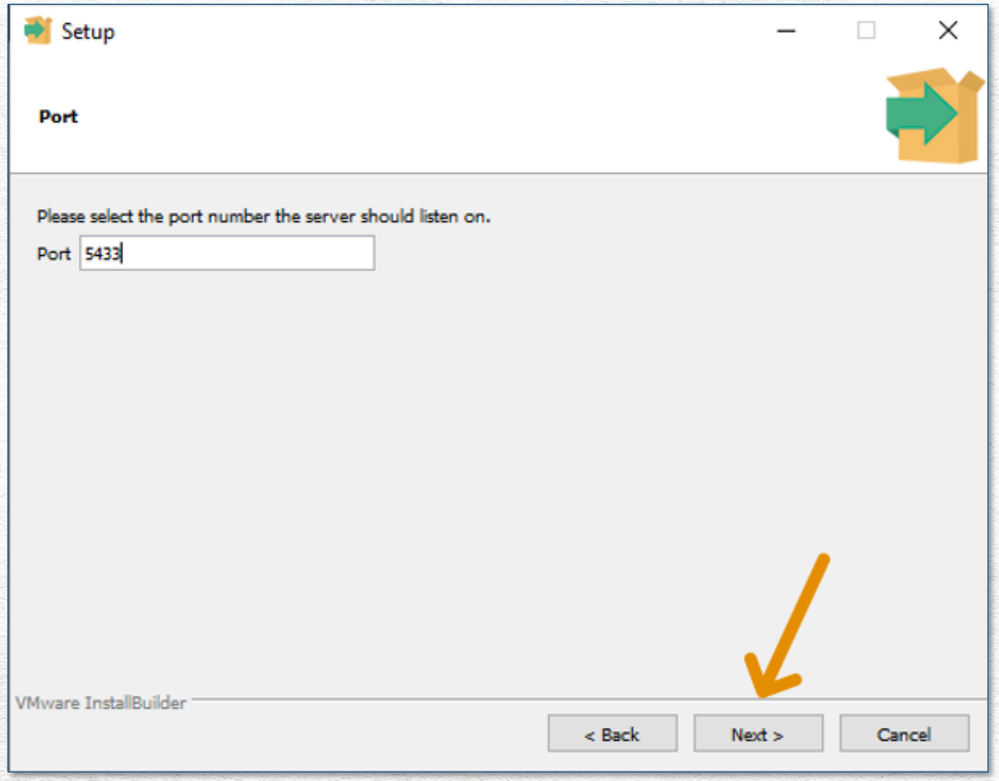
Nesta janela, configure uma senha e clique em “Next”:



Memorize esta senha, pois você precisará dela para acessar seu SGBD.

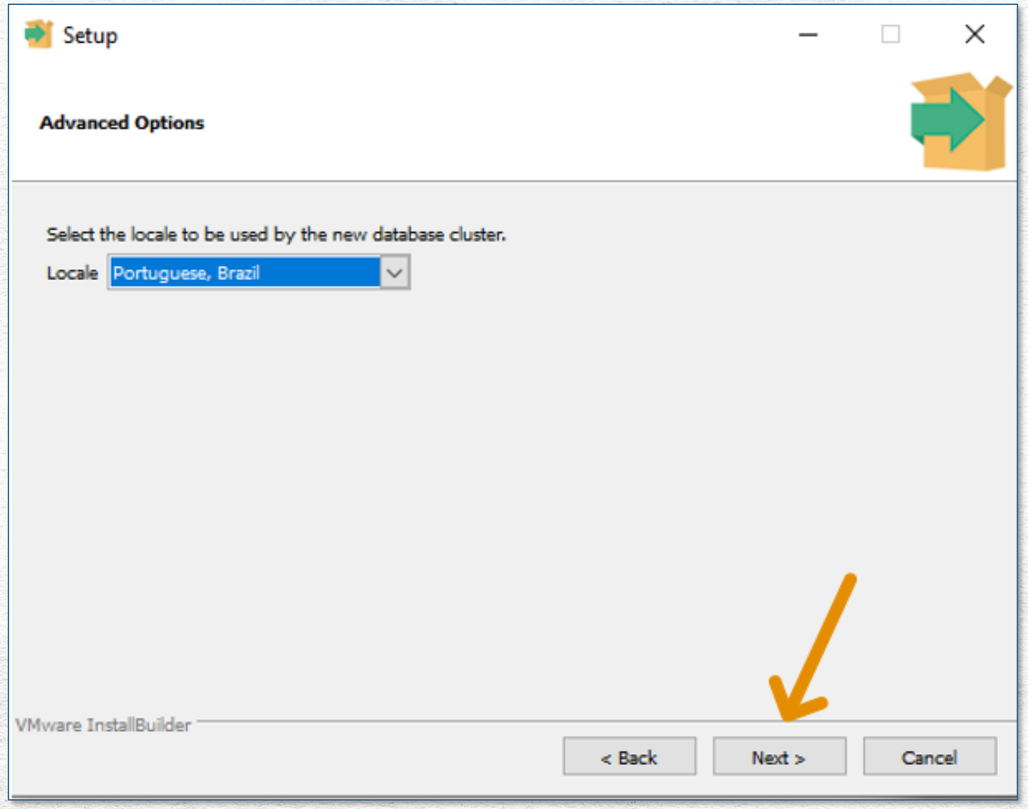
Instalação do PostgreSQL

Na janela seguinte, clique em “Next”:



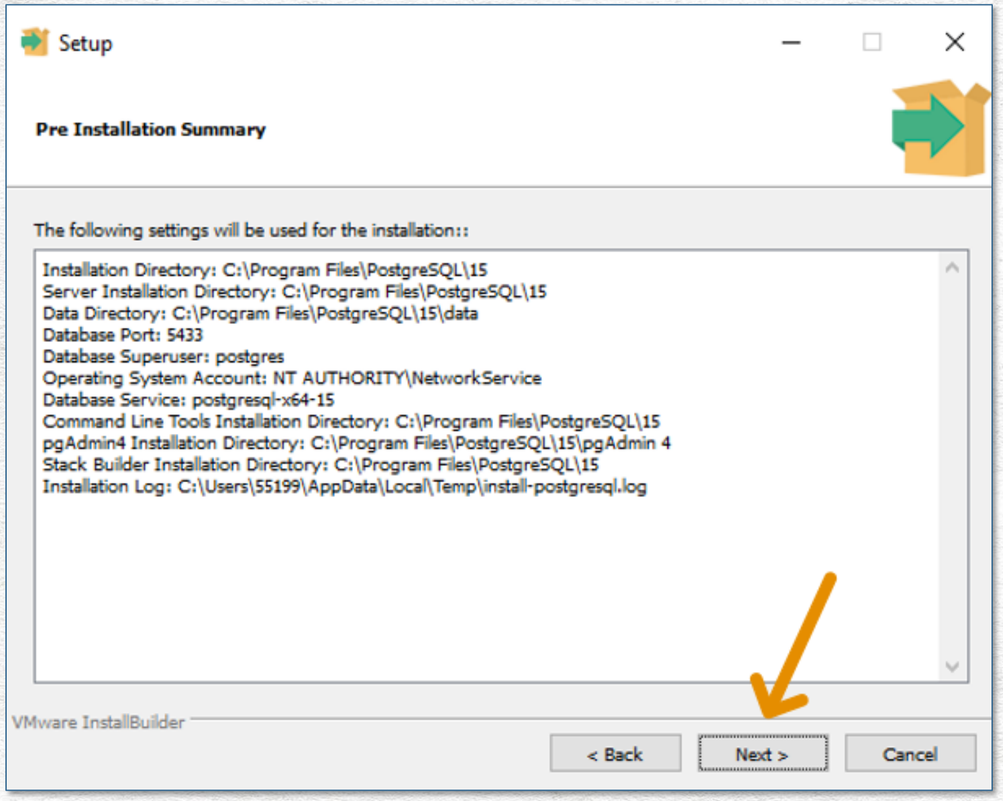
Instalação do PostgreSQL

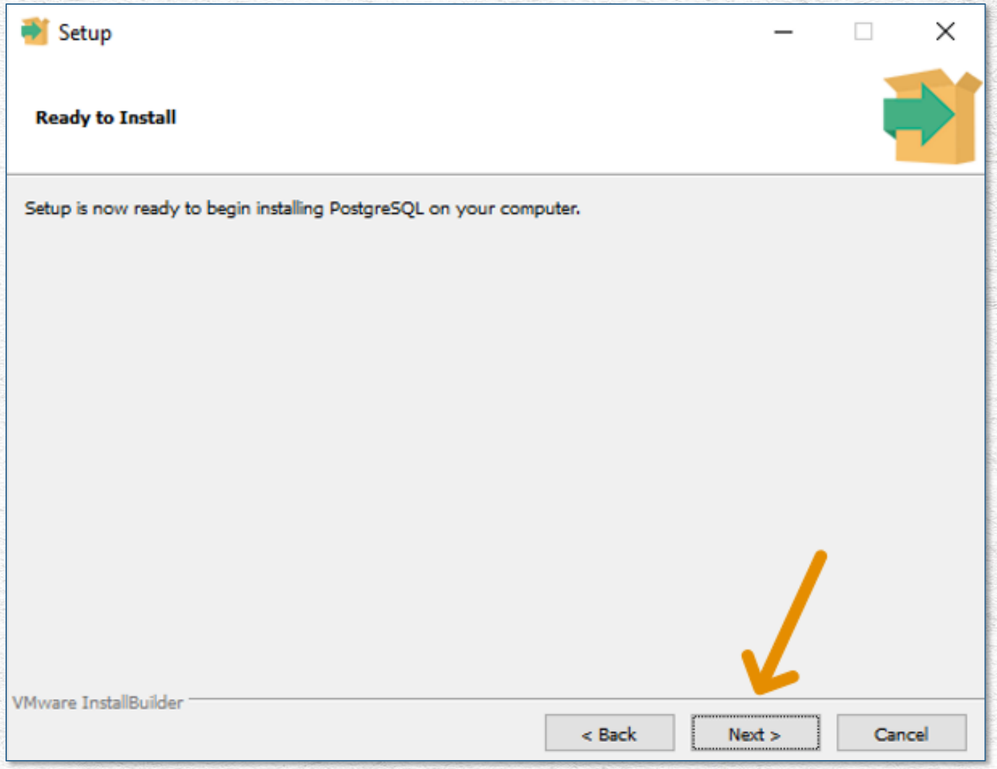
Selecione o idioma (Portuguese, Brazil) e clique em “Next”:



Instalação do PostgreSQL

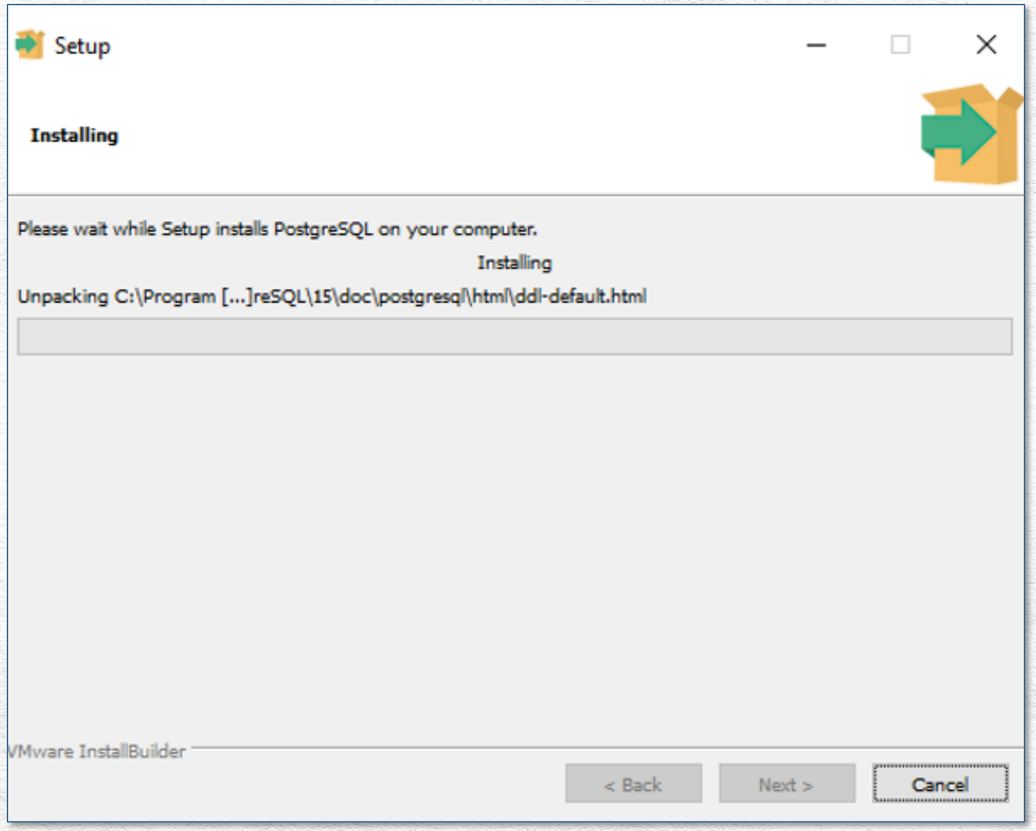
Nas janelas seguintes, clique em “Next”:





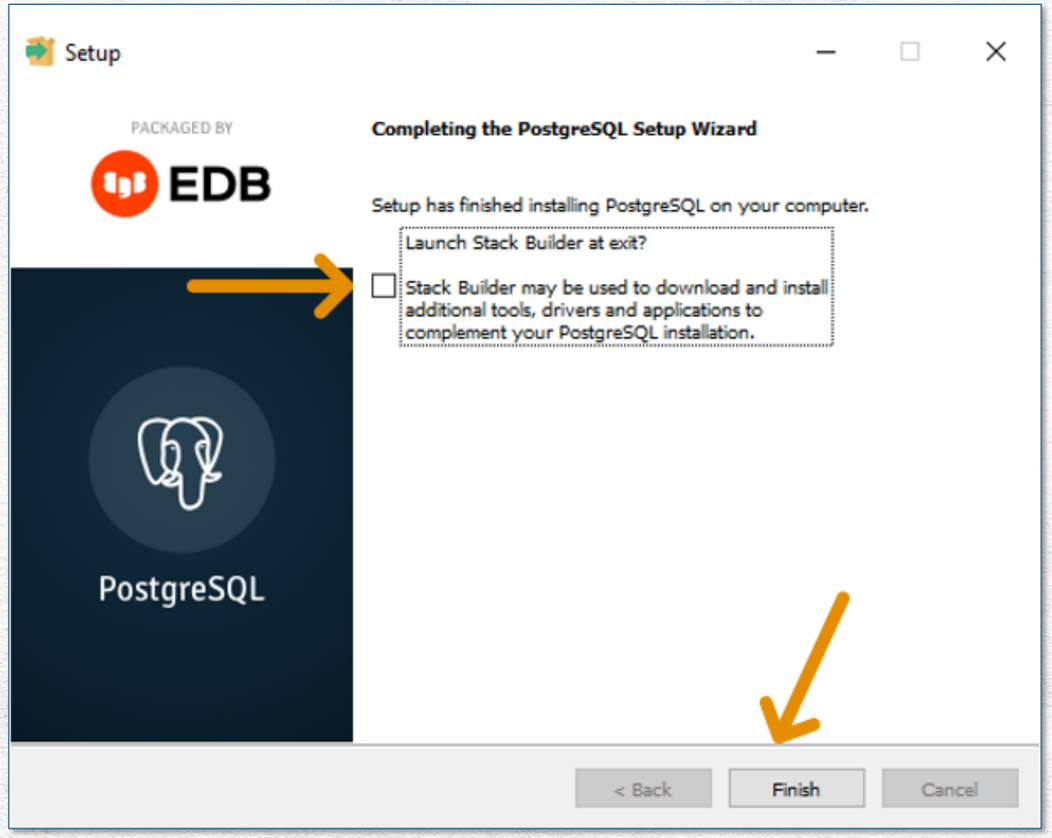
Instalação do PostgreSQL

Deixe que a instalação seja efetuada...



Instalação do PostgreSQL

Remova a seleção “Launch Stack Builder at exit?” e clique em “Finish”:

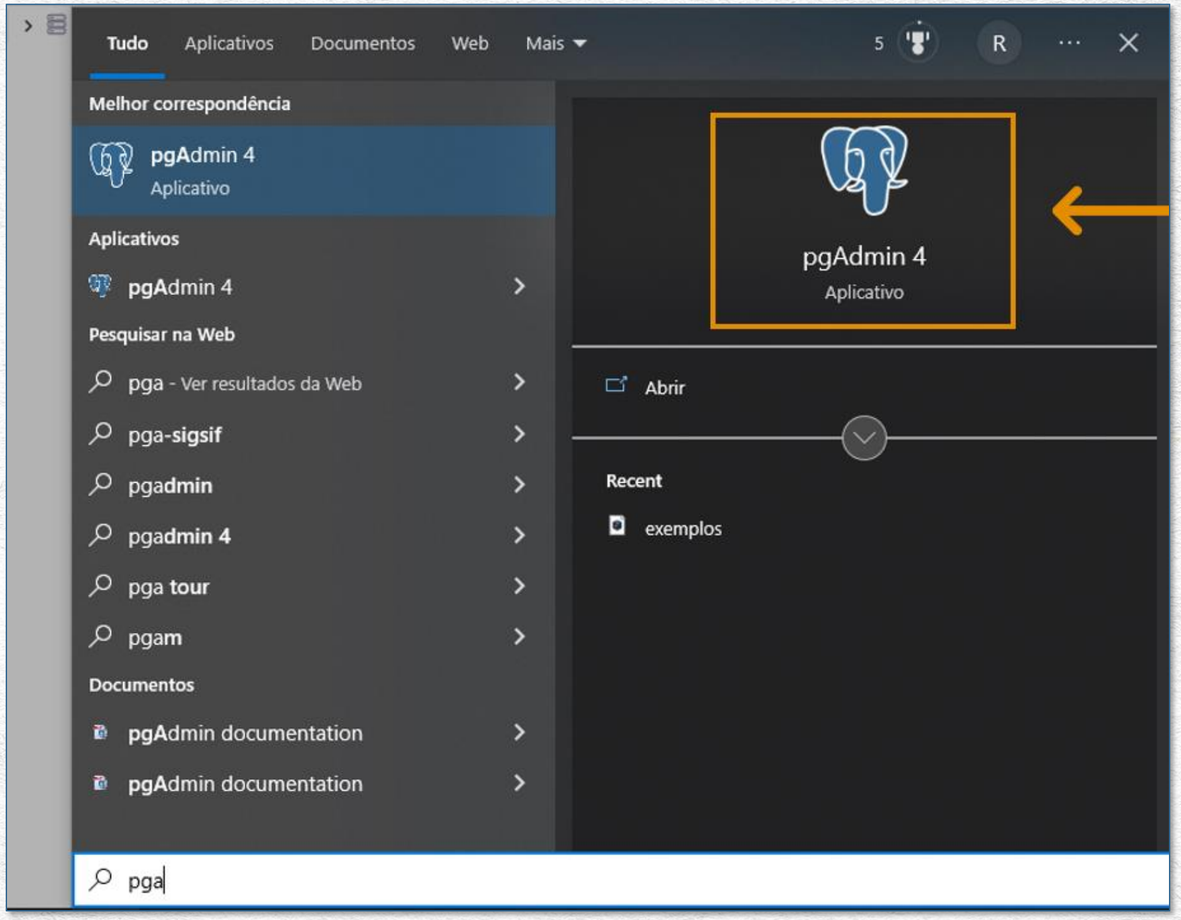


Abrindo o PostgreSQL / pgAdmin 4

Pronto, nosso SGBD está instalado!

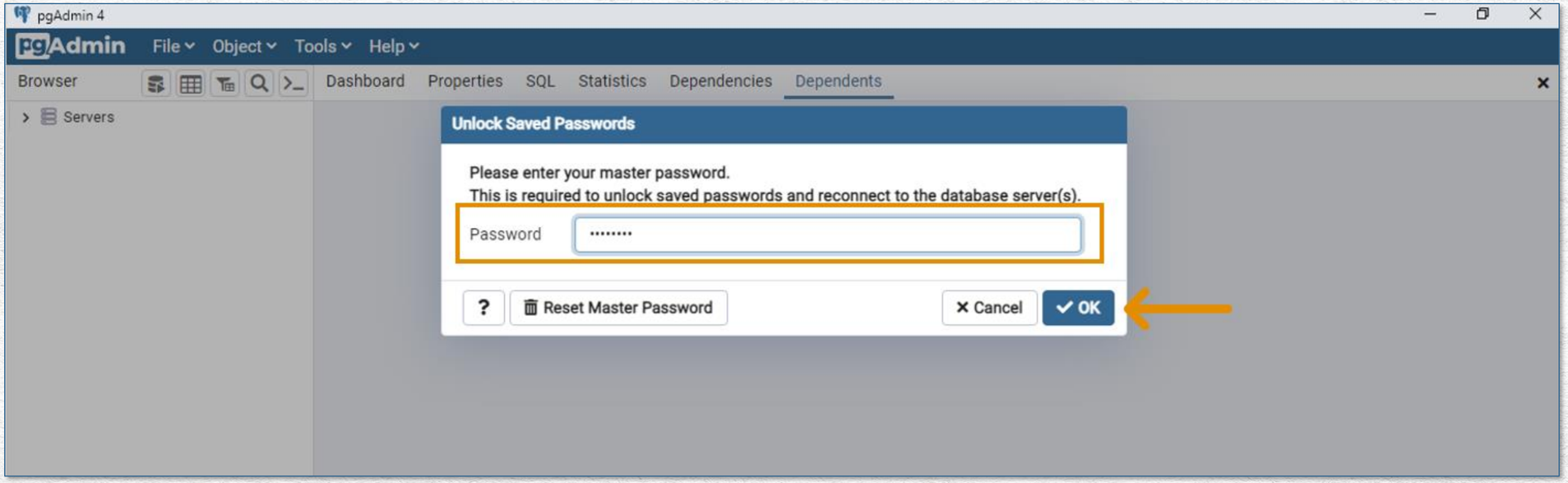
Agora, vamos fazer a importação das tabelas do banco de dados que utilizaremos no curso: o **Northwind**.

Para isso, primeiramente, vamos abrir o **pgAdmin 4**:



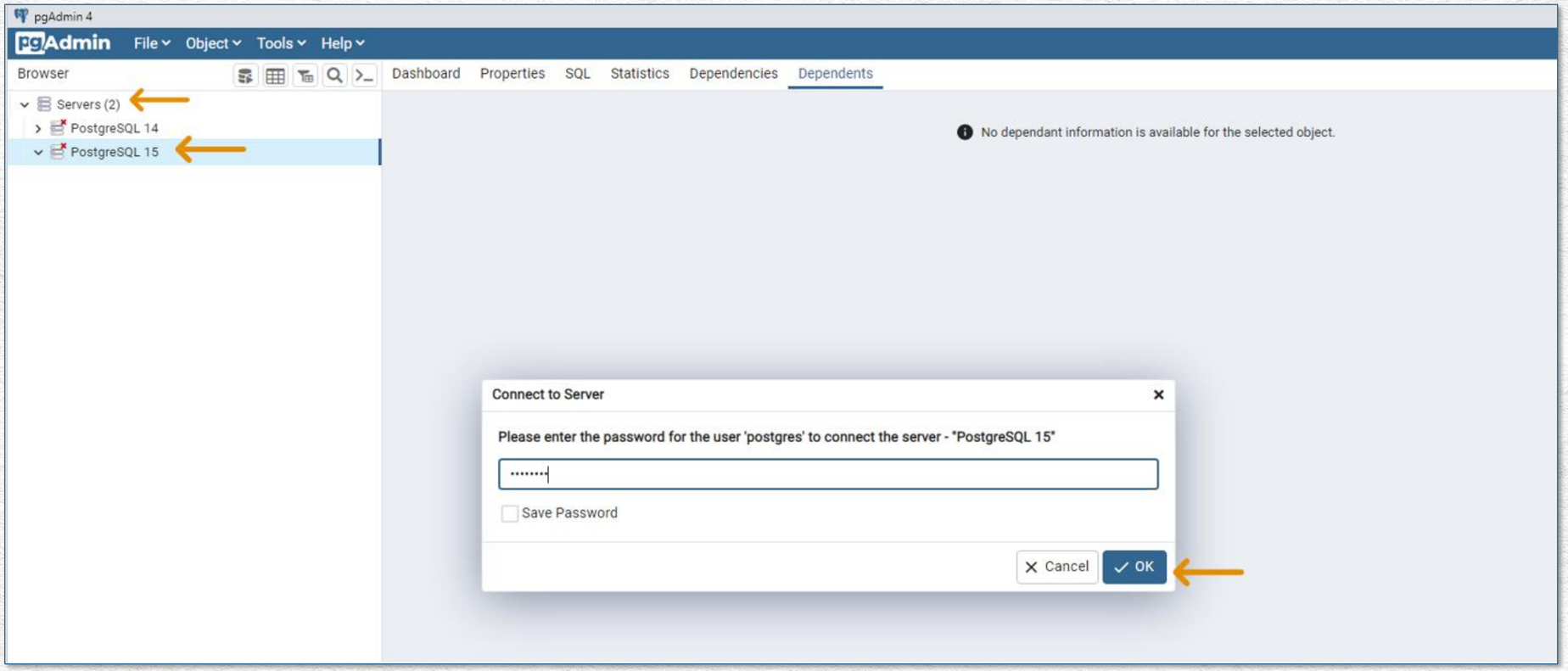
Abrindo o PostgreSQL / pgAdmin 4

Digitamos a senha que configuramos no momento da instalação e clicamos em OK:



Abrindo o PostgreSQL / pgAdmin 4

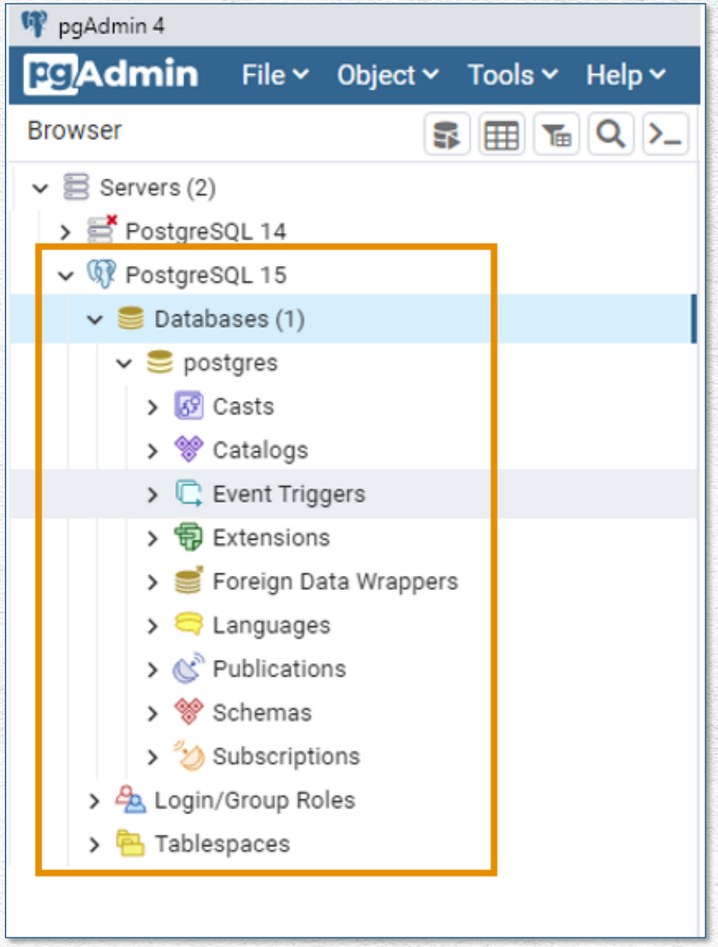
Após, clicamos em Servers, e selecionamos o servidor que queremos acessar.
Solicitada novamente a senha, a digitamos mais uma vez e clicamos em OK:



Abrindo o PostgreSQL / pgAdmin 4

Feito! Estabelecemos conexão com o servidor onde armazenaremos nossos bancos de dados.

Na próxima página, passaremos à importação das tabelas do BD Northwind.

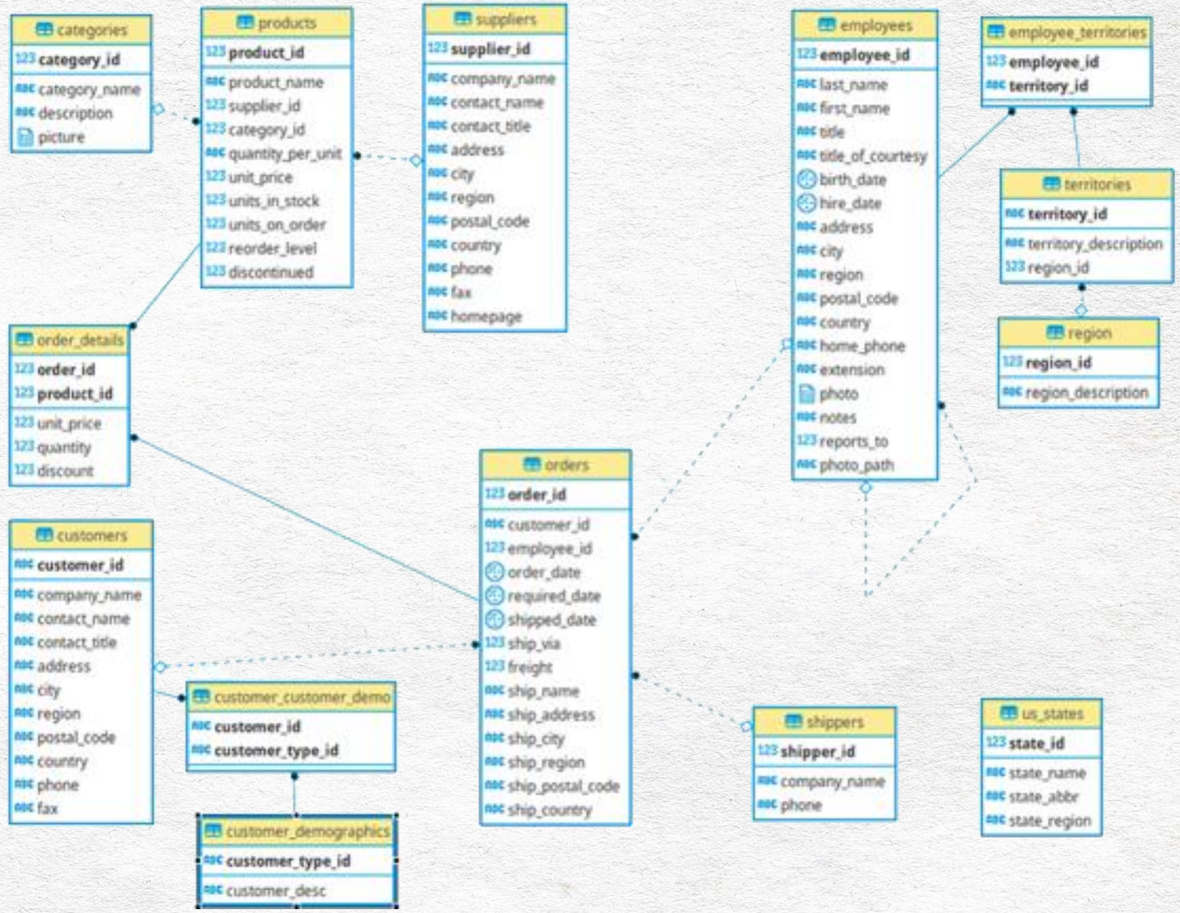


Criando as tabelas do BD Northwind

A Northwind Traders é uma organização fictícia da Microsoft que gerencia pedidos, produtos, clientes, fornecedores e outros aspectos de uma empresa.

Para a parte inicial do curso de PostgreSQL, vamos começar utilizando o seu banco de dados.

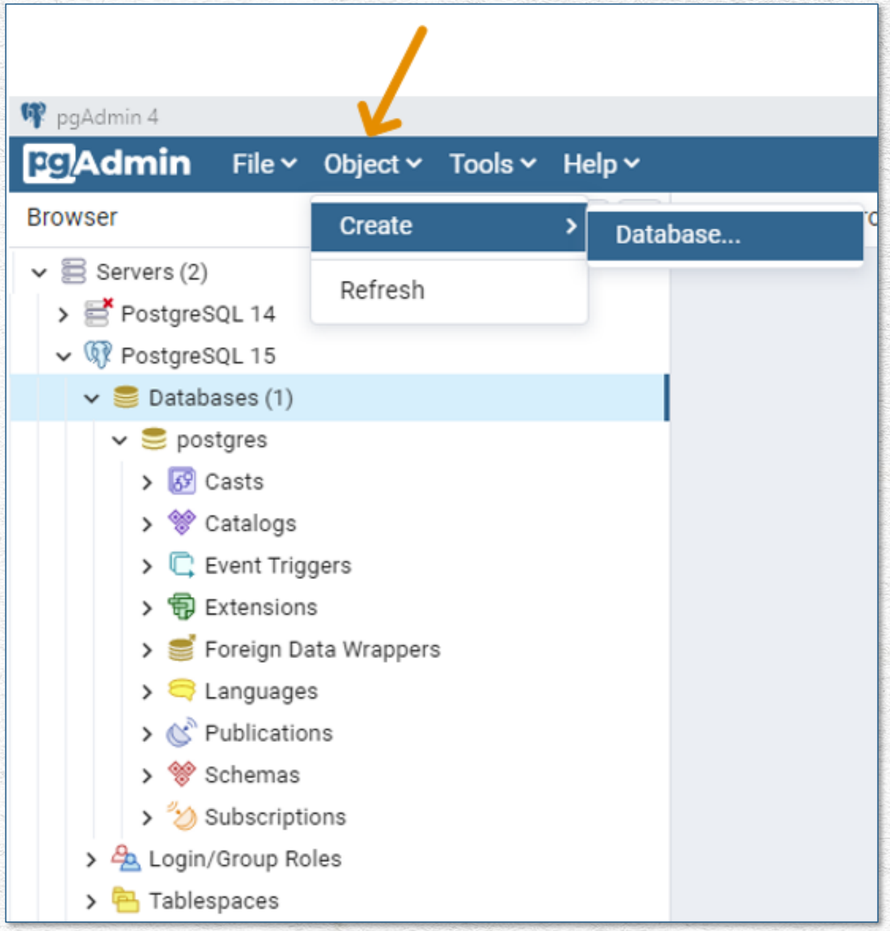
A estrutura do BD é mostrada ao lado.



Criando as tabelas do BD Northwind

Inicialmente, precisamos criar um banco de dados que receberá as tabelas que serão importadas.

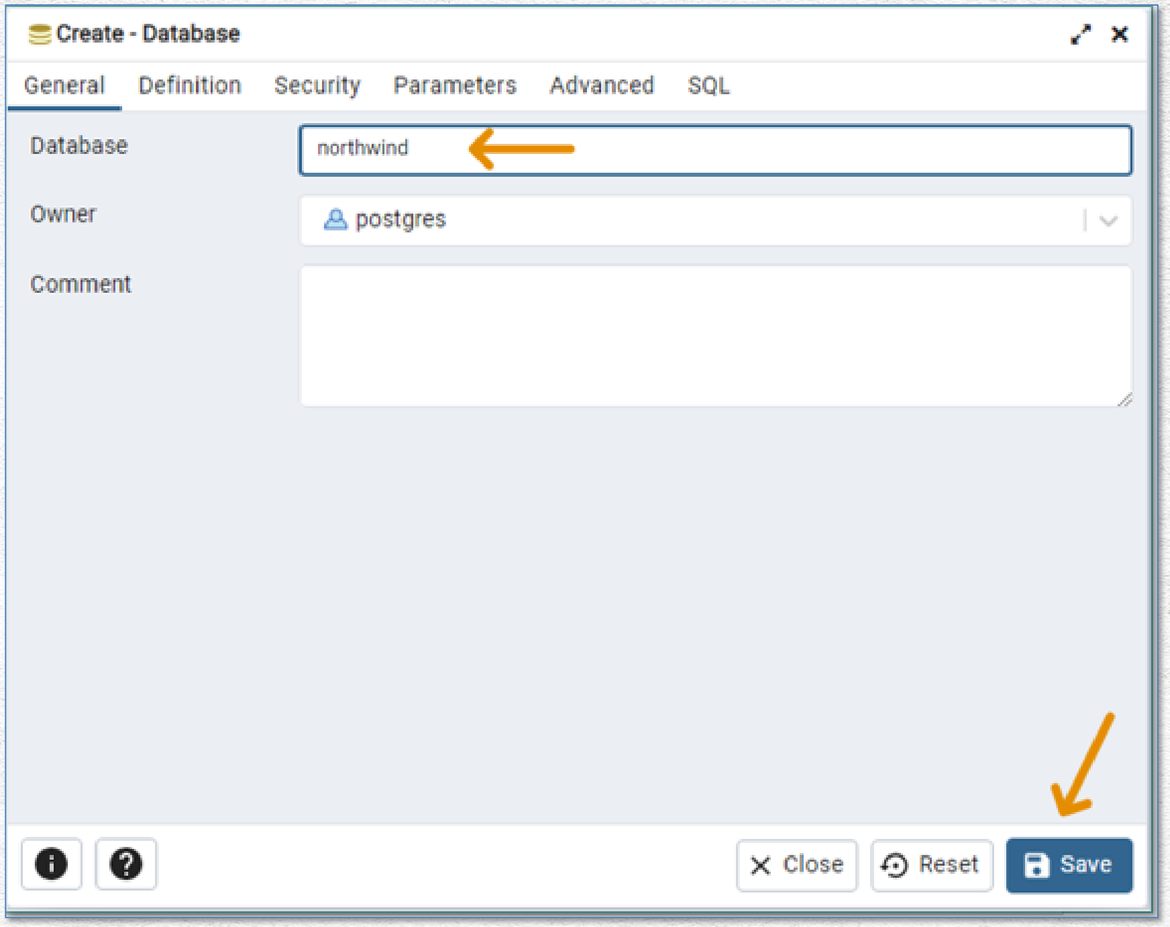
Para isso, clicamos em **Object > Create > Database...**



Criando as tabelas do BD Northwind

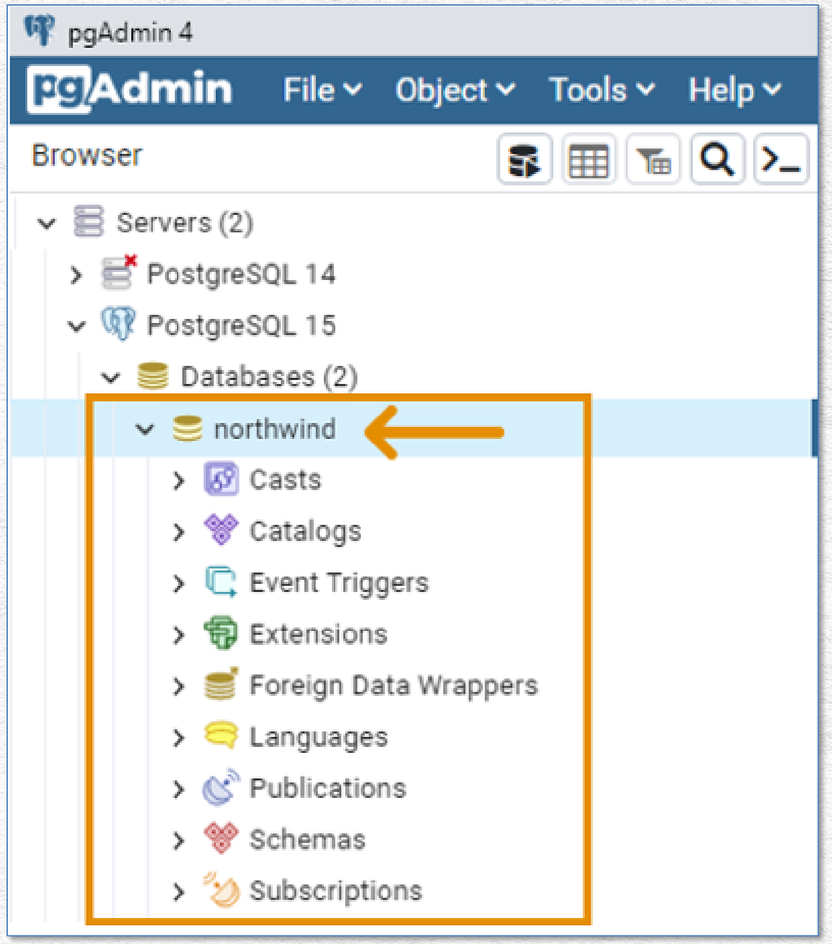
Feito isso, aparecerá na tela uma janela, na qual deveremos dar um nome ao nosso banco de dados, que será “northwind”.

Em seguida, clicamos em “Save”.



Criando as tabelas do BD Northwind

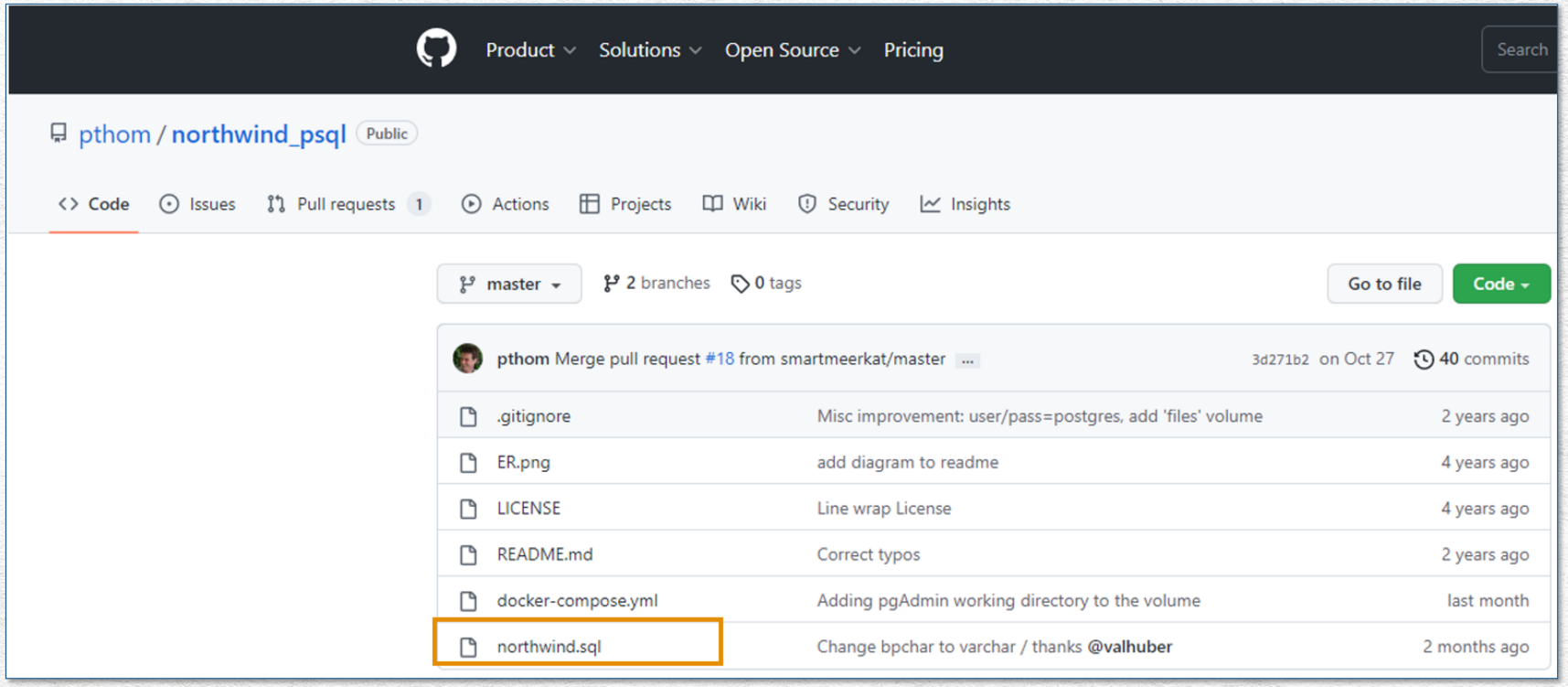
Criamos nosso banco de dados **Northwind**, agora vamos importar suas tabelas.



Criando as tabelas do BD Northwind

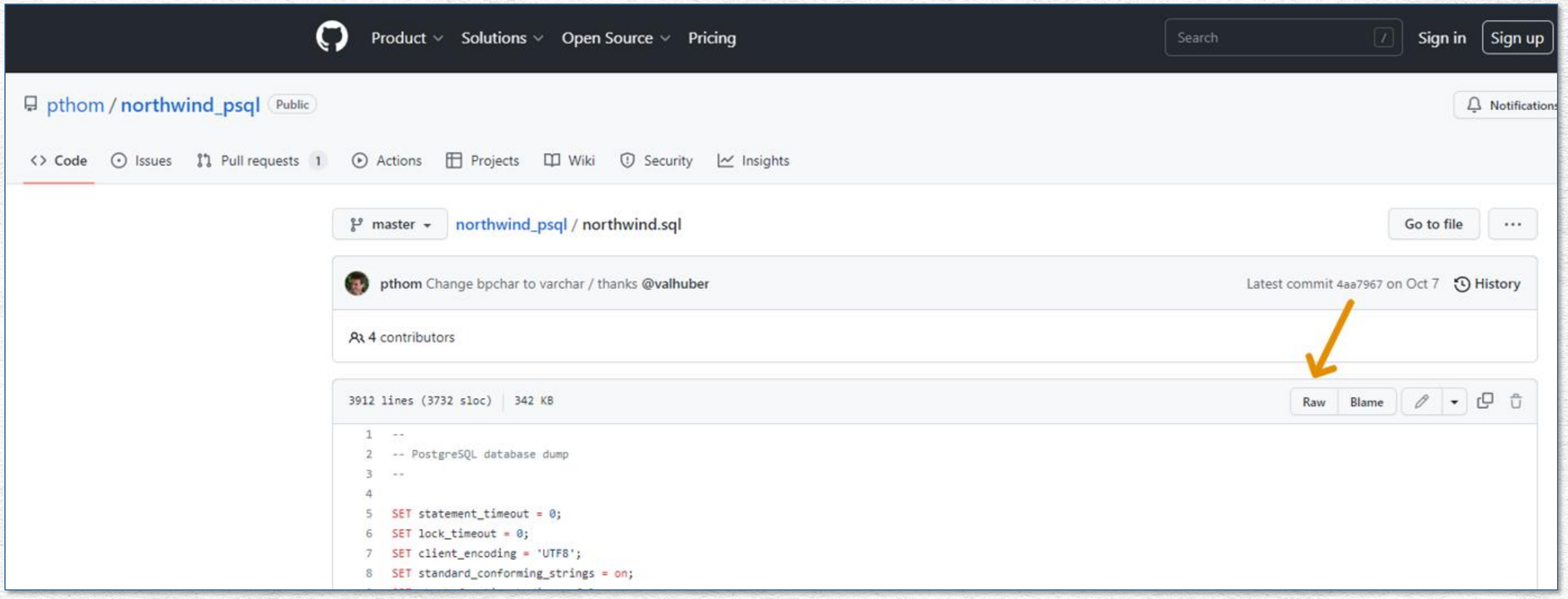
Para isso, acesse: https://github.com/pthom/northwind_psql

Clique em northwind.sql:



Criando as tabelas do BD Northwind

Na página que se abrir, clique em “Raw”:



Criando as tabelas do BD Northwind

Feito isso, teremos acesso ao código para criação das tabelas do banco de dados Northwind.

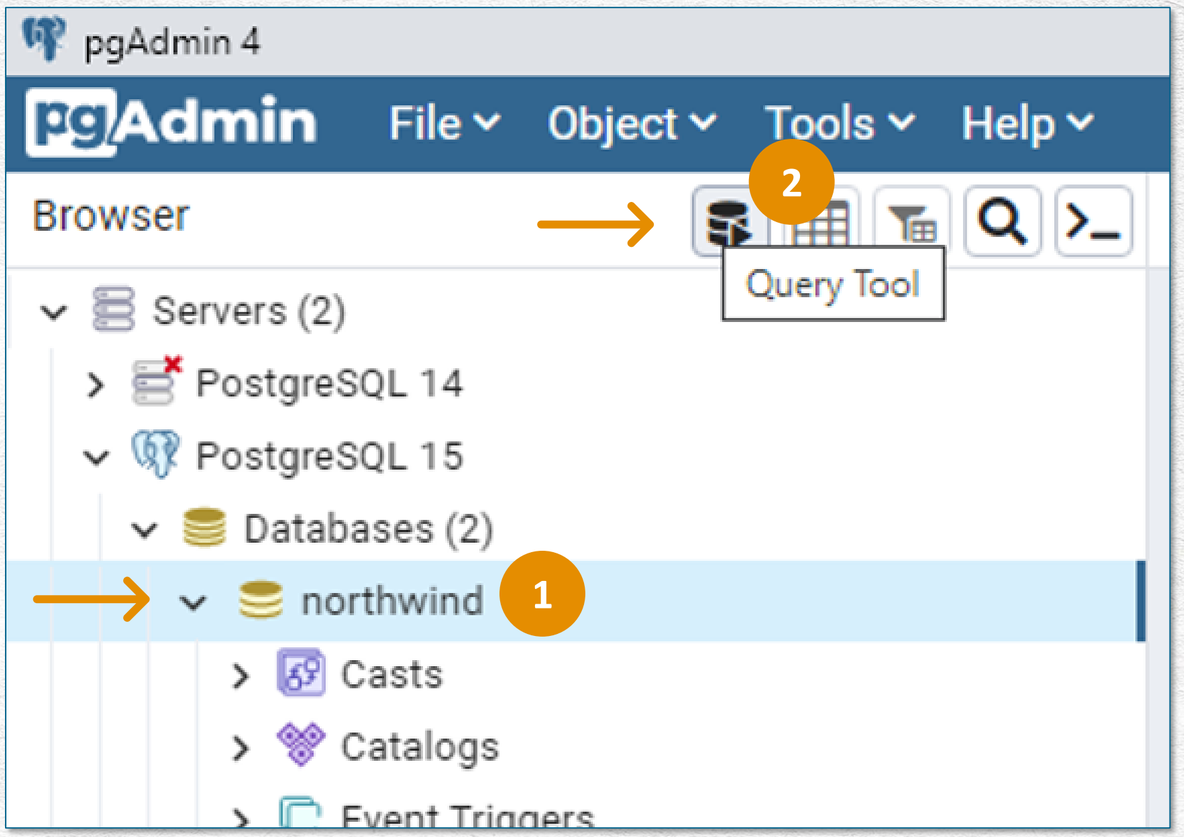
Para selecionar todo o código, aperte as teclas Crtl + A, depois Ctrl + C para copiar:

```
--
-- PostgreSQL database dump
--

SET statement_timeout = 0;
SET lock_timeout = 0;
SET client_encoding = 'UTF8';
SET standard_conforming_strings = on;
SET check_function_bodies = false;
SET client_min_messages = warning;



SET default_tablespace = '';

SET default_with_oids = false;


---
--- drop tables
---


DROP TABLE IF EXISTS customer_customer_demo;
DROP TABLE IF EXISTS customer_demographics;
DROP TABLE IF EXISTS employee_territories;
DROP TABLE IF EXISTS order_details;
DROP TABLE IF EXISTS orders;
DROP TABLE IF EXISTS customers;
DROP TABLE IF EXISTS products;
DROP TABLE IF EXISTS shippers;
DROP TABLE IF EXISTS suppliers;
DROP TABLE IF EXISTS territories;
DROP TABLE IF EXISTS us_states;
DROP TABLE IF EXISTS categories;
DROP TABLE IF EXISTS region;
DROP TABLE IF EXISTS employees;

--
-- Name: categories; Type: TABLE; Schema: public; Owner: -; Tablespace:
--

CREATE TABLE categories (
    category_id smallint NOT NULL,
    category_name character varying(15) NOT NULL,
    description text,
    picture bytea
);


--
```

Criando as tabelas do BD Northwind

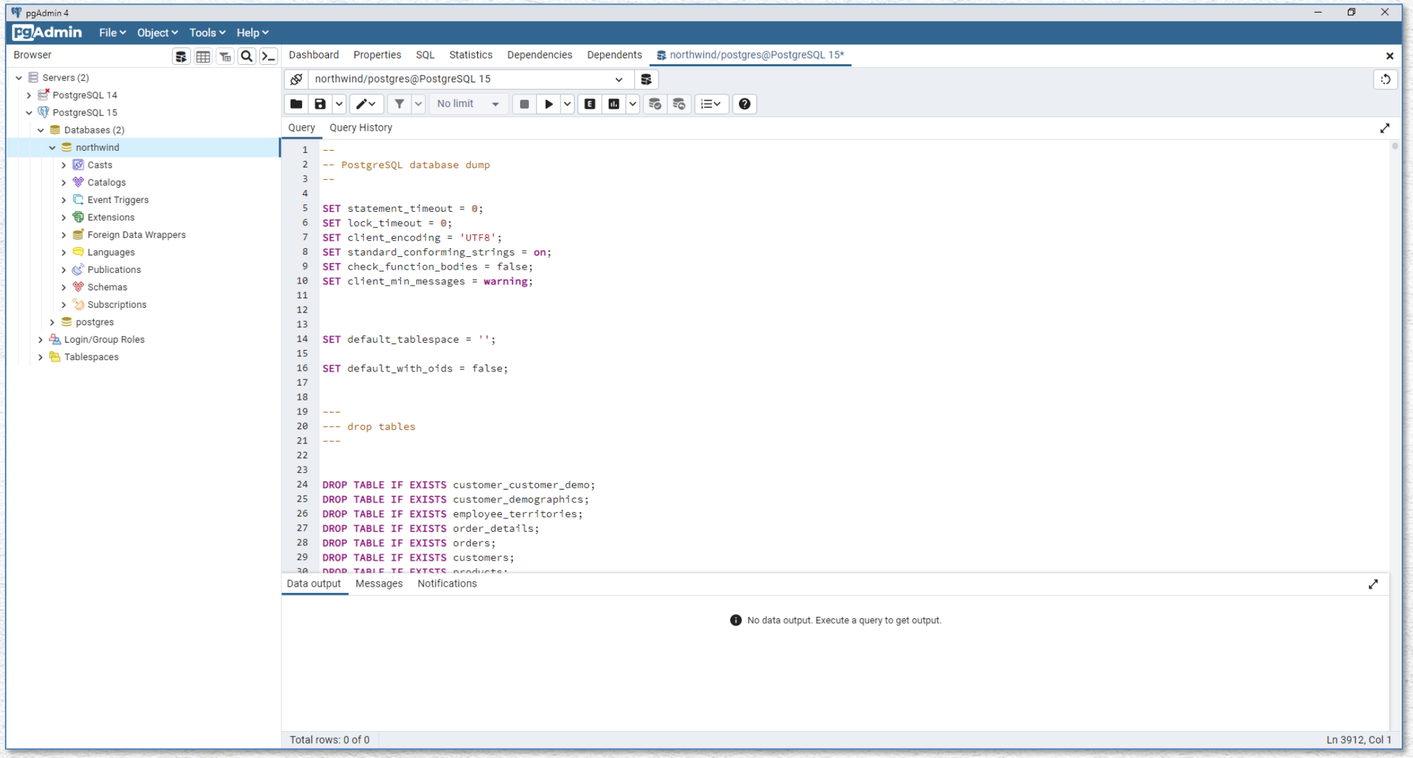
Agora voltamos ao pgAdmin 4, clicamos sobre o nome do banco de dados Northwind (1), em seguida no ícone Query Tool (2):

Dessa forma, será aberto o editor no qual poderemos realizar consultas e manipular nosso banco de dados.



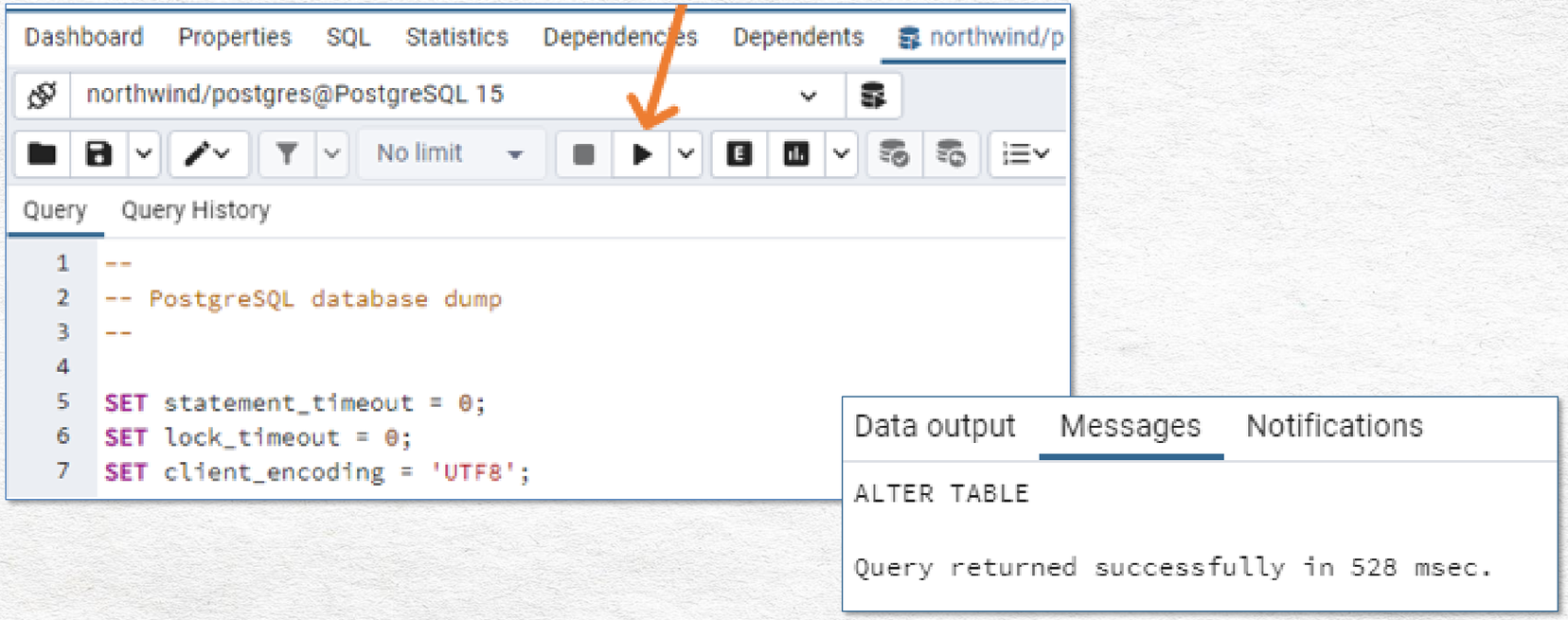
Criando as tabelas do BD Northwind

Dentro do editor, colamos o código das tabelas, utilizando as teclas Ctrl + V:



Criando as tabelas do BD Northwind

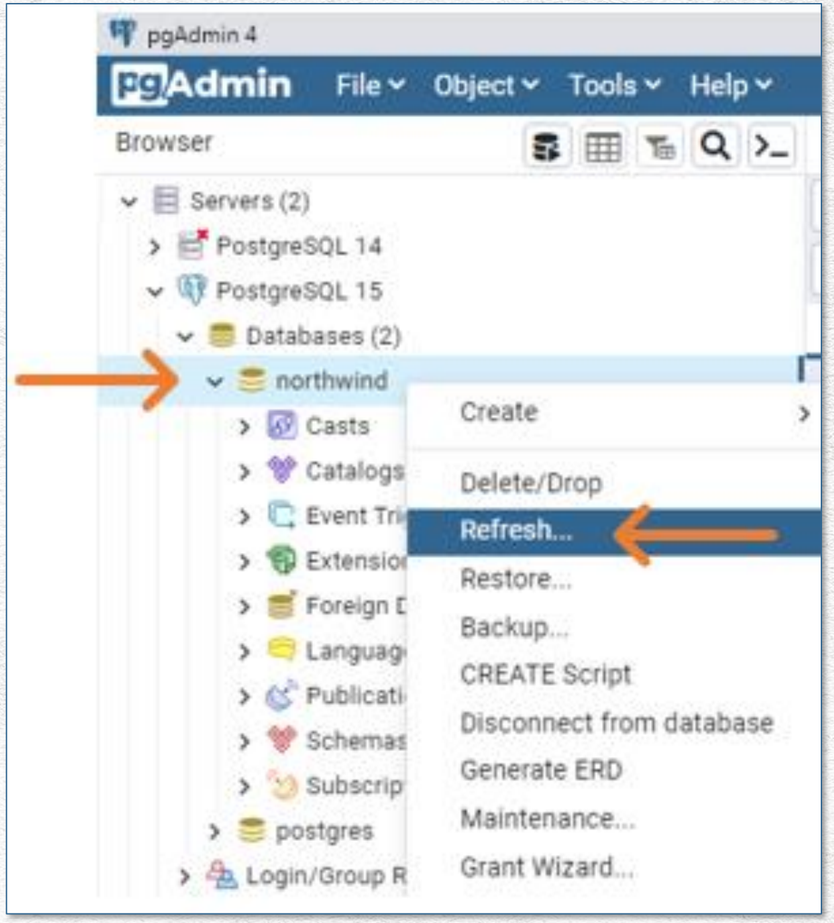
Para executar o código, clicamos no ícone indicado na imagem abaixo:



Criando as tabelas do BD Northwind

Executado o código, devemos atualizar nosso banco de dados para que as tabelas importadas apareçam.

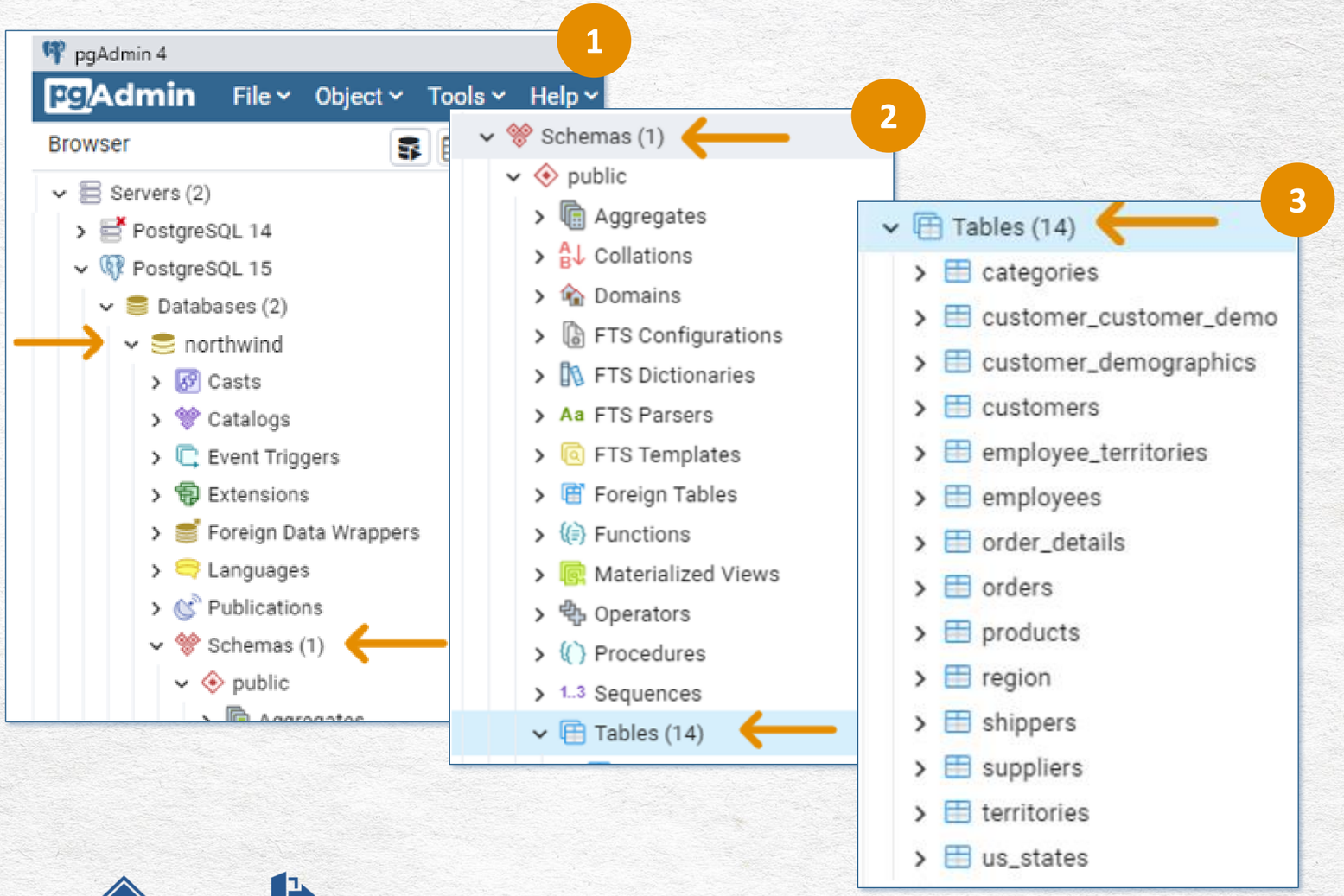
Para isso, clicamos com o botão direito sobre o nome do BD Northwind, em seguida em “Refresh”:



Criando as tabelas do BD Northwind

Pronto! Agora nossas tabelas foram devidamente importadas para o BD Northwind.

Para verificar, podemos clicar em **northwind > Schemas > Tables**. Assim, veremos os nomes das 14 tabelas que foram importadas:



MÓDULO 2

CRIANDO QUERIES BÁSICAS

CRIANDO QUERIES BÁSICAS

CRIANDO QUERIES BÁSICAS

O QUE VEREMOS NESTE MÓDULO

MÓDULO 2 CRIANDO QUERIES BÁSICAS NO POSTGRESQL

01	SELECT FROM - Selecionando dados das tabelas	041
02	Comentários no PostgreSQL	044
03	SELECT AS – Aliasing (renomeando) colunas	045
04	SELECT LIMIT – Limitando a quantidade de linhas de uma query	047
05	SELECT DISTINCT – Selecionando valores distintos de uma coluna	048
06	EXERCÍCIOS	050
07	GABARITO EXPLICATIVO	055

SELECT FROM – Selecionando dados das tabelas

O SELECT é o comando utilizado para selecionar dados de uma tabela.

Com ele, podemos retornar todas as colunas de uma tabela, somente uma ou algumas delas.

1 SELECIONANDO TODAS AS COLUNAS DA TABELA:

A primeira maneira é selecionar todas as colunas de uma determinada tabela. Para isso, usamos o comando **SELECT**, seguido do caractere * (asterisco). Este asterisco significa que queremos exibir todas as colunas.

A seguir, após o comando **FROM**, informamos o nome da tabela da qual queremos visualizar essas colunas.

Imagine que queremos visualizar todas as colunas da tabela categories do BD Northwind. Para isso, utilizamos o código ao lado:

```
-- Selecionando todas as colunas:

SELECT
    *
FROM categories;
```

Data output | Messages | Notifications

	category_id [PK] smallint	category_name character varying (15)	description text	picture bytea
1	1	Beverages	Soft drinks,...	[binary da...
2	2	Condiments	Sweet and ...	[binary da...
3	3	Confections	Desserts, c...	[binary da...
4	4	Dairy Products	Cheeses	[binary da...
5	5	Grains/Cereals	Breads, cra...	[binary da...
6	6	Meat/Poultry	Prepared m...	[binary da...
7	7	Produce	Dried fruit ...	[binary da...
8	8	Seafood	Seaweed a...	[binary da...

SELECT FROM – Selecionando dados das tabelas

O SELECT é o comando utilizado para selecionar dados de uma tabela.

Com ele, podemos retornar todas as colunas de uma tabela, somente uma ou algumas delas.

2 SELECIONANDO APENAS UMA COLUNA DA TABELA:

Também podemos selecionar somente uma coluna de uma determinada tabela. Para isso, usamos o comando **SELECT**, seguido do nome da coluna que queremos selecionar.

A seguir, após o comando **FROM**, informamos o nome da tabela da qual queremos visualizar essa coluna.

Imagine que queremos saber o primeiro nome dos funcionários da Northwind Traders. Para isso, precisamos visualizar a coluna first_name da tabela employees. Portanto, utilizamos o código ao lado:

```
-- Selecionando apenas uma coluna:

SELECT
    first_name
FROM employees;
```

Data output | Messages | Notifications

	first_name character varying (10)
1	Nancy
2	Andrew
3	Janet
4	Margaret
5	Steven
6	Michael
7	Robert
8	Laura
9	Anne

PostgreSQL

SELECT FROM – Selecionando dados das tabelas

O SELECT é o comando utilizado para selecionar dados de uma tabela.

Com ele, podemos retornar todas as colunas de uma tabela, somente uma ou algumas delas.

3 SELECIONANDO MAIS DE UMA COLUNA DA TABELA:

Por fim, podemos também selecionar mais de uma coluna de uma determinada tabela. Para isso, usamos o comando **SELECT**, seguido dos nomes das colunas que queremos selecionar, separados por vírgula.

A seguir, após o comando **FROM**, informamos o nome da tabela da qual queremos visualizar essas colunas.

Imagine que queremos saber o ID, o nome e o preço unitário dos produtos comercializados pela Northwind Traders. Para isso, precisamos visualizar as colunas product_id, product_name e unit_price da tabela products. Portanto, utilizamos o código ao lado:

```sql
-- Selecionando mais de uma coluna:

SELECT
    product_id,
    product_name,
    unit_price
FROM products;
```

Data output | Messages | Notifications

	product_id [PK] smallint	product_name character varying (40)	unit_price real
1	1	Chai	18
2	2	Chang	19
3	3	Aniseed Syrup	10
4	4	Chef Anton's Cajun Se...	22
5	5	Chef Anton's Gumbo ...	21.35
6	6	Grandma's Boysenber...	25
7	7	Uncle Bob's Organic D...	30
8	8	Northwoods Cranberr...	40
9	9	Mishi Kobe Niku	97

Comentários no PostgreSQL

Comentários são uma boa prática para garantir o bom entendimento de um código.

Os comentários não são considerados parte do código quando executado.

Conforme vamos criando consultas cada vez mais complexas, os comentários podem ser muito úteis para ajudar no entendimento do que está sendo feito.

Existem duas formas de comentar códigos:

Ao lado, na opção 1, utilizamos o hífen duplo para comentar uma única linha de código.

Já na opção 2, utilizamos uma barra seguida de um asterisco para identificar onde começa um comentário, e utilizamos o asterisco seguido de uma barra para identificar onde termina um comentário. Assim, podemos fazer comentários com múltiplas linhas.

1

```
-- Este é um comentário de 1 linha
```

2

```
/*

Este

é um comentário

...

de múltiplas linhas

*/
```

SELECT AS – Aliasing (renomeando) colunas

Colunas de uma tabela podem ser renomeadas por meio do comando AS.

Sendo um texto único, o nome da nova coluna pode ser escrito com ou sem as aspas duplas.

Caso precise renomear com um nome composto, utilize as aspas duplas, porém nomes compostos não são considerados uma boa prática.

Também utilize as aspas duplas caso queira preservar as letras maiúsculas do seu alias, pois, sem elas, o SQL considera todas as letras como minúsculas, independentemente da forma como você digitou no seu código. Veja:

```
63  -- sem aspas, o alias fica todo minúsculo
64  SELECT
65      product_id AS ID_Produto,
66      product_name AS Nome_Produto,
67      unit_price AS Preco_Unitario
68  FROM products;
```

Alias sem aspas duplas

Data output | Messages | Notifications

	id_produto smallint	nome_produto character varying (40)	preco_unitario real
1	1	Chai	18
2	2	Chang	19
3	3	Aniseed Syrup	10

Todas as letras minúsculas

```
70  -- com aspas, o alias preserva as letras maiúsculas
71  SELECT
72      product_id AS "ID_Produto",
73      product_name AS "Nome_Produto",
74      unit_price AS "Preco_Unitario"
75  FROM products;
```

Alias com aspas duplas

Data output | Messages | Notifications

	ID_Produto smallint	Nome_Produto character varying (40)	Preco_Unitario real
1	1	Chai	18
2	2	Chang	19
3	3	Aniseed Syrup	10

Preservou as letras maiúsculas

SELECT AS – Aliasing (renomeando) colunas

Podemos também usar o alias com os nomes das tabelas.

Isso será importante quando precisarmos, mais para frente em nossos estudos, informar junto ao nome da coluna à qual tabela ela pertence. Assim, não precisaremos repetir todo o nome da tabela à esquerda dos nomes das colunas, somente seu alias:

```sql
SELECT
    p.product_id AS "ID_Produto",
    p.product_name AS "Nome_Produto",
    p.unit_price AS "Preco_Unitario"
FROM products AS p;
```

O alias não altera os nomes das colunas ou da tabela no banco de dados em si, somente na visualização da *query* (consulta).

SELECT LIMIT – Limitando a quantidade de linhas

O comando LIMIT permite que selecionemos apenas as N primeiras linhas de uma tabela.

É ótimo para fazer o reconhecimento de uma tabela, principalmente se esta for muito extensa, com muitos registros (linhas).

Por exemplo: a tabela orders do BD Northwind possui 830 registros. Vamos supor que queremos retornar apenas as 100 primeiras dentre essas linhas. Para isso, utilizamos o LIMIT ao final do nosso SELECT, desta forma:

```
SELECT * FROM orders
LIMIT 100;
```

Data output | Messages | Notifications

	order_id [PK] smallint	customer_id character varying (5)	employee_id smallint	order_date date	required_date date	shipped_date date	ship_via smallint	freight real	ship_name character varying (40)	ship_address character varying (60)	ship_city character varying (15)	ship_region character varying (15)	ship_postal_code character varying (10
1	10248	VINET	5	1996-07-04	1996-08-01	1996-07-16	3	32.38	Vins et alcools Cheval...	59 rue de l'Abbaye	Reims	[null]	51100
2	10249	TOMSP	6	1996-07-05	1996-08-16	1996-07-10	1	11.61	Toms Spezialitäten	Luisenstr. 48	Münster	[null]	44087
3	10250	HANAR	4	1996-07-08	1996-08-05	1996-07-12	2	65.83	Hanari Carnes	Rua do Paço, 67	Rio de Janeiro	RJ	05454-876
4	10251	VICTE	3	1996-07-08	1996-08-05	1996-07-15	1	41.34	Victuailles en stock	2, rue du Commerce	Lyon	[null]	69004

Total rows: 100 of 100 | Query complete 00:00:00.133 | Ln 76, Col 1

SELECT DISTINCT – Selecionando valores distintos

Quando selecionamos uma coluna de uma tabela, o SQL retorna todas as linhas dessa tabela.

Observe na imagem ao lado: como queremos descobrir as profissões dos clientes da Northwind Traders, criamos um código para trazer a coluna contact_title da tabela customers.

Porém, como todas as 91 linhas da tabela foram retornadas, tivemos várias profissões se repetindo. Isso porque podemos ter vários clientes que exercem a mesma profissão.

Como fazer, então, para retornar apenas as profissões (contact_title) distintas?

```
SELECT
    contact_title
FROM customers;
```

utput | Messages | Notifications

	contact_title character varying (30)
1	Sales Representative
2	Owner
3	Owner
4	Sales Representative
5	Order Administrator
6	Sales Representative
7	Marketing Manager
8	Owner
9	Owner
10	Accounting Manager
11	Sales Representative
12	Sales Agent
13	Marketing Manager
14	Owner
15	Sales Associate
16	Sales Representative
17	Order Administrator

Total rows: 91 of 91 | Query complete 00:00:00.081

PostgreSQL

SELECT DISTINCT – Selecionando valores distintos

É aí que entra o SELECT DISTINCT. Este comando nos permite retornar apenas os valores distintos de uma coluna, e sua aplicação é mostrada na imagem ao lado.

Observe que no resultado temos apenas as profissões (contact_title) distintas (12 no total), nenhuma está se repetindo.

```
SELECT DISTINCT
    contact_title
FROM customers;
```

ages Notifications

	contact_title character varying (30)
1	Sales Manager
2	Owner
3	Owner/Marketing Assistant
4	Accounting Manager
5	Sales Associate
6	Sales Agent
7	Assistant Sales Agent
8	Assistant Sales Representati...
9	Order Administrator
10	Marketing Manager
11	Sales Representative
12	Marketing Assistant

Total rows: 12 of 12 Query complete 00:00:00.088

EXERCÍCIOS

1

Questão 1

Os analistas de vendas desejam revisar as informações de todas as categorias de produtos.

Faça uma consulta que retorne essas informações. **(Utilize a tabela categories)**

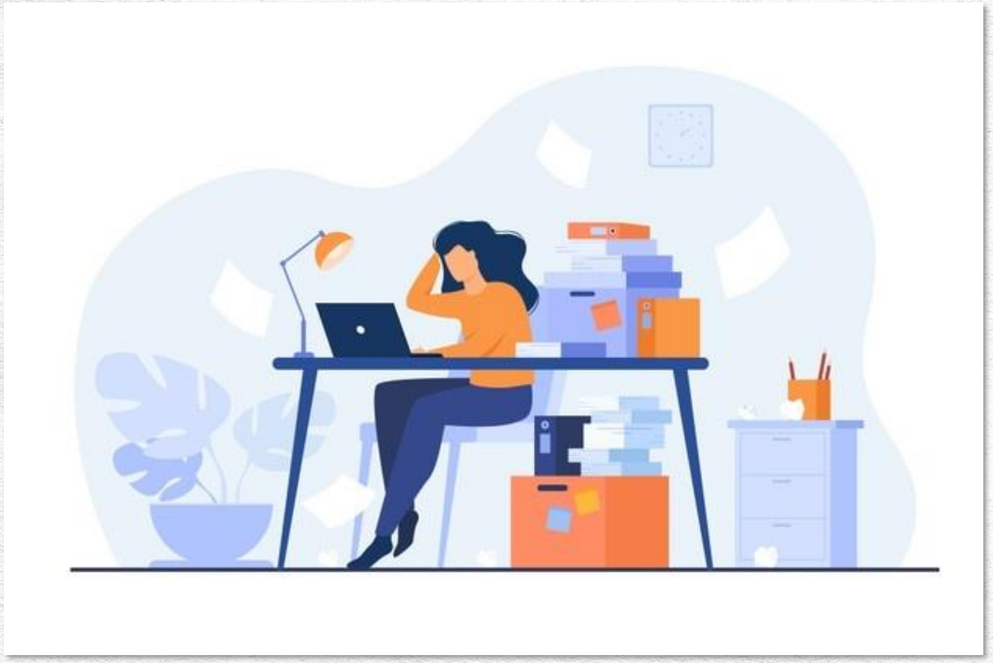
2

Questão 2

A equipe de logística precisa dos nomes de todos os fornecedores cadastrados no sistema.

Faça uma consulta ao banco de dados que liste todos esses nomes. **(Utilize a tabela suppliers)**

EXERCÍCIOS



3

Questão 3

A equipe de atendimento deseja analisar os dados de contato de clientes, mas apenas os 15 primeiros.

Faça uma consulta que retorne as colunas contact_name e contact_title da tabela customers, com limite de 15 registros. **(Utilize a tabela customers)**

4

Questão 4

O time de vendas deseja saber onde os clientes estão localizados.

Faça uma consulta que retorne os nomes únicos de suas cidades. **(Utilize a tabela customers)**

EXERCÍCIOS

5

Questão 5

A diretoria quer analisar os diferentes países onde os fornecedores estão localizados.

Crie uma consulta que retorne os nomes distintos desses países. **(Utilize a tabela suppliers)**

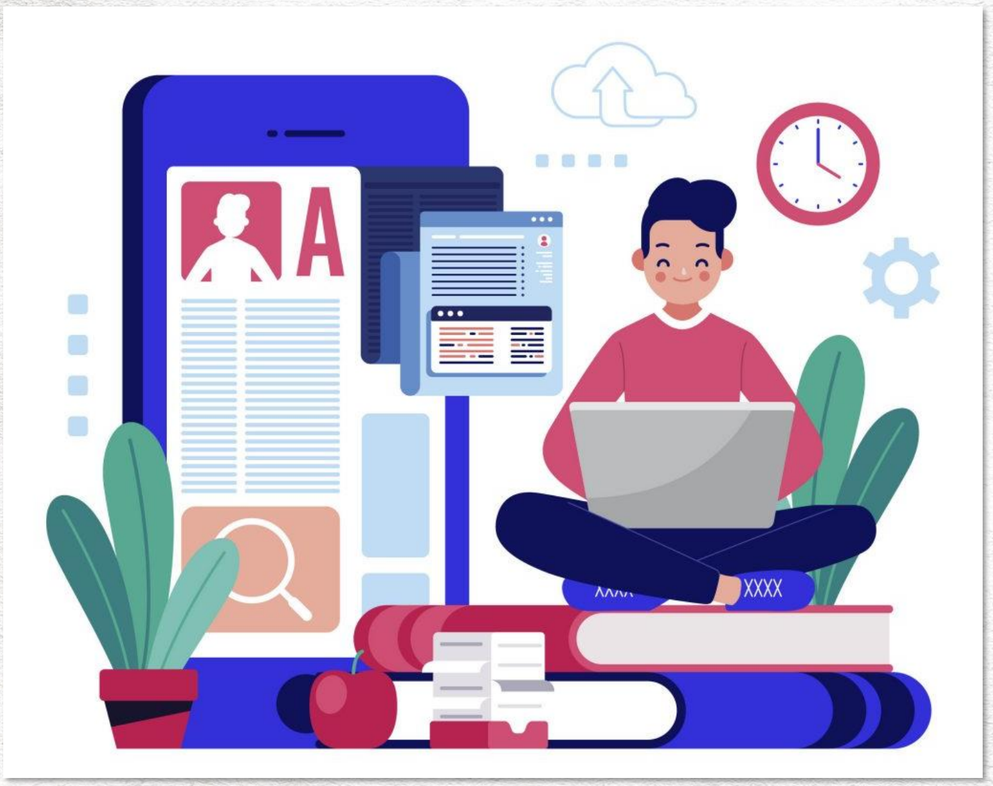
6

Questão 6

A equipe de produtos quer acessar uma lista com os IDs e nomes das categorias, mas renomeando as colunas para "ID_Categoria" e "Categoria".

Construa essa consulta. **(Utilize a tabela categories)**

EXERCÍCIOS



7

Questão 7

Os gerentes precisam verificar os nomes completos dos funcionários e seus títulos (funções) no sistema, com as colunas renomeadas como nome_funcionario, sobrenome_funcionario e funcao.

Crie essa consulta. **(Utilize a tabela employees)**

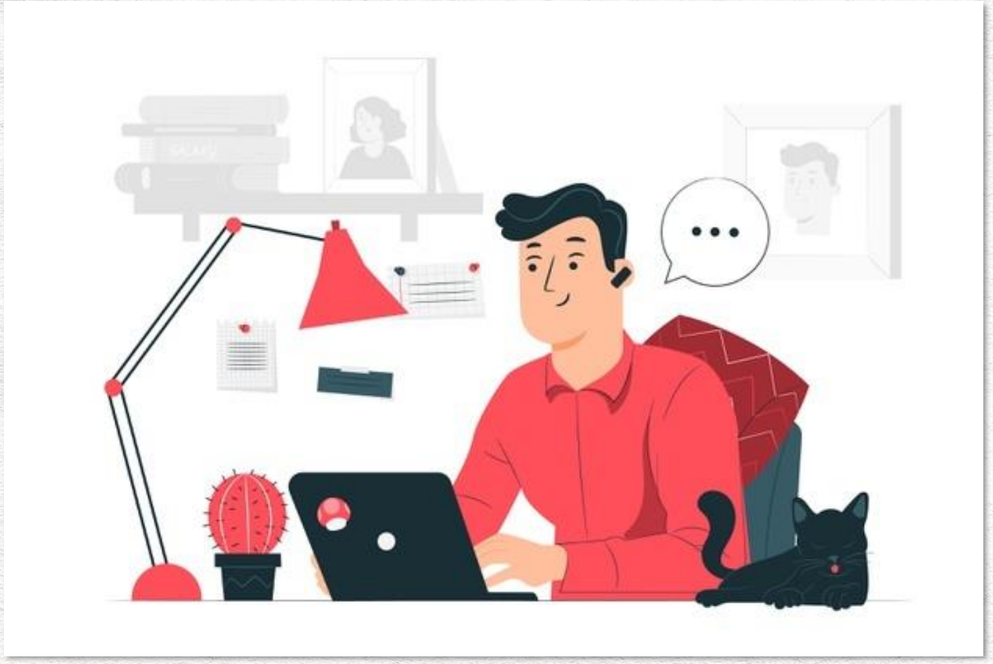
8

Questão 8

O departamento de recursos humanos precisa verificar os países onde os funcionários estão registrados, renomeando a coluna para "País".

Liste esses valores. **(Utilize a tabela employees)**

EXERCÍCIOS



9

Questão 9

A equipe de logística quer verificar os nomes e IDs dos transportadores cadastrados, renomeando as colunas shipper_id e company_name para português (ID_Transportadora e Nome_Transportadora, respectivamente).

Construa essa consulta. **(Utilize a tabela shippers)**

10

Questão 10

A equipe administrativa deseja obter os IDs e os nomes dos fornecedores, bem como seus respectivos países, renomeando as colunas como "ID_Fornecedor", "Fornecedor" e "País", mas limitando o resultado aos 8 primeiros registros.

Crie a consulta que retorna o resultado esperado. **(Utilize a tabela suppliers)**

GABARITO EXPLICATIVO

1

Questão 1

Os analistas de vendas desejam revisar as informações de todas as categorias de produtos.

Faça uma consulta que retorne essas informações. **(Utilize a tabela categories)**

Explicação e Resultado

Para que os analistas possam revisar as **informações de todas as categorias de produtos,** precisamos retornar todas as linhas e colunas da tabela **CATEGORIES**. Para isso, utilizamos o comando **SELECT**, conforme mostrado abaixo:

```
SELECT * FROM categories;
```

Dessa forma, obtemos o seguinte resultado:

	category_id [PK] smallint	category_name character varying (15)	description text	picture bytea
1	1	Beverages	Soft drinks, coffees, teas, beers, and ales	[binary dat...
2	2	Condiments	Sweet and savory sauces, relishes, spreads, and seasonin...	[binary dat...
3	3	Confections	Desserts, candies, and sweet breads	[binary dat...
4	4	Dairy Products	Cheeses	[binary dat...
5	5	Grains/Cereals	Breads, crackers, pasta, and cereal	[binary dat...
6	6	Meat/Poultry	Prepared meats	[binary dat...
7	7	Produce	Dried fruit and bean curd	[binary dat...
8	8	Seafood	Seaweed and fish	[binary dat...

Total rows: 8 of 8 | Query complete 00:00:00.222

GABARITO EXPLICATIVO

2

Questão 2

A equipe de logística precisa dos nomes de todos os fornecedores cadastrados no sistema.

Faça uma consulta ao banco de dados que liste todos esses nomes. **(Utilize a tabela suppliers)**

Explicação e Resultado

As informações sobre fornecedores estão cadastradas na tabela **SUPPLIERS**. Os nomes desses fornecedores estão na coluna **company_name**. Portanto, para retornar seus nomes devemos fazer um **SELECT** à tabela **SUPPLIERS** retornando a coluna **company_name**:

```
SELECT
    company_name
FROM suppliers;
```

Dessa forma, obtemos o resultado ao lado:

	company_name character varying (40)
1	Exotic Liquids
2	New Orleans Cajun Delights
3	Grandma Kelly's Homestead
4	Tokyo Traders
5	Cooperativa de Quesos 'Las Cabras'
6	Mayumi's
7	Pavlova, Ltd.
8	Specialty Biscuits, Ltd.
9	PB Knäckebröd AB
10	Refrescos Americanas LTDA
11	Heli Süßwaren GmbH & Co. KG
12	Plutzer Lebensmittelgroßmärkte AG
13	Nord-Ost-Fisch Handelsgesellschaft mbH
14	Formaggi Fortini s.r.l.
15	Norske Meierier
16	Bigfoot Breweries
17	Svensk Sjöföda AB
18	Aux joyeux ecclésiastiques
19	New England Seafood Cannery
20	Leka Trading
21	Lyngbysild
22	Zaanse Snoepfabriek
23	Karkki Oy
24	G'day, Mate
25	Ma Maison
26	Pasta Buttini s.r.l.
27	Escargots Nouveaux
28	Gai pâturage
29	Forêts d'érables

Total rows: 29 of 29 | Query complete 00:0

GABARITO EXPLICATIVO

3

Questão 3

A equipe de atendimento deseja analisar os dados de contato de clientes, mas apenas os 15 primeiros.

Faça uma consulta que retorne as colunas contact_name e contact_title da tabela customers, com limite de 15 registros. **(Utilize a tabela customers)**

Explicação e Resultado

Para efetuarmos a *query* solicitada no enunciado, devemos usar o comando **SELECT** para consultarmos a tabela **CUSTOMERS**, retornando as colunas **contact_name** e **contact_title**. Como queremos um retorno de apenas 15 registros, usamos um **LIMIT 15** ao final do código:

```
SELECT
    contact_name,
    contact_title
FROM customers
LIMIT 15;
```

Dessa forma, retornamos o resultado ao lado:

	contact_name character varying (30)	contact_title character varying (30)
1	Maria Anders	Sales Representative
2	Ana Trujillo	Owner
3	Antonio Moreno	Owner
4	Thomas Hardy	Sales Representative
5	Christina Berglund	Order Administrator
6	Hanna Moos	Sales Representative
7	Frédérique Citeaux	Marketing Manager
8	Martín Sommer	Owner
9	Laurence Lebihan	Owner
10	Elizabeth Lincoln	Accounting Manager
11	Victoria Ashworth	Sales Representative
12	Patricio Simpson	Sales Agent
13	Francisco Chang	Marketing Manager
14	Yang Wang	Owner
15	Pedro Afonso	Sales Associate
Total rows: 15 of 15		Query complete 00:00:00.092

GABARITO EXPLICATIVO

4

Questão 4

O time de vendas deseja saber onde os clientes estão localizados.

Faça uma consulta que retorne os nomes únicos de suas cidades. **(Utilize a tabela customers)**

Explicação e Resultado

Para retornamos apenas os **nomes únicos (sem duplicidade)** das cidades onde os clientes estão localizados, devemos aplicar um **SELECT DISTINCT** à coluna **city** da tabela **CUSTOMERS**:

```
SELECT DISTINCT
    city
FROM customers;
```

Dessa forma, retornamos um resultado como o mostrado ao lado, com 69 linhas:

	city character varying (15)
1	Butte
2	Paris
3	Cowes
4	Cork
5	Bern
6	Brandenburg
7	Aachen
8	Walla Walla
9	Warszawa
10	Münster
11	San Francisco
12	Eugene
13	Caracas

Total rows: 69 of 69 Query

GABARITO EXPLICATIVO

5

Questão 5

A diretoria quer analisar os diferentes países onde os fornecedores estão localizados.

Crie uma consulta que retorne os nomes distintos desses países. **(Utilize a tabela suppliers)**

Explicação e Resultado

Para retornamos apenas os **nomes distintos (sem duplicidade)** dos países onde os fornecedores estão localizados, devemos aplicar um **SELECT DISTINCT** à coluna **country** da tabela **SUPPLIERS**:

```
SELECT DISTINCT
    country
FROM suppliers;
```

Dessa forma, retornamos um resultado como o mostrado ao lado:

	country character varying (15)
1	Spain
2	Italy
3	Norway
4	Sweden
5	France
6	USA
7	Netherlands
8	Brazil
9	Australia
10	UK
11	Germany
12	Japan
13	Denmark
14	Singapore
15	Canada
16	Finland

Total rows: 16 of 16 Query

GABARITO EXPLICATIVO

6

Questão 6

A equipe de produtos quer acessar uma lista com os IDs e nomes das categorias, mas renomeando as colunas para "ID_Categoria" e "Categoria".

Construa essa consulta. **(Utilize a tabela categories)**

Explicação e Resultado

Para acessarmos uma lista com os IDs e os nomes das categorias, precisamos fazer um **SELECT** à tabela **CATEGORIES** que retorne as colunas **category_id** e **category_name.** Para retornarmos as colunas com outros nomes, utilizamos o comando **AS**, tornando a **category_id** em **ID_Categoria** e **category_name** em **Categoria**. Se quisermos preservar as **letras maiúsculas dos aliases**, devemos colocá-los entre **aspas duplas**:

```
SELECT
    category_id AS "ID_Categoria",
    category_name AS "Categoria"
FROM categories;
```

Dessa forma, retornamos o resultado abaixo, com as colunas devidamente renomeadas:

	ID_Categoria smallint	Categoria character varying (15)
1	1	Beverages
2	2	Condiments
3	3	Confections
4	4	Dairy Products
5	5	Grains/Cereals
6	6	Meat/Poultry
7	7	Produce
8	8	Seafood

Total rows: 8 of 8 | Query complete 00:00:0

GABARITO EXPLICATIVO

7

Questão 7

Os gerentes precisam verificar os nomes completos dos funcionários e seus títulos (funções) no sistema, com as colunas renomeadas como nome_funcionario, sobrenome_funcionario e funcao.

Crie essa consulta. **(Utilize a tabela employees)**

Explicação e Resultado

Para consultarmos os nomes completos e os títulos (funções) dos funcionários, precisamos fazer um **SELECT** à tabela **EMPLOYEES** que retorne as colunas **first_name**, **last_name** e **title.** Para retornarmos as colunas com outros nomes, utilizamos o comando **AS**, tornando a **first_name** em **nome_funcionario**, a **last_name** em **sobrenome_funcionario** e a **title** em **funcao**. Como os **aliases estão todos com letras minúsculas**, envolve-los em **aspas duplas é opcional**:

```
SELECT
    first_name AS nome_funcionario,
    last_name AS sobrenome_funcionario,
    title AS funcao
FROM employees;
```

Dessa forma, retornamos o resultado abaixo, com as colunas devidamente renomeadas:

	nome_funcionario character varying (10)	sobrenome_funcionario character varying (20)	funcao character varying (30)
1	Nancy	Davolio	Sales Representative
2	Andrew	Fuller	Vice President, Sales
3	Janet	Leverling	Sales Representative
4	Margaret	Peacock	Sales Representative
5	Steven	Buchanan	Sales Manager
6	Michael	Suyama	Sales Representative
7	Robert	King	Sales Representative
8	Laura	Callahan	Inside Sales Coordinator
9	Anne	Dodsworth	Sales Representative
Total rows: 9 of 9	Query complete 00:00:00.092		

GABARITO EXPLICATIVO

8

Questão 8

O departamento de recursos humanos precisa verificar os países onde os funcionários estão registrados, renomeando a coluna para "País".

Liste esses valores. **(Utilize a tabela employees)**

Explicação e Resultado

Para verificarmos os países em que os funcionários estão registrados, podemos fazer um **SELECT DISTINCT** à coluna **country** (para não retornarmos nomes de países em duplicidade) da tabela **EMPLOYEES**. Como queremos renomear a coluna country para **País**, usamos o comando **AS** para informar **o alias entre aspas duplas** (pois queremos utilizar um alias que possui letra maiúscula):

```
SELECT DISTINCT
    country AS "País"
FROM employees;
```

Dessa forma, retornamos o resultado abaixo, com a coluna devidamente renomeada:

	País character varying (15)
1	UK
2	USA

Total rows: 2 of 2 Query c

GABARITO EXPLICATIVO

9

Questão 9

A equipe de logística quer verificar os nomes e IDs dos transportadores cadastrados, renomeando as colunas shipper_id e company_name para português (ID_Transportadora e Nome_Transportadora, respectivamente).

Construa essa consulta. **(Utilize a tabela shippers)**

Explicação e Resultado

Para verificarmos os nomes e os IDs dos transportadores cadastrados, precisamos fazer um **SELECT** à tabela **SHIPPERS** que retorne as colunas **shipper_id** e **company_name.** Para retornarmos as colunas com outros nomes, utilizamos o comando **AS**, tornando a **shipper_id** em **ID_Transportadora** e **company_name** em **Nome_Transportadora**. Se quisermos preservar as **letras maiúsculas dos aliases**, devemos colocá-los entre **aspas duplas**:

```
SELECT
    shipper_id AS "ID_Transportadora",
    company_name AS "Nome_Transportadora"
FROM shippers;
```

Dessa forma, retornamos o resultado abaixo, com as colunas devidamente renomeadas:

	ID_Transportadora smallint	Nome_Transportadora character varying (40)
1	1	Speedy Express
2	2	United Package
3	3	Federal Shipping
4	4	Alliance Shippers
5	5	UPS
6	6	DHL

Total rows: 6 of 6 | Query complete 00:00:00.092

GABARITO EXPLICATIVO

10

Questão 10

A equipe administrativa deseja obter os IDs e os nomes dos fornecedores, bem como seus respectivos países, renomeando as colunas como "ID_Fornecedor", "Fornecedor" e "País", mas limitando o resultado aos 8 primeiros registros.

Crie a consulta que retorna o resultado esperado. **(Utilize a tabela suppliers)**

Explicação e Resultado

Para obtermos os IDs, os nomes e os países dos fornecedoress, precisamos fazer um **SELECT** à tabela **SUPPLIERS** que retorne as colunas **supplier_id**, **company_name** e **country.** Para retornarmos as colunas com outros nomes, utilizamos o comando **AS**, tornando a **supplier_id** em **ID_Fornecedor**, a **company_name** em **Fornecedor** e a **country** em **País**. Se quisermos preservar as **letras maiúsculas dos aliases**, devemos colocá-los entre **aspas duplas**. Como queremos um retorno de apenas 8 registros, usamos um **LIMIT 8** ao final do código:

```
SELECT
    supplier_id AS "ID_Fornecedor",
    company_name AS "Fornecedor",
    country AS Pais
FROM suppliers
LIMIT 8;
```

Dessa forma, retornamos o resultado abaixo, com as colunas renomeadas:

	ID_Fornecedor smallint	Fornecedor character varying (40)	pais character varying (15)
1	1	Exotic Liquids	UK
2	2	New Orleans Cajun Delights	USA
3	3	Grandma Kelly's Homestead	USA
4	4	Tokyo Traders	Japan
5	5	Cooperativa de Quesos 'Las Cabras'	Spain
6	6	Mayumi's	Japan
7	7	Pavlova, Ltd.	Australia
8	8	Specialty Biscuits, Ltd.	UK

Total rows: 8 of 8 | Query complete 00:00:00.102

MÓDULO 3

FILTROS

FILTROS
FILTROS

PostgreSQL

Índice Geral
Índice deste Módulo

O QUE VEREMOS NESTE MÓDULO

MÓDULO 3 FILTROS NO POSTGRESQL

01	WHERE – Filtros com textos	067
02	WHERE – Filtros com números	070
03	WHERE – Filtros com datas	073
04	WHERE – Operadores AND e OR	075
05	WHERE e LIKE – Filtros especiais com textos	080
06	WHERE e IN – Uma alternativa aos múltiplos ORs	086
07	WHERE e BETWEEN – Filtrando intervalos	088
08	EXERCÍCIOS	092
09	GABARITO EXPLICATIVO	097

WHERE – Filtros com textos

O comando WHERE nos permite criar filtros nas tabelas dos bancos de dados, seja com colunas de texto, números ou datas.

Vejamos um exemplo de como utilizar este comando com uma coluna de texto.

Na tabela customers do BD Northwind, temos clientes cadastrados de vários países ao redor do mundo. Vamos supor que a gente queira filtrar somente os clientes da França.

Para isso, podemos utilizar o WHERE na nossa consulta, da seguinte forma:

```
SELECT * FROM customers
WHERE country = 'France';
```

WHERE – Filtros com textos

Dessa forma, teremos no nosso resultado somente os clientes que tenham os campos da coluna country preenchidos com ‘France’:

```
SELECT * FROM customers
WHERE country = 'France';
```

Data output | Messages | Notifications

	customer_id [PK] character varying (5)	company_name character varying (40)	contact_name character varying (30)	contact_title character varying (30)	address character varying (60)	city character varying (15)	region character varying (15)	postal_code character varying (10)	country character varying (15)	phone character va
1	BLONP	Blondesddsl père et fils	Frédérique Citeaux	Marketing Manager	24, place Kléber	Strasbourg	[null]	67000	France	88.60.15.31
2	BONAP	Bon app'	Laurence Lebihan	Owner	12, rue des Bouchers	Marseille	[null]	13008	France	91.24.45.40
3	DUMON	Du monde entier	Janine Labrune	Owner	67, rue des Cinquante Otag…	Nantes	[null]	44000	France	40.67.88.88
4	FOLIG	Folies gourmandes	Martine Rancé	Assistant Sales Agent	184, chaussée de Tournai	Lille	[null]	59000	France	20.16.10.16
5	FRANR	France restauration	Carine Schmitt	Marketing Manager	54, rue Royale	Nantes	[null]	44000	France	40.32.21.21
6	LACOR	La corne d'abondance	Daniel Tonini	Sales Representative	67, avenue de l'Europe	Versailles	[null]	78000	France	30.59.84.10
7	LAMAI	La maison d'Asie	Annette Roulet	Sales Manager	1 rue Alsace-Lorraine	Toulouse	[null]	31000	France	61.77.61.10
8	PARIS	Paris spécialités	Marie Bertrand	Owner	265, boulevard Charonne	Paris	[null]	75012	France	(1) 42.34.22
9	SPECD	Spécialités du monde	Dominique Perrier	Marketing Manager	25, rue Lauriston	Paris	[null]	75016	France	(1) 47.55.60
10	VICTE	Victuailles en stock	Mary Saveley	Sales Agent	2, rue du Commerce	Lyon	[null]	69004	France	78.32.54.86
11	VINET	Vins et alcools Chevalier	Paul Henriot	Accounting Manager	59 rue de l'Abbaye	Reims	[null]	51100	France	26.47.15.10

WHERE – Filtros com textos

Mas atenção! O PostgreSQL é *case-sensitive*.

Portanto, ‘france’ não é o mesmo que ‘France’. Cada caractere deve ser filtrado exatamente conforme está registrado na tabela: se alguma letra for maiúscula, como neste caso, utilize-a desta forma, caso contrário, não serão retornados os resultados corretamente. Veja só:

```
SELECT * FROM customers
WHERE country = 'france';
```

Data output | Messages | Notifications

customer_id [PK] character varying (5)	company_name character varying (40)	contact_name character varying (30)	contact_title character varying (30)	address character varying (60)	city character varying (15)	region character varying (15)	postal_code character varying (10)	country character varying (15)	phone character varying (24)

Como o filtro foi informado somente com letras minúsculas (france), mas na tabela estava registrado com a primeira letra maiúscula (France), não foram retornados valores.

WHERE – Filtros com números

Agora, veremos como utilizar o WHERE para filtrar números.

Vamos imaginar que a gente precise saber quais produtos estão com o estoque zerado para poder comunicar ao setor de compras quanto à necessidade de fazer a reposição desses produtos.

Para isso, podemos fazer a seguinte consulta:

```
SELECT * FROM products
WHERE units_in_stock = 0;
```

WHERE – Filtros com números

Assim, teremos no nosso resultado somente os produtos que estejam com o estoque (units_in_stock) igual a zero:

```
SELECT * FROM products
WHERE units_in_stock = 0;
```

Data output | Messages | Notifications

	product_id [PK] smallint	product_name character varying (40)	supplier_id smallint	category_id smallint	quantity_per_unit character varying (20)	unit_price real	units_in_stock smallint	units_on_order smallint	reorder_level smallint	discontinued integer
1	5	Chef Anton's Gumbo ...	2	2	36 boxes	21.35	0	0	0	1
2	17	Alice Mutton	7	6	20 - 1 kg tins	39	0	0	0	1
3	29	Thüringer Rostbratwu...	12	6	50 bags x 30 sausgs.	123.79	0	0	0	1
4	31	Gorgonzola Telino	14	4	12 - 100 g pkgs	12.5	0	70	20	0
5	53	Perth Pasties	24	6	48 pieces	32.8	0	0	0	1

WHERE – Filtros com números

Com dados numéricos, podemos utilizar qualquer operador lógico em nossos filtros:

= (igual) < (menor) > (maior) <= (menor ou igual) >= (maior ou igual) <> (diferente)

Veja só:
Podemos, por exemplo, filtrar somente os produtos cujo valor unitário (unit_price) seja maior ou igual a 50:

```
SELECT * FROM products
WHERE unit_price >= 50;
```

Messages Notifications

	product_id [PK] smallint	product_name character varying (40)	supplier_id smallint	category_id smallint	quantity_per_unit character varying (20)	unit_price real	units_in_stock smallint	units_on_order smallint	reorder_level smallint	discontinued integer
1	9	Mishi Kobe Niku	4	6	18 - 500 g pkgs.	97	29	0	0	1
2	18	Carnarvon Tigers	7	8	16 kg pkg.	62.5	42	0	0	0
3	20	Sir Rodney's Marmala...	8	3	30 gift boxes	81	40	0	0	0
4	29	Thüringer Rostbratwur...	12	6	50 bags x 30 sausgs.	123.79	0	0	0	1
5	38	Côte de Blaye	18	1	12 - 75 cl bottles	263.5	17	0	15	0
6	51	Manjimup Dried Apples	24	7	50 - 300 g pkgs.	53	20	0	10	0
7	59	Raclette Courdavault	28	4	5 kg pkg.	55	79	0	0	0

PostgreSQL

WHERE – Filtros com datas

Também podemos filtrar datas. Nesses filtros, também podemos utilizar qualquer operador lógico:

`=` (igual) `<` (menor) `>` (maior) `<=` (menor ou igual) `>=` (maior ou igual) `<>` (diferente)

Devemos sempre informar a data que queremos utilizar como parâmetro do nosso filtro entre aspas simples, no padrão ‘aaaa-mm-dd’.

Por exemplo: Vamos supor que, agora, precisamos saber quais foram os pedidos (orders) efetuados a partir de 01/01/1998.

Para isso, fazemos assim:

```
SELECT * FROM orders
WHERE order_date >= '1998-01-01';
```

WHERE – Filtros com datas

Dessa forma, obteremos o seguinte resultado, somente com os pedidos efetuados a partir da data informada ao comando WHERE:

```
SELECT * FROM orders
WHERE order_date >= '1998-01-01';
```

Data output | Messages | Notifications

	order_id [PK] smallint	customer_id character varying (5)	employee_id smallint	order_date date	required_date date	shipped_date date	ship_via smallint	freight real	ship_name character varying (40)	ship_address character varying (60)	ship_city character varying (15)	ship_region character varying (15)	ship_postal_code character varying (10
1	10808	OLDWO	2	1998-01-01	1998-01-29	1998-01-09	3	45.53	Old World Delicatessen	2743 Bering St.	Anchorage	AK	99508
2	10809	WELLI	7	1998-01-01	1998-01-29	1998-01-07	1	4.87	Wellington Importado...	Rua do Mercado, 12	Resende	SP	08737-363
3	10810	LAUGB	2	1998-01-01	1998-01-29	1998-01-07	3	4.33	Laughing Bacchus Wi...	2319 Elm St.	Vancouver	BC	V3F 2K1
4	10811	LINOD	8	1998-01-02	1998-01-30	1998-01-08	1	31.22	LINO-Delicateses	Ave. 5 de Mayo Porla...	I. de Margarita	Nueva Esparta	4980
5	10812	REGGC	5	1998-01-02	1998-01-30	1998-01-12	1	59.78	Reggiani Caseifici	Strada Provinciale 124	Reggio Emilia	[null]	42100
6	10813	RICAR	1	1998-01-05	1998-02-02	1998-01-09	1	47.38	Ricardo Adocicados	Av. Copacabana, 267	Rio de Janeiro	RJ	02389-890
7	10814	VICTE	3	1998-01-05	1998-02-02	1998-01-14	3	130.94	Victuailles en stock	2, rue du Commerce	Lyon	[null]	69004
8	10815	SAVEA	2	1998-01-05	1998-02-02	1998-01-14	3	14.62	Save-a-lot Markets	187 Suffolk Ln.	Boise	ID	83720
9	10816	GREAL	4	1998-01-06	1998-02-03	1998-02-04	2	719.78	Great Lakes Food Mar...	2732 Baker Blvd.	Eugene	OR	97403

Total rows: 270 of 270 | Query complete 00:00:00.104 | Ln 144, Col 1

WHERE – Operadores AND e OR

O comando WHERE pode ser combinado com os operadores AND ou OR.

Os operadores AND e OR são usados para filtrar linhas da tabela baseado em mais de uma condição.

- O AND mostra as linhas da tabela se todas as condições forem atendidas.
- O OR mostra as linhas da tabela se pelo menos uma das condições for atendida.

WHERE – Operadores AND e OR

Imaginemos que queiramos selecionar somente os clientes cuja profissão seja 'Owner' do país 'France'.

Perceba que temos duas condições nesta consulta que precisarão ser atendidas:

A coluna contact_title precisa conter a informação 'Owner', assim como a coluna country também precisa conter a informação 'France'. Se uma dessas condições não for preenchida, ou seja, se o cliente for 'Owner', mas não morar no país 'France', ou se ele morar no país 'France', mas não for 'Owner', esse cliente não deve aparecer no resultado. Somente deverá aparecer se ele for 'Owner' E morar no país 'France'.

Para isso, elaboramos o seguinte código:

```
SELECT * FROM customers
WHERE contact_title = 'Owner' AND country = 'France';
```

WHERE – Operadores AND e OR

Repare que, no resultado, foram retornados somente os clientes que preencheram ambas as condições:

```
SELECT * FROM customers
WHERE contact_title = 'Owner' AND country = 'France';
```

Data output | Messages | Notifications

	customer_id [PK] character varying (5)	company_name character varying (40)	contact_name character varying (30)	contact_title character varying (30)	address character varying (60)	city character varying (15)	region character varying (15)	postal_code character varying (10)	country character varying (15)	phone character varying (24)
1	BONAP	Bon app'	Laurence Lebihan	Owner	12, rue des Bouchers	Marseille	[null]	13008	France	91.24.45.40
2	DUMON	Du monde entier	Janine Labrune	Owner	67, rue des Cinquante Otag...	Nantes	[null]	44000	France	40.67.88.88
3	PARIS	Paris spécialités	Marie Bertrand	Owner	265, boulevard Charonne	Paris	[null]	75012	France	(1) 42.34.22.66

WHERE – Operadores AND e OR

E quando queremos que apareça no resultado todos os registros que atendam **pelo menos** um dos critérios?

Neste caso, utilizamos o operador OR.

Imagine que agora, precisamos saber quais são todos os clientes que morem no México ou na França.

Para isso, fazemos o seguinte:

```
SELECT * FROM customers
WHERE country = 'Mexico' OR country = 'France';
```

WHERE – Operadores AND e OR

Assim, teremos todos os clientes que preencheram ao menos uma das condições:

```
SELECT * FROM customers
WHERE country = 'Mexico' OR country = 'France';
```

Data output | Messages | Notifications

	customer_id [PK] character varying (5)	company_name character varying (40)	contact_name character varying (30)	contact_title character varying (30)	address character varying (60)	city character varying (15)	region character varying (15)	postal_code character varying (10)	country character varying (15)	phone character varying (24)
1	ANATR	Ana Trujillo Empareda...	Ana Trujillo	Owner	Avda. de la Constitución 22...	México D.F.	[null]	05021	Mexico	(5) 555-4729
2	ANTON	Antonio Moreno Taqu...	Antonio Moreno	Owner	Mataderos 2312	México D.F.	[null]	05023	Mexico	(5) 555-3932
3	BLONP	Blondesddsl père et fils	Frédérique Citeaux	Marketing Manager	24, place Kléber	Strasbourg	[null]	67000	France	88.60.15.31
4	BONAP	Bon app'	Laurence Lebihan	Owner	12, rue des Bouchers	Marseille	[null]	13008	France	91.24.45.40
5	CENTC	Centro comercial Mo...	Francisco Chang	Marketing Manager	Sierras de Granada 9993	México D.F.	[null]	05022	Mexico	(5) 555-3392
6	DUMON	Du monde entier	Janine Labrune	Owner	67, rue des Cinquante Otag...	Nantes	[null]	44000	France	40.67.88.88
7	FOLIG	Folies gourmandes	Martine Rancé	Assistant Sales Agent	184, chaussée de Tournai	Lille	[null]	59000	France	20.16.10.16
8	FRANR	France restauration	Carine Schmitt	Marketing Manager	54, rue Royale	Nantes	[null]	44000	France	40.32.21.21
9	LACOR	La corne d'abondance	Daniel Tonini	Sales Representative	67, avenue de l'Europe	Versailles	[null]	78000	France	30.59.84.10
10	LAMAI	La maison d'Asie	Annette Roulet	Sales Manager	1 rue Alsace-Lorraine	Toulouse	[null]	31000	France	61.77.61.10
11	PARIS	Paris spécialités	Marie Bertrand	Owner	265, boulevard Charonne	Paris	[null]	75012	France	(1) 42.34.22.66
12	PERIC	Pericles Comidas clá...	Guillermo Fernández	Sales Representative	Calle Dr. Jorge Cash 321	México D.F.	[null]	05033	Mexico	(5) 552-3745
13	SPECD	Spécialités du monde	Dominique Perrier	Marketing Manager	25, rue Lauriston	Paris	[null]	75016	France	(1) 47.55.60.10
14	TORTU	Tortuga Restaurante	Miguel Angel Paolino	Owner	Avda. Azteca 123	México D.F.	[null]	05033	Mexico	(5) 555-2933
15	VICTE	Victuailles en stock	Mary Saveley	Sales Agent	2, rue du Commerce	Lyon	[null]	69004	France	78.32.54.86
16	VINET	Vins et alcools Cheval...	Paul Henriot	Accounting Manager	59 rue de l'Abbaye	Reims	[null]	51100	France	26.47.15.10

WHERE e LIKE – Filtros especiais com textos

O LIKE é usado em conjunto com o WHERE para procurar por um determinado padrão em uma coluna.

Existem 2 caracteres especiais usados em conjunto com o LIKE:

- O sinal de porcentagem (%) representa zero, um ou múltiplos caracteres.
- O *underline* (_) representa um único caractere.

WHERE e LIKE – Filtros especiais com textos

Imagine que a gente queira saber quais produtos são medidos em caixas (boxes).

Se pegarmos uma amostra da nossa tabela products, veremos que não existe uma coluna cujos campos estejam preenchidos somente com o termo 'boxes'. O que temos é a coluna quantity_per_unit que, dentre as medidas, contém a palavra boxes em alguns de seus registros. Veja:

Data output | Messages | Notifications

	product_id [PK] smallint	product_name character varying (40)	supplier_id smallint	category_id smallint	quantity_per_unit character varying (20)	unit_price real	units_in_stock smallint	units_on_order smallint	reorder_level smallint	discontinued integer
1	1	Chai	8	1	10 boxes x 30 bags	18	39	0	10	1
2	2	Chang	1	1	24 - 12 oz bottles	19	17	40	25	1
3	3	Aniseed Syrup	1	2	12 - 550 ml bottles	10	13	70	25	0
4	4	Chef Anton's Cajun Seasoning	2	2	48 - 6 oz jars	22	53	0	0	0
5	5	Chef Anton's Gumbo Mix	2	2	36 boxes	21.35	0	0	0	1
6	6	Grandma's Boysenberry Spre...	3	2	12 - 8 oz jars	25	120	0	25	0
7	7	Uncle Bob's Organic Dried Pe...	3	7	12 - 1 lb pkgs.	30	15	0	10	0
8	8	Northwoods Cranberry Sauce	3	2	12 - 12 oz jars	40	6	0	0	0

WHERE e LIKE – Filtros especiais com textos

Sendo assim, se fizermos um filtro assim...

```
SELECT * FROM products
WHERE quantity_per_unit = 'boxes';
```

Ou até mesmo assim...

```
SELECT * FROM products
WHERE quantity_per_unit LIKE 'boxes';
```

Não obteremos resultado algum:

Data output | Messages | Notifications

product_id [PK] smallint	product_name character varying (40)	supplier_id smallint	category_id smallint	quantity_per_unit character varying (20)	unit_price real	units_in_stock smallint	units_on_order smallint	reorder_level smallint	discontinued integer

WHERE e LIKE – Filtros especiais com textos

Portanto, para informar um parâmetro ao operador LIKE, devemos utilizar a sinal de % quando sabemos que o termo que queremos pesquisar está entre outras informações em determinados campos de uma coluna:

```
SELECT * FROM products
WHERE quantity_per_unit LIKE '%boxes%';
```

Assim, o LIKE entende que, independentemente de quantos caracteres existam antes ou depois do termo ‘boxes’, os registros que contenham este termo devem ser retornados:

Data output | Messages | Notifications

	product_id [PK] smallint	product_name character varying (40)	supplier_id smallint	category_id smallint	quantity_per_unit character varying (20)	unit_price real	units_in_stock smallint	units_on_order smallint	reorder_level smallint	discontinued integer
1	1	Chai	8	1	10 boxes x 30 bags	18	39	0	10	1
2	5	Chef Anton's Gumbo Mix	2	2	36 boxes	21.35	0	0	0	1
3	16	Pavlova	7	3	32 - 500 g boxes	17.45	29	0	10	0
4	19	Teatime Chocolate Biscuits	8	3	10 boxes x 12 pieces	9.2	25	0	5	0
5	20	Sir Rodney's Marmalade	8	3	30 gift boxes	81	40	0	0	0
6	47	Zaanse koeken	22	3	10 - 4 oz boxes	9.5	36	0	0	0
7	52	Filo Mix	24	5	16 - 2 kg boxes	7	38	0	25	0
8	55	Pâté chinois	25	6	24 boxes x 2 pies	24	115	0	20	0
9	68	Scottish Longbreads	8	3	10 boxes x 8 pieces	12.5	6	10	15	0
10	77	Original Frankfurter grüne So...	12	2	12 boxes	13	32	0	15	0

WHERE e LIKE – Filtros especiais com textos

Se nós soubermos exatamente quantos caracteres existem antes ou depois de determinado termos que queremos procurar, podemos utilizar o caractere especial _ em vez de %. Repare nos campos abaixo em destaque:

Data output | Messages | Notifications

	product_id [PK] smallint	product_name character varying (40)	supplier_id smallint	category_id smallint	quantity_per_unit character varying (20)	unit_price real	units_in_stock smallint	units_on_order smallint	reorder_level smallint	discontinued integer
1	1	Chai	8	1	10 boxes x 30 bags	18	39	0	10	1
2	5	Chef Anton's Gumbo Mix	2	2	36 boxes	21.35	0	0	0	1
3	16	Pavlova	7	3	32 - 500 g boxes	17.45	29	0	10	0
4	19	Teatime Chocolate Biscuits	8	3	10 boxes x 12 pieces	9.2	25	0	5	0
5	20	Sir Rodney's Marmalade	8	3	30 gift boxes	81	40	0	0	0
6	47	Zaanse koeken	22	3	10 - 4 oz boxes	9.5	36	0	0	0
7	52	Filo Mix	24	5	16 - 2 kg boxes	7	38	0	25	0
8	55	Pâté chinois	25	6	24 boxes x 2 pies	24	115	0	20	0
9	68	Scottish Longbreads	8	3	10 boxes x 8 pieces	12.5	6	10	15	0
10	77	Original Frankfurter grüne So...	12	2	12 boxes	13	32	0	15	0

Antes do termo ‘boxes’, podemos perceber que existem exatamente três caracteres: dois para os valores (36 ou 12) e mais um para o espaço. Assim, se quisermos retornar, por exemplo, somente esses dois campos, em vez de utilizar o %, podemos utilizar três *underlines* (___), um para cada caractere que vem antes do termo boxes.

WHERE e LIKE – Filtros especiais com textos

Desta forma:

```
SELECT * FROM products
WHERE quantity_per_unit LIKE '___boxes';
```

Veja o resultado:

Data output | Messages | Notifications

	product_id [PK] smallint	product_name character varying (40)	supplier_id smallint	category_id smallint	quantity_per_unit character varying (20)	unit_price real	units_in_stock smallint	units_on_order smallint	reorder_level smallint	discontinued integer
1	5	Chef Anton's Gumbo ...	2	2	36 boxes	21.35	0	0	0	1
2	77	Original Frankfurter gr...	12	2	12 boxes	13	32	0	15	0

WHERE e IN – Uma alternativa aos múltiplos ORs

O operador IN permite que sejam especificados múltiplos critérios dentro do WHERE. O IN é uma alternativa ao uso de múltiplos operadores OR para filtrar dados de uma mesma coluna.

Vejamos um exemplo: vamos supor que precisemos retornar todos os clientes que morem no México, no UK ou no Canadá. Com o que já aprendemos até aqui, poderíamos resolver essa questão utilizando múltiplos ORs:

```
SELECT * FROM customers
WHERE country = 'Mexico' OR country = 'UK' OR country = 'Canada';
```

Data output | Messages | Notif

	customer_id [PK] character varying (5)	company_name character varying (40)	contact_name character varying (30)	contact_title character varying (30)	address character varying (60)	city character varying (15)	region character varying (15)	postal_code character varying (10)	country character varying (15)	phone character va
1	ANATR	Ana Trujillo Emparedados y helad…	Ana Trujillo	Owner	Avda. de la Constitución 22…	México D.F.	[null]	05021	Mexico	(5) 555-472
2	ANTON	Antonio Moreno Taquería	Antonio Moreno	Owner	Mataderos 2312	México D.F.	[null]	05023	Mexico	(5) 555-393
3	AROUT	Around the Horn	Thomas Hardy	Sales Representative	120 Hanover Sq.	London	[null]	WA1 1DP	UK	(171) 555-7
4	BOTTM	Bottom-Dollar Markets	Elizabeth Lincoln	Accounting Manager	23 Tsawassen Blvd.	Tsawassen	BC	T2F 8M4	Canada	(604) 555-4
5	BSBEV	B's Beverages	Victoria Ashworth	Sales Representative	Fauntleroy Circus	London	[null]	EC2 5NT	UK	(171) 555-1
6	CENTC	Centro comercial Moctezuma	Francisco Chang	Marketing Manager	Sierras de Granada 9993	México D.F.	[null]	05022	Mexico	(5) 555-339
7	CONSH	Consolidated Holdings	Elizabeth Brown	Sales Representative	Berkeley Gardens 12 Brewery	London	[null]	WX1 6LT	UK	(171) 555-2
8	EASTC	Eastern Connection	Ann Devon	Sales Agent	35 King George	London	[null]	WX3 6FW	UK	(171) 555-0
9	ISLAT	Island Trading	Helen Bennett	Marketing Manager	Garden House Crowther Way	Cowes	Isle of Wight	PO31 7PJ	UK	(198) 555-8
10	LAUGB	Laughing Bacchus Wine Cellars	Yoshi Tannamuri	Marketing Assistant	1900 Oak St.	Vancouver	BC	V3F 2K1	Canada	(604) 555-3
11	MEREP	Mère Paillarde	Jean Fresnière	Marketing Assistant	43 rue St. Laurent	Montréal	Québec	H1J 1C3	Canada	(514) 555-8
12	NORTS	North/South	Simon Crowther	Sales Associate	South House 300 Queensbr…	London	[null]	SW7 1RZ	UK	(171) 555-7
13	PERIC	Pericles Comidas clásicas	Guillermo Fernández	Sales Representative	Calle Dr. Jorge Cash 321	México D.F.	[null]	05033	Mexico	(5) 552-374
14	SEVES	Seven Seas Imports	Hari Kumar	Sales Manager	90 Wadhurst Rd.	London	[null]	OX15 4NB	UK	(171) 555-1
15	TORTU	Tortuga Restaurante	Miguel Angel Paolino	Owner	Avda. Azteca 123	México D.F.	[null]	05033	Mexico	(5) 555-293

PostgreSQL

Índice Geral

Índice deste Módulo

WHERE e IN – Uma alternativa aos múltiplos ORs

Porém, temos uma maneira mais “elegante” de se retornar o mesmo resultado, fazendo uso do operador IN.

Veja só:

```
SELECT * FROM customers
WHERE country IN ('Mexico', 'UK', 'Canada');
```

Data output | Messages | Notifications

	customer_id [PK] character varying (5)	company_name character varying (40)	contact_name character varying (30)	contact_title character varying (30)	address character varying (60)	city character varying (15)	region character varying (15)	postal_code character varying (10)	country character varying (15)	phone character va
1	ANATR	Ana Trujillo Emparedados y helad…	Ana Trujillo	Owner	Avda. de la Constitución 22…	México D.F.	[null]	05021	Mexico	(5) 555-472
2	ANTON	Antonio Moreno Taquería	Antonio Moreno	Owner	Mataderos 2312	México D.F.	[null]	05023	Mexico	(5) 555-393
3	AROUT	Around the Horn	Thomas Hardy	Sales Representative	120 Hanover Sq.	London	[null]	WA1 1DP	UK	(171) 555-7
4	BOTTM	Bottom-Dollar Markets	Elizabeth Lincoln	Accounting Manager	23 Tsawassen Blvd.	Tsawassen	BC	T2F 8M4	Canada	(604) 555-4
5	BSBEV	B's Beverages	Victoria Ashworth	Sales Representative	Fauntleroy Circus	London	[null]	EC2 5NT	UK	(171) 555-1
6	CENTC	Centro comercial Moctezuma	Francisco Chang	Marketing Manager	Sierras de Granada 9993	México D.F.	[null]	05022	Mexico	(5) 555-339
7	CONSH	Consolidated Holdings	Elizabeth Brown	Sales Representative	Berkeley Gardens 12 Brewery	London	[null]	WX1 6LT	UK	(171) 555-2
8	EASTC	Eastern Connection	Ann Devon	Sales Agent	35 King George	London	[null]	WX3 6FW	UK	(171) 555-0
9	ISLAT	Island Trading	Helen Bennett	Marketing Manager	Garden House Crowther Way	Cowes	Isle of Wight	PO31 7PJ	UK	(198) 555-8
10	LAUGB	Laughing Bacchus Wine Cellars	Yoshi Tannamuri	Marketing Assistant	1900 Oak St.	Vancouver	BC	V3F 2K1	Canada	(604) 555-3
11	MEREP	Mère Paillarde	Jean Fresnière	Marketing Assistant	43 rue St. Laurent	Montréal	Québec	H1J 1C3	Canada	(514) 555-8
12	NORTS	North/South	Simon Crowther	Sales Associate	South House 300 Queensbr…	London	[null]	SW7 1RZ	UK	(171) 555-7
13	PERIC	Pericles Comidas clásicas	Guillermo Fernández	Sales Representative	Calle Dr. Jorge Cash 321	México D.F.	[null]	05033	Mexico	(5) 552-374
14	SEVES	Seven Seas Imports	Hari Kumar	Sales Manager	90 Wadhurst Rd.	London	[null]	OX15 4NB	UK	(171) 555-1
15	TORTU	Tortuga Restaurante	Miguel Angel Paolino	Owner	Avda. Azteca 123	México D.F.	[null]	05033	Mexico	(5) 555-293

PostgreSQL

Índice Geral

Índice deste Módulo

WHERE e BETWEEN – Filtrando intervalos

Se temos uma alternativa mais elegante para múltiplos ORs, que é o operador IN, também a temos para o AND quando precisamos filtrar dados de uma mesma coluna que armazene **números** ou **datas**.

Vamos aos exemplos para entender melhor!

Suponhamos que a gente queira saber quais produtos possuem um unit_price entre 50 e 100.

Com o que aprendemos até aqui, poderíamos fazer assim:

```
SELECT * FROM products
WHERE unit_price >= 50 AND unit_price <= 100;
```

Data output | Messages | Notifications

	product_id [PK] smallint	product_name character varying (40)	supplier_id smallint	category_id smallint	quantity_per_unit character varying (20)	unit_price real	units_in_stock smallint	units_on_order smallint	reorder_level smallint	discontinued integer
1	9	Mishi Kobe Niku	4	6	18 - 500 g pkgs.	97	29	0	0	1
2	18	Carnarvon Tigers	7	8	16 kg pkg.	62.5	42	0	0	0
3	20	Sir Rodney's Marmala...	8	3	30 gift boxes	81	40	0	0	0
4	51	Manjimup Dried Apples	24	7	50 - 300 g pkgs.	53	20	0	10	0
5	59	Raclette Courdavault	28	4	5 kg pkg.	55	79	0	0	0

WHERE e BETWEEN – Filtrando intervalos

Porém, repare que, desta forma, temos que ficar repetindo o nome da mesma coluna:

```
SELECT * FROM products
WHERE unit_price >= 50 AND unit_price <= 100;
```

Portanto, para sintetizar este código, podemos utilizar o operador BETWEEN. Assim, não precisamos repetir o nome da mesma coluna:

```
SELECT * FROM products
WHERE unit_price BETWEEN 50 AND 100;
```

Data output | Messages | Notifications

	product_id [PK] smallint	product_name character varying (40)	supplier_id smallint	category_id smallint	quantity_per_unit character varying (20)	unit_price real	units_in_stock smallint	units_on_order smallint	reorder_level smallint	discontinued integer
1	9	Mishi Kobe Niku	4	6	18 - 500 g pkgs.	97	29	0	0	1
2	18	Carnarvon Tigers	7	8	16 kg pkg.	62.5	42	0	0	0
3	20	Sir Rodney's Marmala...	8	3	30 gift boxes	81	40	0	0	0
4	51	Manjimup Dried Apples	24	7	50 - 300 g pkgs.	53	20	0	10	0
5	59	Raclette Courdavault	28	4	5 kg pkg.	55	79	0	0	0

WHERE e BETWEEN – Filtrando intervalos

O exemplo anterior foi com uma coluna de números, vejamos agora como também é possível utilizar o BETWEEN com datas.

Imagine que precisamos filtrar todos os pedidos efetuados entre 01/01/1997 e 31/12/1997.

Novamente, podemos utilizar o AND, mas para isso, precisamos repetir o nome da coluna utilizada (order_date):

```
SELECT * FROM orders
WHERE order_date >= '1997-01-01' AND order_date <= '1997-12-31';
```

Data output | Messages | Notifications

	order_id [PK] smallint	customer_id character varying (5)	employee_id smallint	order_date date	required_date date	shipped_date date	ship_via smallint	freight real	ship_name character varying (40)	ship_address character varying (60)	ship_city character varying (15)	ship_region character varying (15)	ship_postal_code character varying (10
1	10400	EASTC	1	1997-01-01	1997-01-29	1997-01-16	3	83.93	Eastern Connection	35 King George	London	[null]	WX3 6FW
2	10401	RATTC	1	1997-01-01	1997-01-29	1997-01-10	1	12.51	Rattlesnake Canyon G...	2817 Milton Dr.	Albuquerque	NM	87110
3	10402	ERNSH	8	1997-01-02	1997-02-13	1997-01-10	2	67.88	Ernst Handel	Kirchgasse 6	Graz	[null]	8010
4	10403	ERNSH	4	1997-01-03	1997-01-31	1997-01-09	3	73.79	Ernst Handel	Kirchgasse 6	Graz	[null]	8010
5	10404	MAGAA	2	1997-01-03	1997-01-31	1997-01-08	1	155.97	Magazzini Alimentari ...	Via Ludovico il Moro 22	Bergamo	[null]	24100

Total rows: 408 of 408 | Query complete 00:00:00.094 | Ln 233, Col 1

WHERE e BETWEEN – Filtrando intervalos

Podemos obter este mesmo resultado com o operador BETWEEN, assim não precisamos repetir o nome da coluna order_date:

```
SELECT * FROM orders
WHERE order_date BETWEEN '1997-01-01' AND '1997-12-31';
```

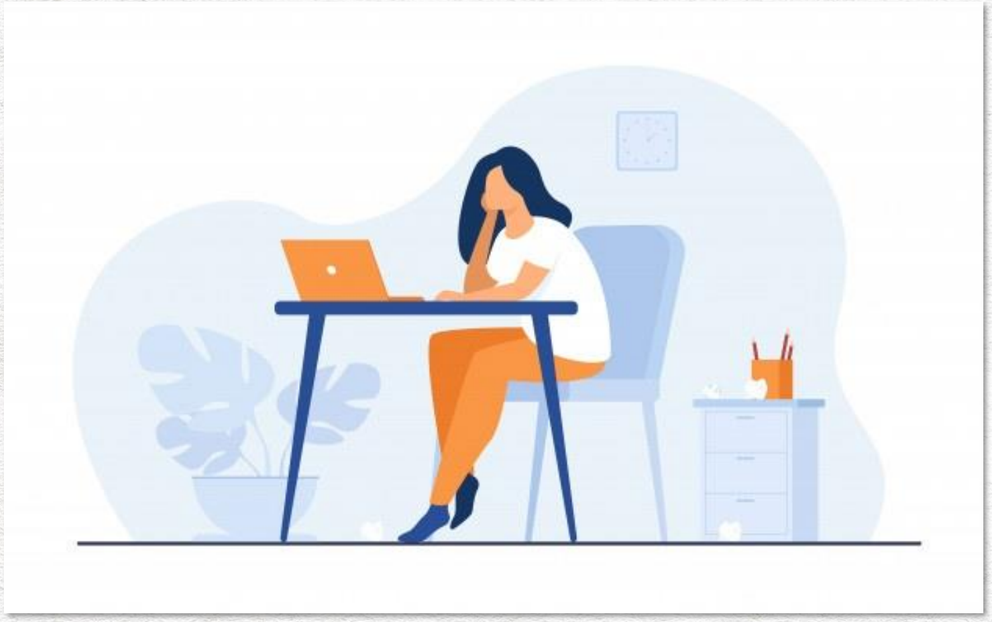
Data output | Messages | Notifications

	order_id [PK] smallint	customer_id character varying (5)	employee_id smallint	order_date date	required_date date	shipped_date date	ship_via smallint	freight real	ship_name character varying (40)	ship_address character varying (60)	ship_city character varying (15)	ship_region character varying (15)	ship_postal_code character varying (10
1	10400	EASTC	1	1997-01-01	1997-01-29	1997-01-16	3	83.93	Eastern Connection	35 King George	London	[null]	WX3 6FW
2	10401	RATTC	1	1997-01-01	1997-01-29	1997-01-10	1	12.51	Rattlesnake Canyon G...	2817 Milton Dr.	Albuquerque	NM	87110
3	10402	ERNSH	8	1997-01-02	1997-02-13	1997-01-10	2	67.88	Ernst Handel	Kirchgasse 6	Graz	[null]	8010
4	10403	ERNSH	4	1997-01-03	1997-01-31	1997-01-09	3	73.79	Ernst Handel	Kirchgasse 6	Graz	[null]	8010
5	10404	MAGAA	2	1997-01-03	1997-01-31	1997-01-08	1	155.97	Magazzini Alimentari ...	Via Ludovico il Moro 22	Bergamo	[null]	24100

Total rows: 408 of 408 | Query complete 00:00:00.094 | Ln 233, Col 1

Detalhe: o operador BETWEEN é inclusivo, ou seja, os valores passados a ele são incluídos no resultado. Portanto, se você não deseja incluir os valores no resultado (somente o que estiver entre tais valores), então o BETWEEN pode não ser a melhor solução.

EXERCÍCIOS



1

Questão 1

O setor de contabilidade está realizando uma revisão das despesas de frete para otimizar os custos de transporte.

Eles precisam identificar quais pedidos possuem um custo de frete superior a $100, para verificar se há alguma inconsistência ou oportunidades de redução de custos.

Liste esses pedidos. **(Utilize a tabela orders)**

2

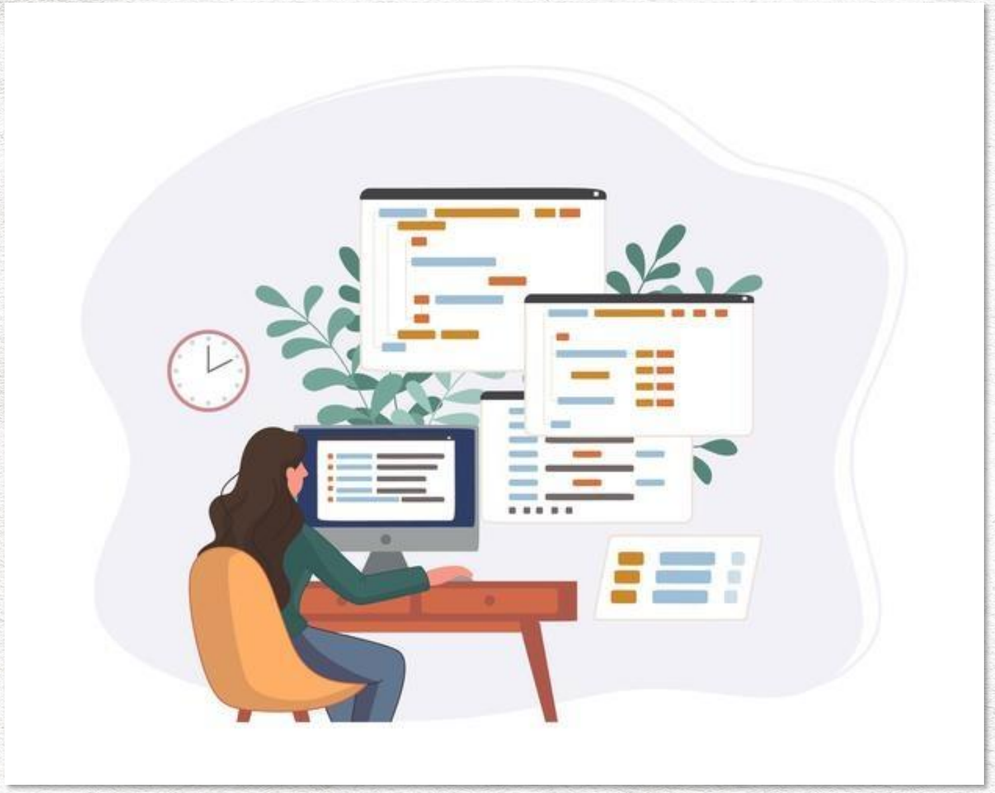
Questão 2

O time de vendas está preparando uma campanha específica para clientes localizados em Londres.

Eles precisam listar os clientes dessa cidade para personalizar ofertas e promover um atendimento especializado.

Crie a consulta que retorne esses clientes que moram em 'London'. **(Utilize a tabela customers)**

EXERCÍCIOS



3

Questão 3

A gerência está fazendo uma análise de desempenho de vendas e logística de 1998 para entender tendências e padrões históricos.

Eles precisam listar os pedidos enviados nesse ano para comparar com os anos seguintes e identificar possíveis melhorias no processo de envio.

Liste os pedidos despachados nesse ano de 1998. **(Utilize a tabela orders)**

4

Questão 4

O time de qualidade está revisando os tipos de embalagem dos produtos para garantir que estão conformes com os padrões de armazenamento e transporte.

Eles precisam identificar os produtos vendidos em pacotes ('pkg') para fazer essa análise.

Liste esses produtos. **(Utilize a tabela products)**

EXERCÍCIOS

5

Questão 5

O setor financeiro está revisando pedidos com frete alto (acima de $200) ou feitos antes de 1997 para garantir que não haja discrepâncias ou pagamentos indevidos.

Eles precisam identificar essas transações específicas para uma auditoria financeira detalhada.

Liste esses pedidos. **(Utilize a tabela orders)**

6

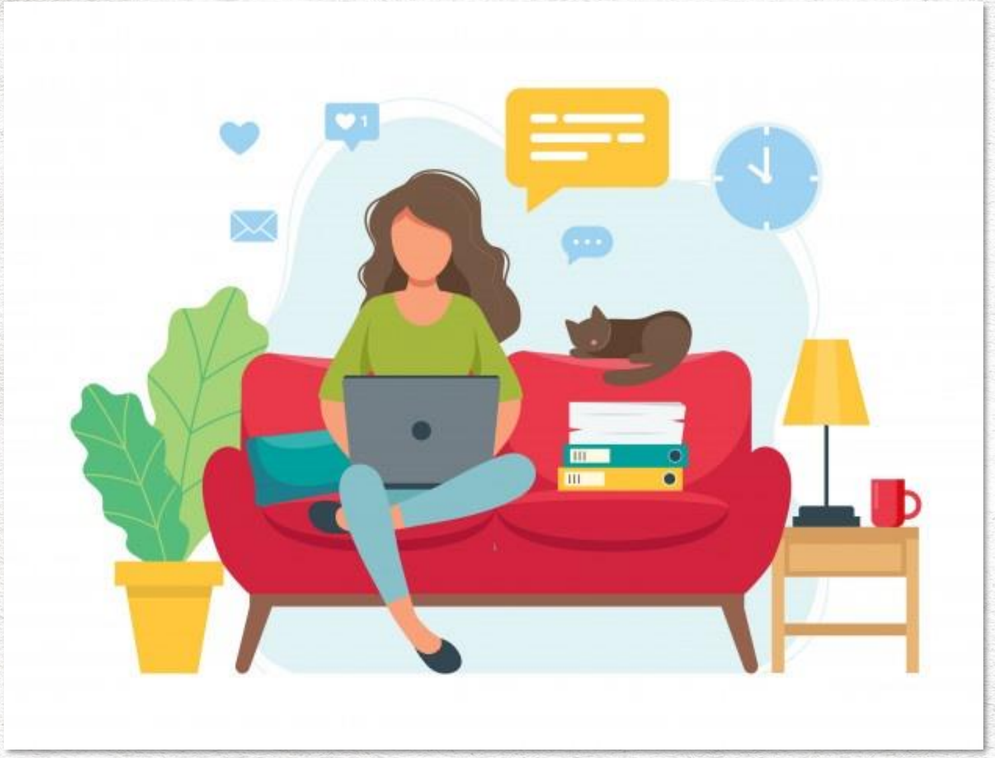
Questão 6

A equipe de inventário está realizando uma análise de estoque para identificar produtos de alta demanda e baixo estoque.

Eles querem verificar produtos com preço superior a $40 e estoque inferior a 50 unidades para planejar reposições e evitar falta de produtos.

Liste esses produtos. **(Utilize a tabela products)**

EXERCÍCIOS



7

Questão 7

O time de suporte ao cliente está tentando filtrar rapidamente empresas cujo nome começa com 'A' para facilitar um atendimento mais ágil a esses clientes, que podem ter solicitações prioritárias ou serem parte de uma campanha especial.

Crie uma consulta que retorne esses clientes. **(Utilize a tabela customers)**

8

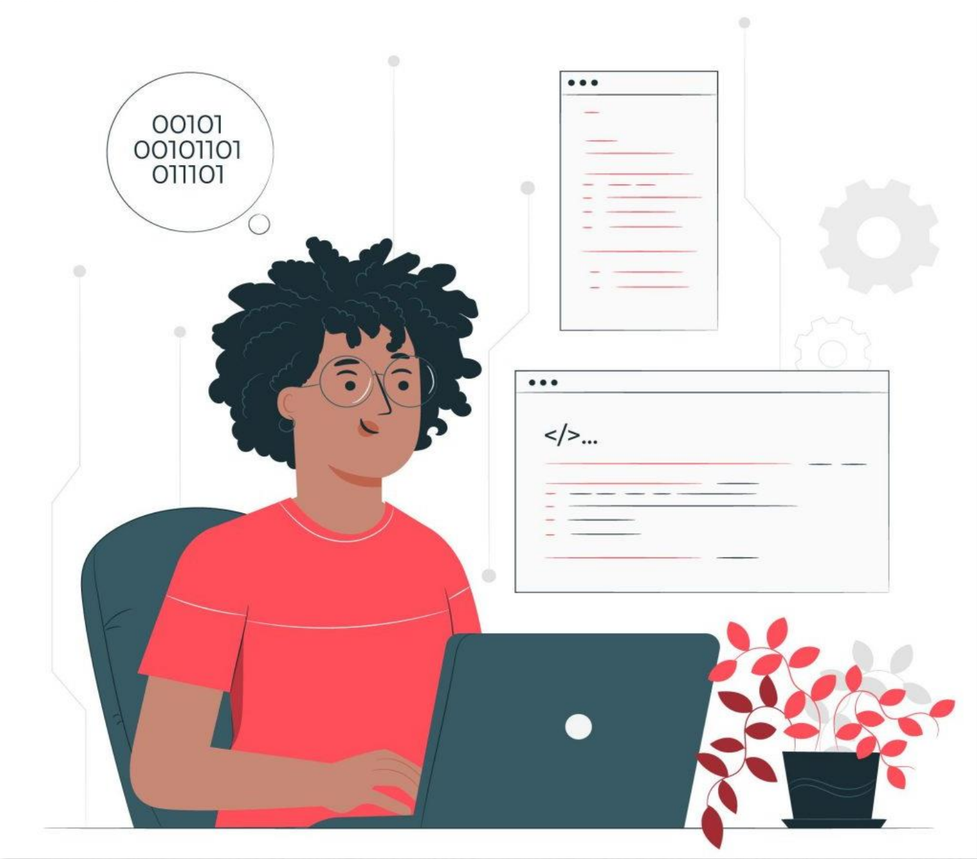
Questão 8

A equipe de análise de dados foi informada sobre possíveis inconsistências em alguns pedidos específicos.

Eles precisam revisar os pedidos com os IDs 10248, 10250 e 10252 para realizar uma investigação detalhada e corrigir possíveis erros nos registros.

Crie uma consulta que retorne esses pedidos. **(Utilize a tabela orders)**

EXERCÍCIOS



9

Questão 9

A gerência está fazendo uma análise de vendas e custos do ano de 1996 para entender como as despesas com frete impactaram as margens de lucro.

Eles precisam identificar pedidos feitos no período de 1996 e com um custo de frete dentro da faixa de $50 a $150 para realizar comparações e ajustes nas estratégias.

Crie uma consulta que retorne esses pedidos. **(Utilize a tabela orders)**

10

Questão 10

O time de marketing está preparando uma campanha direcionada a clientes específicos do Brasil, Estados Unidos e Canadá, com foco nos contatos responsáveis pela área de marketing.

Eles precisam listar esses clientes para personalizar a abordagem de marketing e fortalecer o relacionamento com os gerentes de marketing ('Marketing Manager') e os assistentes de marketing ('Marketing Assistant').

Crie uma consulta que retorne esses clientes. **(Utilize a tabela customers)**

GABARITO EXPLICATIVO

1

Questão 1

O setor de contabilidade está realizando uma revisão das despesas de frete para otimizar os custos de transporte.

Eles precisam identificar quais pedidos possuem um custo de frete superior a $100, para verificar se há alguma inconsistência ou oportunidades de redução de custos.

Liste esses pedidos. **(Utilize a tabela orders)**

Explicação e Resultado

Para identificarmos os pedidos que possuem um custo de frete superior a $100, fazemos um **SELECT** na tabela **ORDERS**, filtrando essa consulta com a cláusula **WHERE** para que retorne as linhas cujos campos da coluna **freight** possua valores **maiores que 100**:

```
SELECT * FROM orders
WHERE freight > 100;
```

Dessa forma, retornamos o resultado abaixo, com **187 pedidos**:

	order_id [PK] smallint	customer_id character	employee_id smallint	order_date date	required_date date	shipped_date date	ship_via smallint	freight real	ship_name character varying (40)	ship_address character varying (60)	ship_city character varying	ship_region character varyi	ship_postal_code character varying (1	ship_country character varying
1	10255	RICSU	9	1996-07-12	1996-08-09	1996-07-15	3	148.33	Richter Supermarkt	Starenweg 5	Genève	[null]	1204	Switzerland
2	10258	ERNSH	1	1996-07-17	1996-08-14	1996-07-23	1	140.51	Ernst Handel	Kirchgasse 6	Graz	[null]	8010	Austria
3	10263	ERNSH	9	1996-07-23	1996-08-20	1996-07-31	3	146.06	Ernst Handel	Kirchgasse 6	Graz	[null]	8010	Austria
4	10267	FRANK	4	1996-07-29	1996-08-26	1996-08-06	1	208.58	Frankenversand	Berliner Platz 43	München	[null]	80805	Germany
5	10270	WARTH	1	1996-08-01	1996-08-29	1996-08-02	1	136.54	Wartian Herkku	Torikatu 38	Oulu	[null]	90110	Finland
6	10277	MORGK	2	1996-08-09	1996-09-06	1996-08-13	3	125.77	Morgenstern Gesundkost	Heerstr. 22	Leipzig	[null]	04179	Germany
7	10286	QUICK	8	1996-08-21	1996-09-18	1996-08-30	3	229.24	QUICK-Stop	Taucherstraße 10	Cunewalde	[null]	01307	Germany
8	10294	RATTC	4	1996-08-30	1996-09-27	1996-09-05	2	147.26	Rattlesnake Canyon Grocery	2817 Milton Dr.	Albuquerque	NM	87110	USA
9	10298	HUNGO	6	1996-09-05	1996-10-03	1996-09-11	2	168.22	Hungry Owl All-Night Grocers	8 Johnstown Road	Cork	Co. Cork	[null]	Ireland
10	10303	GODOS	7	1996-09-11	1996-10-09	1996-09-18	2	107.83	Godos Cocina Típica	C/ Romero, 33	Sevilla	[null]	41101	Spain

Total rows: 187 of 187 | Query complete 00:00:00.211 | Ln 4,

GABARITO EXPLICATIVO

2

Questão 2

O time de vendas está preparando uma campanha específica para clientes localizados em Londres.

Eles precisam listar os clientes dessa cidade para personalizar ofertas e promover um atendimento especializado.

Crie a consulta que retorne esses clientes que moram em 'London'. **(Utilize a tabela customers)**

Explicação e Resultado

Para identificarmos os clientes localizados em Londres, precisamos realizar uma **consulta** na tabela **CUSTOMERS**, filtrando os registros em que a coluna **city** tenha o valor **'London'**.

Para isso, utilizamos a cláusula **WHERE**:

```
SELECT * FROM customers
WHERE city = 'London';
```

Dessa forma, retornamos os **clientes residentes em Londres**, conforme mostrado no resultado abaixo:

	customer_id [PK] character	company_name character varying (40)	contact_name character varying (30)	contact_title character varying (30)	address character varying (60)	city character var	region character var	postal_code character varyin	country character var	phone character varying (2	fax character varying
1	AROUT	Around the Horn	Thomas Hardy	Sales Representative	120 Hanover Sq.	London	[null]	WA1 1DP	UK	(171) 555-7788	(171) 555-6750
2	BSBEV	B's Beverages	Victoria Ashworth	Sales Representative	Fauntleroy Circus	London	[null]	EC2 5NT	UK	(171) 555-1212	[null]
3	CONSH	Consolidated Holdings	Elizabeth Brown	Sales Representative	Berkeley Gardens 12 Brewery	London	[null]	WX1 6LT	UK	(171) 555-2282	(171) 555-9199
4	EASTC	Eastern Connection	Ann Devon	Sales Agent	35 King George	London	[null]	WX3 6FW	UK	(171) 555-0297	(171) 555-3373
5	NORTS	North/South	Simon Crowther	Sales Associate	South House 300 Queensbridge	London	[null]	SW7 1RZ	UK	(171) 555-7733	(171) 555-2530
6	SEVES	Seven Seas Imports	Hari Kumar	Sales Manager	90 Wadhurst Rd.	London	[null]	OX15 4NB	UK	(171) 555-1717	(171) 555-5646

Total rows: 6 of 6 | Query complete 00:00:00.123

PostgreSQL

GABARITO EXPLICATIVO

3

Questão 3

A gerência está fazendo uma análise de desempenho de vendas e logística de 1998 para entender tendências e padrões históricos.

Eles precisam listar os pedidos enviados nesse ano para comparar com os anos seguintes e identificar possíveis melhorias no processo de envio.

Liste os pedidos despachados nesse ano de 1998. **(Utilize a tabela orders)**

Explicação e Resultado

Para listarmos os pedidos enviados no ano de 1998, realizamos uma **consulta** na tabela **ORDERS**, utilizando a cláusula **WHERE** para filtrar os pedidos em que a data de envio (**shipped_date**) esteja **entre '1998-01-01' e '1998-12-31'**.

A consulta é escrita da seguinte forma:

```
SELECT * FROM orders
WHERE shipped_date >= '1998-01-01' AND shipped_date <= '1998-12-31';
```

Com isso, obtemos os **268 pedidos enviados em 1998**, conforme o resultado abaixo:

	order_id [PK] smallint	customer_id character	employee_id smallint	order_date date	required_date date	shipped_date date	ship_via smallint	freight real	ship_name character varying (40)	ship_address character varying (60)	ship_city character varyi	ship_region character varyi	ship_postal_code character varying (1	ship_country character vary
1	10771	ERNSH	9	1997-12-10	1998-01-07	1998-01-02	2	11.19	Ernst Handel	Kirchgasse 6	Graz	[null]	8010	Austria
2	10777	GOURL	7	1997-12-15	1997-12-29	1998-01-21	2	3.01	Gourmet Lanchonetes	Av. Brasil, 442	Campinas	SP	04876-786	Brazil
3	10779	MORGK	3	1997-12-16	1998-01-13	1998-01-14	2	58.13	Morgenstern Gesundkost	Heerstr. 22	Leipzig	[null]	04179	Germany
4	10788	QUICK	1	1997-12-22	1998-01-19	1998-01-19	2	42.7	QUICK-Stop	Taucherstraße 10	Cunewalde	[null]	01307	Germany
5	10791	FRANK	6	1997-12-23	1998-01-20	1998-01-01	2	16.85	Frankenversand	Berliner Platz 43	München	[null]	80805	Germany
6	10793	AROUT	3	1997-12-24	1998-01-21	1998-01-08	3	4.52	Around the Horn	Brook Farm Stratford...	Colchester	Essex	CO7 6JX	UK

Total rows: 268 of 268 | Query complete 00:00:00.135

GABARITO EXPLICATIVO

4

Questão 4

O time de qualidade está revisando os tipos de embalagem dos produtos para garantir que estão conformes com os padrões de armazenamento e transporte.

Eles precisam identificar os produtos vendidos em pacotes ('pkg') para fazer essa análise.

Liste esses produtos. **(Utilize a tabela products)**

Explicação e Resultado

Para identificarmos os produtos vendidos em pacotes, realizamos uma **consulta** na tabela **PRODUCTS**, filtrando os registros em que a coluna **quantity_per_unit contém o texto 'pkg'.**

Para isso, utilizamos a cláusula **LIKE** com o caractere curinga **% antes e depois de 'pkg'.** Assim, todas as linhas cujos campos da coluna quantity_per_unit que tiverem o termo 'pkg', não importando a posição desses caracteres, serão retornadas:

```
SELECT * FROM products
WHERE quantity_per_unit LIKE '%pkg%';
```

O resultado da consulta lista os **24 produtos** cujas embalagens estão descritas com **'pkg'**, conforme mostrado abaixo:

	product_id [PK] smallint	product_name character varying (40)	supplier_id smallint	category_id smallint	quantity_per_unit character varying (2	unit_price real	units_in_stock smallint	units_on_order smallint	reorder_level smallint	discontinued integer
1	7	Uncle Bob's Organic Dried Pears	3	7	12 - 1 lb pkgs.	30	15	0	10	0
2	9	Mishi Kobe Niku	4	6	18 - 500 g pkgs.	97	29	0	0	1
3	11	Queso Cabrales	5	4	1 kg pkg.	21	22	30	30	0
4	12	Queso Manchego La Pastora	5	4	10 - 500 g pkgs.	38	86	0	0	0
5	14	Tofu	6	7	40 - 100 g pkgs.	23.25	35	0	0	0
6	18	Carnarvon Tigers	7	8	16 kg pkg.	62.5	42	0	0	0

Total rows: 24 of 24 Query complete 00:00:00.129

PostgreSQL

GABARITO EXPLICATIVO

5

Questão 5

O setor financeiro está revisando pedidos com frete alto (acima de $200) ou feitos antes de 1997 para garantir que não haja discrepâncias ou pagamentos indevidos.

Eles precisam identificar essas transações específicas para uma auditoria financeira detalhada.

Liste esses pedidos. **(Utilize a tabela orders)**

Explicação e Resultado

Para atendermos à necessidade de revisar pedidos com frete acima de $200 ou realizados antes de 01/01/1997, **consultamos** a tabela **ORDERS** e utilizamos a cláusula **WHERE** com o operador lógico **OR** para filtrar as condições:

```
SELECT * FROM orders
WHERE freight > 200 OR order_date < '1997-01-01';
```

Assim, **obtemos os 217 pedidos específicos que atendem a pelo menos uma das condições**, (OU foram efetuados antes de 1997, OU possuem um frete maior que $200, OU ambos), conforme mostrado abaixo:

	order_id [PK] smallint	customer_id character	employee_id smallint	order_date date	required_date date	shipped_date date	ship_via smallint	freight real	ship_name character varying (40)	ship_address character varying (60)	ship_city character varying	ship_region character vary	ship_postal_code character varying (	ship_country character varyi
146	10393	SAVEA	1	1996-12-25	1997-01-22	1997-01-03	3	126.56	Save-a-lot Markets	187 Suffolk Ln.	Boise	ID	83720	USA
147	10394	HUNGC	1	1996-12-25	1997-01-22	1997-01-03	3	30.34	Hungry Coyote Import ...	City Center Plaza 51...	Elgin	OR	97827	USA
148	10395	HILAA	6	1996-12-26	1997-01-23	1997-01-03	1	184.41	HILARION-Abastos	Carrera 22 con Ave. ...	San Cristóbal	Táchira	5022	Venezuela
149	10396	FRANK	1	1996-12-27	1997-01-10	1997-01-06	3	135.35	Frankenversand	Berliner Platz 43	München	[null]	80805	Germany
150	10397	PRINI	5	1996-12-27	1997-01-24	1997-01-02	1	60.26	Princesa Isabel Vinhos	Estrada da saúde n. ...	Lisboa	[null]	1756	Portugal
151	10398	SAVEA	2	1996-12-30	1997-01-27	1997-01-09	3	89.16	Save-a-lot Markets	187 Suffolk Ln.	Boise	ID	83720	USA
152	10399	VAFFE	8	1996-12-31	1997-01-14	1997-01-08	3	27.36	Vaffeljernet	Smagsloget 45	Århus	[null]	8200	Denmark
153	10424	MEREP	7	1997-01-23	1997-02-20	1997-01-27	2	370.61	Mère Paillarde	43 rue St. Laurent	Montréal	Québec	H1J 1C3	Canada
154	10430	ERNSH	4	1997-01-30	1997-02-13	1997-02-03	1	458.78	Ernst Handel	Kirchgasse 6	Graz	[null]	8010	Austria

Total rows: 217 of 217 | Query complete 00:00:00.142

GABARITO EXPLICATIVO

6

Questão 6

A equipe de inventário está realizando uma análise de estoque para identificar produtos de alta demanda e baixo estoque.

Eles querem verificar produtos com preço superior a $40 e estoque inferior a 50 unidades para planejar reposições e evitar falta de produtos.

Liste esses produtos.
(Utilize a tabela products)

Explicação e Resultado

Para listarmos os produtos cujo preço seja superior a $40 e o estoque seja inferior a 50 unidades, realizamos uma **consulta** na tabela **PRODUCTS**, utilizando a cláusula **WHERE** com o operador lógico **AND** para combinar ambas as condições:

```
SELECT * FROM products
WHERE unit_price > 40 AND units_in_stock < 50;
```

O resultado apresenta os **produtos que atendem a ambos os critérios**, conforme podemos ver abaixo:

	product_id [PK] smallint	product_name character varying (40)	supplier_id smallint	category_id smallint	quantity_per_unit character varying (20)	unit_price real	units_in_stock smallint	units_on_order smallint	reorder_level smallint	discontinued integer
1	9	Mishi Kobe Niku	4	6	18 - 500 g pkgs.	97	29	0	0	1
2	18	Carnarvon Tigers	7	8	16 kg pkg.	62.5	42	0	0	0
3	20	Sir Rodney's Marmalade	8	3	30 gift boxes	81	40	0	0	0
4	27	Schoggi Schokolade	11	3	100 - 100 g pieces	43.9	49	0	30	0
5	28	Rössle Sauerkraut	12	7	25 - 825 g cans	45.6	26	0	0	1
6	29	Thüringer Rostbratwurst	12	6	50 bags x 30 sausgs.	123.79	0	0	0	1
7	38	Côte de Blaye	18	1	12 - 75 cl bottles	263.5	17	0	15	0
8	43	Ipoh Coffee	20	1	16 - 500 g tins	46	17	10	25	0
9	51	Manjimup Dried Apples	24	7	50 - 300 g pkgs.	53	20	0	10	0
10	62	Tarte au sucre	29	3	48 pies	49.3	17	0	0	0
11	63	Vegie-spread	7	2	15 - 625 g jars	43.9	24	0	5	0

Total rows: 11 of 11 | Query complete 00:00:00.137

PostgreSQL

GABARITO EXPLICATIVO

7

Questão 7

O time de suporte ao cliente está tentando filtrar rapidamente empresas cujo nome começa com 'A' para facilitar um atendimento mais ágil a esses clientes, que podem ter solicitações prioritárias ou serem parte de uma campanha especial.

Crie uma consulta que retorne esses clientes. **(Utilize a tabela customers)**

Explicação e Resultado

Para identificarmos empresas cujos nomes começam com a letra 'A', **consultamos** a tabela **CUSTOMERS** e utilizamos a cláusula **LIKE** com o caractere curinga **% após a letra A**.

Assim, o código entenderá que a primeira letra deve ser A, podendo ser seguida de quaisquer outros caracteres:

```
SELECT * FROM customers
WHERE company_name LIKE 'A%';
```

Isso nos retorna a **lista de clientes cujos company_name começam com a letra A,** atendendo ao critério solicitado:

	customer_id [PK] character	company_name character varying (40)	contact_name character varying (3	contact_title character varying (30)	address character varying (60)	city character varyi	region character var	postal_code character varyi	country character var	phone character varying	fax character varying
1	ALFKI	Alfreds Futterkiste	Maria Anders	Sales Representative	Obere Str. 57	Berlin	[null]	12209	Germany	030-0074321	030-0076545
2	ANATR	Ana Trujillo Emparedados y helados	Ana Trujillo	Owner	Avda. de la Constitución 2222	México D.F.	[null]	05021	Mexico	(5) 555-4729	(5) 555-3745
3	ANTON	Antonio Moreno Taquería	Antonio Moreno	Owner	Mataderos 2312	México D.F.	[null]	05023	Mexico	(5) 555-3932	[null]
4	AROUT	Around the Horn	Thomas Hardy	Sales Representative	120 Hanover Sq.	London	[null]	WA1 1DP	UK	(171) 555-7788	(171) 555-6750
Total rows: 4 of 4	Query complete 00:00:00.128										

PostgreSQL

GABARITO EXPLICATIVO

8

Questão 8

A equipe de análise de dados foi informada sobre possíveis inconsistências em alguns pedidos específicos.

Eles precisam revisar os pedidos com os IDs 10248, 10250 e 10252 para realizar uma investigação detalhada e corrigir possíveis erros nos registros.

Crie uma consulta que retorne esses pedidos. **(Utilize a tabela orders)**

Explicação e Resultado

Para localizarmos os pedidos com os IDs 10248, 10250 e 10252, realizamos uma **consulta** na tabela **ORDERS** utilizando a cláusula **WHERE** com o operador **IN** para especificarmos os IDs desejados:

```
SELECT * FROM orders
WHERE order_id IN (10248, 10250, 10252);
```

Com isso, **obtemos as informações desses pedidos específicos**, conforme mostrado abaixo:

	order_id [PK] smallint	customer_id character	employee_id smallint	order_date date	required_date date	shipped_date date	ship_via smallint	freight real	ship_name character varying (40)	ship_address character varying (60)	ship_city character varying	ship_region character var	ship_postal_code character varying (1	ship_country character varyi
1	10248	VINET	5	1996-07-04	1996-08-01	1996-07-16	3	32.38	Vins et alcools Chevalier	59 rue de l'Abbaye	Reims	[null]	51100	France
2	10250	HANAR	4	1996-07-08	1996-08-05	1996-07-12	2	65.83	Hanari Carnes	Rua do Paço, 67	Rio de Janeiro	RJ	05454-876	Brazil
3	10252	SUPRD	4	1996-07-09	1996-08-06	1996-07-11	2	51.3	Suprêmes délices	Boulevard Tirou, 255	Charleroi	[null]	B-6000	Belgium

Total rows: 3 of 3 Query complete 00:00:00.157

GABARITO EXPLICATIVO

9

Questão 9

A gerência está fazendo uma análise de vendas e custos do ano de 1996 para entender como as despesas com frete impactaram as margens de lucro.

Eles precisam identificar pedidos feitos no período de 1996 e com um custo de frete dentro da faixa de $50 a $150 para realizar comparações e ajustes nas estratégias.

Crie uma consulta que retorne esses pedidos. **(Utilize a tabela orders)**

Explicação e Resultado

Para atendermos à análise dos pedidos de 1996 com custo de frete entre $50 e $150, realizamos uma **consulta** na tabela **ORDERS**, filtrando as condições de intervalo de data e frete com a cláusula **WHERE**. Para estabelecer os intervalos de cada condição, utilizamos o operador **BETWEEN** em cada uma delas. E para combinar as duas condições, utilizamos o operador **AND**:

```
SELECT * FROM orders
WHERE order_date BETWEEN '1996-01-01' AND '1996-12-31'
AND freight BETWEEN 50 AND 150;
```

O resultado retorna os **49 pedidos que atendem a ambas as condições**, conforme mostrado abaixo:

	order_id [PK] smallint	customer_id character	employee_id smallint	order_date date	required_date date	shipped_date date	ship_via smallint	freight real	ship_name character varying (40)	ship_address character varying (60)	ship_city character varying	ship_region character vary	ship_postal_code character varying (1	ship_country character vary
1	10250	HANAR	4	1996-07-08	1996-08-05	1996-07-12	2	65.83	Hanari Carnes	Rua do Paço, 67	Rio de Janeiro	RJ	05454-876	Brazil
2	10252	SUPRD	4	1996-07-09	1996-08-06	1996-07-11	2	51.3	Suprêmes délices	Boulevard Tirou, 255	Charleroi	[null]	B-6000	Belgium
3	10253	HANAR	3	1996-07-10	1996-07-24	1996-07-16	2	58.17	Hanari Carnes	Rua do Paço, 67	Rio de Janeiro	RJ	05454-876	Brazil
4	10255	RICSU	9	1996-07-12	1996-08-09	1996-07-15	3	148.33	Richter Supermarkt	Starenweg 5	Genève	[null]	1204	Switzerland
5	10257	HILAA	4	1996-07-16	1996-08-13	1996-07-22	3	81.91	HILARION-Abastos	Carrera 22 con Ave....	San Cristóbal	Táchira	5022	Venezuela

Total rows: 49 of 49 Query complete 00:00:00.166

GABARITO EXPLICATIVO

10

Questão 10

O time de marketing está preparando uma campanha direcionada a clientes específicos do Brasil, Estados Unidos e Canadá, com foco nos contatos responsáveis pela área de marketing.

Eles precisam listar esses clientes para personalizar a abordagem de marketing e fortalecer o relacionamento com os gerentes de marketing ('Marketing Manager') e os assistentes de marketing ('Marketing Assistant'). Crie uma consulta que retorne esses clientes. **(Utilize a tabela customers)**

Explicação e Resultado

Para listarmos os clientes do Brasil, Estados Unidos e Canadá, que tenham como contato o 'Marketing Manager' ou o 'Marketing Assistant' da empresa, realizamos uma **consulta** na tabela **CUSTOMERS**, utilizando a cláusula **WHERE** com o operador **IN** para filtrarmos os países (**country**) e os contatos (**contact_title**) desejados. E para combinarmos ambas as condições filtradas, usamos o operador **AND**:

```
SELECT * FROM customers
WHERE country IN ('Brazil', 'USA', 'Canada')
AND contact_title IN ('Marketing Manager', 'Marketing Assistant');
```

O resultado lista os **clientes específicos para a campanha**, conforme exemplo abaixo:

	customer_id [PK] character	company_name character varying (40)	contact_name character varying (30	contact_title character varying (30)	address character varying (60)	city character varyin	region character var	postal_code character varying	country character var	phone character varying (24	fax character varying (24)
1	FAMIA	Familia Arquibaldo	Aria Cruz	Marketing Assistant	Rua Orós, 92	Sao Paulo	SP	05442-030	Brazil	(11) 555-9857	[null]
2	GREAL	Great Lakes Food Mar...	Howard Snyder	Marketing Manager	2732 Baker Blvd.	Eugene	OR	97403	USA	(503) 555-7555	[null]
3	LAUGB	Laughing Bacchus Win...	Yoshi Tannamuri	Marketing Assistant	1900 Oak St.	Vancouver	BC	V3F 2K1	Canada	(604) 555-3392	(604) 555-7293
4	LAZYK	Lazy K Kountry Store	John Steel	Marketing Manager	12 Orchestra Terr...	Walla Walla	WA	99362	USA	(509) 555-7969	(509) 555-6221
5	MEREP	Mère Paillarde	Jean Fresnière	Marketing Assistant	43 rue St. Laurent	Montréal	Québec	H1J 1C3	Canada	(514) 555-8054	(514) 555-8055
6	QUEEN	Queen Cozinha	Lúcia Carvalho	Marketing Assistant	Alameda dos Can...	Sao Paulo	SP	05487-020	Brazil	(11) 555-1189	[null]
7	THEBI	The Big Cheese	Liz Nixon	Marketing Manager	89 Jefferson Way ...	Portland	OR	97201	USA	(503) 555-3612	[null]
8	THECR	The Cracker Box	Liu Wong	Marketing Assistant	55 Grizzly Peak Rd.	Butte	MT	59801	USA	(406) 555-5834	(406) 555-8083

Total rows: 8 of 8 Query complete 00:00:00.115

PostgreSQL

Índice Geral

Índice deste Módulo

MÓDULO 4

AGRUPAMENTOS

AGRUPAMENTOS
AGRUPAMENTOS

O QUE VEREMOS NESTE MÓDULO

MÓDULO 4 FUNÇÕES DE AGREGAÇÃO E AGRUPAMENTOS

01	COUNT	109
02	SUM	113
03	AVG, MIN e MAX	114
04	GROUP BY – Criando agrupamentos	115
05	GROUP BY, WHERE e HAVING – Filtros em agrupamentos	117
06	EXERCÍCIOS	120
07	GABARITO EXPLICATIVO	125

COUNT

A função de agregação COUNT faz a contagem de valores de uma coluna.

Sua sintaxe é bem simples:

```
SELECT
   COUNT(coluna)
FROM tabela;
```

Repare que, no exemplo ao lado, fizemos um COUNT da coluna contact_name da tabela customers, que retornou o total de 91 clientes.

```
SELECT
    COUNT(contact_name)
FROM customers;
```

Data output | Messages | Notifications

	count bigint
1	91

Tudo perfeito, então, certo?

Sim, entretanto, tome cuidado...

COUNT

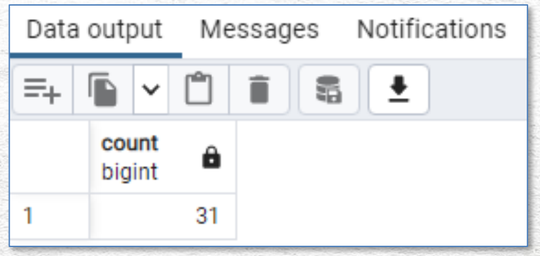
Pois...

Veja só este exemplo:

```
SELECT
    COUNT(region)
FROM customers;
```

Fizemos o mesmo COUNT na tabela customers, mas, desta vez, informamos a coluna region.

Agora, foi retornado um total de apenas 31 clientes.



Por que isso aconteceu?

COUNT

Porque **a função COUNT não considera valores nulos em sua contagem.**

Se você fizer uma consulta às colunas contact_name e region, verá que:

- A coluna contact_name não tem valores nulos.
- Já a coluna region possui valores nulos.

Assim, como o COUNT não considera valores nulos, as consultas retornaram valores diferentes.

```
237 SELECT
238     contact_name
239 FROM customers;
```

Data output | Messages

	contact_name character varying (30)
1	Maria Anders
2	Ana Trujillo
3	Antonio Moreno
4	Thomas Hardy
5	Christina Berglund
6	Hanna Moos
7	Frédérique Citeaux
8	Martín Sommer
9	Laurence Lebihan
10	Elizabeth Lincoln
11	Victoria Ashworth
12	Patricio Simpson
13	Francisco Chang
14	Yang Wang
15	Pedro Afonso
16	Elizabeth Brown

```
247 SELECT
248     region
249 FROM customers;
```

Data output | Messages

	region character varying (15)
25	[null]
26	[null]
27	[null]
28	[null]
29	[null]
30	[null]
31	SP
32	OR
33	DF
34	RJ
35	Táchira
36	OR
37	Co. Cork
38	Isle of Wight
39	[null]
40	[null]
41	[null]

COUNT

Para resolver essa questão dos valores nulos que são desconsiderados pela função COUNT, em vez de especificar qual coluna queremos contar, podemos simplesmente informar um * em seu lugar, conforme no exemplo ao lado.

```
253  SELECT
254      COUNT(*)
255  FROM customers;
256
```

Data output | Messages | Notifications

	count bigint
1	91

Assim, não precisamos nos preocupar se a coluna informada possui valores nulos, uma vez que a função COUNT(*) faz a contagem de todas as linhas de uma tabela, contenham as colunas valores nulos ou não.

SUM

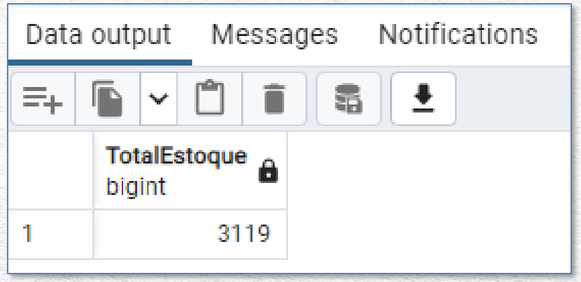
A função de agregação SUM faz a soma dos valores de uma coluna.

Sintaxe:

```
SELECT
   SUM(coluna)
FROM tabela;
```

No exemplo ao lado, fizemos a soma da coluna units_in_stock da tabela products, que retornou o total de unidades dos produtos em estoque.

```
SELECT
    SUM(units_in_stock) AS "TotalEstoque"
FROM products;
```



AVG, MIN, MAX

As funções de agregação AVG, MIN, MAX retornam, respectivamente: a média, o menor e o maior valor de uma coluna.

Sintaxe:

```
SELECT
   SUM(coluna)
FROM tabela;
```

```
SELECT
   MIN(coluna)
FROM tabela;
```

```
SELECT
   MAX(coluna)
FROM tabela;
```

```
SELECT
    AVG(unit_price) AS preco_medio,
    MIN(unit_price) AS menor_preco,
    MAX(unit_price) AS maior_preco
FROM products;
```

No exemplo ao lado, em relação à coluna unit_price da tabela products:

→ Calculamos sua media (AVG);
→ Descobrimos seu menor valor (MIN);
→ Descobrimos seu maior valor (MAX).

Data output | Messages | Notifications

	preco_medio double precision	menor_preco real	maior_preco real
1	28.8338960920061	2.5	263.5

GROUP BY – Criando agrupamentos

O comando GROUP BY permite agrupar valores de acordo com uma coluna.

O GROUP BY é usado junto com funções de agregação (COUNT(), MAX(), MIN(), SUM(), AVG()) para agrupar valores de acordo com uma ou mais colunas.

Abaixo, vemos um exemplo do comando GROUP BY aplicado em conjunto com a função de agregação SUM para retornar a soma total de estoque (units_in_stock) por supplier_id.

```
SELECT
    supplier_id,
    SUM(units_in_stock)
FROM products
GROUP BY supplier_id;
```

Data output | Messages | Notifications

	supplier_id smallint	sum bigint
1	29	130
2	4	64
3	10	20
4	9	165
5	7	110
6	15	164
7	6	98
8	26	57

GROUP BY – Criando agrupamentos

O comando GROUP BY combinado com o ORDER BY permite que a tabela agrupada seja também ordenada.

Veja agora como agrupamos a quantidade total de clientes por país, ordenando esse agrupamento pela contagem em ordem crescente:

```
SELECT
    country,
    COUNT(*)
FROM customers
GROUP BY country
ORDER BY COUNT(*);
```

	country character varying (15)	count bigint
1	Poland	1
2	Norway	1
3	Ireland	1
4	Portugal	2
5	Finland	2
6	Belgium	2
7	Denmark	2
8	Sweden	2
9	Austria	2
10	Switzerland	2
11	Argentina	3
12	Italy	3
13	Canada	3
14	Venezuela	4
15	Spain	5
16	Mexico	5
17	UK	7
18	Brazil	9
19	Germany	11
20	France	11
21	USA	13

GROUP BY, WHERE e HAVING – Filtros em agrupamentos

A combinação **GROUP BY + WHERE** nos permite criar filtros antes de agrupar uma tabela, para depois fazer o agrupamento de dados a partir de uma ou mais colunas dessa tabela.

Vamos a um exemplo:

Faça um agrupamento da quantidade total de clientes por país, considerando apenas os clientes cujo contact_title seja igual a “Owner”.

Neste caso, primeiro a gente vai precisar filtrar a tabela customers para retornar apenas os clientes que tenham o contact_title = ‘Owner’. Para isso, usamos o WHERE.

Assim, podemos fazer uma contagem (COUNT(*)) apenas dos registros retornados (os clientes cujo contact_title seja igual a “Owner”), agrupando-os (GROUP BY) pelo país (country).

Ao lado, veja como fica a nossa consulta e seu resultado:

```sql
SELECT
    country,
    COUNT(*)
FROM customers
WHERE contact_title = 'Owner'
GROUP BY country;
```

Data output | Messages | Notifications

	country character varying (15)	count bigint
1	Spain	1
2	Mexico	3
3	Switzerland	1
4	Poland	1
5	Germany	1
6	Venezuela	2
7	Denmark	1
8	Norway	1
9	Sweden	1
10	USA	2
11	France	3

PostgreSQL

Índice Geral

Índice deste Módulo

GROUP BY, WHERE e HAVING – Filtros em agrupamentos

Já a combinação **GROUP BY + HAVING** nos permite primeiro fazer o agrupamento de dados a partir de uma ou mais colunas de uma tabela, para depois criar filtros a partir desse agrupamento.

Vamos a um exemplo:

Faça um agrupamento da quantidade total de clientes por país, e retorne apenas os países que tenham mais de 10 clientes.

Aqui, primeiro vamos fazer uma contagem (COUNT(*)) do total de clientes, agrupando-os (GROUP BY) pelo país (country).

Feito isso, utilizamos o HAVING para extrair deste agrupamento criado somente aqueles países cuja contagem retornou um valor maior que 10.

Ao lado, veja como fica a nossa consulta e seu resultado:

```sql
SELECT
    country,
    COUNT(*)
FROM customers
GROUP BY country
HAVING COUNT(*) > 10;
```

Data output | Messages | Notifications

	country character varying (15)	count bigint
1	USA	13
2	France	11
3	Germany	11

GROUP BY, WHERE e HAVING – Filtros em agrupamentos

Portanto, para saber quando utilizar o **GROUP BY** associado à cláusula WHERE ou à cláusula HAVING, você deve se perguntar:

Este filtro precisa ser feito na tabela que já existe ou no agrupamento que estou criando?

NA TABELA:

UTILIZE O WHERE ANTES DO GROUP BY

SE SUA RESPOSTA FOR:

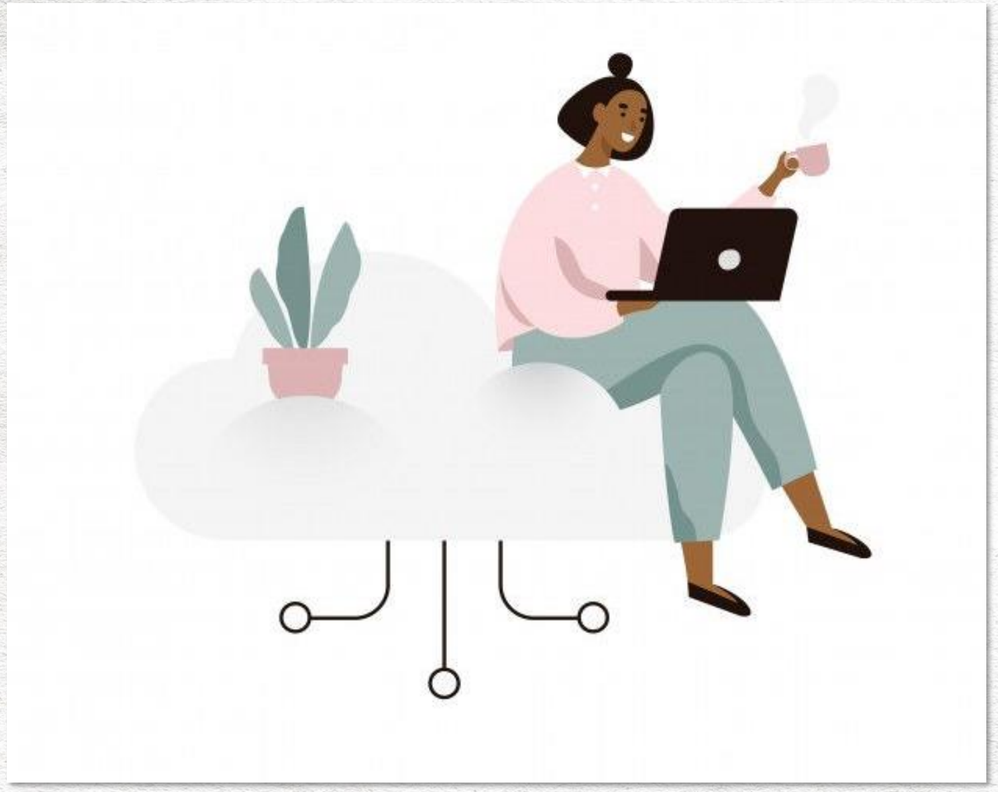
« »

NO AGRUPAMENTO:

UTILIZE O HAVING DEPOIS DO GROUP BY

PostgreSQL

EXERCÍCIOS



1

Questão 1

A gerência deseja saber quantos fornecedores estão cadastrados no sistema para avaliar as possibilidades de novas parcerias.

Crie uma consulta que retorne a quantidade total de fornecedores. **(Utilize a tabela suppliers)**

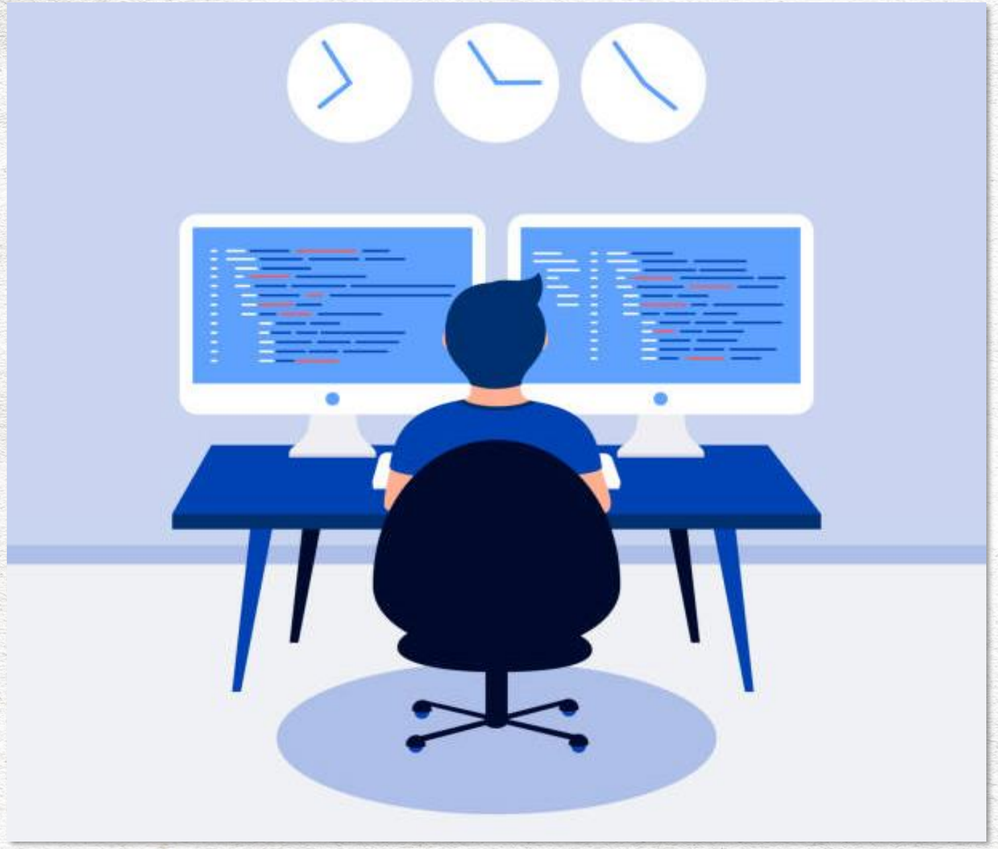
2

Questão 2

O gerente financeiro precisa saber o preço médio dos produtos no sistema para avaliar o custo médio dos itens vendidos.

Crie uma consulta para calcular a média de preço dos produtos. **(Utilize a tabela products)**

EXERCÍCIOS



3

Questão 3

A equipe de vendas deseja saber qual é o menor e o maior valor de frete já cobrado em pedidos para entender a variabilidade nos custos de entrega.

Crie a consulta que retorna esses valores. **(Utilize a tabela orders)**

4

Questão 4

O setor de Recursos Humanos está realizando um levantamento regional para identificar a quantidade de funcionários que residem na região "WA". Essa informação é fundamental para a organização de um evento corporativo local.

Desenvolva uma consulta que mostre a contagem total de funcionários nessa região. **(Utilize a tabela employees)**

EXERCÍCIOS

5

Questão 5

O setor de logística está avaliando o volume de produtos encomendados que são embalados em garrafas. Para isso, eles precisam saber o total de unidades desses produtos que estão em pedidos pendentes.

Crie uma consulta que retorne a soma das unidades encomendadas (units_on_order) de todos os produtos cuja descrição no campo "quantity_per_unit" mencione "bottles". **(Utilize a tabela products)**

6

Questão 6

O setor de vendas deseja saber quantos produtos estão disponíveis em cada categoria.

Crie uma consulta que retorne o ID da categoria e a quantidade de produtos associados a ela. Ordene o resultado pela contagem da quantidade de produtos em ordem decrescente. **(Utilize a tabela products)**

EXERCÍCIOS

7

Questão 7

O setor financeiro está analisando os custos logísticos relacionados ao envio de pedidos para diferentes países. Para isso, eles precisam saber o total acumulado de frete para cada país de destino, a fim de identificar quais regiões geram os maiores custos de transporte.

Elabore uma consulta que exiba o nome do país de envio e a soma total dos fretes, ordenando o resultado do maior para o menor valor. **(Utilize a tabela orders)**

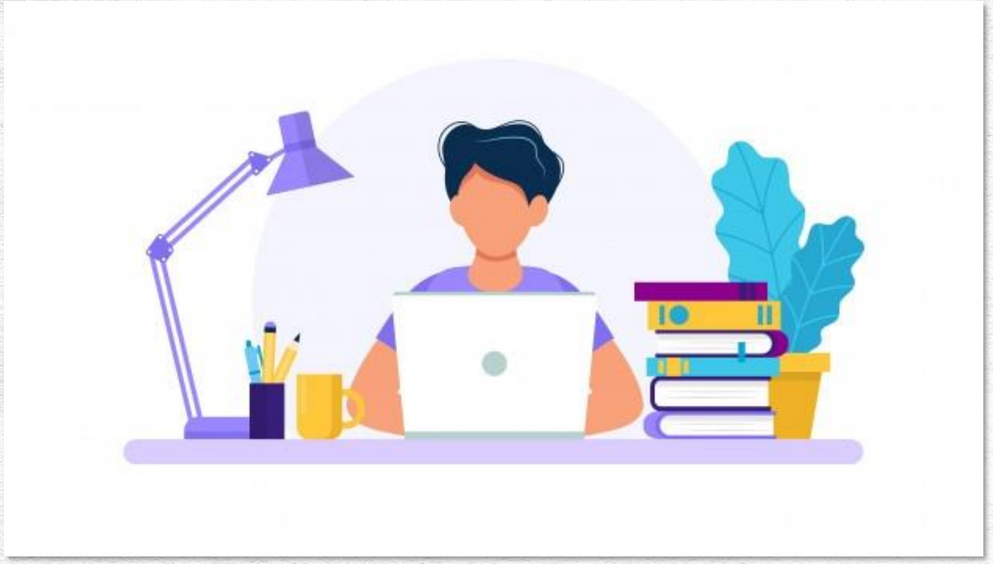
8

Questão 8

A equipe de gestão de inventário está buscando uma forma de monitorar o estoque de cada categoria de produtos. Eles precisam entender quantos produtos existem em cada categoria e qual o total de unidades em estoque. Esse controle é essencial para que possam planejar reabastecimentos, identificar possíveis excessos ou faltas de estoque e tomar decisões informadas sobre estratégias de vendas e promoções.

Crie uma consulta que retorne o ID da categoria, a contagem de produtos e a soma do estoque por categoria. **(Utilize a tabela products)**

EXERCÍCIOS



9

Questão 9

A equipe de análise de vendas da empresa recebeu uma solicitação para avaliar o desempenho médio de preço dos produtos nas categorias de bebidas (category_id = 1) e laticínios (category_id = 4). O objetivo é entender a variação de preço dessas categorias para tomar decisões estratégicas sobre promoções e ajustes de preços.

Sua tarefa é criar uma consulta SQL que calcule o preço médio por unidade para as categorias de bebidas e laticínios, pois isso ajudará a equipe a tomar futuras decisões comerciais. **(Utilize a tabela products)**

10

Questão 10

A equipe de vendas da empresa está analisando o desempenho dos produtos para entender quais itens estão gerando maior volume de vendas. Para ajudar na tomada de decisões sobre estoque e estratégias de marketing, eles precisam obter um levantamento detalhado das quantidades vendidas de cada produto.

Crie uma consulta SQL que retorne a soma total de unidades vendidas (quantity) por produto, mas somente para os produtos que já alcançaram no mínimo 1000 unidades vendidas. A consulta deve exibir os produtos com o maior volume de vendas primeiro, para que a equipe de vendas possa priorizar os itens mais vendidos. **(Utilize a tabela order_details)**

GABARITO EXPLICATIVO

1

Questão 1

A gerência deseja saber quantos fornecedores estão cadastrados no sistema para avaliar as possibilidades de novas parcerias.

Crie uma consulta que retorne a quantidade total de fornecedores. **(Utilize a tabela suppliers)**

Explicação e Resultado

Para contarmos quantos fornecedores estão cadastrados no sistema, fazemos um **SELECT** na tabela **SUPPLIERS**. Como cada registro (linha) dessa tabela contém as informações de um fornecedor, podemos **contar sua quantidade de linhas**. Para isso, usamos a função de agregação **COUNT(*)**. Para que no resultado fique claro a que se refere essa contagem, podemos **fornecer um alias ao COUNT(*)**, chamando-a de **'total_fornecedores'**:

```
SELECT COUNT(*) AS total_fornecedores
FROM suppliers;
```

Dessa forma, obtemos o número total de fornecedores cadastrados no sistema:

	total_fornecedores bigint
1	29

GABARITO EXPLICATIVO

2

Questão 2

O gerente financeiro precisa saber o preço médio dos produtos no sistema para avaliar o custo médio dos itens vendidos.

Crie uma consulta para calcular a média de preço dos produtos. **(Utilize a tabela products)**

Explicação e Resultado

Para calcularmos o preço médio dos produtos no sistema, fazemos um **SELECT** na tabela **PRODUCTS**. A coluna **unit_price** contém os preços individuais de cada produto, e para calcularmos a média desses valores, utilizamos a função de agregação **AVG**. Para tornar o resultado mais claro, atribuímos um alias à função, chamando-a de **preco_medio_produtos**:

```
SELECT AVG(unit_price) AS preco_medio_produtos
FROM products;
```

Dessa forma, obtemos o preço médio de todos os produtos cadastrados no sistema:

	preco_medio_produtos double precision
1	28.83389609200614

GABARITO EXPLICATIVO

3

Questão 3

A equipe de vendas deseja saber qual é o menor e o maior valor de frete já cobrado em pedidos para entender a variabilidade nos custos de entrega.

Crie a consulta que retorna esses valores. **(Utilize a tabela orders)**

Explicação e Resultado

Para descobrirmos o menor e o maior valor de frete já cobrado em pedidos, utilizamos a função **MIN** para identificar o menor valor e a função **MAX** para identificar o maior valor. Essas funções são aplicadas à coluna **freight** da tabela **ORDERS**. Para facilitar a leitura, fornecemos aliases aos resultados como **menor_frete** e **maior_frete**:

```sql
SELECT
    MIN(freight) AS menor_frete,
    MAX(freight) AS maior_frete
FROM orders;
```

Dessa forma, obtemos os valores mínimo e máximo cobrados em fretes, permitindo a análise de variabilidade nos custos de entrega:

	menor_frete real	maior_frete real
1	0.02	1007.64

GABARITO EXPLICATIVO

4

Questão 4

O setor de Recursos Humanos está realizando um levantamento regional para identificar a quantidade de funcionários que residem na região "WA". Essa informação é fundamental para a organização de um evento corporativo local.

Desenvolva uma consulta que mostre a contagem total de funcionários nessa região. **(Utilize a tabela employees)**

Explicação e Resultado

Para contarmos o número de funcionários que residem na região "WA", fazemos um **SELECT** na tabela **EMPLOYEES** e aplicamos um filtro com a cláusula **WHERE**, limitando a consulta à coluna **region** com valor igual a "WA". Utilizamos a função **COUNT(*)** para contar essas ocorrências e fornecemos um alias como **funcionarios_em_wa**:

```
SELECT COUNT(*) AS funcionarios_em_wa
FROM employees
WHERE region = 'WA';
```

Assim, obtemos a quantidade total de funcionários dessa região, permitindo o planejamento do evento corporativo:

	funcionarios_em_wa bigint
1	5

GABARITO EXPLICATIVO

5

Questão 5

O setor de logística está avaliando o volume de produtos encomendados que são embalados em garrafas. Para isso, eles precisam saber o total de unidades desses produtos que estão em pedidos pendentes.

Crie uma consulta que retorne a soma das unidades encomendadas (units_on_order) de todos os produtos cuja descrição no campo "quantity_per_unit" mencione "bottles". **(Utilize a tabela products)**

Explicação e Resultado

Para calcular o total de unidades encomendadas de produtos embalados em garrafas, fazemos um **SELECT** na tabela **PRODUCTS** e utilizamos a cláusula **WHERE** para filtrar os registros cuja coluna **quantity_per_unit** contém o texto 'bottles', usando o operador **LIKE** para tanto, passando como seu parâmetro **'%bottles%'**. Em seguida, somamos as quantidades encomendadas com a função **SUM** aplicada à coluna **units_on_order**. Para tornar o resultado mais claro, atribuímos um alias à função, chamando-a de **total_encomendado**:

```
SELECT SUM(units_on_order) AS total_encomendado
FROM products
WHERE quantity_per_unit LIKE '%bottles%';
```

Dessa forma, obtemos o total de unidades encomendadas desses produtos, atendendo à solicitação do setor de logística.

	total_encomendado bigint
1	120

GABARITO EXPLICATIVO

6

Questão 6

O setor de vendas deseja saber quantos produtos estão disponíveis em cada categoria.

Crie uma consulta que retorne o ID da categoria e a quantidade de produtos associados a ela. Ordene o resultado pela contagem da quantidade de produtos em ordem decrescente. **(Utilize a tabela products)**

Explicação e Resultado

Para saber quantos produtos existem em cada categoria, utilizamos a tabela **PRODUCTS**. Contamos os produtos com a função **COUNT(*)**, formando a coluna **quantidade_produtos**, e agrupamos os resultados pela coluna **category_id** usando **GROUP BY**. Para exibir as categorias com mais produtos primeiro, adicionamos a cláusula **ORDER BY** em ordem decrescente:

```sql
SELECT
    category_id,
    COUNT(*) AS quantidade_produtos
FROM products
GROUP BY category_id
ORDER BY quantidade_produtos DESC;
```

	category_id smallint	quantidade_produtos bigint
1	3	13
2	1	12
3	8	12
4	2	12
5	4	10
6	5	7
7	6	6
8	7	5
Total rows: 8 of 8	Query complete 00:00:	

Assim, obtemos uma lista das categorias e suas respectivas quantidades de produtos organizadas em ordem decrescente:

GABARITO EXPLICATIVO

7

Questão 7

O setor financeiro está analisando os custos logísticos relacionados ao envio de pedidos para diferentes países. Para isso, eles precisam saber o total acumulado de frete para cada país de destino, a fim de identificar quais regiões geram os maiores custos de transporte.

Elabore uma consulta que exiba o nome do país de envio e a soma total dos fretes, ordenando o resultado do maior para o menor valor. **(Utilize a tabela orders)**

Explicação e Resultado

Para calcular o custo total de frete por país de destino, utilizamos a tabela **ORDERS**. Somamos os valores da coluna **freight** com a função **SUM** e agrupamos os resultados pela coluna **ship_country**. Para facilitar a análise, ordenamos os resultados em ordem decrescente de fretes acumulados:

```sql
SELECT
    ship_country,
    SUM(freight) AS total_frete
FROM orders
GROUP BY ship_country
ORDER BY total_frete DESC;
```

	ship_country character varying (15)	total_frete real
1	USA	13771.285
2	Germany	11283.278
3	Austria	7391.5005
4	Brazil	4880.19
5	France	4237.8394
6	Sweden	3237.6003
7	UK	2954.27
8	Ireland	2755.24

Total rows: 21 of 21 | Query complete 00

Dessa forma, identificamos a soma total dos fretes para cada país de envio:

GABARITO EXPLICATIVO

8

Questão 8

A equipe de gestão de inventário está buscando uma forma de monitorar o estoque de cada categoria de produtos. Eles precisam entender quantos produtos existem em cada categoria e qual o total de unidades em estoque. Esse controle é essencial para que possam planejar reabastecimentos, identificar possíveis excessos ou faltas de estoque e tomar decisões informadas sobre estratégias de vendas e promoções.

Crie uma consulta que retorne o ID da categoria, a contagem de produtos e a soma do estoque por categoria. **(Utilize a tabela products)**

Explicação e Resultado

Para monitorar o estoque de produtos por categoria, fazemos um **SELECT** na tabela **PRODUCTS**. Contamos os produtos com a função **COUNT(*)** e calculamos o total de unidades em estoque com a função **SUM** aplicada à coluna **units_in_stock**. Agrupamos os resultados pela coluna **category_id**:

```sql
SELECT
    category_id,
    COUNT(*) AS qtd_produtos_categoria,
    SUM(units_in_stock) AS estoque_total
FROM products
GROUP BY category_id;
```

Assim, obtemos o total de produtos e unidades disponíveis por categoria, auxiliando o planejamento de inventário:

	category_id smallint	qtd_produtos_categoria bigint	estoque_total bigint
1	8	12	701
2	7	5	100
3	1	12	559
4	5	7	308
5	4	10	393
6	2	12	507
7	6	6	165
8	3	13	386

Total rows: 8 of 8 | Query complete 00:00:00.080

GABARITO EXPLICATIVO

9

Questão 9

A equipe de análise de vendas da empresa recebeu uma solicitação para avaliar o desempenho médio de preço dos produtos nas categorias de bebidas (category_id = 1) e laticínios (category_id = 4). O objetivo é entender a variação de preço dessas categorias para tomar decisões estratégicas sobre promoções e ajustes de preços.

Sua tarefa é criar uma consulta SQL que calcule o preço médio por unidade para as categorias de bebidas e laticínios, pois isso ajudará a equipe a tomar futuras decisões comerciais. **(Utilize a tabela products)**

Explicação e Resultado

Para calcularmos o preço médio de produtos nas categorias de bebidas **(category_id = 1)** e laticínios **(category_id = 4)**, utilizamos a tabela **PRODUCTS**. Filtramos as categorias com a cláusula **WHERE** e o operador **IN**, e aplicamos a função **AVG** à coluna **unit_price**. Agrupamos os resultados pela coluna **category_id** para obter a média separadamente para cada categoria:

```sql
SELECT
    category_id,
    AVG(unit_price)
FROM products
WHERE category_id IN (1, 4)
GROUP BY category_id;
```

Dessa forma, a equipe de vendas pode analisar e comparar os preços médios das categorias selecionadas:

	category_id smallint	avg double precision
1	1	37.979166666666664
2	4	28.729999923706053

GABARITO EXPLICATIVO

10

Questão 10

A equipe de vendas da empresa está analisando o desempenho dos produtos para entender quais itens estão gerando maior volume de vendas. Para ajudar na tomada de decisões sobre estoque e estratégias de marketing, eles precisam obter um levantamento detalhado das quantidades vendidas de cada produto.

Crie uma consulta SQL que retorne a soma total de unidades vendidas (quantity) por produto, mas somente para os produtos que já alcançaram no mínimo 1000 unidades vendidas. A consulta deve exibir os produtos com o maior volume de vendas primeiro, para que a equipe de vendas possa priorizar os itens mais vendidos. **(Utilize a tabela order_details)**

Explicação e Resultado

Para identificar os produtos que alcançaram pelo menos 1000 unidades vendidas, utilizamos a tabela **ORDER_DETAILS**. Somamos as quantidades vendidas com a função **SUM** aplicada à coluna **quantity** e agrupamos os resultados pela coluna **product_id**. Filtramos os produtos com a cláusula **HAVING**, garantindo que apenas aqueles com 1000 unidades ou mais sejam incluídos. Ordenamos os resultados em ordem decrescente:

```sql
SELECT
    product_id AS id_produto,
    SUM(quantity) AS total_vendido
FROM order_details
GROUP BY product_id
HAVING SUM(quantity) >= 1000
ORDER BY total_vendido DESC;
```

Assim, obtemos os produtos com maior volume de vendas, permitindo à equipe priorizar estratégias de marketing e estoque para os itens mais vendidos:

	id_produto smallint	total_vendido bigint
1	60	1577
2	59	1496
3	31	1397
4	56	1263
5	16	1158
6	75	1155
7	24	1125
8	40	1103
9	62	1083
10	71	1057
11	2	1057
12	21	1016
Total rows: 12 of 12		Query compl

MÓDULO 5

JOINS

JOINS
JOINS

O QUE VEREMOS NESTE MÓDULO

MÓDULO 5 JOINS NO POSTGRESQL

01	Introdução	137
02	Chave Primária vs. Chave Estrangeira	138
03	Tabela Dimensão vs. Tabela Fato	139
04	Sintaxe	141
05	LEFT JOIN	143
06	INNER JOIN	144
07	RIGHT JOIN	145
08	FULL JOIN	146
09	LEFT JOIN ou INNER JOIN?	147
10	Exemplos Práticos	148
11	JOIN, GROUP BY e ORDER BY – Exemplo prático	160
12	JOIN, GROUP BY, WHERE e HAVING – Exemplo prático	166
13	EXERCÍCIOS	170
14	GABARITO EXPLICATIVO	175

Introdução

Os JOINs no SQL têm como objetivo relacionar as diferentes tabelas dos nossos bancos de dados.

Com eles, conseguimos dar um passo além nas nossas análises, permitindo cruzar informações de diferentes tabelas.

Para criar JOINs, o primeiro passo é descobrir qual coluna as tabelas que queremos relacionar têm em comum.
Será através dessa coluna que o SQL saberá a forma como ele deve cruzar os dados.

Exemplo: as tabelas "products" e "order_details" possuem uma coluna em comum, chamada "product_id".

É daí que vêm os conceitos de CHAVE PRIMÁRIA e CHAVE ESTRANGEIRA, que veremos a seguir.

Chave Primária vs. Chave Estrangeira

Uma **Chave Primária** é uma coluna que identifica as informações distintas em uma tabela. Geralmente é uma coluna de ID. Toda tabela terá uma, e somente uma, chave primária. Essa chave é utilizada como identificador único da tabela, sendo representada por uma coluna que não receberá valores repetidos.
Já uma **Chave Estrangeira** é uma coluna que permite relacionar as linhas de uma segunda tabela com a Chave Primária de uma primeira tabela.

Ao lado, vemos que a tabela **products** possui uma coluna chamada product_id, com valores que não se repetem. Essa será a Chave Primária.

Já na tabela **order_details**, a coluna de product_id também aparece, mas os valores se repetem. Isso porque podemos ter mais de um pedido para o mesmo produto.

Na tabela order_details, a coluna de product_id vai ser a Chave Estrangeira e nos permitirá relacionar os valores dessa coluna com a Chave Primária da tabela products.

TABELA PRODUCTS

Data output | Messages | Notifications

	product_id [PK] smallint	product_name character varying (40)
1	1	Chai
2	2	Chang
3	3	Aniseed Syrup
4	4	Chef Anton's Cajun Se...
5	5	Chef Anton's Gumbo ...
6	6	Grandma's Boysenber...
7	7	Uncle Bob's Organic D...
8	8	Northwoods Cranberr...

Chave Primária

TABELA ORDER_DETAILS

Data output | Messages | Notifications

	order_id [PK] smallint	product_id [PK] smallint	unit_price real	quantity smallint	discount real
1	10248	11	14	12	0
2	10248	42	9.8	10	0
3	10248	72	34.8	5	0
4	10249	14	18.6	9	0
5	10249	51	42.4	40	0
6	10250	41	7.7	10	0
7	10250	51	42.4	35	0.15

Chave Estrangeira

Tabela Dimensão vs. Tabela Fato

Uma **Tabela Dimensão** é uma tabela que contém características de um determinado elemento: lojas, produtos, funcionários, clientes, etc.

Nessa tabela, nenhum dos elementos principais irá se repetir. **É onde vamos encontrar nossas chaves primárias.**

Já uma **Tabela Fato** é aquela que vai registrar os fatos ou acontecimentos de uma empresa/negócio em determinados períodos de tempo (vendas, devoluções, aberturas de chamados, receitas, despesas, etc.)

Geralmente, é uma tabela com milhares de informações, composta essencialmente por colunas de ID, conhecidas como **chaves estrangeiras**, usadas para buscar as informações complementares de uma tabela dimensão.

No exemplo da página anterior, a tabela **products** é a tabela Dimensão e a **order_details** é a tabela Fato.

Tabela Dimensão vs. Tabela Fato

Não necessariamente uma relação acontece entre uma fato e uma dimensão.

Duas tabelas dimensão também podem se relacionar, como é o caso do exemplo ao lado.

O que não fazemos é uma relação entre duas tabelas fato.

```
412  SELECT * FROM categories;
```

Data output | Messages | Notifications

	category_id [PK] smallint	category_name character varying (15)	description text
1	1	Beverages	Soft drinks,...
2	2	Condiments	Sweet and ...
3	3	Confections	Desserts, c...
4	4	Dairy Products	Cheeses
5	5	Grains/Cereals	Breads, cra...
6	6	Meat/Poultry	Prepared m...
7	7	Produce	Dried fruit ...
8	8	Seafood	Seaweed a...

Chave Primária

```
SELECT * FROM products;
```

Chave Estrangeira

	category_id smallint
1	1
2	1
3	2
4	2
5	2
6	2
7	7

Sintaxe

A sintaxe mais simples para relacionar 2 tabelas (que tenham a 'Coluna1' em comum é a seguinte):

```
SELECT
   *
FROM
   Tabela_A
(xxxx) JOIN Tabela_B
ON Tabela_A.Coluna1 = Tabela_B.Coluna1;
```

Com a opção acima, trazemos em uma mesma consulta TODAS as colunas das duas tabelas relacionadas, isso porque usamos o *.

Sintaxe

Caso a gente queira escolher colunas específicas para visualizar na consulta final, seguimos a seguinte estrutura:

Opção 1:

```
SELECT
   Tabela_A.Coluna1,
   Tabela_A.Coluna2,
   Tabela_A.Coluna3,
   Tabela_B.Coluna4
FROM
   Tabela_A
(xxxx) JOIN Tabela_B
   ON Tabela_A.Coluna1 = Tabela_B.Coluna1;
```

Opção 2 (utilizando aliases):

```
SELECT
   ta.Coluna1,
   ta.Coluna2,
   ta.Coluna3,
   tb.Coluna4
 FROM
   Tabela_A ta
(xxxx) JOIN Tabela_B tb
   ON ta.Coluna1 = tb.Coluna1;
```

LEFT JOIN

O **LEFT JOIN** estabelece o relacionamento entre as tabelas, retornando as linhas que são comuns entre as duas tabelas e também as linhas que existem apenas na tabela da ESQUERDA.

No exemplo abaixo, repare que o ID_Produto = 4 não está cadastrado na tabela Produtos (da direita), porém existe na tabela Vendas (da esquerda). Portanto, foi retornado na tabela Final, mesmo sem ter informações na coluna Produto (NULL):

TABELA VENDAS

ID_Vendas	ID_Produto	Quantidade
001	1	10
002	3	20
003	4	40

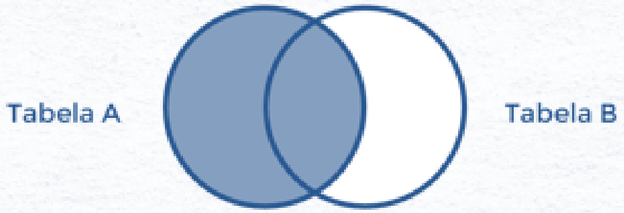


TABELA PRODUTOS

ID_Produto	Produto
1	A
2	B
3	C

TABELA FINAL

ID_Vendas	ID_Produto	Quantidade	Produto
001	1	10	A
002	3	20	C
003	4	40	NULL

INNER JOIN

O **INNER JOIN** realiza o relacionamento entre as tabelas e retorna apenas as linhas que são comuns entre as duas tabelas.

Abaixo, repare que somente os produtos de ID_Produto = 1 e 3 (A e C) foram retornados na tabela Final, pois são os únicos em comum entre as duas tabelas [pois o ID_Produto = 4 só existe na tabela Vendas, enquanto que o ID_Produto = 2 (B) só existe na tabela Produtos]:

TABELA VENDAS

ID_Vendas	ID_Produto	Quantidade
001	1	10
002	3	20
003	4	40

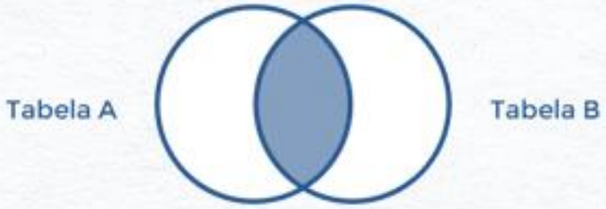


TABELA PRODUTOS

ID_Produto	Produto
1	A
2	B
3	C

TABELA FINAL

ID_Vendas	ID_Produto	Quantidade	Produto
001	1	10	A
002	3	20	C

RIGHT JOIN

O **RIGHT JOIN** estabelece o relacionamento entre as tabelas, retornando as linhas que são comuns entre as duas tabelas e também as linhas que existem apenas na tabela da DIREITA.

Agora, repare que, além dos produtos de ID_Produto = 1 e 3 (A e C), comum entre as tabelas, também foi retornado na tabela Final o Produto B, por estar cadastrado na tabela Produtos (da direita), mesmo não existindo na tabela Vendas (NULL). Já o ID_Produto = 4 não foi retornado, pois só existe na tabela Vendas (da esquerda).

TABELA VENDAS

ID_Vendas	ID_Produto	Quantidade
001	1	10
002	3	20
003	4	40

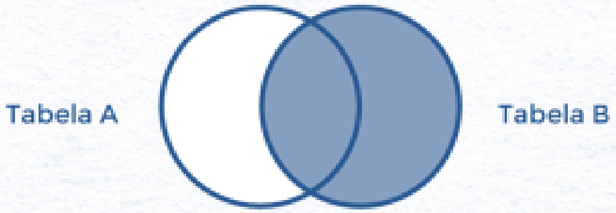


TABELA PRODUTOS

ID_Produto	Produto
1	A
2	B
3	C

TABELA FINAL

ID_Vendas	ID_Produto	Quantidade	Produto
001	1	10	A
NULL	NULL	NULL	B
002	3	20	C

PostgreSQL

FULL JOIN

O **FULL JOIN** estabelece o relacionamento entre as tabelas e retorna TODAS as linhas das tabelas.

Agora, repare que, além dos produtos de ID_Produto = 1 e 3 (A e C), comum entre as tabelas, também foram retornados na tabela Final o Produto B, por estar cadastrado na tabela Produtos (da direita), bem como o ID_Produto = 4 por estar cadastrado na tabela Vendas (da esquerda).

TABELA VENDAS

ID_Vendas	ID_Produto	Quantidade
001	1	10
002	3	20
003	4	40

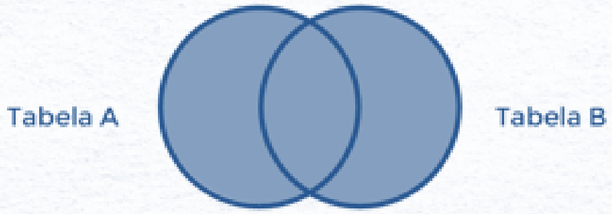


TABELA PRODUTOS

ID_Produto	Produto
1	A
2	B
3	C

TABELA FINAL

ID_Vendas	ID_Produto	Quantidade	Produto
001	1	10	A
002	3	20	C
003	4	40	NULL
NULL	NULL	NULL	B

LEFT JOIN ou INNER JOIN?

Muitas vezes, o LEFT JOIN e o INNER JOIN, que são os dois tipos de JOINs mais utilizados, retornarão o mesmo resultado.

Tomando como exemplo nossas tabelas Vendas e Produtos, isso acontecerá quando tanto todos os produtos vendidos na tabela Vendas também estiverem cadastrados na tabela Produtos, quanto todos os produtos cadastrados na tabela Produtos tiverem sido vendidos na tabela Vendas. Veja só:

TABELA VENDAS

ID_Vendas	ID_Produto	Quantidade
001	1	10
002	3	20
003	4	40
004	2	50

TABELA PRODUTOS

ID_Produto	Produto
1	A
2	B
3	C
4	D

TABELA FINAL

ID_Vendas	ID_Produto	Quantidade	Produto
001	1	10	A
002	3	20	C
003	4	40	D
004	2	50	B

PostgreSQL

Exemplos Práticos

Voltando ao nosso banco de dados Northwind, vamos agora fazer um JOIN para relacionar as tabelas products e categories.

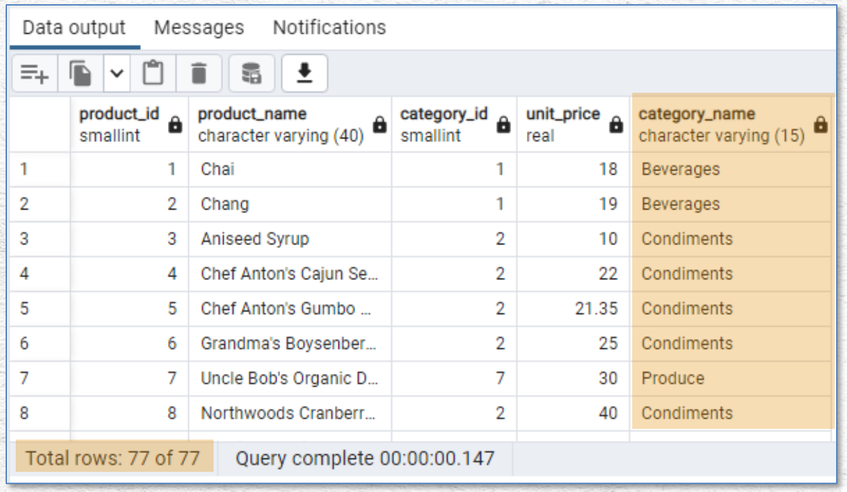
Neste JOIN, queremos retornar as colunas product_id, product_name, category_id, unit_price (da tabela products) e category_name (da tabela categories).

Façamos um LEFT JOIN para ver como fica:

```sql
SELECT
    products.product_id,
    products.product_name,
    products.category_id,
    products.unit_price,
    categories.category_name
FROM products
LEFT JOIN categories
    ON products.category_id = categories.category_id;
```

Exemplos Práticos

Repare que foram retornadas 77 linhas no nosso resultado, que é exatamente a quantidade de produtos cadastrados na tabela da esquerda: a products. Repare que a coluna category_name, da tabela da direita (categories), também trouxe as informações que queríamos acrescentar ao nosso resultado, graças ao LEFT JOIN que efetuamos:



Data output | Messages | Notifications

	product_id smallint	product_name character varying (40)	category_id smallint	unit_price real	category_name character varying (15)
1	1	Chai	1	18	Beverages
2	2	Chang	1	19	Beverages
3	3	Aniseed Syrup	2	10	Condiments
4	4	Chef Anton's Cajun Se...	2	22	Condiments
5	5	Chef Anton's Gumbo ...	2	21.35	Condiments
6	6	Grandma's Boysenber...	2	25	Condiments
7	7	Uncle Bob's Organic D...	7	30	Produce
8	8	Northwoods Cranberr...	2	40	Condiments

Total rows: 77 of 77 | Query complete 00:00:00.147

Como não existe nenhum produto cadastrado na tabela products sem uma categoria (category_id) relacionada a ele, a coluna retornada da tabela categories (category_name) está toda preenchida, sem nenhum valor nulo (NULL).

Porém, como utilizamos o LEFT JOIN, caso existisse algum produto sem categoria, ele seria retornado mesmo assim, pois encontra-se registrado na tabela da esquerda (products); apenas as informações de categoria (category_id e category_name) seriam nulas.

Exemplos Práticos

Entretanto, como todos os produtos da tabela products estão relacionados a alguma categoria da tabela categories, assim como cada categoria da tabela categories está relacionada a um ou mais produtos da tabela products, neste caso, se utilizarmos um INNER JOIN no lugar de um LEFT JOIN, obteremos exatamente o mesmo resultado. Veja só:

```sql
SELECT
    products.product_id,
    products.product_name,
    products.category_id,
    products.unit_price,
    categories.category_name
FROM products
INNER JOIN categories
    ON products.category_id = categories.category_id;
```

Data output | Messages | Notifications

	product_id smallint	product_name character varying (40)	category_id smallint	unit_price real	category_name character varying (15)
1	1	Chai	1	18	Beverages
2	2	Chang	1	19	Beverages
3	3	Aniseed Syrup	2	10	Condiments
4	4	Chef Anton's Cajun Seasoning	2	22	Condiments
5	5	Chef Anton's Gumbo Mix	2	21.35	Condiments
6	6	Grandma's Boysenberry Spread	2	25	Condiments
7	7	Uncle Bob's Organic Dried Pears	7	30	Produce
8	8	Northwoods Cranberry Sauce	2	40	Condiments

Total rows: 77 of 77 | Query complete 00:00:00.079

Exemplos Práticos

Repare também que, se preferirmos, em vez de repetir os nomes das tabelas em todo o código, podemos atribuir aliases a essas tabelas, e utilizá-los para se referir a elas, deixando o código mais enxuto.

No exemplo abaixo, chamamos a tabela products de p e a tabela categories de c. Dessa forma, em todas as linhas do código em que foi necessário mencionar seus nomes, utilizamos seus respectivos aliases.

Repare que, ao executar o código abaixo, obtivemos exatamente o mesmo resultado:

```sql
SELECT
    p.product_id,
    p.product_name,
    p.category_id,
    p.unit_price,
    c.category_name
FROM products p
LEFT JOIN categories c
    ON p.category_id = c.category_id;
```

Data output | Messages | Notifications

	product_id smallint	product_name character varying (40)	category_id smallint	unit_price real	category_name character varying (15)
1	1	Chai	1	18	Beverages
2	2	Chang	1	19	Beverages
3	3	Aniseed Syrup	2	10	Condiments
4	4	Chef Anton's Cajun Seasoning	2	22	Condiments
5	5	Chef Anton's Gumbo Mix	2	21.35	Condiments
6	6	Grandma's Boysenberry Spread	2	25	Condiments
7	7	Uncle Bob's Organic Dried Pears	7	30	Produce
8	8	Northwoods Cranberry Sauce	2	40	Condiments

Total rows: 77 of 77 | Query complete 00:00:00.088

Exemplos Práticos

Vejamos mais um exemplo:

Sabemos que na nossa tabela de pedidos (orders), temos 830 registros:

```
493  SELECT * FROM orders;
```

Data output | Messages | Notifications

	order_id [PK] smallint	customer_id character varying (5)	employee_id smallint	order_date date	required_date date	shipped_date date	ship_via smallint	freight real	ship_name character varying (40)	ship_address character varying (60
1	10248	VINET	5	1996-07-04	1996-08-01	1996-07-16	3	32.38	Vins et alcools Cheval...	59 rue de l'Abbaye
2	10249	TOMSP	6	1996-07-05	1996-08-16	1996-07-10	1	11.61	Toms Spezialitäten	Luisenstr. 48

Total rows: 830 of 830 | Query complete 00:00:00.122 | Ln 491, Col 25

Exemplos Práticos

Se fizermos um SELECT DISTINCT customer_id nesta tabela, veremos que esses 830 pedidos foram efetuados por 89 clientes:

```
SELECT
    DISTINCT customer_id
FROM orders;
```

Data output | Messages

	customer_id character varying (5)
1	TOMSP
2	LONEP
3	OLDWO
4	WARTH
5	MAGAA
6	QUEEN
7	VINET
8	ANTON
9	MORGK
10	GOURL

Total rows: 89 of 89 Que

Exemplos Práticos

E se fizermos um SELECT DISTINCT customer_id na tabela customers, veremos que temos 91 clientes cadastrados na nossa base de dados:

```
SELECT
    DISTINCT customer_id
FROM customers;
```

Data output | Messages | Not

	customer_id [PK] character varying (5)
1	TOMSP
2	OLDWO
3	LONEP
4	WARTH
5	MAGAA
6	QUEEN
7	VINET
8	MORGK
9	ANTON
10	GOURL

Total rows: 91 of 91 | Query c

PostgreSQL

Exemplos Práticos

Ou seja: temos dois clientes registrados na tabela customers que nunca efetuaram pedido algum.

Será que conseguimos descobrir quem são esses dois clientes utilizando JOINs?

Vejamos:

Se tentarmos efetuar um LEFT JOIN ou um INNER JOIN, não encontraremos o resultado que esperamos, porque, uma vez que a tabela orders é a nossa tabela da esquerda:

a) O LEFT JOIN retornaria os 830 registros existentes na tabela orders, ignorando as linhas que aparecessem somente na tabela da direita (a customers). Portanto, os dois clientes que nunca efetuaram pedidos, não apareceriam no resultado, uma vez que, obviamente, se eles nunca efetuaram pedidos, não aparecem na tabela orders (da esquerda), somente na customers (da direita).

b) O INNER JOIN também não retornaria os nomes que queremos, uma vez que seu resultado mostraria somente os registros em comum entre as duas tabelas (orders e customers). Como os dois clientes que nunca efetuaram pedidos só aparecem na tabela da direita (customers), não seriam considerados como “em comum” com a tabela da esquerda (orders) pelo INNER JOIN.

Exemplos Práticos

E o RIGHT JOIN? Será que daria certo?

Vamos ver:

```
SELECT
    orders.order_id,
    orders.customer_id,
    orders.order_date,
    customers.contact_name
FROM orders
RIGHT JOIN customers
    ON orders.customer_id = customers.customer_id;
```

Data output | Messages | Notifications

	order_id smallint	customer_id character varying (5)	order_date date	contact_name character varying (30)
1	10248	VINET	1996-07-04	Paul Henriot
2	10249	TOMSP	1996-07-05	Karin Josephs
3	10250	HANAR	1996-07-08	Mario Pontes

Total rows: 832 of 832 Query complete 00:00:00.082

Perceba que, com o RIGHT JOIN, foram retornados 832 registros.

Exemplos Práticos

Destes, 830 fazem referência aos pedidos da tabela orders, que tem 830 pedidos registrados nela. E os outros 2 registros retornados?

Se verificarmos ao final do resultado retornado, veremos o seguinte:

Data output | Messages | Notifications

	order_id smallint	customer_id character varying (5)	order_date date	contact_name character varying (30)
830	11077	RATTC	1998-05-06	Paula Wilson
831	[null]	[null]	[null]	Marie Bertrand
832	[null]	[null]	[null]	Diego Roel

Total rows: 832 of 832 Query complete 00:00:00.082

Repare que apareceram os nomes de dois clientes (contact_name) que não possuem informações nas colunas que foram retornadas pela tabela da esquerda (order_id, customer_id, order_date).

Portanto, Marie Bertrand e Diego Roel são os nomes dos clientes que temos cadastrados na tabela customers (da direita), mas nunca efetuaram pedidos, portanto, não possuem registro algum na tabela da esquerda (orders).

Exemplos Práticos

Assim, com o RIGHT JOIN, conseguimos chegar ao resultado desejado.

Mas, por curiosidade, vamos ver se com o FULL JOIN também conseguimos encontrar os nomes desses dois clientes?

```
SELECT
    orders.order_id,
    orders.customer_id,
    orders.order_date,
    customers.contact_name
FROM orders
FULL JOIN customers
    ON orders.customer_id = customers.customer_id;
```

Data output | Messages | Notifications

	order_id smallint	customer_id character varying (5)	order_date date	contact_name character varying (30)
1	10248	VINET	1996-07-04	Paul Henriot
2	10249	TOMSP	1996-07-05	Karin Josephs

Total rows: 832 of 832 | Query complete 00:00:00.094

Veja que, com o FULL JOIN, também foram retornados 832 registros.

Exemplos Práticos

Novamente, destes registros, 830 fazem referência aos pedidos da tabela orders, que tem 830 pedidos registrados nela. E, verificando ao final do resultado retornado, veremos que os outros 2 registros referem-se aos dois clientes que nunca efetuaram pedidos:

Data output | Messages | Notifications

	order_id smallint	customer_id character varying (5)	order_date date	contact_name character varying (30)
830	11077	RATTC	1998-05-06	Paula Wilson
831	[null]	[null]	[null]	Marie Bertrand
832	[null]	[null]	[null]	Diego Roel

Total rows: 832 of 832 Query complete 00:00:00.094

Como o FULL JOIN é capaz de trazer todas as linhas de todas as tabelas relacionadas no JOIN, ele também conseguiu nos ajudar a descobrir quem eram esses dois clientes.

JOIN, GROUP BY e ORDER BY – Exemplo Prático

Vejamos agora um exemplo de como fazer um JOIN utilizando os comandos GROUP BY e ORDER BY em conjunto.

Vamos supor que a gente queira criar um agrupamento (GROUP BY) que retorne como resultado a quantidade total vendida (SUM(quantity)) para cada produto (product_name). Além disso, queremos ordenar (ORDER BY) o resultado do produto mais vendido para o menos vendido (ordem decrescente).

Sabemos que na tabela order_details não temos a coluna product_name.

O que temos é a coluna product_id que se relaciona com sua equivalente na tabela products.

Por isso, precisaremos fazer um JOIN para associar as tabelas order_details e products, assim poderemos trazer ao resultado os nomes dos produtos (product_name) em vez de seu ID (product_id).

JOIN, GROUP BY e ORDER BY – Exemplo Prático

Para isso, executaremos o seguinte código:

```
SELECT
    products.product_name,
    SUM(order_details.quantity) AS quantidade_total
FROM order_details
LEFT JOIN products
    ON order_details.product_id = products.product_id
GROUP BY products.product_name
ORDER BY quantidade_total DESC;
```

JOIN, GROUP BY e ORDER BY – Exemplo Prático

Para isso, executaremos o seguinte código:

```
SELECT
    products.product_name,
    SUM(order_details.quantity) AS quantidade_total
FROM order_details
LEFT JOIN products
    ON order_details.product_id = products.product_id
GROUP BY products.product_name
ORDER BY quantidade_total DESC;
```

1) Faremos um LEFT JOIN entre as tabelas products (da esquerda) com a order_details (da direita)...

JOIN, GROUP BY e ORDER BY – Exemplo Prático

Para isso, executaremos o seguinte código:

```
SELECT
    products.product_name,
    SUM(order_details.quantity) AS quantidade_total
FROM order_details
LEFT JOIN products
    ON order_details.product_id = products.product_id
GROUP BY products.product_name
ORDER BY quantidade_total DESC;
```

2) ... Somaremos a quantidade total de produtos vendidos...

3) ... Agrupando essa soma pela coluna product_name da tabela products...

JOIN, GROUP BY e ORDER BY – Exemplo Prático

Para isso, executaremos o seguinte código:

```
SELECT
    products.product_name,
    SUM(order_details.quantity) AS quantidade_total
FROM order_details
LEFT JOIN products
    ON order_details.product_id = products.product_id
GROUP BY products.product_name
ORDER BY quantidade_total DESC;
```

4) ... E ordenaremos o resultado pela soma efetuada (quantidade_total) em ordem decrescente (do produto mais vendido para o menos vendido).

JOIN, GROUP BY e ORDER BY – Exemplo Prático

Com isso, teremos o resultado ao lado, mostrando os nomes dos 77 produtos registrados na tabela products (product_name), bem como a soma de sua quantidade total já vendida, registrada na tabela orders_details (quantidade_total):

```sql
SELECT
    products.product_name,
    SUM(order_details.quantity) AS quantidade_total
FROM order_details
LEFT JOIN products
    ON order_details.product_id = products.product_id
GROUP BY products.product_name
ORDER BY quantidade_total DESC;
```

Data output | Messages | Notifications

	product_name character varying (40)	quantidade_total bigint
1	Camembert Pierrot	1577
2	Raclette Courdavault	1496
3	Gorgonzola Telino	1397
4	Gnocchi di nonna Alice	1263
5	Pavlova	1158
6	Rhönbräu Klosterbier	1155
7	Guaraná Fantástica	1125
8	Boston Crab Meat	1103
9	Tarte au sucre	1083
10	Chang	1057
11	Flotemysost	1057
12	Sir Rodney's Scones	1016
13	Jack's New England Clam Cho...	981
14	Lakkalikööri	981
15	Alice Mutton	978
16	Pâté chinois	903
17	Konbu	891

Total rows: 77 of 77 | Query complete 00:00:00.059

JOIN, GROUP BY, WHERE e HAVING – Exemplo Prático

Vamos imaginar que nós precisamos fazer o mesmo agrupamento anterior, só que agora devemos considerar somente os produtos da classe Luxo (ou seja, os produtos com preço acima de R$ 80,00).

Para isso, devemos aplicar um filtro ao nosso código. Qual filtro utilizar: WHERE ou HAVING?

Se temos que primeiro selecionar todos os produtos cujo preço (unit_price) seja acima de R$ 80,00, então precisamos utilizar o WHERE, que filtra a tabela original. Não temos como utilizar o HAVING, pois ele utiliza o agrupamento já efetuado e filtra alguma condição nesse agrupamento. Como queremos filtrar uma informação constante da tabela original (coluna unit_price > 80), devemos utilizar o WHERE.

JOIN, GROUP BY, WHERE e HAVING – Exemplo Prático

Portanto, no nosso código, antes do GROUP BY, devemos acrescentar o filtro WHERE:

```
SELECT
    p.product_name,
    SUM(od.quantity) AS quantidade_total
FROM order_details od
LEFT JOIN products p
    ON od.product_id = p.product_id
WHERE p.unit_price > 80
GROUP BY p.product_name
ORDER BY quantidade_total DESC;
```

Data output | Messages | Notifications

	product_name character varying (40)	quantidade_total bigint
1	Thüringer Rostbratwu...	746
2	Côte de Blaye	623
3	Sir Rodney's Marmala...	313
4	Mishi Kobe Niku	95

Repare que, agora, nosso código retornou apenas os quatro produtos cujo unit_price é maior que R$80,00 na tabela products, efetuando a soma de suas quantidades vendidas, ordenando-os do mais vendido para o menos vendido.

JOIN, GROUP BY, WHERE e HAVING – Exemplo Prático

Pensemos em outra situação: e se quisermos, considerando todos os produtos, retornar na tela apenas aqueles que tiveram mais de 1000 unidades vendidas?

Neste caso, precisaremos primeiro efetuar o agrupamento com a soma da quantidade total vendida de cada produto, para depois conseguir descobrir quais tiveram mais de 1000 unidades vendidas e retornarmos somente esses produtos no resultado. Certo?

Portanto, o filtro que precisamos utilizar agora é o HAVING, já que ele é o comando capaz de filtrar o agrupamento.

JOIN, GROUP BY, WHERE e HAVING – Exemplo Prático

Sendo assim, no nosso código, após o GROUP BY, devemos acrescentar o filtro HAVING:

```
SELECT
    p.product_name,
    SUM(od.quantity) AS quantidade_total
FROM order_details od
LEFT JOIN products p
    ON od.product_id = p.product_id
GROUP BY p.product_name
HAVING SUM(od.quantity) >= 1000
ORDER BY quantidade_total DESC;
```

Data output | Messages | Notifications

	product_name character varying (40)	quantidade_total bigint
1	Camembert Pierrot	1577
2	Raclette Courdavault	1496
3	Gorgonzola Telino	1397
4	Gnocchi di nonna Alice	1263
5	Pavlova	1158
6	Rhönbräu Klosterbier	1155
7	Guaraná Fantástica	1125
8	Boston Crab Meat	1103
9	Tarte au sucre	1083
10	Chang	1057
11	Flotemysost	1057
12	Sir Rodney's Scones	1016

Total rows: 12 of 12 | Query complete 00:00:0

Repare que, agora, nosso código efetuou a soma das quantidades vendidas para cada produto e retornou apenas os 12 que tiveram mais de 1000 unidades vendidas, ordenando-os do mais vendido para o menos vendido.

EXERCÍCIOS

1

Questão 1

A equipe de compras está analisando os fornecedores para otimizar a logística de importação e transporte. Eles precisam saber em quais países estão localizados os fornecedores dos produtos disponíveis no estoque. Essa informação será usada para negociar melhores condições de frete internacional e tempo de entrega.

Crie uma consulta usando um LEFT JOIN para mostrar o nome do produto, o nome do fornecedor e o país do fornecedor. **(Utilize as tabelas products e suppliers)**

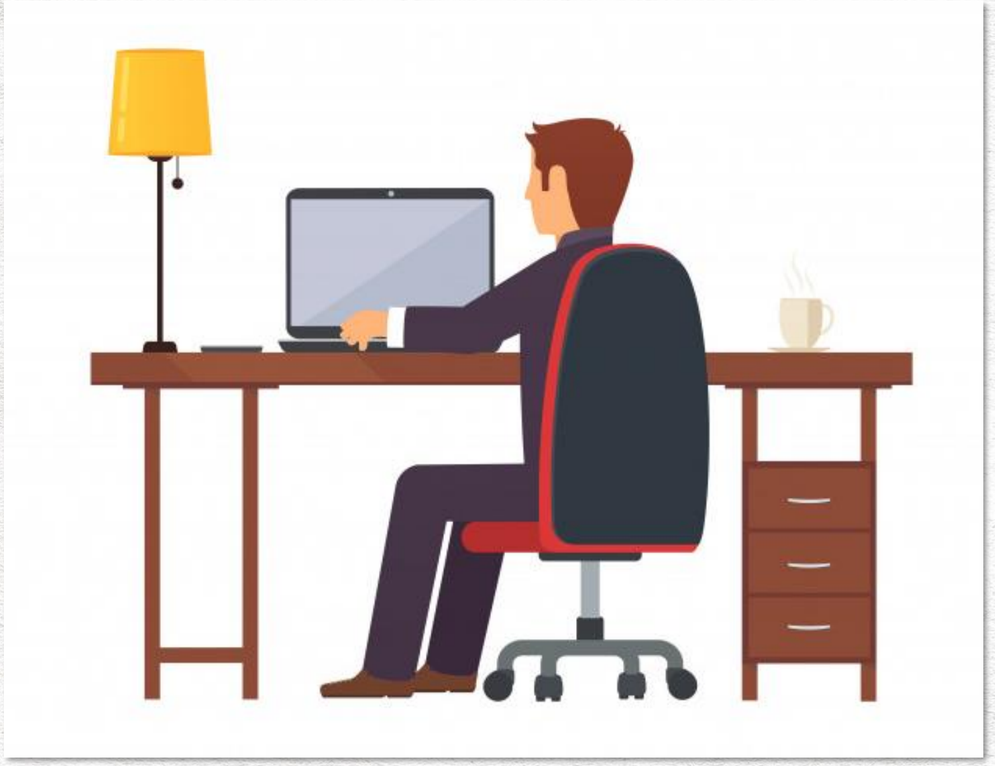
2a

Questão 2-a

O setor de análise está desenvolvendo um relatório de fidelidade para identificar os clientes mais engajados com a empresa. Eles precisam saber quantos pedidos cada cliente realizou para criar estratégias personalizadas de retenção, como programas de benefícios e descontos progressivos.

Crie uma consulta que mostre o ID do cliente, o nome do cliente e a quantidade total de pedidos realizados por cliente. Ordene a consulta do cliente que mais efetuou pedidos para o que menos comprou na empresa. **(Utilize as tabelas customers e orders)**

EXERCÍCIOS



2b

Questão 2-b

Existem clientes que ainda não efetuaram nenhum pedido e o setor de análise também deseja identificá-los, com o objetivo de direcionar campanhas de marketing específicas para motivá-los a realizar suas primeiras compras.

Altere o código do item A para retornar apenas esses clientes. **(Utilize as tabelas customers e orders)**

2c

Questão 2-c

A equipe de análise também quer avaliar o mercado nacional e entender o comportamento de compra dos clientes brasileiros. Eles precisam de um relatório que mostre a quantidade de pedidos efetuados apenas por clientes localizados no Brasil, ajudando a ajustar estratégias de vendas locais.

Altere a consulta do item A para que retorne o resultado esperado. **(Utilize as tabelas customers e orders)**

EXERCÍCIOS

3

Questão 3

O setor financeiro está criando um relatório para monitorar a rentabilidade por cliente. Eles precisam calcular a receita total gerada por cada cliente com base nos pedidos realizados. Essa informação será utilizada para identificar os clientes mais lucrativos e propor ações de fidelização específicas.

Crie uma consulta que liste o ID do cliente, o nome do cliente e a receita de seus pedidos (unit_price * quantity). Dica: você precisará fazer dois INNER JOINs nesta consulta. **(Utilize as tabelas customers, orders e order_details)**

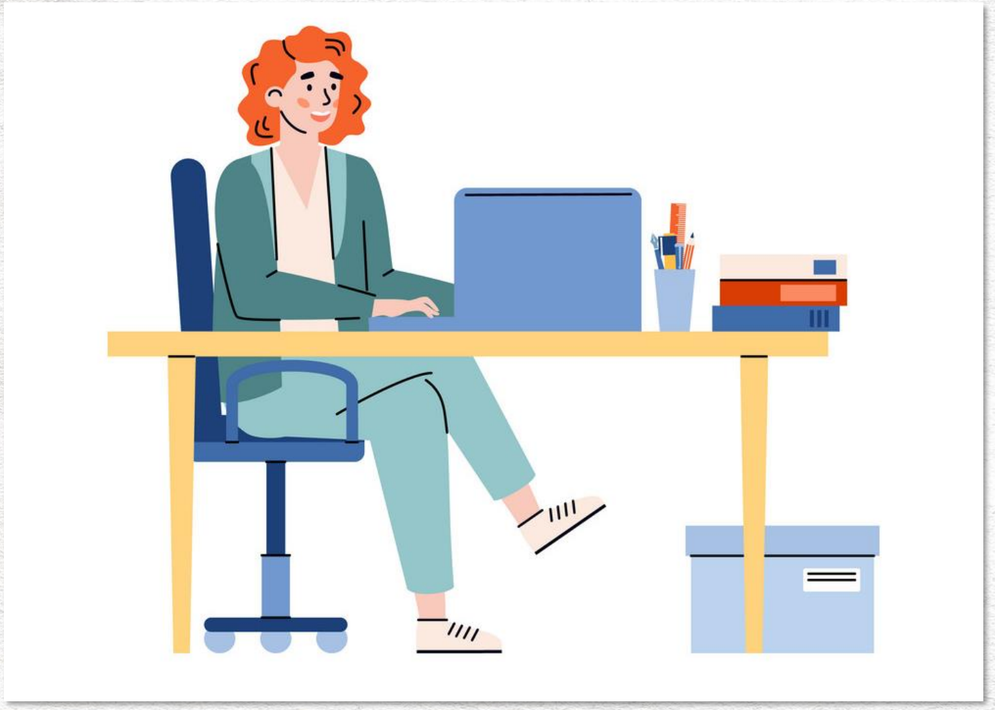
4

Questão 4

O setor de atendimento ao cliente está analisando o comportamento de clientes que realizam mais de uma compra no mesmo dia. Eles querem identificar esses padrões para propor soluções que melhorem a experiência do cliente, como combos de produtos ou programas de assinatura.

Crie uma consulta que retorne o nome do cliente e a data do pedido quando o cliente tiver realizado mais de um pedido na mesma data. **(Utilize as tabelas orders e customers)**

EXERCÍCIOS



5

Questão 5

A equipe financeira quer priorizar negociações com fornecedores que oferecem produtos com preços médios acima de R$30. Esses fornecedores serão tratados como parceiros estratégicos para produtos de maior valor agregado, visando maximizar a margem de lucro da empresa.

Crie uma consulta que retorne o nome do fornecedor e a média de preço dos produtos fornecidos, considerando apenas os fornecedores cuja média de preço seja superior a R$30. **(Utilize as tabelas products e suppliers)**

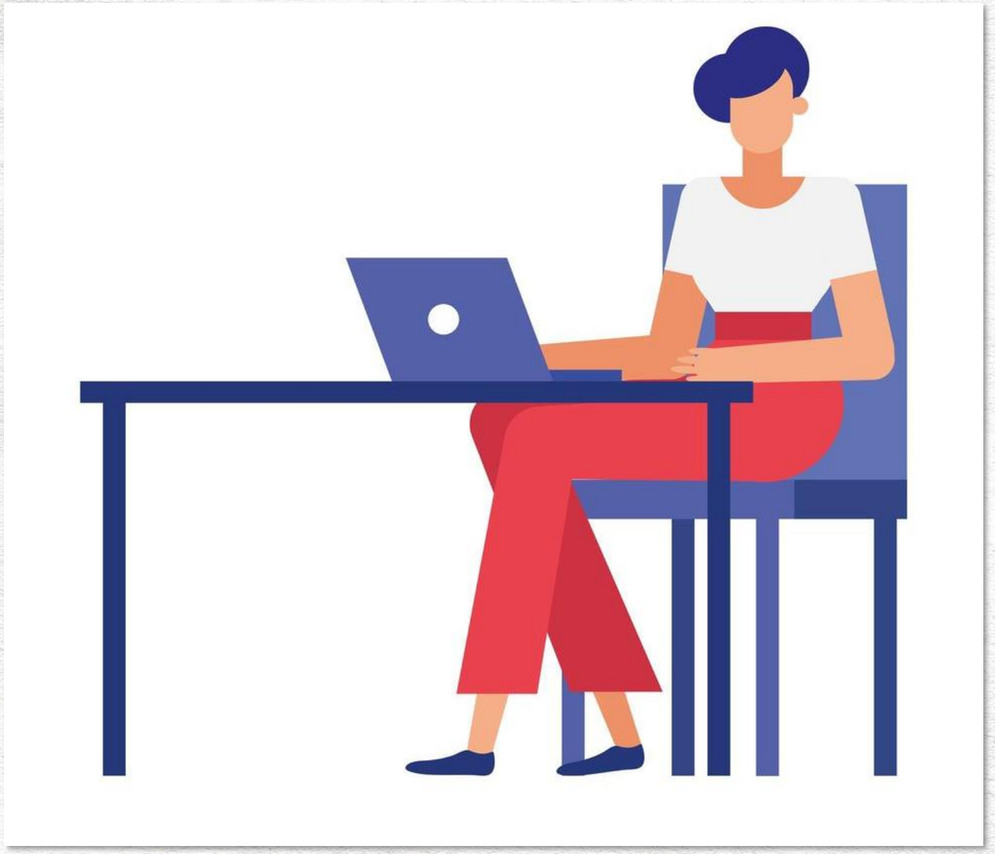
6

Questão 6

A gerência deseja premiar os funcionários que gerenciaram mais de 10 pedidos ao longo do ano de 1997 como forma de motivação e reconhecimento. Para isso, eles precisam de um relatório detalhado com os nomes dos colaboradores e o número total de pedidos gerenciados por cada um.

Crie uma consulta que liste os nomes completos dos funcionários e a quantidade total de pedidos que cada um gerenciou em 1997, ordenados do maior para o menor número de pedidos. Considere apenas os funcionários com mais de 10 pedidos. **(Utilize as tabelas orders e employees)**

EXERCÍCIOS



7

Questão 7

A equipe de logística está avaliando o desempenho das transportadoras para otimizar as operações de entrega. Eles precisam identificar quais transportadoras realizaram mais entregas nos últimos meses para manter ou renegociar contratos baseados em volume e eficiência.

Liste os nomes das transportadoras e a quantidade de pedidos entregues por cada uma. Ordene o resultado pela quantidade de pedidos em ordem decrescente. **(Utilize as tabelas orders e shippers)**

8

Questão 8

A diretoria quer identificar quais categorias de produtos geraram o maior valor médio de vendas por produto. Esse relatório será usado para decidir em quais categorias investir mais em marketing, promoções e estoque, maximizando o retorno financeiro.

Liste os nomes das categorias e o valor médio das vendas por categoria (média de unit_price * quantity). Ordene os resultados pelo valor médio em ordem decrescente. Dica: você precisará fazer dois INNER JOINs nesta consulta. **(Utilize as tabelas order_details, products e categories)**

GABARITO EXPLICATIVO

1

Questão 1

A equipe de compras está analisando os fornecedores para otimizar a logística de importação e transporte. Eles precisam saber em quais países estão localizados os fornecedores dos produtos disponíveis no estoque. Essa informação será usada para negociar melhores condições de frete internacional e tempo de entrega.

Crie uma consulta usando um LEFT JOIN para mostrar o nome do produto, o nome do fornecedor e o país do fornecedor. **(Utilize as tabelas products e suppliers)**

Explicação e Resultado

Para exibir o nome do produto, o nome do fornecedor e o país do fornecedor, utilizamos um **LEFT JOIN** entre as tabelas **PRODUCTS** e **SUPPLIERS**.

A correspondência entre as tabelas é feita com base na coluna **supplier_id**, presente nas duas tabelas.

Os dados retornados incluem o nome do produto (**product_name**), o nome da empresa fornecedora (**company_name**) e o país onde o fornecedor está localizado (**country**):

```sql
SELECT
    p.product_name AS "Produto",
    s.company_name AS "Fornecedor",
    s.country AS "País"
FROM products p
LEFT JOIN suppliers s
    ON p.supplier_id = s.supplier_id;
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

1

Questão 1

A equipe de compras está analisando os fornecedores para otimizar a logística de importação e transporte. Eles precisam saber em quais países estão localizados os fornecedores dos produtos disponíveis no estoque. Essa informação será usada para negociar melhores condições de frete internacional e tempo de entrega.

Crie uma consulta usando um LEFT JOIN para mostrar o nome do produto, o nome do fornecedor e o país do fornecedor. **(Utilize as tabelas products e suppliers)**

Explicação e Resultado

Dessa forma, obtemos a lista dos 77 produtos e seus respectivos fornecedores, junto com o país de cada fornecedor, permitindo a análise para otimização da logística de importação e transporte:

	Produto character varying (40)	Fornecedor character varying (40)	País character var
1	Chai	Specialty Biscuits, Ltd.	UK
2	Chang	Exotic Liquids	UK
3	Aniseed Syrup	Exotic Liquids	UK
4	Chef Anton's Cajun Seasoning	New Orleans Cajun Delights	USA
5	Chef Anton's Gumbo Mix	New Orleans Cajun Delights	USA
6	Grandma's Boysenberry Spread	Grandma Kelly's Homestead	USA
7	Uncle Bob's Organic Dried Pears	Grandma Kelly's Homestead	USA
8	Northwoods Cranberry Sauce	Grandma Kelly's Homestead	USA
9	Mishi Kobe Niku	Tokyo Traders	Japan
10	Ikura	Tokyo Traders	Japan

Total rows: 77 of 77 Query complete 00:00:00.128

GABARITO EXPLICATIVO

2a

Questão 2-a

O setor de análise está desenvolvendo um relatório de fidelidade para identificar os clientes mais engajados com a empresa. Eles precisam saber quantos pedidos cada cliente realizou para criar estratégias personalizadas de retenção, como programas de benefícios e descontos progressivos.

Crie uma consulta que mostre o ID do cliente, o nome do cliente e a quantidade total de pedidos realizados por cliente. Ordene a consulta do cliente que mais efetuou pedidos para o que menos comprou na empresa. **(Utilize as tabelas customers e orders)**

Explicação e Resultado

Para identificar quantos pedidos cada cliente realizou, utilizamos um **LEFT JOIN** entre as tabelas **CUSTOMERS** e **ORDERS**. Isso nos permite listar todos os clientes, incluindo aqueles que ainda não fizeram nenhum pedido. A correspondência é feita com base na coluna **customer_id**.

Usamos a função de agregação **COUNT(o.order_id)** para contar o número total de pedidos realizados por cada cliente. Em seguida, agrupamos os dados pelo ID do cliente (**customer_id**) e pelo nome da empresa (**company_name**) com a cláusula **GROUP BY**.

Por fim, ordenamos os resultados em ordem decrescente (**DESC**) pelo total de pedidos (**quantidade_pedidos**) para que os clientes mais engajados apareçam no topo:

```sql
SELECT
    c.customer_id,
    c.company_name,
    COUNT(o.order_id) AS quantidade_pedidos
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.company_name
ORDER BY quantidade_pedidos DESC;
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

2a

Questão 2-a

O setor de análise está desenvolvendo um relatório de fidelidade para identificar os clientes mais engajados com a empresa. Eles precisam saber quantos pedidos cada cliente realizou para criar estratégias personalizadas de retenção, como programas de benefícios e descontos progressivos.

Crie uma consulta que mostre o ID do cliente, o nome do cliente e a quantidade total de pedidos realizados por cliente. Ordene a consulta do cliente que mais efetuou pedidos para o que menos comprou na empresa. **(Utilize as tabelas customers e orders)**

Explicação e Resultado

Dessa forma, obtemos o relatório de fidelidade com o total de pedidos efetuados por cada um dos 91 clientes, permitindo à equipe de análise desenvolver estratégias personalizadas de retenção:

	customer_id [PK] character	company_name character varying (40)	quantidade_pedidos bigint
1	SAVEA	Save-a-lot Markets	31
2	ERNSH	Ernst Handel	30
3	QUICK	QUICK-Stop	28
4	HUNGO	Hungry Owl All-Night Grocers	19
5	FOLKO	Folk och fä HB	19
6	BERGS	Berglunds snabbköp	18
7	RATTC	Rattlesnake Canyon Grocery	18
8	HILAA	HILARION-Abastos	18
9	BONAP	Bon app'	17
10	FRANK	Frankenversand	15

Total rows: 91 of 91 Query complete 00:00:00.089

GABARITO EXPLICATIVO

2b

Questão 2-b

Existem clientes que ainda não efetuaram nenhum pedido e o setor de análise também deseja identificá-los, com o objetivo de direcionar campanhas de marketing específicas para motivá-los a realizar suas primeiras compras.

Altere o código do item A para retornar apenas esses clientes. **(Utilize as tabelas customers e orders)**

Explicação e Resultado

Para adaptar o código do item A para atender à necessidade deste item B, fizemos apenas uma alteração principal: incluímos a cláusula **HAVING COUNT(o.order_id) = 0**. Essa cláusula é usada após o **GROUP BY** para filtrar os resultados e exibir apenas os clientes cuja contagem de pedidos seja igual a zero.

Essa modificação garante que, na lista final, apareçam somente os clientes que não realizaram nenhum pedido. O restante do código permanece igual ao do item A, incluindo o **LEFT JOIN** para assegurar que todos os clientes sejam incluídos, mesmo os que não possuem correspondência na tabela **ORDERS**. Opcionalmente, podemos remover o **ORDER BY**, já que o retorno de todas as linhas da coluna calculada **quantidade_pedidos** será 0:

```sql
SELECT
    c.customer_id,
    c.company_name,
    COUNT(o.order_id) AS quantidade_pedidos
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.company_name
HAVING COUNT(o.order_id) = 0;
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

2b

Questão 2-b

Existem clientes que ainda não efetuaram nenhum pedido e o setor de análise também deseja identificá-los, com o objetivo de direcionar campanhas de marketing específicas para motivá-los a realizar suas primeiras compras.

Altere o código do item A para retornar apenas esses clientes. **(Utilize as tabelas customers e orders)**

Explicação e Resultado

Com essa alteração, obtemos uma lista dos clientes que ainda não efetuaram pedidos, permitindo que o setor de análise direcione campanhas específicas para motivá-los a realizar sua primeira compra:

	customer_id [PK] character	company_name character varying (40)	quantidade_pedidos bigint
1	FISSA	FISSA Fabrica Inter. Salchichas S.A.	0
2	PARIS	Paris spécialités	0

Total rows: 2 of 2 Query complete 00:00:00.087

GABARITO EXPLICATIVO

2c

Questão 2-c

A equipe de análise também quer avaliar o mercado nacional e entender o comportamento de compra dos clientes brasileiros. Eles precisam de um relatório que mostre a quantidade de pedidos efetuados apenas por clientes localizados no Brasil, ajudando a ajustar estratégias de vendas locais.

Altere a consulta do item A para que retorne o resultado esperado. **(Utilize as tabelas customers e orders)**

Explicação e Resultado

Para adaptar o código do item A para atender à necessidade deste item C, foi adicionada a cláusula **WHERE c.country = 'Brazil'**. Essa modificação restringe a consulta para considerar apenas os clientes localizados no Brasil, filtrando os dados logo após o **LEFT JOIN**.

Além disso, incluímos a coluna **c.country** no **SELECT** para exibir explicitamente o país do cliente no resultado. O restante do código permanece o mesmo, incluindo o agrupamento por cliente e a ordenação dos clientes com base na quantidade de pedidos:

```sql
SELECT
    c.customer_id,
    c.company_name,
    c.country,
    COUNT(o.order_id) AS quantidade_pedidos
FROM customers c
LEFT JOIN orders o
    ON c.customer_id = o.customer_id
WHERE c.country = 'Brazil'
GROUP BY c.customer_id, c.company_name
ORDER BY quantidade_pedidos DESC;
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

2c

Questão 2-c

A equipe de análise também quer avaliar o mercado nacional e entender o comportamento de compra dos clientes brasileiros. Eles precisam de um relatório que mostre a quantidade de pedidos efetuados apenas por clientes localizados no Brasil, ajudando a ajustar estratégias de vendas locais.

Altere a consulta do item A para que retorne o resultado esperado. **(Utilize as tabelas customers e orders)**

Explicação e Resultado

Com essa alteração, o relatório passa a exibir apenas os clientes brasileiros e a quantidade de pedidos realizados por eles, permitindo que a equipe de análise foque no mercado nacional e desenvolva estratégias direcionadas ao público local:

	customer_id [PK] character	company_name character varying (40)	country character varyi	quantidade_pedidos bigint
1	HANAR	Hanari Carnes	Brazil	14
2	QUEEN	Queen Cozinha	Brazil	13
3	RICAR	Ricardo Adocicados	Brazil	11
4	GOURL	Gourmet Lanchonetes	Brazil	9
5	QUEDE	Que Delícia	Brazil	9
6	WELLI	Wellington Importadora	Brazil	9
7	FAMIA	Familia Arquibaldo	Brazil	7
8	TRADH	Tradição Hipermercados	Brazil	6
9	COMMI	Comércio Mineiro	Brazil	5

Total rows: 9 of 9 Query complete 00:00:00.083

GABARITO EXPLICATIVO

3

Questão 3

O setor financeiro está criando um relatório para monitorar a rentabilidade por cliente. Eles precisam calcular a receita total gerada por cada cliente com base nos pedidos realizados. Essa informação será utilizada para identificar os clientes mais lucrativos e propor ações de fidelização específicas.

Crie uma consulta que liste o ID do cliente, o nome do cliente e a receita de seus pedidos (unit_price * quantity). Dica: você precisará fazer dois INNER JOINs nesta consulta. **(Utilize as tabelas customers, orders e order_details)**

Explicação e Resultado

Para atender à solicitação do setor financeiro e calcular a receita total gerada por cada cliente, seguimos estes passos:

Selecionamos as informações necessárias: o **customer_id** e o **company_name** da tabela **CUSTOMERS** e a soma da receita gerada pelos pedidos de cada cliente, calculada como **unit_price * quantity** da tabela **ORDER_DETAILS**.

Realizamos **dois INNER JOINs**: o **primeiro** conecta **CUSTOMERS** com **ORDERS** pela coluna **customer_id**, para vincular os clientes aos seus pedidos. O **segundo** conecta **ORDERS** com **ORDER_DETAILS** pela coluna **order_id**, para obter os detalhes de cada pedido.

Calculamos a receita total por cliente: usamos a função de agregação **SUM** para **somar o resultado de unit_price * quantity** para cada cliente.

Agrupamos os resultados: aplicamos o **GROUP BY** ao **customer_id** e ao **company_name**, garantindo que a soma da receita seja calculada individualmente para cada cliente:

```sql
SELECT
    c.customer_id,
    c.company_name,
    SUM(od.unit_price * od.quantity) AS receita_por_cliente
FROM customers c
INNER JOIN orders o
    ON c.customer_id = o.customer_id
    INNER JOIN order_details od
        ON o.order_id = od.order_id
GROUP BY c.customer_id, c.company_name;
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

3

Questão 3

O setor financeiro está criando um relatório para monitorar a rentabilidade por cliente. Eles precisam calcular a receita total gerada por cada cliente com base nos pedidos realizados. Essa informação será utilizada para identificar os clientes mais lucrativos e propor ações de fidelização específicas.

Crie uma consulta que liste o ID do cliente, o nome do cliente e a receita de seus pedidos (unit_price * quantity). Dica: você precisará fazer dois INNER JOINs nesta consulta. **(Utilize as tabelas customers, orders e order_details)**

Explicação e Resultado

Essa consulta gera um relatório que lista o ID, o nome do cliente e a receita total gerada por 89 clientes, ajudando o setor financeiro a identificar os clientes mais lucrativos:

	customer_id [PK] character	company_name character varying (40)	receita_por_cliente double precision
1	TOMSP	Toms Spezialitäten	4954.000060558319
2	LONEP	Lonesome Pine Restaurant	4258.599994659424
3	OLDWO	Old World Delicatessen	16325.15002822876
4	WARTH	Wartian Herkku	16617.10011959076
5	MAGAA	Magazzini Alimentari Riuniti	7603.849960327148
6	QUEEN	Queen Cozinha	30226.10017967224
7	VINET	Vins et alcools Chevalier	1480.0000019073486
8	ANTON	Antonio Moreno Taquería	7515.349945068359
9	MORGK	Morgenstern Gesundkost	5042.200035095215
Total rows: 89 of 89		Query complete 00:00:00.175	

Lembre-se: na tabela **CUSTOMERS** temos 91 clientes cadastrados, porém, **2 deles nunca efetuaram compras**. Portanto, não existem pedidos registrados com os **customer_id** desses dois clientes na tabela **ORDERS**.

Como efetuamos um **INNER JOIN** entre as tabelas **CUSTOMERS** e **ORDERS**, tais clientes não apareceram no resultado, já que o **INNER JOIN** retorna somente as linhas em comum entre as tabelas. **Por isso o resultado acima retornou somente 89 linhas** (em vez de 91).

GABARITO EXPLICATIVO

4

Questão 4

O setor de atendimento ao cliente está analisando o comportamento de clientes que realizam mais de uma compra no mesmo dia. Eles querem identificar esses padrões para propor soluções que melhorem a experiência do cliente, como combos de produtos ou programas de assinatura.

Crie uma consulta que retorne o nome do cliente e a data do pedido quando o cliente tiver realizado mais de um pedido na mesma data. **(Utilize as tabelas orders e customers)**

Explicação e Resultado

Identificamos clientes que realizaram mais de um pedido no mesmo dia da seguinte forma:

Selecionamos as informações necessárias: o nome do cliente (**c.company_name**) e a data do pedido (**o.order_date**), além de contar a quantidade de pedidos realizados nessa data (**COUNT(o.order_id)**).

Usamos um **INNER JOIN**: conectamos as tabelas **ORDERS** e **CUSTOMERS** pela coluna **customer_id**, vinculando os pedidos às informações dos clientes.

Agrupamos os dados: aplicamos o **GROUP BY** ao nome do cliente (**c.company_name**) e à data do pedido (**o.order_date**), consolidando os pedidos realizados por cada cliente por data.

Filtramos os resultados com **HAVING**: mantivemos apenas os grupos em que a contagem de pedidos **(COUNT(o.order_id)) é maior que 1**, ou seja, casos em que o cliente fez mais de um pedido no mesmo dia:

```sql
SELECT
    c.company_name,
    o.order_date,
    COUNT(o.order_id) AS total_pedidos
FROM orders o
INNER JOIN customers c
    ON o.customer_id = c.customer_id
GROUP BY c.company_name, o.order_date
HAVING COUNT(o.order_id) > 1;
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

4

Questão 4

O setor de atendimento ao cliente está analisando o comportamento de clientes que realizam mais de uma compra no mesmo dia. Eles querem identificar esses padrões para propor soluções que melhorem a experiência do cliente, como combos de produtos ou programas de assinatura.

Crie uma consulta que retorne o nome do cliente e a data do pedido quando o cliente tiver realizado mais de um pedido na mesma data. **(Utilize as tabelas orders e customers)**

Explicação e Resultado

Essa consulta ajuda o setor de atendimento a identificar padrões de comportamento que podem ser utilizados para criar soluções personalizadas e melhorar a experiência do cliente:

	company_name character varying (40)	order_date date	total_pedidos bigint
1	Königlich Essen	1997-02-25	2
2	Save-a-lot Markets	1997-10-22	2
3	Bottom-Dollar Markets	1997-01-10	2
4	Save-a-lot Markets	1998-04-17	2
5	La corne d'abondance	1998-03-24	2
6	LINO-Delicateses	1998-01-19	2
7	Great Lakes Food Market	1997-07-31	2

Total rows: 7 of 7 | Query complete 00:00:00.076

GABARITO EXPLICATIVO

5

Questão 5

A equipe financeira quer priorizar negociações com fornecedores que oferecem produtos com preços médios acima de R$30. Esses fornecedores serão tratados como parceiros estratégicos para produtos de maior valor agregado, visando maximizar a margem de lucro da empresa.

Crie uma consulta que retorne o nome do fornecedor e a média de preço dos produtos fornecidos, considerando apenas os fornecedores cuja média de preço seja superior a R$30. **(Utilize as tabelas products e suppliers)**

Explicação e Resultado

Para identificar os fornecedores cujos produtos têm um preço médio acima de R$30, fazemos o seguinte:

Selecionamos os dados necessários: incluímos o nome do fornecedor (**s.company_name**) e calculamos a média de preço dos produtos (**AVG(p.unit_price)**), renomeando o resultado como **preco_medio**.

Usamos um **INNER JOIN**: conectamos as tabelas **PRODUCTS** e **SUPPLIERS** pela coluna **supplier_id**, associando os produtos aos seus respectivos fornecedores.

Agrupamos os dados por fornecedor: utilizamos **GROUP BY s.company_name** para calcular a média de preço de todos os produtos fornecidos agrupando por cada fornecedor.

Filtramos com a cláusula **HAVING**: incluímos apenas os fornecedores cuja média de preço dos produtos (**AVG(p.unit_price)**) seja maior que **R$30**:

```sql
SELECT
    s.company_name,
    AVG(p.unit_price) AS preco_medio
FROM products p
INNER JOIN suppliers s
    ON p.supplier_id = s.supplier_id
GROUP BY s.company_name
HAVING AVG(p.unit_price) > 30;
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

5

Questão 5

A equipe financeira quer priorizar negociações com fornecedores que oferecem produtos com preços médios acima de R$30. Esses fornecedores serão tratados como parceiros estratégicos para produtos de maior valor agregado, visando maximizar a margem de lucro da empresa.

Crie uma consulta que retorne o nome do fornecedor e a média de preço dos produtos fornecidos, considerando apenas os fornecedores cuja média de preço seja superior a R$30. **(Utilize as tabelas products e suppliers)**

Explicação e Resultado

Essa consulta fornece uma visão clara dos fornecedores que contribuem para produtos de maior valor agregado, ajudando a equipe a tomar decisões estratégicas de negociação:

	company_name character varying (40)	preco_medio double precision
1	Grandma Kelly's Homestead	31.666666666666668
2	Pavlova, Ltd.	35.57000045776367
3	Gai pâturage	44.5
4	Forêts d'érables	38.89999961853027
5	Plutzer Lebensmittelgroßmärkte AG	44.677999877929686
6	G'day, Mate	30.933333079020183
7	Tokyo Traders	46
8	Aux joyeux ecclésiastiques	140.75

Total rows: 8 of 8 | Query complete 00:00:00.069

GABARITO EXPLICATIVO

6

Questão 6

A gerência deseja premiar os funcionários que gerenciaram mais de 10 pedidos ao longo do ano de 1997 como forma de motivação e reconhecimento. Para isso, eles precisam de um relatório detalhado com os nomes dos colaboradores e o número total de pedidos gerenciados por cada um.

Crie uma consulta que liste os nomes completos dos funcionários e a quantidade total de pedidos que cada um gerenciou em 1997, ordenados do maior para o menor número de pedidos. Considere apenas os funcionários com mais de 10 pedidos. **(Utilize as tabelas orders e employees)**

Explicação e Resultado

Para identificar os funcionários que gerenciaram mais de 10 pedidos em 1997, fazemos o seguinte:

Selecionamos os dados necessários: o nome completo do funcionário (**e.first_name** e **e.last_name**) e o total de pedidos gerenciados (**COUNT(o.order_id)**), que é calculado e renomeado como **total_pedidos**.

Realizamos um **INNER JOIN**: conectamos as tabelas **ORDERS** e **EMPLOYEES** pela coluna **employee_id**, associando cada pedido ao funcionário responsável.

Aplicamos um filtro por ano: utilizamos **WHERE o.order_date BETWEEN '1997-01-01' AND '1997-12-31'** para considerar apenas os pedidos feitos em 1997.

Agrupamos por funcionário: utilizamos **GROUP BY e.first_name, e.last_name** para calcular a quantidade de pedidos gerenciados por cada funcionário.

Filtramos os funcionários com **mais de 10 pedidos**: adicionamos a cláusula **HAVING COUNT(o.order_id) > 10** para incluir apenas aqueles que atenderam o critério.

Ordenamos o resultado: com **ORDER BY total_pedidos DESC**, organizamos os funcionários do maior para o menor número de pedidos gerenciados.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

6

Questão 6

A gerência deseja premiar os funcionários que gerenciaram mais de 10 pedidos ao longo do ano de 1997 como forma de motivação e reconhecimento. Para isso, eles precisam de um relatório detalhado com os nomes dos colaboradores e o número total de pedidos gerenciados por cada um.

Crie uma consulta que liste os nomes completos dos funcionários e a quantidade total de pedidos que cada um gerenciou em 1997, ordenados do maior para o menor número de pedidos. Considere apenas os funcionários com mais de 10 pedidos. **(Utilize as tabelas orders e employees)**

Explicação e Resultado

Veja como fica o código:

```sql
SELECT
    e.first_name,
    e.last_name,
    COUNT(o.order_id) AS total_pedidos
FROM orders o
INNER JOIN employees e
    ON o.employee_id = e.employee_id
WHERE o.order_date BETWEEN '1997-01-01' AND '1997-12-31'
GROUP BY e.first_name, e.last_name
HAVING COUNT(o.order_id) > 10
ORDER BY total_pedidos DESC;
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

6

Questão 6

A gerência deseja premiar os funcionários que gerenciaram mais de 10 pedidos ao longo do ano de 1997 como forma de motivação e reconhecimento. Para isso, eles precisam de um relatório detalhado com os nomes dos colaboradores e o número total de pedidos gerenciados por cada um.

Crie uma consulta que liste os nomes completos dos funcionários e a quantidade total de pedidos que cada um gerenciou em 1997, ordenados do maior para o menor número de pedidos. Considere apenas os funcionários com mais de 10 pedidos. **(Utilize as tabelas orders e employees)**

Explicação e Resultado

Esse relatório ajuda a gerência a identificar e reconhecer os funcionários que se destacaram na gestão de pedidos ao longo de 1997:

	first_name character varying (10)	last_name character varying (20)	total_pedidos bigint
1	Margaret	Peacock	81
2	Janet	Leverling	71
3	Nancy	Davolio	55
4	Laura	Callahan	54
5	Andrew	Fuller	41
6	Robert	King	36
7	Michael	Suyama	33
8	Anne	Dodsworth	19
9	Steven	Buchanan	18

Total rows: 9 of 9 Query complete 00:00:00.094

GABARITO EXPLICATIVO

7

Questão 7

A equipe de logística está avaliando o desempenho das transportadoras para otimizar as operações de entrega. Eles precisam identificar quais transportadoras realizaram mais entregas nos últimos meses para manter ou renegociar contratos baseados em volume e eficiência.

Liste os nomes das transportadoras e a quantidade de pedidos entregues por cada uma. Ordene o resultado pela quantidade de pedidos em ordem decrescente. **(Utilize as tabelas orders e shippers)**

Explicação e Resultado

Para listar as transportadoras e a quantidade de pedidos entregues por cada uma, fazemos o seguinte:

Selecionamos os dados necessários: o nome da transportadora (**s.company_name**) e o total de pedidos entregues (**COUNT(o.order_id)**), renomeado como **total_pedidos**.

Realizamos um **RIGHT JOIN** entre as tabelas **ORDERS** e **SHIPPERS** por meio das colunas **ship_via** da tabela **ORDERS** e **shipper_id** da tabela **SHIPPERS (colunas estas que identificam as transportados nas tabelas)**, relacionando o pedido à transportadora.

O **RIGHT JOIN** garante que todas as transportadoras sejam listadas, mesmo que não tenham entregado pedidos, e que a contagem seja feita corretamente apenas para pedidos entregues.

Agrupamos os resultados por transportadora: utilizamos **GROUP BY s.company_name** para contar o número de pedidos entregues por cada transportadora.

Ordenamos o resultado: a cláusula **ORDER BY total_pedidos DESC** organiza os resultados pela quantidade de pedidos em ordem decrescente, permitindo que as transportadoras mais ativas sejam listadas primeiro.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

7

Questão 7

A equipe de logística está avaliando o desempenho das transportadoras para otimizar as operações de entrega. Eles precisam identificar quais transportadoras realizaram mais entregas nos últimos meses para manter ou renegociar contratos baseados em volume e eficiência.

Liste os nomes das transportadoras e a quantidade de pedidos entregues por cada uma. Ordene o resultado pela quantidade de pedidos em ordem decrescente. **(Utilize as tabelas orders e shippers)**

Explicação e Resultado

Veja como fica o código:

```
SELECT
    s.company_name,
    COUNT(o.order_id) AS total_pedidos
FROM orders o
    RIGHT JOIN shippers s ON o.ship_via = s.shipper_id
GROUP BY s.company_name
ORDER BY total_pedidos DESC;
```

Essa consulta permite à equipe de logística avaliar o desempenho das transportadoras e tomar decisões baseadas no volume de entregas realizadas:

	company_name character varying (40)	total_pedidos bigint
1	United Package	326
2	Federal Shipping	255
3	Speedy Express	249
4	UPS	0
5	Alliance Shippers	0
6	DHL	0

Total rows: 6 of 6 | Query complete 00:00:00

GABARITO EXPLICATIVO

8

Questão 8

A diretoria quer identificar quais categorias de produtos geraram o maior valor médio de vendas por produto. Esse relatório será usado para decidir em quais categorias investir mais em marketing, promoções e estoque, maximizando o retorno financeiro.

Liste os nomes das categorias e o valor médio das vendas por categoria (média de unit_price * quantity). Ordene os resultados pelo valor médio em ordem decrescente. Dica: você precisará fazer dois INNER JOINs nesta consulta. **(Utilize as tabelas order_details, products e categories)**

Explicação e Resultado

Para identificar quais categorias de produtos geraram o maior valor médio de vendas por produto, fazemos o seguinte:

Selecionamos os dados necessários: o nome da categoria (**c.category_name**) e calculamos o valor médio das vendas (**AVG(od.unit_price * od.quantity)**), renomeado como **media_vendas**.

Realizamos **dois INNER JOINs**: o **primeiro** conecta a tabela **ORDER_DETAILS** com a tabela **PRODUCTS**, relacionando os produtos com os detalhes do pedido pela coluna **product_id**. O **segundo** conecta a tabela **PRODUCTS** com a tabela **CATEGORIES**, relacionando os produtos com suas respectivas categorias pela coluna **category_id**.

Perceba que, aqui, **o primeiro JOIN é necessário**, pois, apesar de precisarmos de informações que estão na tabela CATEGORIES (não na PRODUCTS), não existe nenhuma coluna na ORDER_DETAILS que a relacione diretamente com a CATEGORIES. Mas a **PRODUCTS** se relaciona tanto com a ORDER_DETAILS quanto com a CATEGORIES, podendo, dessa forma, **servir como uma tabela intermediária (uma "ponte") entre as tabelas ORDER_DETAILS e CATEGORIES.**

Agrupamos os resultados por categoria: utilizamos **GROUP BY c.category_name** para calcular a média de vendas por categoria.

Ordenamos o resultado: a cláusula **ORDER BY media_vendas DESC** organiza os resultados pela média de vendas em ordem decrescente, permitindo que as categorias com o maior valor médio de vendas sejam listadas primeiro.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

8

Questão 8

A diretoria quer identificar quais categorias de produtos geraram o maior valor médio de vendas por produto. Esse relatório será usado para decidir em quais categorias investir mais em marketing, promoções e estoque, maximizando o retorno financeiro.

Liste os nomes das categorias e o valor médio das vendas por produto (média de unit_price * quantity). Ordene os resultados pelo valor médio em ordem decrescente. Dica: você precisará fazer dois INNER JOINs nesta consulta. **(Utilize as tabelas order_details, products e categories)**

Explicação e Resultado

Veja como fica o código:

```sql
SELECT
    c.category_name,
    AVG(od.unit_price * od.quantity) AS media_vendas
FROM order_details od
INNER JOIN products p
    ON od.product_id = p.product_id
    INNER JOIN categories c
        ON p.category_id = c.category_id
GROUP BY c.category_name
ORDER BY media_vendas DESC;
```

Essa consulta permite à diretoria visualizar quais categorias têm o maior potencial de vendas, orientando as decisões sobre onde concentrar esforços em marketing, promoções e estoque:

	category_name character varying (15)	media_vendas double precision
1	Meat/Poultry	1029.993069282157
2	Produce	774.0338248084573
3	Beverages	709.2251239991424
4	Dairy Products	686.695354633644
5	Confections	530.2368281458666
6	Condiments	526.3645818586702
7	Grains/Cereals	513.912244516976
8	Seafood	429.16087631023294
Total rows: 8 of 8		Query complete 00:00:00.087

PostgreSQL

MÓDULO 6

VIEWS

VIEWS
VIEWS

O QUE VEREMOS NESTE MÓDULO

MÓDULO 6 VIEWS NO POSTGRESQL

01	Introdução	198
02	CREATE or REPLACE View	201
03	ALTER View	205
04	DROP (IF EXISTS) View	207
05	EXERCÍCIOS	209
06	GABARITO EXPLICATIVO	214

PostgreSQL

Introdução

Até aqui vimos como criar diferentes consultas aos bancos de dados. Para isso, aprendemos a utilizar comandos como o SELECT, o GROUP BY, JOINs, etc.

Mas onde foram parar todas esses resultados que criamos? Eles estão em algum lugar?

A resposta é: eles não estão em lugar algum!

Tudo o que fizemos até agora foi apenas visualizar alguns dados das nossas tabelas do banco de dados, nada além disso. Quando executamos um SELECT e logo em seguida executamos outro SELECT, o resultado do primeiro é perdido.

Nenhuma das consultas que fizemos ficou salvo em algum lugar. Inclusive, diversas vezes precisamos criar as mesmas consultas, pois elas se perdem a cada novo SELECT, ou quando fechamos uma consulta e abrimos uma nova.

Existe uma solução para conseguirmos salvar essas *queries* em algum lugar, e esta solução é a **View**.

Introdução

- Uma View (ou, traduzindo, uma exibição), é uma tabela virtual criada a partir de uma consulta a uma ou mais tabelas (ou até mesmo a outras Views) do banco de dados.

- Ela contém linhas e colunas, assim como uma tabela real. Nela, podemos utilizar comandos como o JOIN, o WHERE, e diversas outras funções.

- As Views sempre mostram resultados atualizados dos dados, ou seja, uma vez criadas, caso haja alterações no banco de dados, elas são atualizadas automaticamente.

- Caso o servidor seja desligado (ou o pgAdmin 4 fechado), a View continua armazenada no sistema.

Através de uma View, conseguimos armazenar uma consulta e acessá-la sempre que precisar, como se fosse uma tabela, com a vantagem de não precisar recriar esse SELECT do zero.

Introdução

São muitas as vantagens de uma View. Abaixo temos algumas das principais:

Reutilização

Sempre que necessário, podemos consultar aquela View, pois ela fica armazenada no sistema.

Segurança

Ao criar uma View, estamos ocultando linhas ou colunas da tabela original do banco de dados. Desta forma, apenas algumas informações relevantes serão visualizadas na View.

Ganho de tempo

Quando criamos Views, estamos poupando o tempo de recriar vários SELECTs, o que aumenta a produtividade.

CREATE or REPLACE View

Vamos a um exemplo bem simples de como criar uma View.

Vamos supor que fizemos uma consulta às colunas product_id, product_name e unit_price da tabela products e queremos armazenar essa consulta em uma View para utilizá-la posteriormente.

Para isso, utilizamos o comando CREATE or REPLACE View:

```
CREATE OR REPLACE VIEW vwprodutos AS
SELECT
    product_id,
    product_name,
    unit_price
FROM products;
```

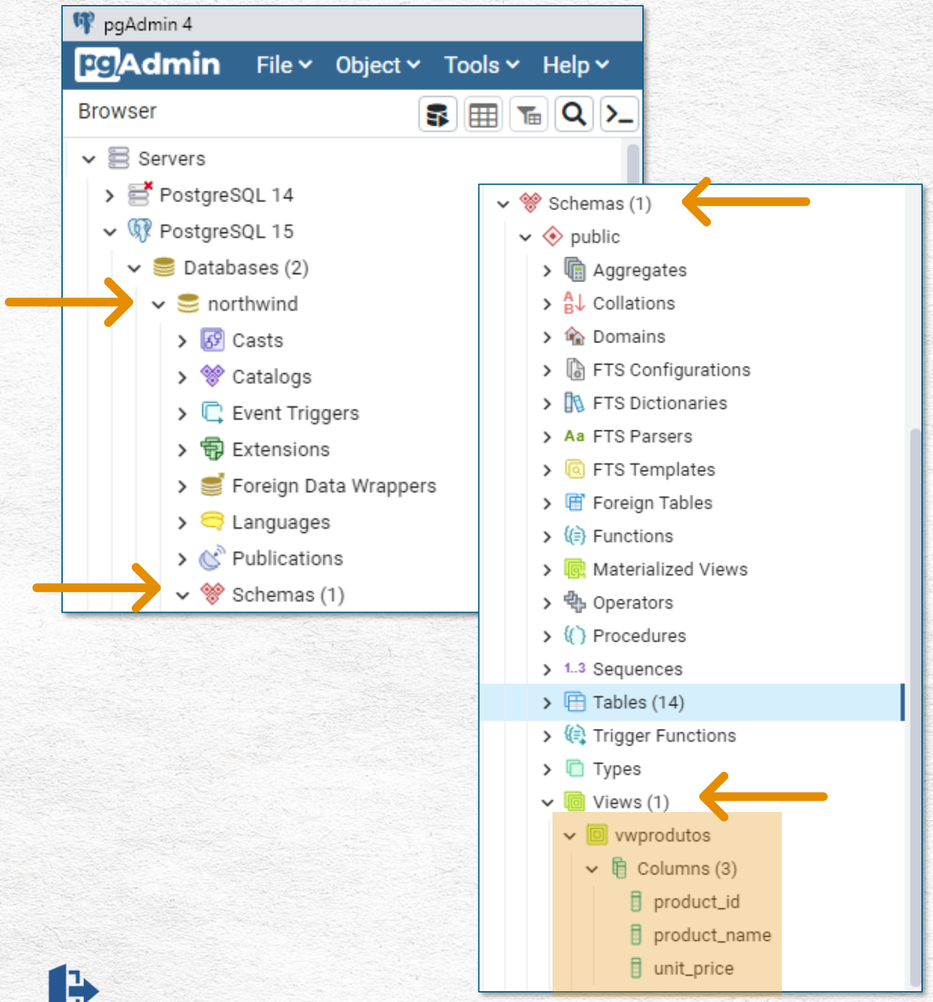
Data output | Messages | Notifications

```
CREATE VIEW

Query returned successfully in 62 msec.
```

PostgreSQL

CREATE or REPLACE View

Uma vez criada, a View ficará armazenada no banco de dados, dentro de Schemas > Views:



CREATE or REPLACE View

Para consultar uma VIEW, utilizamos o comando SELECT e informamos o nome da VIEW:

```
SELECT * FROM vwprodutos;
```

Data output | Messages | Notifications

	product_id smallint	product_name character varying (40)	unit_price real
1	1	Chai	18
2	2	Chang	19
3	3	Aniseed Syrup	10
4	4	Chef Anton's Cajun Se...	22
5	5	Chef Anton's Gumbo ...	21.35
6	6	Grandma's Boysenber...	25
7	7	Uncle Bob's Organic D...	30
8	8	Northwoods Cranberr...	40
9	9	Mishi Kobe Niku	97
10	10	Ikura	31
11	11	Queso Cabrales	21

Total rows: 77 of 77 | Query complete 00:00:00.062

CREATE or REPLACE View

Imagine que nos esquecemos de incluir a coluna units_in_stock na View, por isso precisamos alterá-la.

Para isso, utilizamos novamente o comando CREATE or REPLACE View, acrescentando a coluna desejada no nosso SELECT:

```
CREATE OR REPLACE VIEW vwprodutos AS
SELECT
    product_id,
    product_name,
    unit_price,
    units_in_stock
FROM products;
```

Data output | Messages | Notifications

```
CREATE VIEW

Query returned successfully in 73 msec.
```

Observe o resultado com a nova coluna incluída:

Data output | Messages | Notifications

	product_id smallint	product_name character varying (40)	unit_price real	units_in_stock smallint
1	1	Chai	18	39
2	2	Chang	19	17
3	3	Aniseed Syrup	10	13
4	4	Chef Anton's Cajun Seasoning	22	53
5	5	Chef Anton's Gumbo Mix	21.35	0
6	6	Grandma's Boysenberry Spread	25	120

Total rows: 77 of 77 | Query complete 00:00:00.072

Se utilizamos os dois comandos no mesmo código (CREATE e REPLACE), fica mais prático, pois, caso a VIEW ainda não exista em nosso banco de dados, ela é criada; se ela já existir, é substituída.

ALTER View

Nomeamos nossa View como “vwprodutos”. E se quisermos alterar para “vw_prod”, como fazemos?

Para isso, existe o comando ALTER VIEW:

```
ALTER VIEW vwprodutos RENAME TO vw_prod;
```

Data output | Messages | Notifications

```
ALTER VIEW

Query returned successfully in 97 msec.
```

ALTER View

Agora, quando quisermos consultá-la, devemos utilizar seu novo nome:

```
SELECT * FROM vw_prod;
```

Data output | Messages | Notifications

	product_id smallint	product_name character varying (40)	unit_price real	units_in_stock smallint
1	1	Chai	18	39
2	2	Chang	19	17
3	3	Aniseed Syrup	10	13

Total rows: 77 of 77 | Query complete 00:00:00.092

Se tentarmos utilizar seu antigo nome, será retornado um erro, pois o sistema entende que aquela View não existe mais:

```
SELECT * FROM vwprodutos;
```

Data output | Messages | Notifications

```
ERROR:  relation "vwprodutos" does not exist
LINE 1: SELECT * FROM vwprodutos;
                      ^

SQL state: 42P01
Character: 15
```

DROP (IF EXISTS) View

E se quisermos excluir a View, como devemos proceder?

Para isso, existem os comandos DROP IF EXISTS VIEW e DROP VIEW.

O DROP IF EXISTS VIEW primeiro verifica se a View existe: se existir, ele procede à exclusão:

```
DROP VIEW IF EXISTS vw_prod;
```

Data output | Messages | Notifications

```
DROP VIEW

Query returned successfully in 78 msec.
```

Se ele verificar que a View não existe, retorna uma mensagem de alerta na tela, mas o sistema permanece em execução:

```
DROP VIEW IF EXISTS vw_prod;
```

Data output | Messages | Notifications

```
NOTICE:  view "vw_prod" does not exist, skipping
DROP VIEW

Query returned successfully in 46 msec.
```

DROP (IF EXISTS) View

Neste ponto, o comando DROP IF EXISTS VIEW acaba sendo melhor que o comando DROP VIEW, pois este último não verifica antes se a View existe:

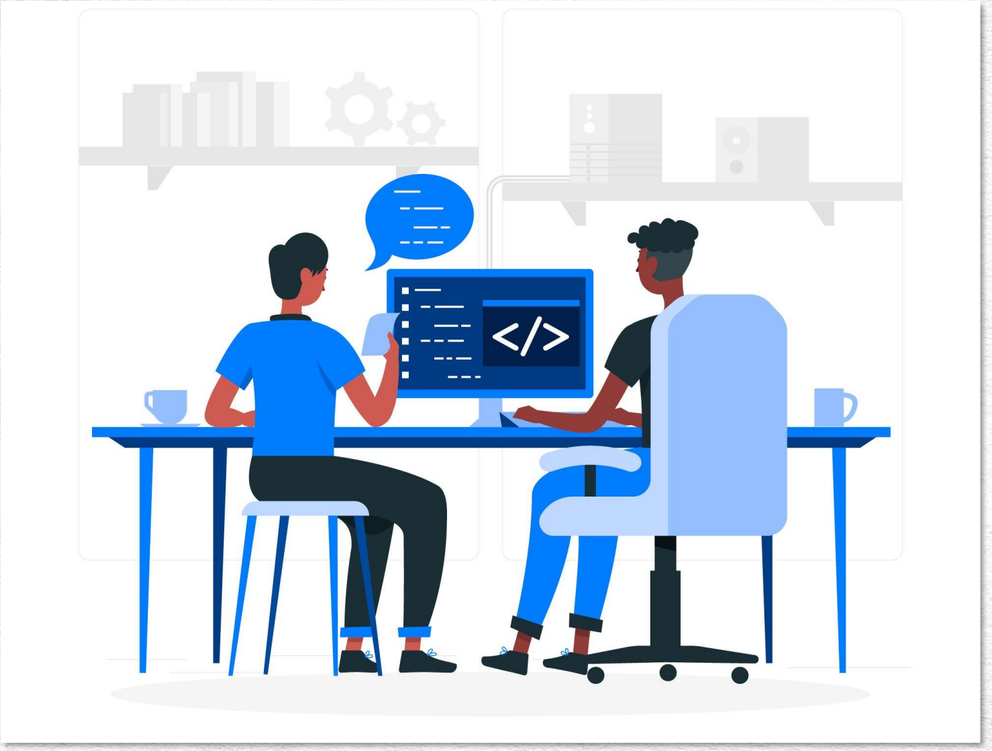
- Se a View existir, ele a exclui;
- Caso a View não exista, é retornado um erro na tela e o sistema é pausado, o que pode ser prejudicial para bancos de dados funcionando em tempo real:

```
DROP VIEW vwprodutos;
```

Data output | Messages | Notifications

```
ERROR:  view "vwprodutos" does not exist
SQL state: 42P01
```

EXERCÍCIOS



1

Questão 1

A equipe de vendas frequentemente precisa entrar em contato com os clientes para discutir propostas ou resolver problemas relacionados a pedidos. Para facilitar esse processo, é necessário ter acesso rápido às informações de contato de cada cliente.

Crie uma view chamada vw_clientes_contato que retorne o ID, o nome da empresa, o nome do contato, a função do contato e o telefone de todos os clientes. **(Utilize a tabela customers)**

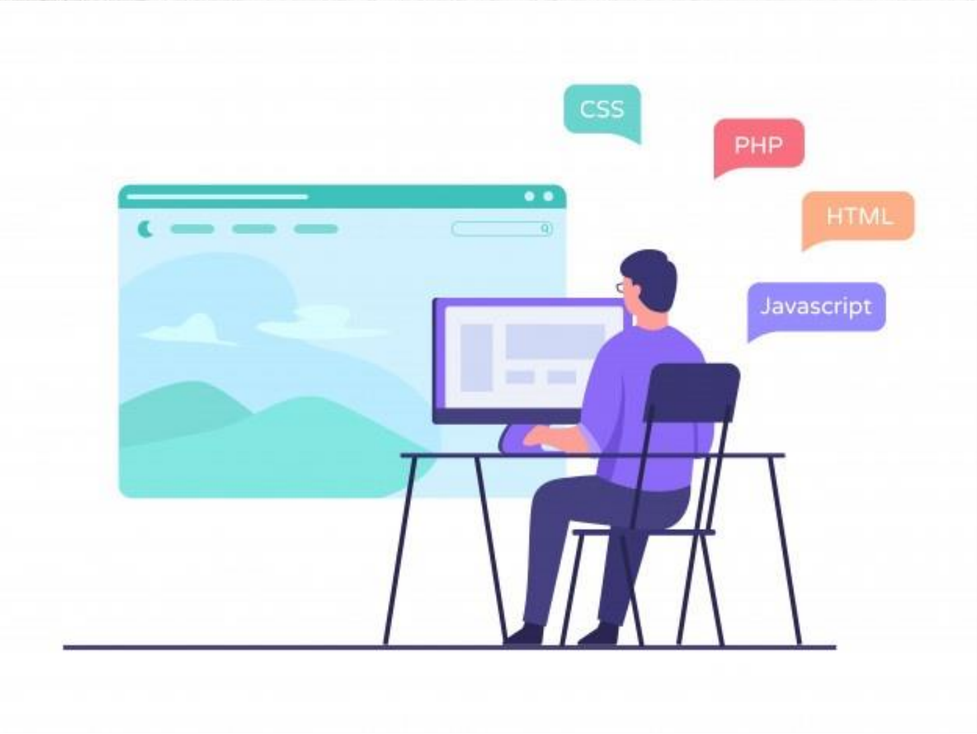
2

Questão 2

O setor de logística tem enfrentado desafios para equilibrar os níveis de estoque das diferentes categorias de produtos. Para otimizar a reposição e evitar excessos ou faltas, eles precisam de uma visão consolidada que relacione cada categoria à quantidade total de itens em estoque.

Crie uma view chamada vw_estoque_por_categoria para atender a essa necessidade. **(Utilize as tabelas categories e products)**

EXERCÍCIOS



3

Questão 3

O gerente comercial está focado em entender o desempenho do mercado na América do Norte (México, EUA e Canadá). Isso inclui monitorar os pedidos realizados por clientes dessa região, identificando padrões de consumo e áreas de potencial crescimento.

Crie uma view chamada vw_pedidos_america_norte para retornar o ID e a data do pedido, o nome e o país do cliente, considerando-se somente os países da América do Norte. **(Utilize as tabelas orders e customers)**

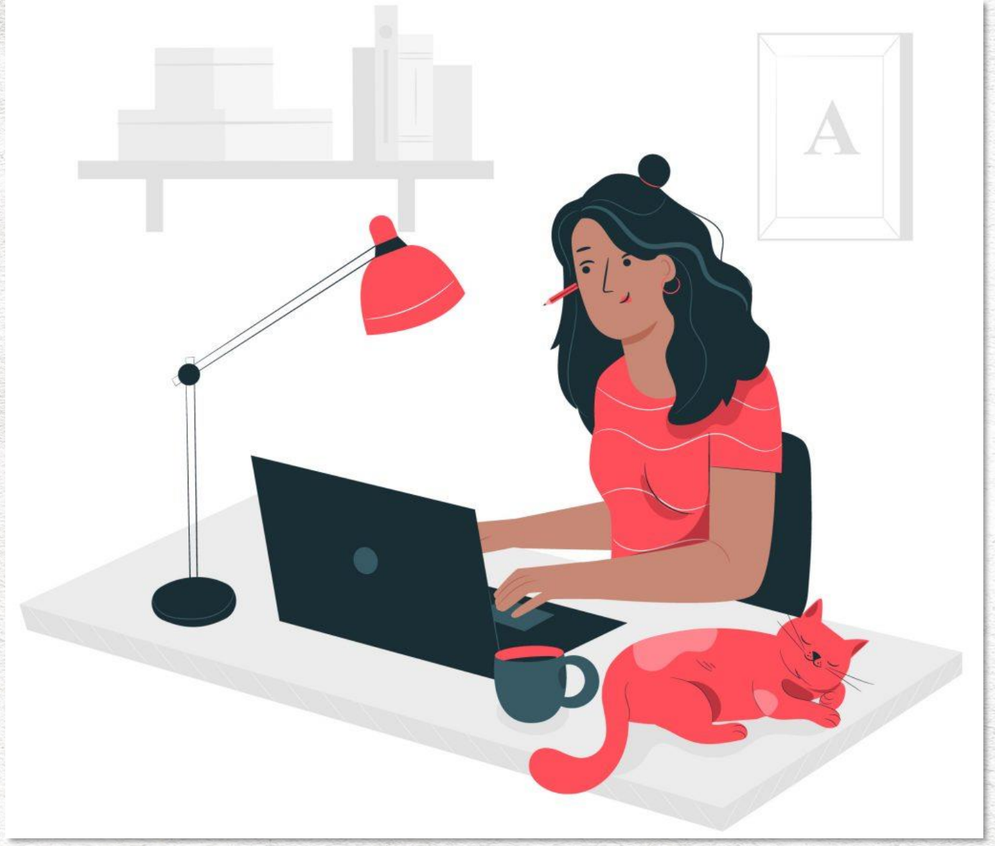
4

Questão 4

O gerente deseja ter acesso rápido aos pedidos mais recentes para acompanhar o fluxo de vendas e identificar tendências ou possíveis gargalos no processo de pedidos.

Crie uma view chamada vw_ultimos_pedidos para atender a esse pedido. **(Utilize a tabela orders)**

EXERCÍCIOS



5

Questão 5

A equipe de vendas está priorizando clientes que geram alto volume de receita. Por isso, é essencial identificar aqueles cujos pedidos somam uma receita total acima de R$30.000, para oferecer um atendimento mais dedicado e estratégico.

Crie uma view chamada vw_clientes_receita_total para exibir esses clientes e suas respectivas receitas totais (soma de unit_price * quantity). **(Utilize as tabelas customers, orders e orders_details)**

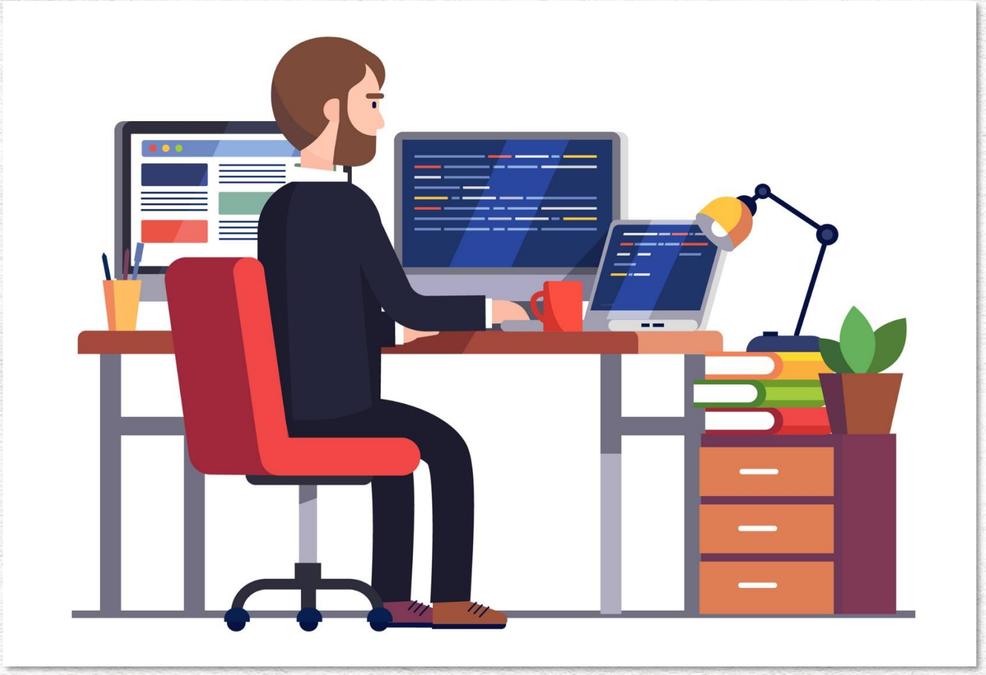
6

Questão 6

O setor de análise está conduzindo uma pesquisa para entender quais categorias de produtos têm maior representatividade no portfólio. Essa informação ajudará a direcionar esforços de marketing e otimizar o mix de produtos oferecidos.

Crie uma view chamada vw_top5_categorias para exibir as cinco categorias com maior quantidade de produtos. **(Utilize as tabelas categories e products)**

EXERCÍCIOS



7a

Questão 7-a

A gerência quer identificar os mercados mais lucrativos ao redor do mundo. Para isso, é necessário calcular a média de vendas por país, permitindo que os recursos sejam direcionados para os mercados com maior retorno financeiro.

Crie uma view chamada vw_media_vendas_dos_paises que mostre os nomes dos países e suas respectivas medias de vendas (média de quantity * unit_price). **(Utilize as tabelas orders e order_details)**

7b

Questão 7-b

Para facilitar a comunicação entre os setores, o nome da view criada no item A dever ser ajustado para um padrão que reflete melhor seu propósito: vw_media_vendas_por_pais.

Faça essa alteração no banco de dados. **(Não é necessário utilizar tabelas)**

EXERCÍCIOS

8

Questão 8

A diretoria quer analisar os pedidos realizados no ano de 1998 para identificar tendências históricas e avaliar o desempenho da empresa em períodos passados, o que pode ajudar na projeção de vendas futuras.

Crie uma view chamada vw_pedidos_1998 para listar esses pedidos. **(Utilize a tabela orders)**

9

Questão 9

Exclua todas as views criadas nesta lista de exercícios:

vw_clientes_contato

vw_estoque_por_categoria

vw_pedidos_america_norte

vw_ultimos_pedidos

vw_clientes_receita_total

vw_top5_categorias

vw_media_vendas_por_pais

vw_pedidos_1998

(Não é necessário utilizar tabelas)

GABARITO EXPLICATIVO

1

Questão 1

A equipe de vendas frequentemente precisa entrar em contato com os clientes para discutir propostas ou resolver problemas relacionados a pedidos. Para facilitar esse processo, é necessário ter acesso rápido às informações de contato de cada cliente.

Crie uma view chamada vw_clientes_contato que retorne o ID, o nome da empresa, o nome do contato, a função do contato e o telefone de todos os clientes. **(Utilize a tabela customers)**

Explicação e Resultado

Para **criar uma View** com o ID, o nome da empresa, o nome do contato, a função do contato e o telefone de todos os clientes, seguimos os passos abaixo:

Utilizamos o comando **CREATE OR REPLACE VIEW** para criar uma nova View chamada **vw_clientes_contato**.

No comando **SELECT** dentro da definição da View, selecionamos as colunas **customer_id, company_name, contact_name, contact_title** e **phone** da tabela **CUSTOMERS**.

Executamos o código para criar a View no banco de dados:

```
CREATE OR REPLACE VIEW vw_clientes_contato AS
SELECT
    customer_id,
    company_name,
    contact_name,
    contact_title,
    phone
FROM customers;
```

Data Output | Messages | Notifications

```
CREATE VIEW

Query returned successfully in 74 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

1

Questão 1

A equipe de vendas frequentemente precisa entrar em contato com os clientes para discutir propostas ou resolver problemas relacionados a pedidos. Para facilitar esse processo, é necessário ter acesso rápido às informações de contato de cada cliente.

Crie uma view chamada vw_clientes_contato que retorne o ID, o nome da empresa, o nome do contato, a função do contato e o telefone de todos os clientes. **(Utilize a tabela customers)**

Explicação e Resultado

Após criar a View, utilizamos o comando **SELECT * FROM vw_clientes_contato** para verificar o conteúdo da View:

```
SELECT * FROM vw_clientes_contato;
```

Dessa forma, retornamos o resultado abaixo, com as informações dos 91 clientes cadastrados:

	customer_id character	company_name character varying (40)	contact_name character varying (30)	contact_title character varying (30)	phone character varying (
1	ALFKI	Alfreds Futterkiste	Maria Anders	Sales Representative	030-0074321
2	ANATR	Ana Trujillo Emparedados ...	Ana Trujillo	Owner	(5) 555-4729
3	ANTON	Antonio Moreno Taquería	Antonio Moreno	Owner	(5) 555-3932
4	AROUT	Around the Horn	Thomas Hardy	Sales Representative	(171) 555-7788
5	BERGS	Berglunds snabbköp	Christina Berglund	Order Administrator	0921-12 34 65

Total rows: 91 of 91 Query complete 00:00:00.206

GABARITO EXPLICATIVO

2

Questão 2

O setor de logística tem enfrentado desafios para equilibrar os níveis de estoque das diferentes categorias de produtos. Para otimizar a reposição e evitar excessos ou faltas, eles precisam de uma visão consolidada que relacione cada categoria à quantidade total de itens em estoque.

Crie uma view chamada vw_estoque_por_categoria para atender a essa necessidade. **(Utilize as tabelas categories e products)**

Explicação e Resultado

Para **criar uma View** que mostre a relação entre cada categoria e a quantidade total de itens em estoque, seguimos os seguintes passos:

Utilizamos o comando **CREATE OR REPLACE VIEW** para criar a View chamada **vw_estoque_por_categoria**.

Dentro da definição da View, fazemos um **INNER JOIN** entre as tabelas **CATEGORIES** e **PRODUCTS**, relacionando-as pela coluna **category_id**.

Selecionamos a coluna **category_name** da tabela **CATEGORIES** e utilizamos a função agregada **SUM(p.units_in_stock)** para somar a quantidade total de itens em estoque por categoria.

Agrupamos os resultados pela coluna **category_name** com o comando **GROUP BY**.

Executamos o código para criar a View no banco de dados:

```
CREATE OR REPLACE VIEW vw_estoque_por_categoria AS
SELECT
    c.category_name,
    SUM(p.units_in_stock) AS total_em_estoque
FROM categories c
INNER JOIN products p
    ON c.category_id = p.category_id
GROUP BY c.category_name;
```

Data Output | Messages | Notifications

```
CREATE VIEW

Query returned successfully in 109 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

2

Questão 2

O setor de logística tem enfrentado desafios para equilibrar os níveis de estoque das diferentes categorias de produtos. Para otimizar a reposição e evitar excessos ou faltas, eles precisam de uma visão consolidada que relacione cada categoria à quantidade total de itens em estoque.

Crie uma view chamada vw_estoque_por_categoria para atender a essa necessidade. **(Utilize as tabelas categories e products)**

Explicação e Resultado

Após criar a View, utilizamos o comando **SELECT * FROM vw_estoque_por_categoria** para verificar o conteúdo da View:

```
SELECT * FROM vw_estoque_por_categoria;
```

Dessa forma, retornamos o resultado abaixo, com as informações de estoque por categoria:

	category_name character varying (15)	total_em_estoque bigint
1	Beverages	559
2	Produce	100
3	Condiments	507
4	Grains/Cereals	308
5	Meat/Poultry	165
6	Confections	386
7	Dairy Products	393
8	Seafood	701
Total rows: 8 of 8	Query complete 00:00:00.13	

GABARITO EXPLICATIVO

3

Questão 3

O gerente comercial está focado em entender o desempenho do mercado na América do Norte (México, EUA e Canadá). Isso inclui monitorar os pedidos realizados por clientes dessa região, identificando padrões de consumo e áreas de potencial crescimento.

Crie uma view chamada vw_pedidos_america_norte para retornar o ID e a data do pedido, o nome e o país do cliente, considerando-se somente os países da América do Norte. **(Utilize as tabelas orders e customers)**

Explicação e Resultado

Para **criar uma View** que retorna o ID e a data do pedido, o nome e o país do cliente, considerando somente os países da América do Norte, seguimos os seguintes passos:

Utilizamos o comando **CREATE OR REPLACE VIEW** para criar a View chamada **vw_pedidos_america_norte**.

Fazemos um INNER JOIN entre as tabelas **ORDERS** e **CUSTOMERS** relacionando-as pela coluna **customer_id**.

Utilizamos a cláusula **WHERE** para filtrar os países desejados: **México, EUA e Canadá**.

Executamos o código para criar a View no banco de dados:

```sql
CREATE OR REPLACE VIEW vw_pedidos_america_norte AS
SELECT
    o.order_id,
    o.order_date,
    c.company_name,
    c.country
FROM orders o
INNER JOIN customers c
    ON o.customer_id = c.customer_id
WHERE c.country IN ('Mexico', 'USA', 'Canada');
```

Data Output | Messages | Notifications

```
CREATE VIEW

Query returned successfully in 72 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

3

Questão 3

O gerente comercial está focado em entender o desempenho do mercado na América do Norte (México, EUA e Canadá). Isso inclui monitorar os pedidos realizados por clientes dessa região, identificando padrões de consumo e áreas de potencial crescimento.

Crie uma view chamada vw_pedidos_america_norte para retornar o ID e a data do pedido, o nome e o país do cliente, considerando-se somente os países da América do Norte. **(Utilize as tabelas orders e customers)**

Explicação e Resultado

Após criar a View, utilizamos o comando **SELECT * FROM vw_pedidos_america_norte** para verificar o conteúdo da View:

```
SELECT * FROM vw_pedidos_america_norte;
```

Dessa forma, retornamos o resultado abaixo, com os 180 pedidos da América do Norte:

	order_id smallint	order_date date	company_name character varying (40)	country character var
1	10259	1996-07-18	Centro comercial Moctezuma	Mexico
2	10262	1996-07-22	Rattlesnake Canyon Grocery	USA
3	10269	1996-07-31	White Clover Markets	USA
4	10271	1996-08-01	Split Rail Beer & Ale	USA
5	10272	1996-08-02	Rattlesnake Canyon Grocery	USA
6	10276	1996-08-08	Tortuga Restaurante	Mexico
7	10293	1996-08-29	Tortuga Restaurante	Mexico
8	10294	1996-08-30	Rattlesnake Canyon Grocery	USA

Total rows: 180 of 180 Query complete 00:00:00.085

GABARITO EXPLICATIVO

4

Questão 4

O gerente deseja ter acesso rápido aos pedidos mais recentes para acompanhar o fluxo de vendas e identificar tendências ou possíveis gargalos no processo de pedidos.

Crie uma view chamada vw_ultimos_pedidos para atender a esse pedido. **(Utilize a tabela orders)**

Explicação e Resultado

Para **criar uma View** que retorna os 5 pedidos mais recentes, seguimos os seguintes passos:

Utilizamos o comando **CREATE OR REPLACE VIEW** para criar a View chamada **vw_ultimos_pedidos**.

Selecionamos todos os dados da tabela **ORDERS** e ordenamos pelo campo **order_date** de forma decrescente, utilizando **ORDER BY order_date DESC**.

Limitamos o resultado aos 5 pedidos mais recentes com o comando **LIMIT 5**.

Executamos o código para criar a View no banco de dados:

```sql
CREATE OR REPLACE VIEW vw_ultimos_pedidos AS
SELECT
    *
FROM orders
ORDER BY order_date DESC
LIMIT 5;
```

Data Output | Messages | Notifications

```
CREATE VIEW

Query returned successfully in 118 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

4

Questão 4

O gerente deseja ter acesso rápido aos pedidos mais recentes para acompanhar o fluxo de vendas e identificar tendências ou possíveis gargalos no processo de pedidos.

Crie uma view chamada vw_ultimos_pedidos para atender a esse pedido. **(Utilize a tabela orders)**

Explicação e Resultado

Após criar a View, utilizamos o comando **SELECT * FROM vw_ultimos_pedidos** para verificar o conteúdo da View:

```
SELECT * FROM vw_ultimos_pedidos;
```

Dessa forma, retornamos o resultado abaixo, com os 5 pedidos mais recentes:

	order_id smallint	customer_id character	employee_id smallint	order_date date	required_date date	shipped_date date	ship_via smallint	freight real	ship_name character varying (40)	ship_address character varying (60)	ship_city character varyin	ship_region character var	ship_postal_code character varying (1	ship_country character varyin
1	11076	BONAP	4	1998-05-06	1998-06-03	[null]	2	38.28	Bon app'	12, rue des Bouchers	Marseille	[null]	13008	France
2	11077	RATTC	1	1998-05-06	1998-06-03	[null]	2	8.53	Rattlesnake Canyon Grocery	2817 Milton Dr.	Albuquerque	NM	87110	USA
3	11074	SIMOB	7	1998-05-06	1998-06-03	[null]	2	18.44	Simons bistro	Vinbæltet 34	Kobenhavn	[null]	1734	Denmark
4	11075	RICSU	8	1998-05-06	1998-06-03	[null]	2	6.19	Richter Supermarkt	Starenweg 5	Genève	[null]	1204	Switzerland
5	11073	PERIC	2	1998-05-05	1998-06-02	[null]	2	24.95	Pericles Comidas clásicas	Calle Dr. Jorge Cash 321	México D.F.	[null]	05033	Mexico

Total rows: 5 of 5 Query complete 00:00:00.165

GABARITO EXPLICATIVO

5

Questão 5

A equipe de vendas está priorizando clientes que geram alto volume de receita. Por isso, é essencial identificar aqueles cujos pedidos somam uma receita total acima de R$30.000, para oferecer um atendimento mais dedicado e estratégico.

Crie uma view chamada vw_clientes_receita_total para exibir esses clientes e suas respectivas receitas totais (soma de unit_price * quantity). **(Utilize as tabelas customers, orders e orders_details)**

Explicação e Resultado

Para **criar uma View** que retorna os clientes que geram receitas totais acima de R$30.000, seguimos os seguintes passos:

Utilizamos o comando **CREATE OR REPLACE VIEW** para criar a View chamada **vw_clientes_receita_total**.

Fazemos **dois INNER JOINs**: **um** entre as tabelas **CUSTOMERS** e **ORDERS**, relacionando-as pela coluna **customer_id**; **outro** entre as tabelas **ORDERS** e **ORDER_DETAILS**, relacionando-as pela coluna **order_id**.

Utilizamos a função **SUM(od.unit_price * od.quantity)** para calcular a **receita_total** de cada cliente, **agrupando** pelas colunas **customer_id** e **company_name**, aplicando, em seguida, a cláusula **HAVING** para filtrar os clientes cuja **receita gerada seja superior a R$30.000**.

Executamos o código para criar a View no banco de dados:

```sql
CREATE OR REPLACE VIEW vw_clientes_receita_total AS
SELECT
    c.customer_id,
    c.company_name,
    SUM(od.unit_price * od.quantity) AS receita_total
FROM customers c
INNER JOIN orders o
    ON c.customer_id = o.customer_id
INNER JOIN order_details od
    ON o.order_id = od.order_id
GROUP BY c.customer_id, c.company_name
HAVING SUM(od.unit_price * od.quantity) > 30000;
```

Data Output | Messages | Notifications

```
CREATE VIEW

Query returned successfully in 89 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

5

Questão 5

A equipe de vendas está priorizando clientes que geram alto volume de receita. Por isso, é essencial identificar aqueles cujos pedidos somam uma receita total acima de R$30.000, para oferecer um atendimento mais dedicado e estratégico.

Crie uma view chamada vw_clientes_receita_total para exibir esses clientes e suas respectivas receitas totais (soma de unit_price * quantity). **(Utilize as tabelas customers, orders e orders_details)**

Explicação e Resultado

Após criar a View, utilizamos o comando **SELECT * FROM vw_clientes_receita_total** para verificar o conteúdo da View:

```
SELECT * FROM vw_clientes_receita_total;
```

Dessa forma, retornamos o resultado abaixo, com os clientes e suas respectivas receitas totais:

	customer_id character	company_name character varying (40)	receita_total double precision
1	QUEEN	Queen Cozinha	30226.10017967224
2	RATTC	Rattlesnake Canyon Grocery	52245.90034675598
3	QUICK	QUICK-Stop	117483.390147686
4	HUNGO	Hungry Owl All-Night Grocers	57317.39016246796
5	KOENE	Königlich Essen	31745.749893188477
6	MEREP	Mère Paillarde	32203.900234222412
7	HANAR	Hanari Carnes	34101.149973869324
8	FOLKO	Folk och fä HB	32555.55001926422
9	SAVEA	Save-a-lot Markets	115673.38964271545
10	ERNSH	Ernst Handel	113236.67978191376

Total rows: 10 of 10 | Query complete 00:00:00.094

GABARITO EXPLICATIVO

6

Questão 6

O setor de análise está conduzindo uma pesquisa para entender quais categorias de produtos têm maior representatividade no portfólio. Essa informação ajudará a direcionar esforços de marketing e otimizar o mix de produtos oferecidos.

Crie uma view chamada vw_top5_categorias para exibir as cinco categorias com maior quantidade de produtos. **(Utilize as tabelas categories e products)**

Explicação e Resultado

Para **criar uma View** que retorna as 5 categorias com maior quantidade de produtos, seguimos os seguintes passos:

Utilizamos o comando **CREATE OR REPLACE VIEW** para criar a View chamada **vw_top5_categorias**.

Fazemos um **INNER JOIN** entre as tabelas **CATEGORIES** e **PRODUCTS**, relacionando-as pela coluna **category_id**.

Utilizamos a função **COUNT(p.product_id)** para contar o número de produtos por categoria (**GROUP BY c.category_name**) e ordenamos os resultados pela quantidade de produtos de forma decrescente, utilizando **ORDER BY total_produtos DESC**.

Limitamos o resultado às 5 categorias com mais produtos com o comando **LIMIT 5**.

Executamos o código para criar a View no banco de dados:

```sql
CREATE OR REPLACE VIEW vw_top5_categorias AS
SELECT
    c.category_name,
    COUNT(p.product_id) AS total_produtos
FROM categories c
INNER JOIN products p
    ON c.category_id = p.category_id
GROUP BY c.category_name
ORDER BY total_produtos DESC
LIMIT 5;
```

Data Output | Messages | Notifications

```
CREATE VIEW

Query returned successfully in 78 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

6

Questão 6

O setor de análise está conduzindo uma pesquisa para entender quais categorias de produtos têm maior representatividade no portfólio. Essa informação ajudará a direcionar esforços de marketing e otimizar o mix de produtos oferecidos.

Crie uma view chamada vw_top5_categorias para exibir as cinco categorias com maior quantidade de produtos. **(Utilize as tabelas categories e products)**

Explicação e Resultado

Após criar a View, utilizamos o comando **SELECT * FROM vw_top5_categorias** para verificar o conteúdo da View:

```
SELECT * FROM vw_top5_categorias;
```

Dessa forma, retornamos o resultado abaixo, com as 5 categorias com maior quantidade de produtos:

	category_name character varying (15)	total_produtos bigint
1	Confections	13
2	Condiments	12
3	Beverages	12
4	Seafood	12
5	Dairy Products	10

Total rows: 5 of 5 | Query complete 00:00:00.

GABARITO EXPLICATIVO

7a

Questão 7-a

A gerência quer identificar os mercados mais lucrativos ao redor do mundo. Para isso, é necessário calcular a média de vendas por país, permitindo que os recursos sejam direcionados para os mercados com maior retorno financeiro.

Crie uma view chamada vw_media_vendas_dos_paises que mostre os nomes dos países e suas respectivas medias de vendas (média de quantity * unit_price). **(Utilize as tabelas orders e order_details)**

Explicação e Resultado

Para **criar uma View** que retorna a média de vendas por país, seguimos os seguintes passos:

Utilizamos o comando **CREATE OR REPLACE VIEW** para criar a View chamada **vw_media_vendas_dos_paises**.

Fazemos um **INNER JOIN** entre as tabelas **ORDERS** e **ORDER_DETAILS**, relacionando-as pela coluna **order_id**.

Utilizamos a função **AVG(od.quantity * od.unit_price)** para calcular a **media_vendas** por país, agrupando os resultados por país com **GROUP BY o.ship_country**.

Ordenamos os resultados pela média de vendas de forma decrescente com **ORDER BY media_vendas DESC**.

Executamos o código para criar a View no banco de dados:

```sql
CREATE OR REPLACE VIEW vw_media_vendas_dos_paises AS
SELECT
    o.ship_country,
    AVG(od.quantity * od.unit_price) AS media_vendas
FROM orders o
INNER JOIN order_details od
    ON o.order_id = od.order_id
GROUP BY o.ship_country
ORDER BY media_vendas DESC;
```

Data Output | Messages | Notifications

```
CREATE VIEW

Query returned successfully in 76 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

7a

Questão 7-a

A gerência quer identificar os mercados mais lucrativos ao redor do mundo. Para isso, é necessário calcular a média de vendas por país, permitindo que os recursos sejam direcionados para os mercados com maior retorno financeiro.

Crie uma view chamada vw_media_vendas_dos_paises que mostre os nomes dos países e suas respectivas medias de vendas (média de quantity * unit_price). **(Utilize as tabelas orders e order_details)**

Explicação e Resultado

Após criar a View, utilizamos o comando **SELECT * FROM vw_media_vendas_dos_paises** para verificar o conteúdo da View:

```
SELECT * FROM vw_media_vendas_dos_paises;
```

Dessa forma, retornamos o resultado abaixo, com as médias de vendas por país dos 21 países cadastrados:

	ship_country character varying (15)	media_vendas double precision
1	Austria	1115.9730389328004
2	Ireland	1042.1343665903264
3	Denmark	756.1358697207078
4	USA	748.7698295929215
5	Germany	745.8555791726926
6	Canada	737.7880024973551
7	Switzerland	633.067307646458
8	Belgium	627.4103628737586

Total rows: 21 of 21 | Query complete 00:00:00.07

GABARITO EXPLICATIVO

7b

Questão 7-b

Para facilitar a comunicação entre os setores, o nome da view criada no item A dever ser ajustado para um padrão que reflete melhor seu propósito: vw_media_vendas_por_pais.

Faça essa alteração no banco de dados. **(Não é necessário utilizar tabelas)**

Explicação e Resultado

Para **alterar o nome da View** criada no exercício anterior, seguimos os seguintes passos:

Utilizamos o comando **ALTER VIEW** para renomear a View de vw_media_vendas_dos_paises para **vw_media_vendas_por_pais**.

Executamos o código para realizar a alteração:

```
ALTER VIEW vw_media_vendas_dos_paises
RENAME TO vw_media_vendas_por_pais;
```

Data Output | Messages | Notifications

```
ALTER VIEW

Query returned successfully in 56 msec.
```

Após a alteração, utilizamos o comando **SELECT * FROM vw_media_vendas_por_pais** para verificar o conteúdo da View com o novo nome:

```
SELECT * FROM vw_media_vendas_por_pais;
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

7b

Questão 7-b

Para facilitar a comunicação entre os setores, o nome da view criada no item A dever ser ajustado para um padrão que reflete melhor seu propósito: vw_media_vendas_por_pais.

Faça essa alteração no banco de dados. **(Não é necessário utilizar tabelas)**

Explicação e Resultado

Dessa forma, verificamos que a View foi renomeada com sucesso e retornou o resultado esperado (as médias de vendas por país dos 21 países cadastrados):

	ship_country character varying (15)	media_vendas double precision
1	Austria	1115.9730389328004
2	Ireland	1042.1343665903264
3	Denmark	756.1358697207078
4	USA	748.7698295929215
5	Germany	745.8555791726926
6	Canada	737.7880024973551
7	Switzerland	633.067307646458
8	Belgium	627.4103628737586

Total rows: 21 of 21 | Query complete 00:00:00.07

GABARITO EXPLICATIVO

8

Questão 8

A diretoria quer analisar os pedidos realizados no ano de 1998 para identificar tendências históricas e avaliar o desempenho da empresa em períodos passados, o que pode ajudar na projeção de vendas futuras.

Crie uma view chamada vw_pedidos_1998 para listar esses pedidos. **(Utilize a tabela orders)**

Explicação e Resultado

Para **criar uma View** que retorna os pedidos realizados no ano de 1998, seguimos os seguintes passos:

Utilizamos o comando **CREATE OR REPLACE VIEW** para criar a View chamada **vw_pedidos_1998**.

Selecionamos os campos **order_id**, **customer_id** e **order_date** da tabela **ORDERS** e aplicamos a cláusula **WHERE** para filtrar os pedidos realizados entre **1º de janeiro e 31 de dezembro de 1998**.

Executamos o código para criar a View no banco de dados:

```sql
CREATE OR REPLACE VIEW vw_pedidos_1998 AS
SELECT
    order_id,
    customer_id,
    order_date
FROM orders
WHERE order_date BETWEEN '1998-01-01' AND '1998-12-31';
```

Data Output | Messages | Notifications

```
CREATE VIEW

Query returned successfully in 65 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

8

Questão 8

A diretoria quer analisar os pedidos realizados no ano de 1998 para identificar tendências históricas e avaliar o desempenho da empresa em períodos passados, o que pode ajudar na projeção de vendas futuras.

Crie uma view chamada vw_pedidos_1998 para listar esses pedidos. **(Utilize a tabela orders)**

Explicação e Resultado

Após criar a View, utilizamos o comando **SELECT * FROM vw_pedidos_1998** para verificar o conteúdo da View:

```
SELECT * FROM vw_pedidos_1998;
```

Dessa forma, retornamos o resultado abaixo, com os 270 pedidos de 1998:

	order_id smallint	customer_id character	order_date date
1	10808	OLDWO	1998-01-01
2	10809	WELLI	1998-01-01
3	10810	LAUGB	1998-01-01
4	10811	LINOD	1998-01-02
5	10812	REGGC	1998-01-02
6	10813	RICAR	1998-01-05
7	10814	VICTE	1998-01-05
8	10815	SAVEA	1998-01-05

Total rows: 270 of 270 Query complete 00:00

GABARITO EXPLICATIVO

9

Questão 9

Exclua todas as views criadas nesta lista de exercícios:

vw_clientes_contato

vw_estoque_por_categoria

vw_pedidos_america_norte

vw_ultimos_pedidos

vw_clientes_receita_total

vw_top5_categorias

vw_media_vendas_por_pais

vw_pedidos_1998

(Não é necessário utilizar tabelas)

Explicação e Resultado

Para **excluir todas as views criadas**, utilizamos o comando **DROP VIEW IF EXISTS** para cada uma das views. Executando cada código, veremos que todas as views são excluídas com sucesso:

```
DROP VIEW IF EXISTS vw_clientes_contato;
```

```
DROP VIEW

Query returned successfully in 68 msec.
```

```
DROP VIEW IF EXISTS vw_estoque_por_categoria;
```

```
DROP VIEW

Query returned successfully in 75 msec.
```

```
DROP VIEW IF EXISTS vw_pedidos_america_norte;
```

```
DROP VIEW

Query returned successfully in 67 msec.
```

```
DROP VIEW IF EXISTS vw_ultimos_pedidos;
```

```
DROP VIEW

Query returned successfully in 71 msec.
```

```
DROP VIEW IF EXISTS vw_clientes_receita_total;
```

```
DROP VIEW

Query returned successfully in 63 msec.
```

```
DROP VIEW IF EXISTS vw_top5_categorias;
```

```
DROP VIEW

Query returned successfully in 83 msec.
```

```
DROP VIEW IF EXISTS vw_media_vendas_por_pais;
```

```
DROP VIEW

Query returned successfully in 66 msec.
```

```
DROP VIEW IF EXISTS vw_pedidos_1998;
```

```
DROP VIEW

Query returned successfully in 72 msec.
```

MÓDULO 7

CRUD
CRUD
CRUD

O QUE VEREMOS NESTE MÓDULO

MÓDULO 7 OPERAÇÕES CRUD NO POSTGRESQL

01	Introdução, CREATE e DROP Database	235
02	Tipos de Dados, CREATE e DROP Table	246
03	CONSTRAINTS (Restrições)	251
04	INSERT INTO	256
05	UPDATE	259
06	DELETE	260
07	TRUNCATE	261
08	DROP (Cascade)	262
09	EXERCÍCIOS	265
10	GABARITO EXPLICATIVO	273

Introdução, CREATE e DROP Database

Operações CRUD são operações que conseguimos fazer em um banco de dados.

Essa sigla significa o seguinte:

→ **C**reate: permite criar bancos de dados, tabelas ou views;

→ **R**ead: permite ler os dados do banco de dados. Basicamente, foi o que mais fizemos no curso, através do SELECT;

→ **U**pdate: permite atualizar os dados do banco de dados, tabelas ou views;

→ **D**elete: permite deletar dados de um banco de dados, tabelas ou views.

Introdução, CREATE e DROP Database

Vamos iniciar este módulo aprendendo a utilizar os comandos CREATE DATABASE e DROP DATABASE para criar e excluir bancos de dados, respectivamente.

Inicialmente, vamos criar um banco de dados chamado Teste:

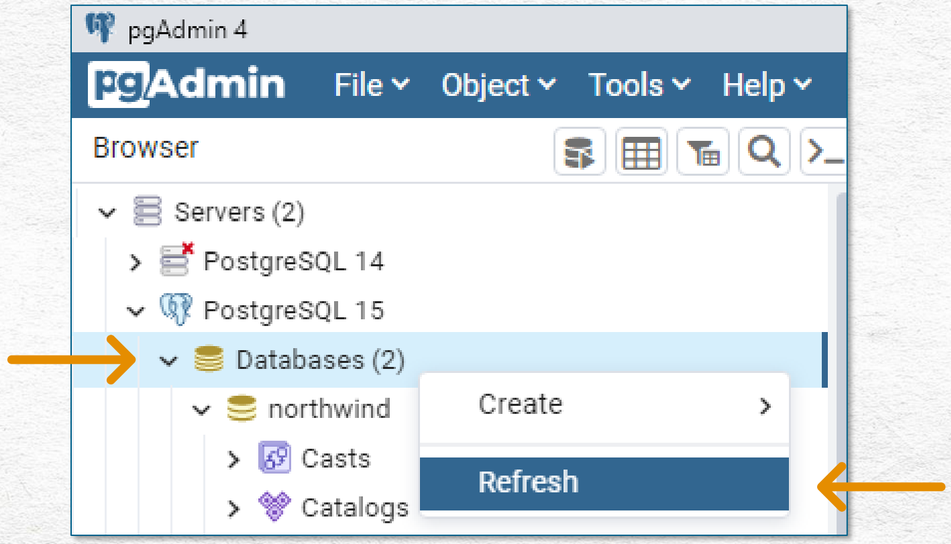
```
CREATE DATABASE teste;
```

Data output | Messages | Notifications

```
CREATE DATABASE

Query returned successfully in 867 msec.
```

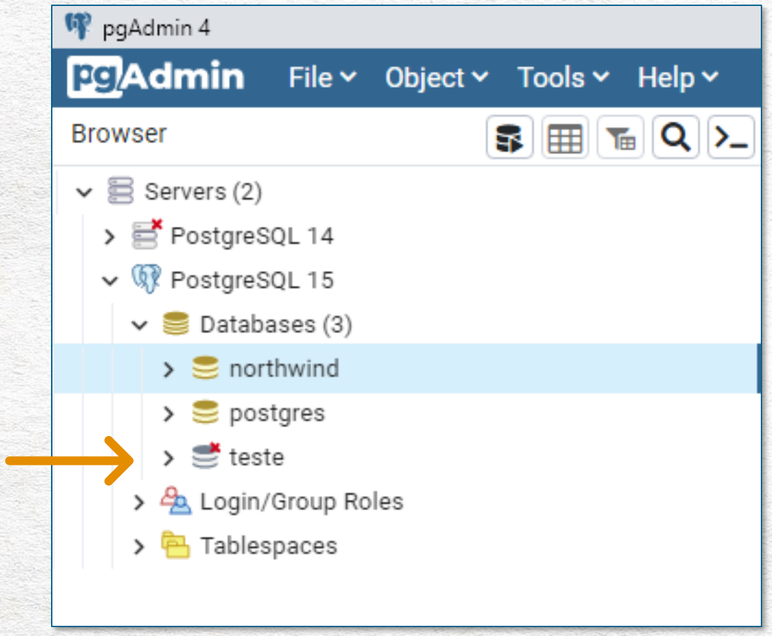
Introdução, CREATE e DROP Database

Para que nosso novo banco de dados apareça no menu do lado esquerdo da tela do pgAdmin 4, devemos clicar com o botão direito sobre Databases e, em seguida, selecionar “Refresh”:

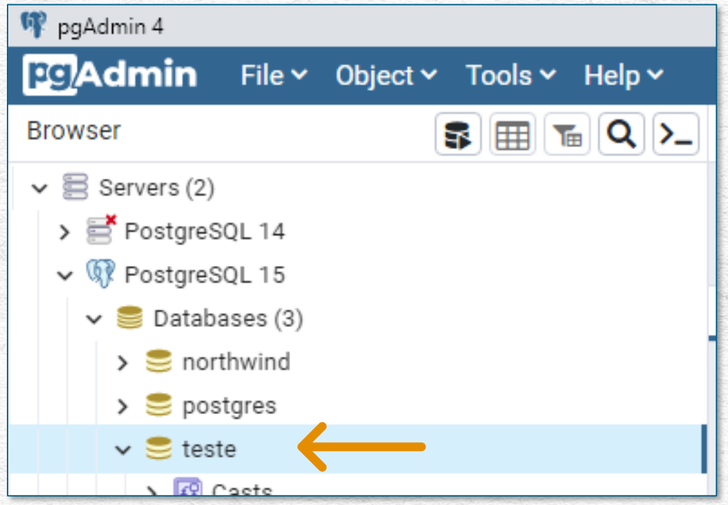


Introdução, CREATE e DROP Database

Repare que, agora, o BD Teste apareceu entre os bancos de dados existentes, porém está com uma cor cinza e um “x” vermelho sobre seu ícone. Isso significa que ele não está conectado. Para utilizá-lo, precisamos conectá-lo. Para fazer isto, basta clicar sobre ele:



Perceba que agora ele aparece com essa cor esverdeada, o que significa que a conexão foi efetuada, estando pronto para uso:



Introdução, CREATE e DROP Database

Se quisermos excluí-lo, vamos precisar utilizar o comando DROP DATABASE. Mas antes disso, precisamos desconectá-lo, pois, se tentarmos excluir um banco de dados que esteja em uso (conectado), o sistema retornará o seguinte erro:

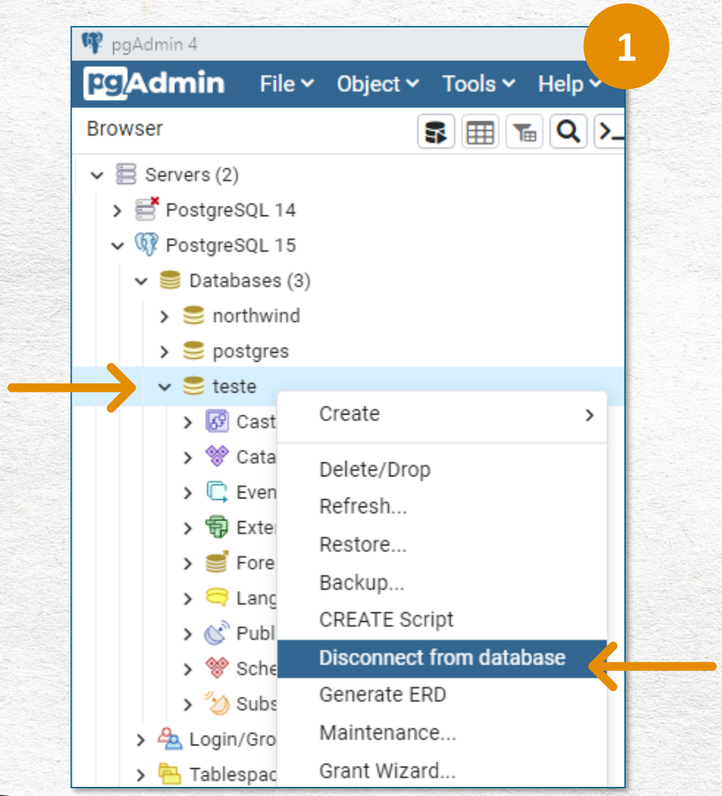
```
DROP DATABASE teste;
```

Data output | Messages | Notifications

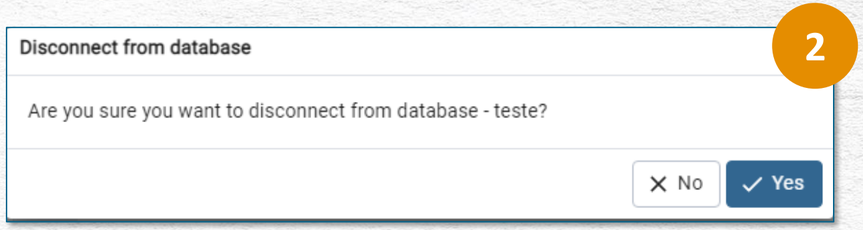
```
ERROR:  database "teste" is being accessed by other users
DETAIL:  There is 1 other session using the database.
SQL state: 55006
```

Introdução, CREATE e DROP Database

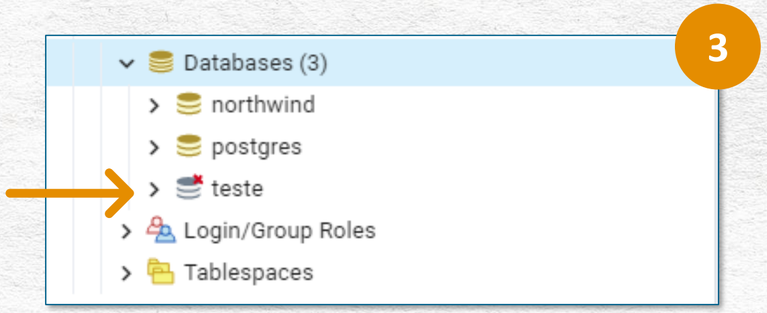
Para desconectar um banco de dados, clicamos com o botão direito sobre seu nome no menu lateral esquerdo e, em seguida, selecionamos “Disconnect from database”:



Confirmamos que desejamos desconectar clicando em “Yes”:



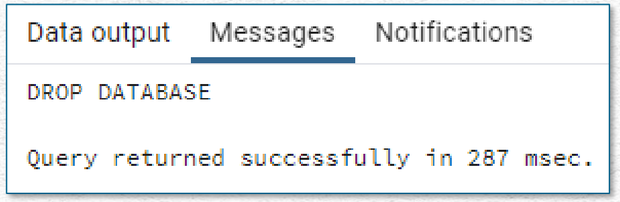
Repare que, dessa forma, o banco de dados volta a ficar cinza com um “x” vermelho sobre seu ícone:



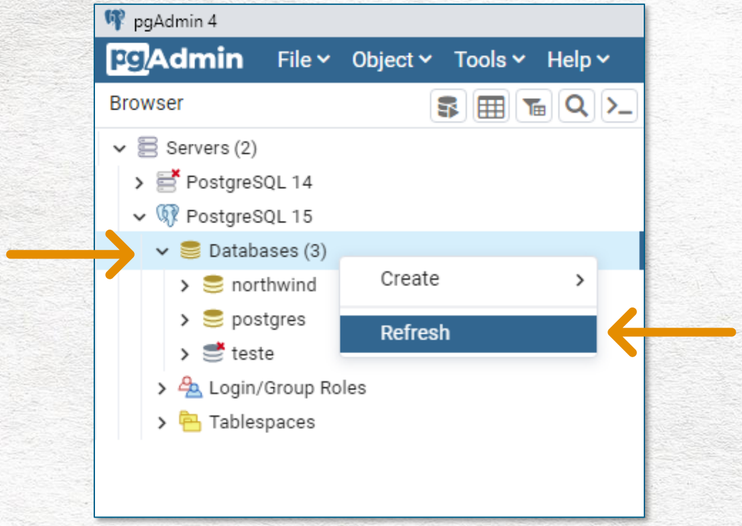
Introdução, CREATE e DROP Database

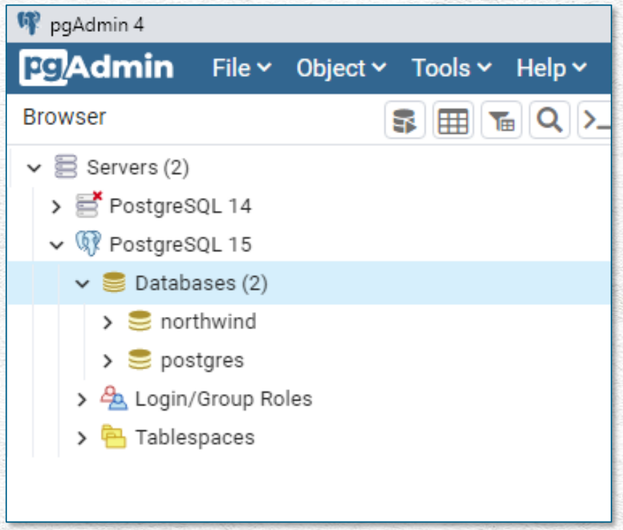
Feito isso, podemos executar o comando DROP DATABASE e o banco de dados será excluído:

```
DROP DATABASE teste;
```



Damos um novo “Refresh” em Databases, e veremos que o banco de dados Teste realmente foi excluído:





Introdução, CREATE e DROP Database

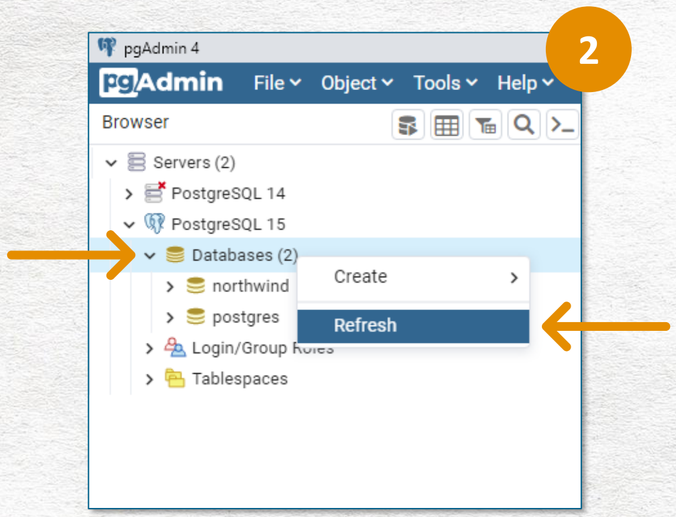
Para os próximos exemplos, utilizaremos um banco de dados chamado Hashtag, portanto vamos criá-lo:

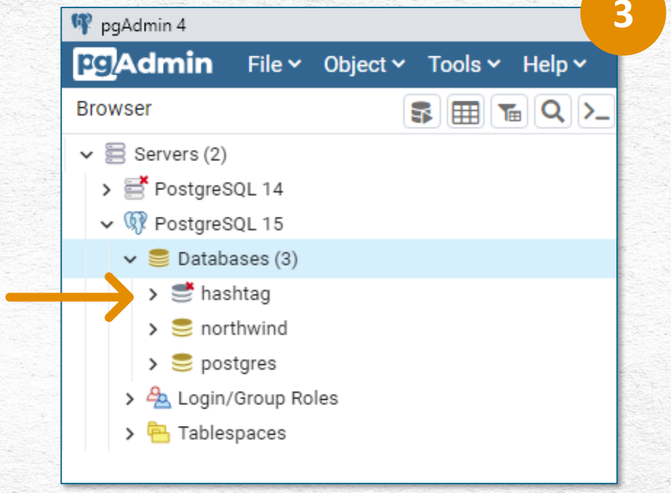
```
CREATE DATABASE hashtag;
```

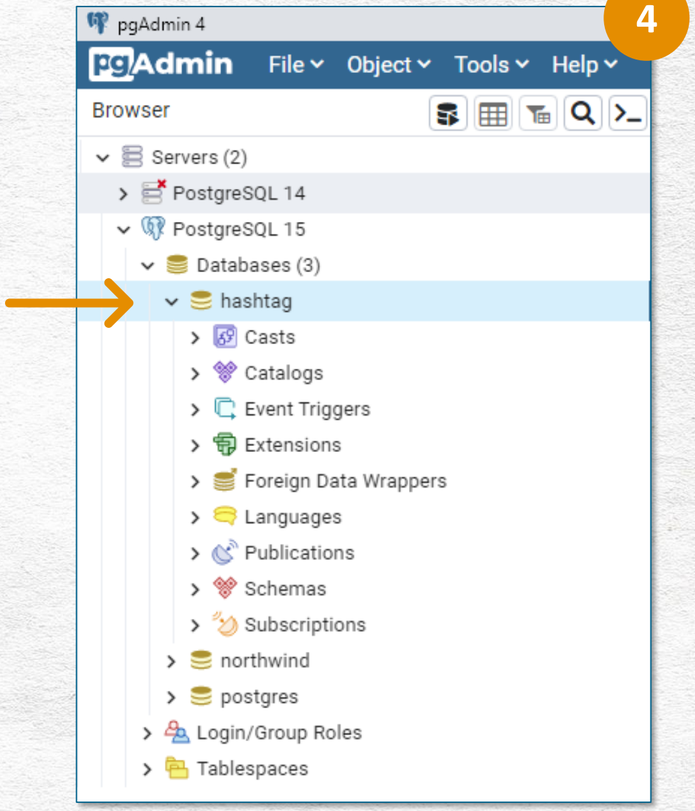
1

Data output | Messages | Notifications

```
CREATE DATABASE

Query returned successfully in 516 msec.
```





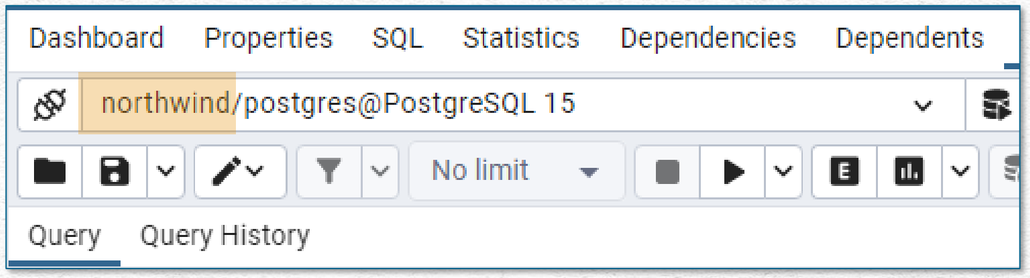


Introdução, CREATE e DROP Database

Criado nosso novo banco de dados Hashtag, precisamos abrir uma nova Query associada a ele.

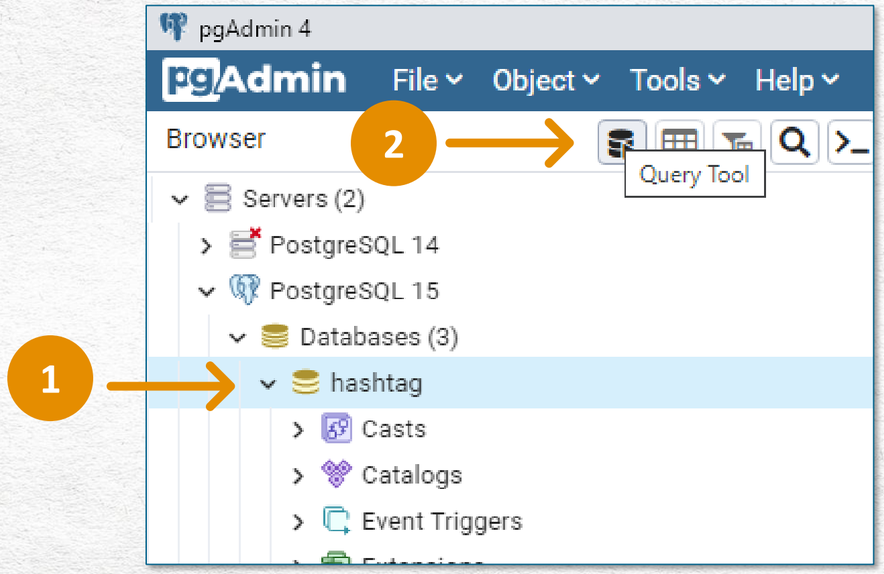
Assim, todos os comandos que executarmos serão efetuados no BD Hashtag.

Se continuarmos usando a Query que temos utilizado até agora, os comandos serão executados no banco de dados Northwind:



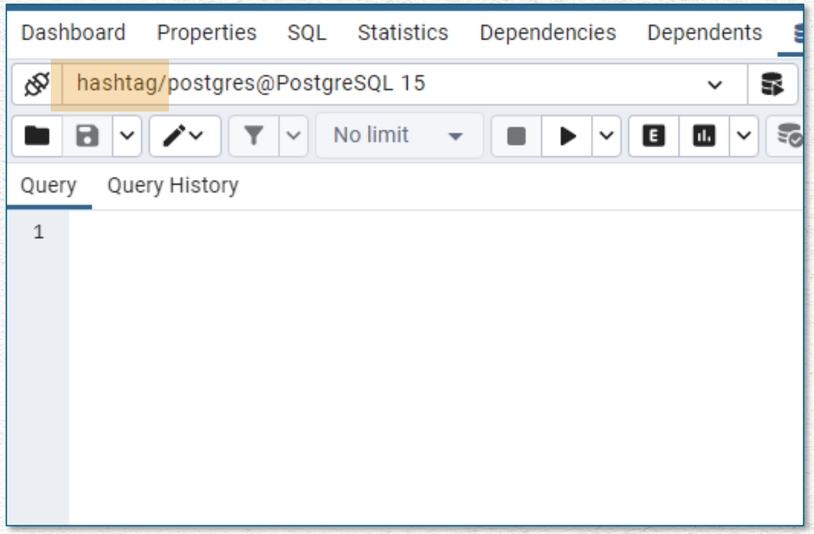
Introdução, CREATE e DROP Database

Portanto, para criar uma nova Query associada ao banco de dados Hashtag, clicamos sobre o BD Hashtag e, em seguida, no ícone Query Tool, conforme mostrado abaixo:



Introdução, CREATE e DROP Database

Feito isso, será aberta uma nova Query em branco associada, desta vez, ao banco de dados Hashtag:



Vamos utilizar esta nova Query para executar os próximos comandos.

Tipos de Dados, CREATE e DROP Table

Uma tabela tem como objetivo armazenar informações dispostas em diferentes colunas.

Quando criamos uma nova tabela, precisamos especificar quais são as colunas que essa tabela deve conter.

Cada uma dessas colunas vai armazenar um tipo de dados específico:

```
CREATE TABLE tabela (
   coluna1 TIPO1,
   coluna2 TIPO2
);
```

Tipos de Dados, CREATE e DROP Table

Os principais tipos de dados são os listados abaixo:

INT: um número inteiro.

NUMERIC(M, D): um número decimal. M é o número total de dígitos e D é a quantidade de casas decimais permitidas.

VARCHAR(N): uma *string* de comprimento VARIÁVEL (pode contar letras, números e caracteres especiais).
O parâmetro N especifica o comprimento máximo da coluna em caracteres.

DATE: uma data no formato YYYY-MM-DD.

TIMESTAMP: uma combinação de data e hora no formato YYYY-MM-DD HH:MM:SS.

Tipos de Dados, CREATE e DROP Table

Sabendo os principais tipos de dados, a criação das tabelas do nosso banco de dados Hashtag seguirá a estrutura abaixo:

```
CREATE TABLE alunos(
    ID_Aluno INT,
    Nome_Aluno VARCHAR(100),
    Email VARCHAR(100)
);
```

Data output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 43 msec.
```

```
CREATE TABLE cursos(
    ID_Curso INT,
    Nome_Curso VARCHAR(100),
    Preco_Curso NUMERIC(10, 2)
);
```

Data output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 51 msec.
```

```
CREATE TABLE matriculas(
    ID_Matricula INT,
    ID_Aluno INT,
    ID_Curso INT,
    Data_Cadastro DATE
);
```

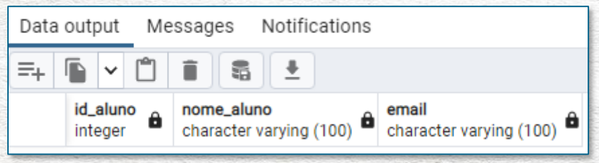
Data output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 87 msec.
```

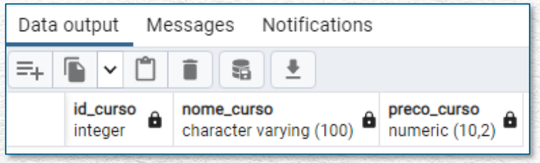
Tipos de Dados, CREATE e DROP Table

Se fizermos uma consulta às tabelas alunos, cursos e matriculas que acabamos de criar, veremos que elas estão vazias, pois ainda não inserimos dados nelas:

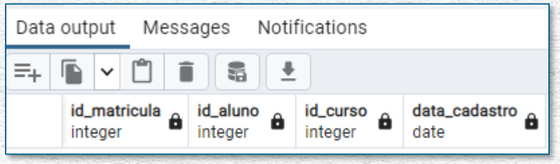
```
SELECT * FROM alunos;
```




```
SELECT * FROM cursos;
```




```
SELECT * FROM matriculas;
```



Tipos de Dados, CREATE e DROP Table

Mas... ainda podemos incluir CONSTRAINTS, ou seja, restrições na criação das tabelas.

Apesar de não serem obrigatórias, as CONSTRAINTS são muito importantes para garantir a integridade de nosso banco de dados, conforme veremos no tópico seguinte.

Como tais restrições são definidas no momento da criação das tabelas, precisamos excluir as tabelas que acabamos de criar, para criá-las novamente, desta vez, incluindo as CONSTRAINTS que veremos a seguir.

Portanto, para excluí-las, basta utilizarmos o comando DROP TABLE:

```
DROP TABLE alunos;
DROP TABLE cursos;
DROP TABLE matriculas;
```

Data output | Messages | Notifications

```
DROP TABLE

Query returned successfully in 78 msec.
```

CONSTRAINTS (Restrições)

CONSTRAINTS no SQL são regras (restrições) que podemos definir para as colunas de uma tabela. Essas regras garantem integridade ao banco de dados, pois é através delas que garantimos que apenas os valores que atendam às regras pré-estabelecidas sejam incluídos em cada coluna.

Por exemplo:

→ Podemos especificar que uma coluna não pode ter valores nulos;
→ Podemos especificar que uma coluna deverá ser uma chave primária ou chave estrangeira.
→ Etc.

As CONSTRAINTS são usadas para limitar os tipos de dados que serão inseridos.

CONSTRAINTS (Restrições)

As principais CONSTRAINTS são as seguintes:

NOT NULL:
- A Constraint NOT NULL faz com que uma coluna não aceite valores nulos.
- Ela identifica que nenhum valor foi definido, obrigando que um campo sempre possua um valor.
- Dessa forma, uma coluna com restrição NOT NULL não aceita valores vazios.

PRIMARY KEY (Chave Primária):
- A PRIMARY KEY identifica de forma única cada registro em uma tabela do banco de dados.
- Chaves primárias devem conter valores únicos.
- Uma coluna de chave primária não pode conter valores NULL.
- Cada tabela deve conter uma, e apenas uma, chave primária.

FOREIGN KEY (Chave Estrangeira):
- Uma FOREING KEY em uma tabela é um campo que aponta para uma chave primária de outra tabela.

CONSTRAINTS (Restrições)

Agora, vamos refazer as tabelas alunos, cursos e matriculas do nosso banco de dados Hashtag, que havíamos criado e excluído anteriormente, só que desta vez, aplicando as restrições necessárias:

Comecemos pela tabela alunos:

```
CREATE TABLE alunos(
    ID_Aluno INT ,
    Nome_Aluno VARCHAR(100) NOT NULL,
    Email VARCHAR(100) NOT NULL,
    PRIMARY KEY(ID_Aluno)
);
```

1

2

Data output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 58 msec.
```

1. Estabelecemos que as colunas Nome_Aluno e Email não podem conter valores nulos (**NOT NULL**);

2. Também definimos a coluna ID_Aluno como a chave primária (**PRIMARY KEY**) desta tabela. Esta coluna também acaba sendo do tipo NOT NULL, já que, por definição, uma PK não pode conter valores nulos.

CONSTRAINTS (Restrições)

Vejamos agora a tabela cursos:

```
CREATE TABLE cursos(
    ID_Curso INT,
    Nome_Curso VARCHAR(100) NOT NULL,
    Preco_Curso NUMERIC(10, 2) NOT NULL,
    PRIMARY KEY(ID_Curso)
);
```

1

2

Data output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 110 msec.
```

1. Estabelecemos que as colunas Nome_Curso e Preco_Curso não podem conter valores nulos (**NOT NULL**);

2. Também definimos a coluna ID_Curso como a chave primária (**PRIMARY KEY**) desta tabela. Esta coluna também acaba sendo do tipo NOT NULL, já que, por definição, uma PK não pode conter valores nulos.

CONSTRAINTS (Restrições)

Por fim, a tabela matriculas:

```
CREATE TABLE matriculas(
    ID_Matricula INT,
    ID_Aluno INT NOT NULL,
    ID_Curso INT NOT NULL,
    Data_Cadastro DATE NOT NULL,
    PRIMARY KEY(ID_Matricula),
    FOREIGN KEY(ID_Aluno) REFERENCES alunos(ID_Aluno),
    FOREIGN KEY(ID_Curso) REFERENCES cursos(ID_Curso)
);
```

Data output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 50 msec.
```

1. Estabelecemos que as colunas ID_Aluno, ID_Curso e Data_Cadastro não podem conter valores nulos (**NOT NULL**);

2. Também definimos a coluna ID_Matricula como a chave primária (**PRIMARY KEY**) desta tabela. Esta coluna também acaba sendo do tipo NOT NULL, já que, por definição, uma PK não pode conter valores nulos.

3. Por fim, também informamos que as colunas ID_Aluno e ID_Curso são chaves estrangeiras (**FOREIGN KEYs**) que fazem referência às colunas ID_Aluno da tabela alunos e ID_Curso da tabela cursos, respectivamente.

INSERT INTO

Para inserir dados nas tabelas, utilizamos a seguinte sintaxe:

```
INSERT INTO tabela(coluna1, coluna2, colunaN)
VALUES
   (valor_coluna1, valor_coluna2, valor_colunaN),
   (valor_coluna1, valor_coluna2, valor_colunaN),
   (valor_coluna1, valor_coluna3, valor_colunaN);
```

INSERT INTO

Vamos inserir dados nas nossas tabelas alunos, cursos e matriculas:

```
INSERT INTO alunos(ID_Aluno, Nome_Aluno, Email)
VALUES
    (1, 'Ana'    , 'ana123@gmail.com'          ),
    (2, 'Bruno'  , 'bruno_vargas@outlook.com' ),
    (3, 'Carla'  , 'carlinha1999@gmail.com'    ),
    (4, 'Diego'  , 'diicastroneves@gmail.com' );
```

Data output | Messages | Notifications

```
INSERT 0 4

Query returned successfully in 44 msec.
```

```
INSERT INTO cursos(ID_Curso, Nome_Curso, Preco_Curso)
VALUES
    (1, 'Excel'      , 100),
    (2, 'VBA'        , 200),
    (3, 'PowerBI'    , 150);
```

Data output | Messages | Notifications

```
INSERT 0 3

Query returned successfully in 65 msec.
```

```
INSERT INTO matriculas(ID_Matricula, ID_Aluno, ID_Curso, Data_Cadastro)
VALUES
    (1, 1, 1, '2021/03/11'),
    (2, 1, 2, '2021/06/21'),
    (3, 2, 3, '2021/01/08'),
    (4, 3, 1, '2021/04/03'),
    (5, 4, 1, '2021/05/10'),
    (6, 4, 3, '2021/05/10');
```

Data output | Messages | Notifications

```
INSERT 0 6

Query returned successfully in 45 msec.
```

PostgreSQL

INSERT INTO

Vejamos como ficaram as tabelas agora que possuem dados:

```
SELECT * FROM alunos;
```

Data output | Messages | Notifications

	id_aluno [PK] integer	nome_aluno character varying (100)	email character varying (100)
1	1	Ana	ana123@gmail.com
2	2	Bruno	bruno_vargas@outlook.com
3	3	Carla	carlinha1999@gmail.com
4	4	Diego	diicastroneves@gmail.com

```
SELECT * FROM cursos;
```

Data output | Messages | Notifications

	id_curso [PK] integer	nome_curso character varying (100)	preco_curso numeric (10,2)
1	1	Excel	100.00
2	2	VBA	200.00
3	3	PowerBI	150.00

```
SELECT * FROM matriculas;
```

Data output | Messages | Notifications

	id_matricula [PK] integer	id_aluno integer	id_curso integer	data_cadastro date
1	1	1	1	2021-03-11
2	2	1	2	2021-06-21
3	3	2	3	2021-01-08
4	4	3	1	2021-04-03
5	5	4	1	2021-05-10
6	6	4	3	2021-05-10

PostgreSQL

Índice Geral

Índice deste Módulo

UPDATE

O valor do curso de Excel aumentou de R$ 100,00 para R$ 300,00.

Precisamos, então, alterar este valor na tabela cursos.

Para isso, utilizamos o comando UPDATE:

```
UPDATE cursos
SET Preco_Curso = 300
WHERE ID_Curso = 1;
```

Data output | Messages | Notifications

```
UPDATE 1

Query returned successfully in 80 msec.
```

Repare que o preço do curso de Excel foi alterado na tabela, conforme queríamos:

```
837  SELECT * FROM cursos;
```

Data output | Messages | Notifications

	id_curso [PK] integer	nome_curso character varying (100)	preco_curso numeric (10,2)
1	1	Excel	300.00
2	2	VBA	200.00
3	3	PowerBI	150.00

DELETE

A matrícula de ID = 6 foi cancelada, pois o aluno pediu reembolso.

Dessa forma, precisamos excluí-la da tabela matriculas.

Para isso, utilizamos o comando DELETE:

```
DELETE FROM matriculas
WHERE ID_Matricula = 6;
```

Data output | Messages | Notifications

```
DELETE 1

Query returned successfully in 57 msec.
```

Repare que o registro de ID_Matricula = 6 não existe mais na tabela matriculas:

```
847  SELECT * FROM matriculas;
```

Data output | Messages | Notifications

	id_matricula [PK] integer	id_aluno integer	id_curso integer	data_cadastro date
1	1	1	1	2021-03-11
2	2	1	2	2021-06-21
3	3	2	3	2021-01-08
4	4	3	1	2021-04-03
5	5	4	1	2021-05-10

TRUNCATE

Outro comando que temos é o TRUNCATE. Com ele, conseguimos deletar todos os registros de uma tabela de uma só vez, mas a tabela continua existindo.

Vamos utilizar o TRUNCATE com a tabela matriculas:

```
TRUNCATE TABLE matriculas;
```

Data output | Messages | Notifications

```
TRUNCATE TABLE

Query returned successfully in 48 msec.
```

Repare que, se consultarmos a tabela matriculas, veremos que ela continua existindo, porém sem registros:

```
857  SELECT * FROM matriculas;
```

Data output | Messages | Notifications

id_matricula [PK] integer	id_aluno integer	id_curso integer	data_cadastro date

DROP (Cascade)

Por fim, utilizamos o comando DROP TABLE para excluir uma tabela.
Porém, atenção: cuidado ao excluir uma tabela que tenha restrições de PRIMARY KEY!

Perceba que, se tentarmos excluir a tabela alunos ou a tabela cursos, será retornado um erro:

```
DROP TABLE alunos;
```

Data output | Messages | Notifications

```
ERROR:  cannot drop table alunos because other objects depend on it
DETAIL:  constraint matriculas_id_aluno_fkey on table matriculas depends on table alunos
HINT:  Use DROP ... CASCADE to drop the dependent objects too.
SQL state: 2BP01
```

```
DROP TABLE cursos;
```

Data output | Messages | Notifications

```
ERROR:  cannot drop table cursos because other objects depend on it
DETAIL:  constraint matriculas_id_curso_fkey on table matriculas depends on table cursos
HINT:  Use DROP ... CASCADE to drop the dependent objects too.
SQL state: 2BP01
```

PostgreSQL

DROP (Cascade)

Isso aconteceu porque, como a tabela matriculas depende das tabelas alunos e cursos, pois faz referência a elas por meio das chaves estrangeiras que foram definidas na tabela matriculas, as tabelas alunos e cursos não podem ser excluídas enquanto as chaves estrangeiras da tabela matriculas existirem.

Assim, ou começamos excluindo a tabela matriculas, para depois excluir as tabelas alunos e cursos, ou utilizamos o comando CASCADE nas tabelas alunos e cursos:

```
DROP TABLE alunos CASCADE;
DROP TABLE cursos CASCADE;
DROP TABLE matriculas;
```

Data output | Messages | Notifications

```
NOTICE:  drop cascades to constraint matriculas_id_aluno_fkey on table matriculas
NOTICE:  drop cascades to constraint matriculas_id_curso_fkey on table matriculas
DROP TABLE

Query returned successfully in 45 msec.
```

Com o CASCADE, independentemente da tabela que estivermos excluindo possuir alguma relação com outras tabelas ou não, ela será excluída, como aconteceu acima com as tabelas alunos e cursos.

DROP (Cascade)

Se tentarmos visualizar as três tabelas, veremos que, agora, elas foram de fato excluídas:

```
SELECT * FROM alunos;
```

Data output | Messages | Notifications

```
ERROR:  relation "alunos" does not exist
LINE 1: SELECT * FROM alunos;
                      ^

SQL state: 42P01
Character: 15
```

```
SELECT * FROM cursos;
```

Data output | Messages | Notifications

```
ERROR:  relation "cursos" does not exist
LINE 1: SELECT * FROM cursos;
                      ^

SQL state: 42P01
Character: 15
```

```
SELECT * FROM matriculas;
```

Data output | Messages | Notifications

```
ERROR:  relation "matriculas" does not exist
LINE 1: SELECT * FROM matriculas;
                      ^

SQL state: 42P01
Character: 15
```

PostgreSQL

EXERCÍCIOS

1

Questão 1

Você foi contratado para realizar testes iniciais em um ambiente de desenvolvimento do Hotel Hashtag. Para garantir que o sistema está configurado corretamente, siga os passos abaixo:

- ✓ Crie um banco de dados chamado **dbTeste**;
- ✓ Após, exclua-o completamente.

2

Questão 2

O Hotel Hashtag está modernizando seu sistema de gestão e decidiu implementar um banco de dados para gerenciar as informações de seus clientes, quartos e estadias.

Para isso, crie um banco de dados chamado **dbHotelHashtag**, que será utilizado para centralizar e organizar todas as informações necessárias para a operação eficiente do hotel.

EXERCÍCIOS

3a

Questão 3-a

Com o banco de dados dbHotelHashtag em uso, crie as seguintes tabelas:

a) Tabela clientes

NOME DA COLUNA	ESPECIFICAÇÕES
id_cliente	INT, será a chave primária da tabela.
nome	VARCHAR(100), não pode ser nulo.
email	VARCHAR(50), não pode ser nulo.
data_nascimento	DATE, pode ser nula.

3b

Questão 3-b

Com o banco de dados dbHotelHashtag em uso, crie as seguintes tabelas:

b) Tabela quartos

NOME DA COLUNA	ESPECIFICAÇÕES
id_quarto	INT, será a chave primária da tabela.
tipo	VARCHAR(30), não pode ser nulo.
qtd_pessoas	INT, não pode ser nulo.
preco_diaria	NUMERIC(10, 2), não pode ser nulo.

EXERCÍCIOS

3c

Questão 3-c

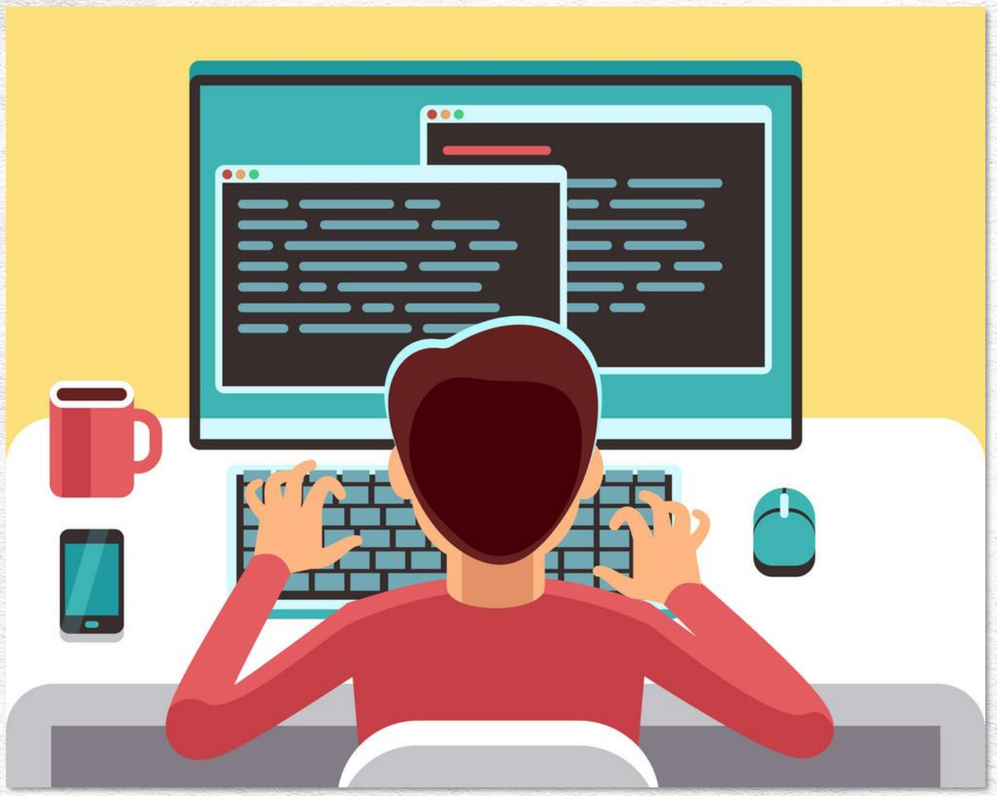
Com o banco de dados dbHotelHashtag em uso, crie as seguintes tabelas:

a) Tabela estadias

NOME DA COLUNA	ESPECIFICAÇÕES
id_estadia	INT, será a chave primária da tabela.
id_cliente	INT, será a chave estrangeira que fará referência à coluna id_cliente da tabela clientes, não pode ser nula.
id_quarto	INT, será a chave estrangeira que fará referência à coluna id_quarto da tabela quartos, não pode ser nula.
data_entrada	TIMESTAMP, não pode ser nula.
data_saida	TIMESTAMP, pode ser nula.

EXERCÍCIOS

4a



Questão 4-a

Agora que as tabelas foram criadas, insira os dados dos clientes, quartos e estadias:

a) Insira **10 clientes** na **tabela clientes** e faça uma consulta para verificar se os dados foram inseridos corretamente:

id_cliente [PK] integer	nome character varying (100)	email character varying (50)	data_nascimento date
1	Ana Silva	ana.silva@email.com	1990-03-15
2	Carlos Souza	carlos.souza@email.com	1985-07-20
3	Maria Oliveira	maria.oliveira@email.com	1993-11-05
4	João Santos	joao.santos@email.com	1978-02-25
5	Beatriz Lima	beatriz.lima@email.com	2000-01-12
6	Lucas Alves	lucas.alves@email.com	[null]
7	Fernanda Rocha	fernanda.rocha@email.com	1995-06-10
8	Gabriel Mendes	gabriel.mendes@email.com	1989-08-08
9	Sofia Costa	sofia.costa@email.com	1991-12-24
10	Rafael Silva	rafael.silva@email.com	1997-04-18

EXERCÍCIOS

4b

Questão 4-b

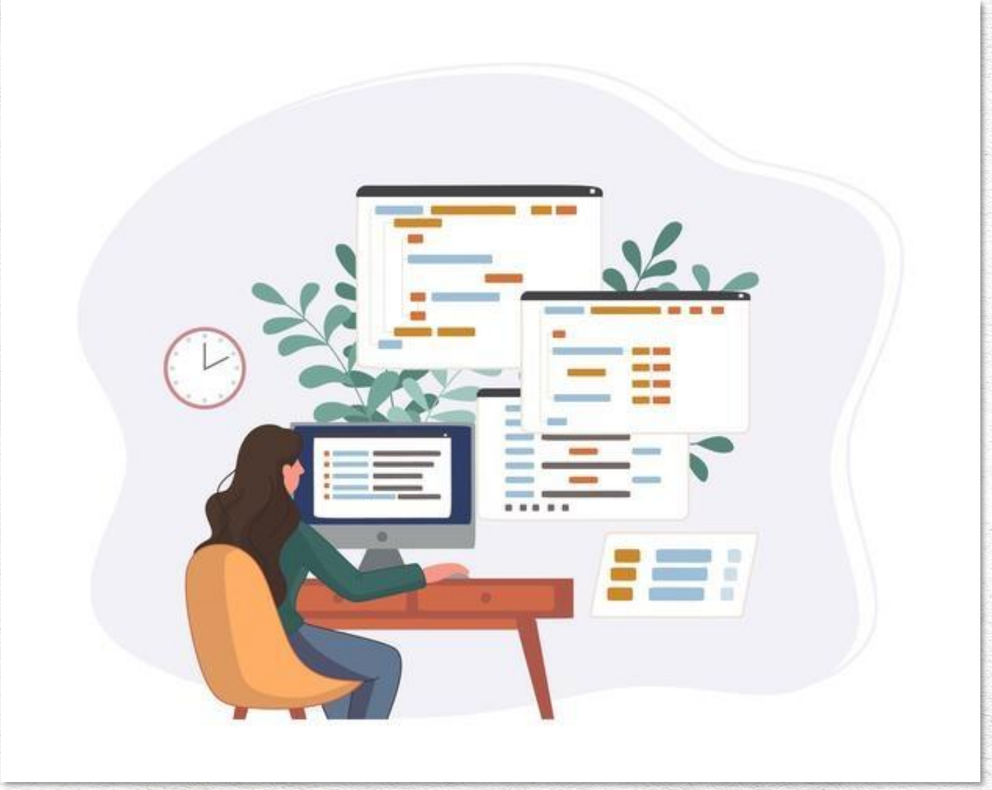
Agora que as tabelas foram criadas, insira os dados dos clientes, quartos e estadias:

b) Insira **10 quartos** na **tabela quartos** e faça uma consulta para verificar se os dados foram inseridos corretamente:

id_quarto [PK] integer	tipo character varying (30)	qtd_pessoas integer	preco_diaria numeric (10,2)
1	Single	1	150.00
2	Double	2	250.00
3	Suite	4	500.00
4	Single	1	140.00
5	Double	2	230.00
6	Suite	4	550.00
7	Single	1	160.00
8	Double	2	240.00
9	Suite	4	600.00
10	Single	1	170.00

EXERCÍCIOS

4c



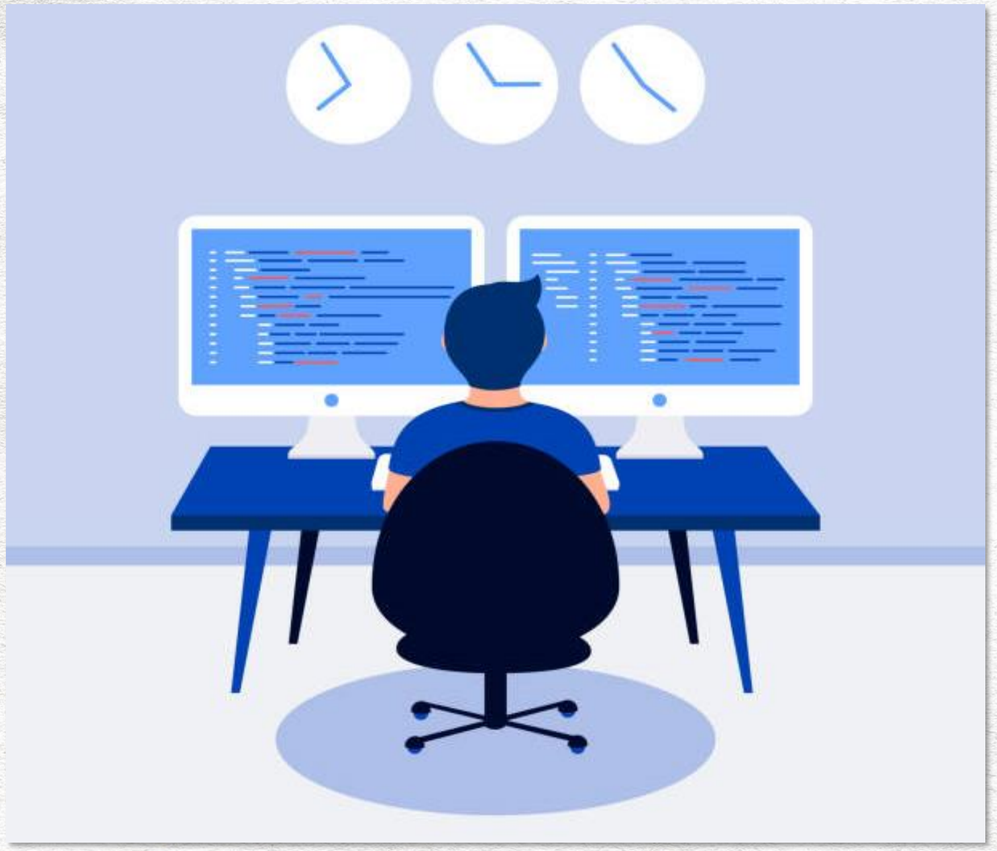
Questão 4-c

Agora que as tabelas foram criadas, insira os dados dos clientes, quartos e estadias:

c) Insira **15 estadias** na **tabela estadias** e faça uma consulta para verificar se os dados foram inseridos corretamente:

id_estadia [PK] integer	id_cliente integer	id_quarto integer	data_entrada timestamp without time zone	data_saida timestamp without time zone
1	1	2	2024-01-15 14:00:00	2024-01-16 12:00:00
2	3	5	2024-02-16 16:00:00	2024-02-18 11:00:00
3	4	1	2024-03-17 13:00:00	2024-03-19 11:00:00
4	2	3	2024-04-18 15:00:00	2024-05-10 10:00:00
5	5	7	2024-05-19 12:30:00	2024-05-23 12:00:00
6	6	9	2024-06-20 18:00:00	2024-06-22 14:00:00
7	8	4	2024-07-21 11:00:00	2024-07-23 09:00:00
8	9	10	2024-08-22 14:00:00	2024-08-24 12:00:00
9	7	6	2024-09-23 10:00:00	2024-09-26 10:00:00
10	10	8	2024-10-24 16:00:00	2024-11-06 11:00:00
11	1	5	2024-11-25 13:00:00	2024-11-27 10:00:00
12	3	2	2024-12-26 17:00:00	2024-12-28 13:00:00
13	4	1	2024-12-27 12:00:00	[null]
14	8	7	2024-12-28 15:00:00	[null]
15	9	3	2024-12-29 14:00:00	[null]

EXERCÍCIOS



5

Questão 5

O cliente da estadia de id_estadia = 13 desocupou o quarto em 04/01/2025 ao meio-dia.

Atualize a tabela para fazer constar essa informação.

6

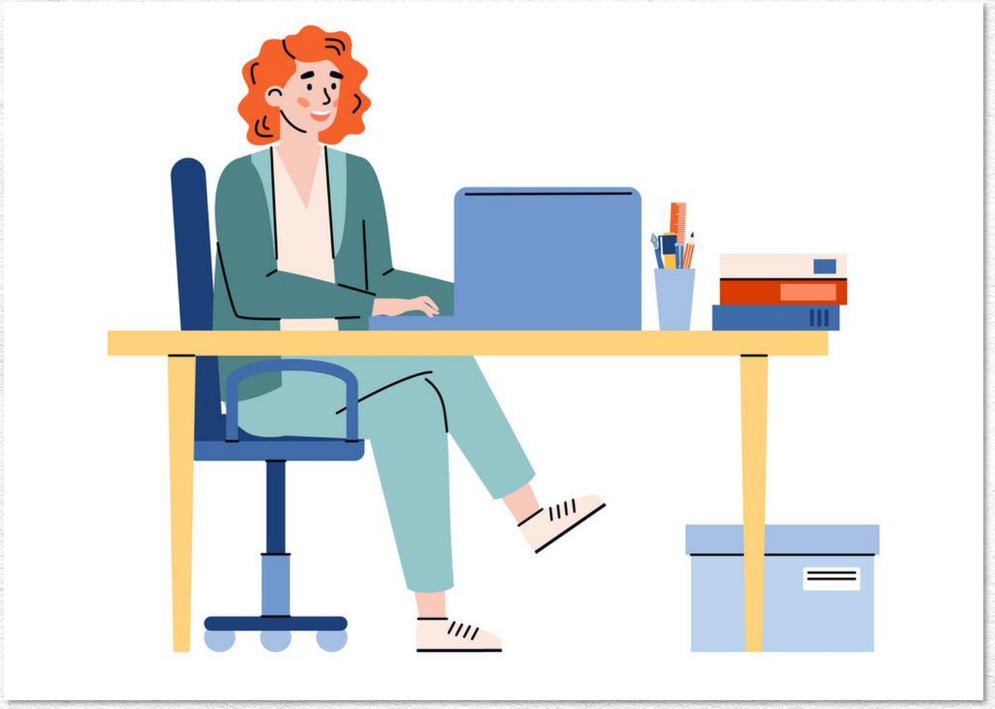
Questão 6

A cliente Beatriz Lima (id_cliente = 5) solicitou a exclusão de seus dados de nosso banco de dados. Como esta cliente se hospedou no hotel anteriormente, ela está referenciada na tabela estadias. Portanto, é necessário seguir os seguintes passos para realizar a exclusão de seus dados:

a) Excluir a(s) estadia(s) que faz(em) referência a esta cliente na tabela estadias.

b) Após a exclusão das estadias, excluir a cliente Beatriz Lima (id_cliente = 5) da tabela clientes.

EXERCÍCIOS



7

Questão 7

A gerência do Hotel Hashtag decidiu iniciar um novo ano fiscal reorganizando o sistema e limpando os registros antigos de estadias. Isso permitirá começar o ano com dados atualizados e espaço no banco de dados para novos registros. O backup dos registros do ano anterior já foi efetuado para que os dados não sejam perdidos.

Agora, você já pode esvaziar todos os registros da tabela estadias, garantindo que a estrutura da tabela seja mantida.

8

Questão 8

Após uma reestruturação no sistema de TI, o Hotel Hashtag contratou um novo software que substituirá o atual banco de dados. Todos os dados do dbHotelHashtag já foram migrados para esse novo software.

Agora, como parte da transição, as tabelas clientes, quartos e estadias precisam ser excluídas completamente, removendo qualquer dependência existente. Realize essa exclusão.

GABARITO EXPLICATIVO

1

Questão 1

Você foi contratado para realizar testes iniciais em um ambiente de desenvolvimento do Hotel Hashtag. Para garantir que o sistema está configurado corretamente, siga os passos abaixo:

- ✓ Crie um banco de dados chamado **dbTeste**;
- ✓ Após, exclua-o completamente.

Explicação e Resultado

Para criar o banco de dados **dbTeste**, devemos executar o seguinte comando:

```
CREATE DATABASE dbTeste;
```

Data Output | Messages | Notifications

```
CREATE DATABASE

Query returned successfully in 756 msec.
```

E para exclui-lo, o comando abaixo:

```
DROP DATABASE dbTeste;
```

Data Output | Messages | Notifications

```
DROP DATABASE

Query returned successfully in 143 msec.
```

GABARITO EXPLICATIVO

2

Questão 2

O Hotel Hashtag está modernizando seu sistema de gestão e decidiu implementar um banco de dados para gerenciar as informações de seus clientes, quartos e estadias.

Para isso, crie um banco de dados chamado **dbHotelHashtag**, que será utilizado para centralizar e organizar todas as informações necessárias para a operação eficiente do hotel.

Explicação e Resultado

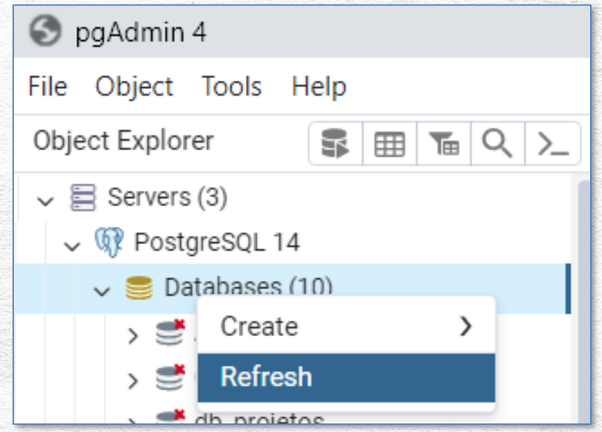
Para criar o banco de dados **dbHotelHashtag**, devemos executar o seguinte comando:

```
CREATE DATABASE dbHotelHashtag;
```

```
Data Output  Messages  Notifications
CREATE DATABASE

Query returned successfully in 635 msec.
```

Para garantir que os comandos dos próximos exercícios serão executados no banco de dados que acabamos de criar, **devemos colocá-lo em uso**.

Para fazer isso, no menu lateral esquerdo do pgAdmin 4, podemos clicar com o botão direito do mouse em **Databases** e selecionar **Refresh**:



CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

2

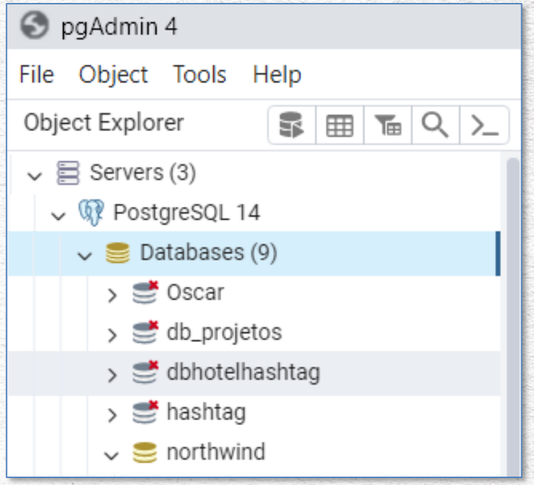
Questão 2

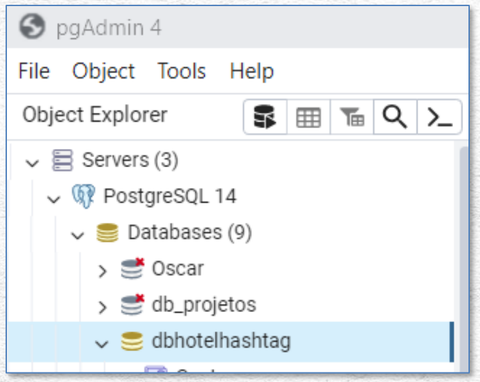
O Hotel Hashtag está modernizando seu sistema de gestão e decidiu implementar um banco de dados para gerenciar as informações de seus clientes, quartos e estadias.

Para isso, crie um banco de dados chamado **dbHotelHashtag**, que será utilizado para centralizar e organizar todas as informações necessárias para a operação eficiente do hotel.

Explicação e Resultado

Dessa forma, o banco de dados **dbHotelHashtag aparecerá** no menu, porém **desconectado** (repare que o ícone ao lado de seu nome está cinza com um X vermelho):





Clicamos sobre ele para conectá-lo (repare que agora ele aparece com o ícone em amarelo):

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

2

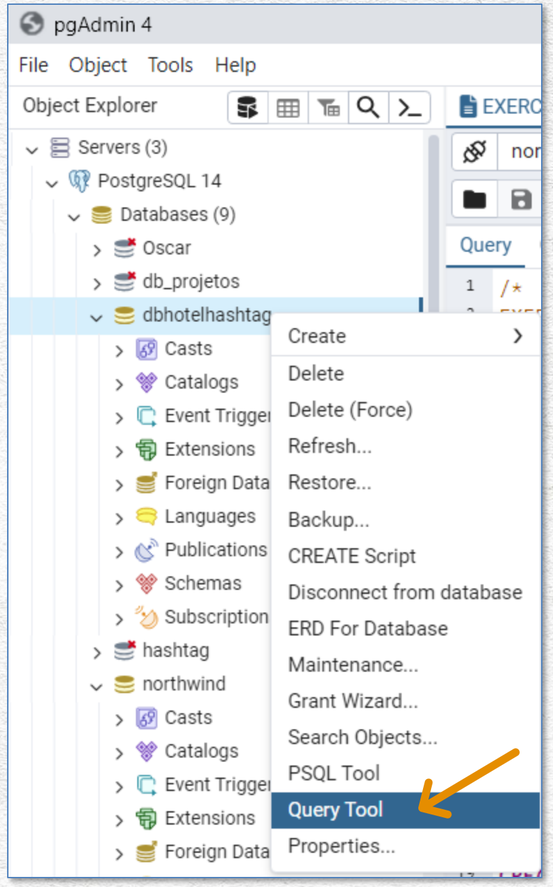
Questão 2

O Hotel Hashtag está modernizando seu sistema de gestão e decidiu implementar um banco de dados para gerenciar as informações de seus clientes, quartos e estadias.

Para isso, crie um banco de dados chamado **dbHotelHashtag**, que será utilizado para centralizar e organizar todas as informações necessárias para a operação eficiente do hotel.

Explicação e Resultado

Clicamos com o botão direito do mouse sobre o nome do banco de dados e selecionamos **Query Tool**:



CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

2

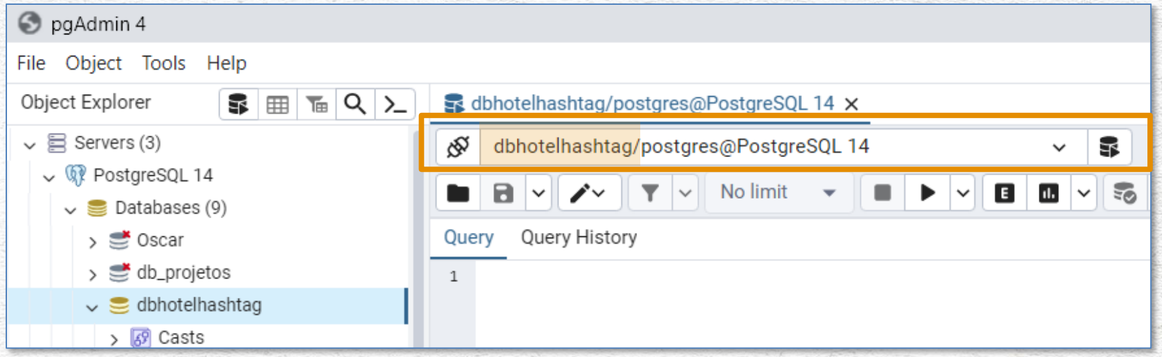
Questão 2

O Hotel Hashtag está modernizando seu sistema de gestão e decidiu implementar um banco de dados para gerenciar as informações de seus clientes, quartos e estadias.

Para isso, crie um banco de dados chamado **dbHotelHashtag**, que será utilizado para centralizar e organizar todas as informações necessárias para a operação eficiente do hotel.

Explicação e Resultado

Assim **colocamos o banco de dados dbHotelHashtag em uso, abrindo uma janela** para executarmos os comandos que deverão ser efetuados nele:



Agora estamos prontos para criarmos os comandos dos próximos exercícios nesta janela.

GABARITO EXPLICATIVO

3a

Questão 3-a

Com o banco de dados dbHotelHashtag em uso, crie as seguintes tabelas:

a) Tabela clientes

NOME DA COLUNA	ESPECIFICAÇÕES
id_cliente	INT, será a chave primária da tabela.
nome	VARCHAR(100), não pode ser nulo.
email	VARCHAR(50), não pode ser nulo.
data_nascimento	DATE, pode ser nula.

Explicação e Resultado

Para criar a tabela **clientes** com as colunas solicitadas e com as especificações apresentadas ao lado, executamos um **CREATE TABLE** passando para o comando os **nomes das colunas** e as **especificações** de cada uma, ou seja, seus **tipos de dados e constraints**:

```
-- a) Criando a tabela clientes

CREATE TABLE clientes (
    id_cliente INT,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(50) NOT NULL,
    data_nascimento DATE,
    PRIMARY KEY(id_cliente)
);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 118 msec.
```

PostgreSQL

GABARITO EXPLICATIVO

3b

Questão 3-b

Com o banco de dados dbHotelHashtag em uso, crie as seguintes tabelas:

b) Tabela quartos

NOME DA COLUNA	ESPECIFICAÇÕES
id_quarto	INT, será a chave primária da tabela.
tipo	VARCHAR(30), não pode ser nulo.
qtd_pessoas	INT, não pode ser nulo.
preco_diaria	NUMERIC(10, 2), não pode ser nulo.

Explicação e Resultado

Para criar a tabela **quartos** com as colunas solicitadas e com as especificações apresentadas ao lado, executamos um **CREATE TABLE** passando para o comando os **nomes das colunas** e as **especificações** de cada uma, ou seja, seus **tipos de dados e constraints**:

```
-- b) Criando a tabela quartos

CREATE TABLE quartos (
    id_quarto INT,
    tipo VARCHAR(30) NOT NULL,
    qtd_pessoas INT NOT NULL,
    preco_diaria NUMERIC(10, 2) NOT NULL,
    PRIMARY KEY(id_quarto)
);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 41 msec.
```

GABARITO EXPLICATIVO

3c

Questão 3-c

Com o banco de dados dbHotelHashtag em uso, crie as seguintes tabelas:

c) Tabela estadias

NOME DA COLUNA	ESPECIFICAÇÕES
id_estadia	INT, será a chave primária da tabela.
id_cliente	INT, será a chave estrangeira que fará referência à coluna id_cliente da tabela clientes, não pode ser nula.
id_quarto	INT, será a chave estrangeira que fará referência à coluna id_quarto da tabela quartos, não pode ser nula.
data_entrada	TIMESTAMP, não pode ser nula.
data_saida	TIMESTAMP, pode ser nula.

Explicação e Resultado

Para criar a tabela **estadias** com as colunas solicitadas e com as especificações apresentadas ao lado, executamos um **CREATE TABLE** passando para o comando os **nomes das colunas** e as **especificações** de cada uma, ou seja, seus **tipos de dados e constraints**:

```
-- c) Criando a tabela estadias

CREATE TABLE estadias (
    id_estadia INT,
    id_cliente INT NOT NULL,
    id_quarto INT NOT NULL,
    data_entrada TIMESTAMP NOT NULL,
    data_saida TIMESTAMP,
    PRIMARY KEY(id_estadia),
    FOREIGN KEY(id_cliente) REFERENCES clientes(id_cliente),
    FOREIGN KEY(id_quarto) REFERENCES quartos(id_quarto)
);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 102 msec.
```

GABARITO EXPLICATIVO

4a

Questão 4-a

Agora que as tabelas foram criadas, insira os dados dos clientes, quartos e estadias:

a) Insira **10 clientes** na **tabela clientes** e faça uma consulta para verificar se os dados foram inseridos corretamente:

id_cliente [PK] integer	nome character varying (100)	email character varying (50)	data_nascimento date
1	Ana Silva	ana.silva@email.com	1990-03-15
2	Carlos Souza	carlos.souza@email.com	1985-07-20
3	Maria Oliveira	maria.oliveira@email.com	1993-11-05
4	João Santos	joao.santos@email.com	1978-02-25
5	Beatriz Lima	beatriz.lima@email.com	2000-01-12
6	Lucas Alves	lucas.alves@email.com	[null]
7	Fernanda Rocha	fernanda.rocha@email.com	1995-06-10
8	Gabriel Mendes	gabriel.mendes@email.com	1989-08-08
9	Sofia Costa	sofia.costa@email.com	1991-12-24
10	Rafael Silva	rafael.silva@email.com	1997-04-18

Explicação e Resultado

Para inserir as **10 linhas** mostradas ao lado na **tabela clientes**, executamos o comando **INSERT INTO** conforme mostrado abaixo:

```
INSERT INTO clientes (id_cliente, nome, email, data_nascimento) VALUES
(1, 'Ana Silva', 'ana.silva@email.com', '1990-03-15'),
(2, 'Carlos Souza', 'carlos.souza@email.com', '1985-07-20'),
(3, 'Maria Oliveira', 'maria.oliveira@email.com', '1993-11-05'),
(4, 'João Santos', 'joao.santos@email.com', '1978-02-25'),
(5, 'Beatriz Lima', 'beatriz.lima@email.com', '2000-01-12'),
(6, 'Lucas Alves', 'lucas.alves@email.com', NULL),
(7, 'Fernanda Rocha', 'fernanda.rocha@email.com', '1995-06-10'),
(8, 'Gabriel Mendes', 'gabriel.mendes@email.com', '1989-08-08'),
(9, 'Sofia Costa', 'sofia.costa@email.com', '1991-12-24'),
(10, 'Rafael Silva', 'rafael.silva@email.com', '1997-04-18');
```

Data Output | Messages | Notifications

```
INSERT 0 10

Query returned successfully in 42 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

4a

Questão 4-a

Agora que as tabelas foram criadas, insira os dados dos clientes, quartos e estadias:

a) Insira **10 clientes** na **tabela clientes** e faça uma consulta para verificar se os dados foram inseridos corretamente:

id_cliente [PK] integer	nome character varying (100)	email character varying (50)	data_nascimento date
1	Ana Silva	ana.silva@email.com	1990-03-15
2	Carlos Souza	carlos.souza@email.com	1985-07-20
3	Maria Oliveira	maria.oliveira@email.com	1993-11-05
4	João Santos	joao.santos@email.com	1978-02-25
5	Beatriz Lima	beatriz.lima@email.com	2000-01-12
6	Lucas Alves	lucas.alves@email.com	[null]
7	Fernanda Rocha	fernanda.rocha@email.com	1995-06-10
8	Gabriel Mendes	gabriel.mendes@email.com	1989-08-08
9	Sofia Costa	sofia.costa@email.com	1991-12-24
10	Rafael Silva	rafael.silva@email.com	1997-04-18

Explicação e Resultado

Assim, ao consultarmos a **tabela clientes**, vemos que **as 10 linhas foram inseridas**:

```
SELECT * FROM clientes;
```

	id_cliente [PK] integer	nome character varying (100)	email character varying (50)	data_nascimento date
1	1	Ana Silva	ana.silva@email.com	1990-03-15
2	2	Carlos Souza	carlos.souza@email.com	1985-07-20
3	3	Maria Oliveira	maria.oliveira@email.com	1993-11-05
4	4	João Santos	joao.santos@email.com	1978-02-25
5	5	Beatriz Lima	beatriz.lima@email.com	2000-01-12
6	6	Lucas Alves	lucas.alves@email.com	[null]
7	7	Fernanda Rocha	fernanda.rocha@email.com	1995-06-10
8	8	Gabriel Mendes	gabriel.mendes@email.com	1989-08-08
9	9	Sofia Costa	sofia.costa@email.com	1991-12-24
10	10	Rafael Silva	rafael.silva@email.com	1997-04-18

Total rows: 10 of 10 Query complete 00:00:00.443

GABARITO EXPLICATIVO

4b

Questão 4-b

Agora que as tabelas foram criadas, insira os dados dos clientes, quartos e estadias:

b) Insira **10 quartos** na **tabela quartos** e faça uma consulta para verificar se os dados foram inseridos corretamente:

id_quarto [PK] integer	tipo character varying (30)	qtd_pessoas integer	preco_diaria numeric (10,2)
1	Single	1	150.00
2	Double	2	250.00
3	Suite	4	500.00
4	Single	1	140.00
5	Double	2	230.00
6	Suite	4	550.00
7	Single	1	160.00
8	Double	2	240.00
9	Suite	4	600.00
10	Single	1	170.00

Explicação e Resultado

Para inserir as **10 linhas** mostradas ao lado na **tabela quartos**, executamos o comando **INSERT INTO** conforme mostrado abaixo:

```
INSERT INTO quartos (id_quarto, tipo, qtd_pessoas, preco_diaria) VALUES
(1, 'Single', 1, 150.00),
(2, 'Double', 2, 250.00),
(3, 'Suite', 4, 500.00),
(4, 'Single', 1, 140.00),
(5, 'Double', 2, 230.00),
(6, 'Suite', 4, 550.00),
(7, 'Single', 1, 160.00),
(8, 'Double', 2, 240.00),
(9, 'Suite', 4, 600.00),
(10, 'Single', 1, 170.00);
```

Data Output | Messages | Notifications

```
INSERT 0 10

Query returned successfully in 106 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

4b

Questão 4-b

Agora que as tabelas foram criadas, insira os dados dos clientes, quartos e estadias:

b) Insira **10 quartos** na **tabela quartos** e faça uma consulta para verificar se os dados foram inseridos corretamente:

id_quarto [PK] integer	tipo character varying (30)	qtd_pessoas integer	preco_diaria numeric (10,2)
1	Single	1	150.00
2	Double	2	250.00
3	Suite	4	500.00
4	Single	1	140.00
5	Double	2	230.00
6	Suite	4	550.00
7	Single	1	160.00
8	Double	2	240.00
9	Suite	4	600.00
10	Single	1	170.00

Explicação e Resultado

Assim, ao consultarmos a **tabela quartos**, vemos que **as 10 linhas foram inseridas**:

```
SELECT * FROM quartos;
```

	id_quarto [PK] integer	tipo character varying (30)	qtd_pessoas integer	preco_diaria numeric (10,2)
1	1	Single	1	150.00
2	2	Double	2	250.00
3	3	Suite	4	500.00
4	4	Single	1	140.00
5	5	Double	2	230.00
6	6	Suite	4	550.00
7	7	Single	1	160.00
8	8	Double	2	240.00
9	9	Suite	4	600.00
10	10	Single	1	170.00

Total rows: 10 of 10 Query complete 00:00:00.082

GABARITO EXPLICATIVO

4c

Questão 4-c

Agora que as tabelas foram criadas, insira os dados dos clientes, quartos e estadias:

c) Insira **15 estadias** na **tabela estadias** e faça uma consulta para verificar se os dados foram inseridos corretamente:

id_estadia [PK] integer	id_cliente integer	id_quarto integer	data_entrada timestamp without time zone	data_saida timestamp without time zone
1	1	2	2024-01-15 14:00:00	2024-01-16 12:00:00
2	3	5	2024-02-16 16:00:00	2024-02-18 11:00:00
3	4	1	2024-03-17 13:00:00	2024-03-19 11:00:00
4	2	3	2024-04-18 15:00:00	2024-05-10 10:00:00
5	5	7	2024-05-19 12:30:00	2024-05-23 12:00:00
6	6	9	2024-06-20 18:00:00	2024-06-22 14:00:00
7	8	4	2024-07-21 11:00:00	2024-07-23 09:00:00
8	9	10	2024-08-22 14:00:00	2024-08-24 12:00:00
9	7	6	2024-09-23 10:00:00	2024-09-26 10:00:00
10	10	8	2024-10-24 16:00:00	2024-11-06 11:00:00
11	1	5	2024-11-25 13:00:00	2024-11-27 10:00:00
12	3	2	2024-12-26 17:00:00	2024-12-28 13:00:00
13	4	1	2024-12-27 12:00:00	[null]
14	8	7	2024-12-28 15:00:00	[null]
15	9	3	2024-12-29 14:00:00	[null]

Explicação e Resultado

Para inserir as **15 linhas** mostradas ao lado na **tabela estadias**, executamos o comando **INSERT INTO** conforme mostrado abaixo:

```
INSERT INTO estadias (id_estadia, id_cliente, id_quarto, data_entrada, data_saida) VALUES
(1, 1, 2, '2024-01-15 14:00:00', '2024-01-16 12:00:00'),
(2, 3, 5, '2024-02-16 16:00:00', '2024-02-18 11:00:00'),
(3, 4, 1, '2024-03-17 13:00:00', '2024-03-19 11:00:00'),
(4, 2, 3, '2024-04-18 15:00:00', '2024-05-10 10:00:00'),
(5, 5, 7, '2024-05-19 12:30:00', '2024-05-23 12:00:00'),
(6, 6, 9, '2024-06-20 18:00:00', '2024-06-22 14:00:00'),
(7, 8, 4, '2024-07-21 11:00:00', '2024-07-23 09:00:00'),
(8, 9, 10, '2024-08-22 14:00:00', '2024-08-24 12:00:00'),
(9, 7, 6, '2024-09-23 10:00:00', '2024-09-26 10:00:00'),
(10, 10, 8, '2024-10-24 16:00:00', '2024-11-06 11:00:00'),
(11, 1, 5, '2024-11-25 13:00:00', '2024-11-27 10:00:00'),
(12, 3, 2, '2024-12-26 17:00:00', '2024-12-28 13:00:00'),
(13, 4, 1, '2024-12-27 12:00:00', NULL),
(14, 8, 7, '2024-12-28 15:00:00', NULL),
(15, 9, 3, '2024-12-29 14:00:00', NULL);
```

Data Output | Messages | Notifications

```
INSERT 0 15

Query returned successfully in 93 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

4c

Questão 4-c

Agora que as tabelas foram criadas, insira os dados dos clientes, quartos e estadias:

c) Insira **15 estadias** na **tabela estadias** e faça uma consulta para verificar se os dados foram inseridos corretamente:

id_estadia [PK] integer	id_cliente integer	id_quarto integer	data_entrada timestamp without time zone	data_saida timestamp without time zone
1	1	2	2024-01-15 14:00:00	2024-01-16 12:00:00
2	3	5	2024-02-16 16:00:00	2024-02-18 11:00:00
3	4	1	2024-03-17 13:00:00	2024-03-19 11:00:00
4	2	3	2024-04-18 15:00:00	2024-05-10 10:00:00
5	5	7	2024-05-19 12:30:00	2024-05-23 12:00:00
6	6	9	2024-06-20 18:00:00	2024-06-22 14:00:00
7	8	4	2024-07-21 11:00:00	2024-07-23 09:00:00
8	9	10	2024-08-22 14:00:00	2024-08-24 12:00:00
9	7	6	2024-09-23 10:00:00	2024-09-26 10:00:00
10	10	8	2024-10-24 16:00:00	2024-11-06 11:00:00
11	1	5	2024-11-25 13:00:00	2024-11-27 10:00:00
12	3	2	2024-12-26 17:00:00	2024-12-28 13:00:00
13	4	1	2024-12-27 12:00:00	[null]
14	8	7	2024-12-28 15:00:00	[null]
15	9	3	2024-12-29 14:00:00	[null]

Explicação e Resultado

Assim, ao consultarmos a **tabela estadias**, vemos que **as 15 linhas foram inseridas**:

```
SELECT * FROM estadias;
```

	id_estadia [PK] integer	id_cliente integer	id_quarto integer	data_entrada timestamp without time zone	data_saida timestamp without time zone
1	1	1	2	2024-01-15 14:00:00	2024-01-16 12:00:00
2	2	3	5	2024-02-16 16:00:00	2024-02-18 11:00:00
3	3	4	1	2024-03-17 13:00:00	2024-03-19 11:00:00
4	4	2	3	2024-04-18 15:00:00	2024-05-10 10:00:00
5	5	5	7	2024-05-19 12:30:00	2024-05-23 12:00:00
6	6	6	9	2024-06-20 18:00:00	2024-06-22 14:00:00
7	7	8	4	2024-07-21 11:00:00	2024-07-23 09:00:00
8	8	9	10	2024-08-22 14:00:00	2024-08-24 12:00:00
9	9	7	6	2024-09-23 10:00:00	2024-09-26 10:00:00
10	10	10	8	2024-10-24 16:00:00	2024-11-06 11:00:00
11	11	1	5	2024-11-25 13:00:00	2024-11-27 10:00:00
12	12	3	2	2024-12-26 17:00:00	2024-12-28 13:00:00
13	13	4	1	2024-12-27 12:00:00	[null]
14	14	8	7	2024-12-28 15:00:00	[null]
15	15	9	3	2024-12-29 14:00:00	[null]

Total rows: 15 of 15 Query complete 00:00:00.076

PostgreSQL

GABARITO EXPLICATIVO

5

Questão 5

O cliente da estadia de id_estadia = 13 desocupou o quarto em 04/01/2025 ao meio-dia.

Atualize a tabela para fazer constar essa informação.

Explicação e Resultado

Para **atualizarmos a data em que o cliente da id_estadia = 13 desocupou o quarto (dia 04/01/2025 ao meio-dia)**, executamos o comando **UPDATE** abaixo, passando para ele o nome da tabela a ser atualizada (**estadias**), seguido do comando **SET** com a alteração a ser efetuada (**SET data_saida = '2025-01-04 12:00:00'**) e a condição que deve ser respeitada para que somente a linha referente à estadia de id_estadia = 13 receba a atualização (**WHERE id_estadia = 13**):

```
UPDATE estadias
SET data_saida = '2025-01-04 12:00:00'
WHERE id_estadia = 13;
```

Data Output | Messages | Notifications

```
UPDATE 1

Query returned successfully in 85 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

5

Questão 5

O cliente da estadia de id_estadia = 13 desocupou o quarto em 04/01/2025 ao meio-dia.

Atualize a tabela para fazer constar essa informação.

Explicação e Resultado

Dessa forma, ao **consultarmos a tabela estadias** novamente, vemos que **a alteração foi de fato efetuada**:

```
SELECT * FROM estadias;
```

	id_estadia [PK] integer	id_cliente integer	id_quarto integer	data_entrada timestamp without time zone	data_saida timestamp without time zone
1	1	1	2	2024-01-15 14:00:00	2024-01-16 12:00:00
2	2	3	5	2024-02-16 16:00:00	2024-02-18 11:00:00
3	3	4	1	2024-03-17 13:00:00	2024-03-19 11:00:00
4	4	2	3	2024-04-18 15:00:00	2024-05-10 10:00:00
5	5	5	7	2024-05-19 12:30:00	2024-05-23 12:00:00
6	6	6	9	2024-06-20 18:00:00	2024-06-22 14:00:00
7	7	8	4	2024-07-21 11:00:00	2024-07-23 09:00:00
8	8	9	10	2024-08-22 14:00:00	2024-08-24 12:00:00
9	9	7	6	2024-09-23 10:00:00	2024-09-26 10:00:00
10	10	10	8	2024-10-24 16:00:00	2024-11-06 11:00:00
11	11	1	5	2024-11-25 13:00:00	2024-11-27 10:00:00
12	12	3	2	2024-12-26 17:00:00	2024-12-28 13:00:00
13	14	8	7	2024-12-28 15:00:00	[null]
14	15	9	3	2024-12-29 14:00:00	[null]
15	13	4	1	2024-12-27 12:00:00	2025-01-04 12:00:00

Total rows: 15 of 15 Query complete 00:00:00.085

GABARITO EXPLICATIVO

6

Questão 6

A cliente Beatriz Lima (id_cliente = 5) solicitou a exclusão de seus dados de nosso banco de dados. Como esta cliente se hospedou no hotel anteriormente, ela está referenciada na tabela estadias. Portanto, é necessário seguir os seguintes passos para realizar a exclusão de seus dados:

a) Excluir a(s) estadia(s) que faz(em) referência a esta cliente na tabela estadias.

b) Após a exclusão das estadias, excluir a cliente Beatriz Lima (id_cliente = 5) da tabela clientes.

Explicação e Resultado

a) Para **excluirmos a(s) estadia(s)** que faz(em) referência à cliente **Beatriz Lima**, primeiro executamos o comando **DELETE** na **tabela estadias**, passando como **condição** o ID referente à essa cliente (**id_cliente = 5**):

```
DELETE FROM estadias
WHERE id_cliente = 5;
```

Data Output | Messages | Notifications

```
DELETE 1

Query returned successfully in 74 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

6

Questão 6

A cliente Beatriz Lima (id_cliente = 5) solicitou a exclusão de seus dados de nosso banco de dados. Como esta cliente se hospedou no hotel anteriormente, ela está referenciada na tabela estadias. Portanto, é necessário seguir os seguintes passos para realizar a exclusão de seus dados:

a) Excluir a(s) estadia(s) que faz(em) referência a esta cliente na tabela estadias.

b) Após a exclusão das estadias, excluir a cliente Beatriz Lima (id_cliente = 5) da tabela clientes.

Explicação e Resultado

Consultando a **tabela estadias** novamente, vemos que **a linha** que se referia à **única** estadia dessa cliente **foi excluída**, restando agora **14 linhas** na tabela (nenhuma delas se referindo à cliente id_cliente = 5):

```
SELECT * FROM estadias;
```

	id_estadia [PK] integer	id_cliente integer	id_quarto integer	data_entrada timestamp without time zone	data_saida timestamp without time zone
1	1	1	2	2024-01-15 14:00:00	2024-01-16 12:00:00
2	2	3	5	2024-02-16 16:00:00	2024-02-18 11:00:00
3	3	4	1	2024-03-17 13:00:00	2024-03-19 11:00:00
4	4	2	3	2024-04-18 15:00:00	2024-05-10 10:00:00
5	6	6	9	2024-06-20 18:00:00	2024-06-22 14:00:00
6	7	8	4	2024-07-21 11:00:00	2024-07-23 09:00:00
7	8	9	10	2024-08-22 14:00:00	2024-08-24 12:00:00
8	9	7	6	2024-09-23 10:00:00	2024-09-26 10:00:00
9	10	10	8	2024-10-24 16:00:00	2024-11-06 11:00:00
10	11	1	5	2024-11-25 13:00:00	2024-11-27 10:00:00
11	12	3	2	2024-12-26 17:00:00	2024-12-28 13:00:00
12	14	8	7	2024-12-28 15:00:00	[null]
13	15	9	3	2024-12-29 14:00:00	[null]
14	13	4	1	2024-12-27 12:00:00	2025-01-04 12:00:00

Total rows: 14 of 14 | Query complete 00:00:00.078

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

6

Questão 6

A cliente Beatriz Lima (id_cliente = 5) solicitou a exclusão de seus dados de nosso banco de dados. Como esta cliente se hospedou no hotel anteriormente, ela está referenciada na tabela estadias. Portanto, é necessário seguir os seguintes passos para realizar a exclusão de seus dados:

a) Excluir a(s) estadia(s) que faz(em) referência a esta cliente na tabela estadias.

b) Após a exclusão das estadias, excluir a cliente Beatriz Lima (id_cliente = 5) da tabela clientes.

Explicação e Resultado

b) Agora, para **excluirmos** a cliente **Beatriz Lima** da tabela clientes, executamos o comando **DELETE** na **tabela clientes**, passando como **condição** o ID referente à essa cliente (**id_cliente = 5**):

```
DELETE FROM clientes
WHERE id_cliente = 5;
```

Data Output | Messages | Notifications

```
DELETE 1

Query returned successfully in 86 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

6

Questão 6

A cliente Beatriz Lima (id_cliente = 5) solicitou a exclusão de seus dados de nosso banco de dados. Como esta cliente se hospedou no hotel anteriormente, ela está referenciada na tabela estadias. Portanto, é necessário seguir os seguintes passos para realizar a exclusão de seus dados:

a) Excluir a(s) estadia(s) que faz(em) referência a esta cliente na tabela estadias.

b) Após a exclusão das estadias, excluir a cliente Beatriz Lima (id_cliente = 5) da tabela clientes.

Explicação e Resultado

Consultando a **tabela clientes** novamente, vemos que **a linha** que se referia a essa cliente foi **excluída**, restando somente **9 linhas** na tabela (nenhuma delas se referindo à cliente Beatriz Lima, de id_cliente = 5):

```
SELECT * FROM clientes;
```

	id_cliente [PK] integer	nome character varying (100)	email character varying (50)	data_nascimento date
1	1	Ana Silva	ana.silva@email.com	1990-03-15
2	2	Carlos Souza	carlos.souza@email.com	1985-07-20
3	3	Maria Oliveira	maria.oliveira@email.com	1993-11-05
4	4	João Santos	joao.santos@email.com	1978-02-25
5	6	Lucas Alves	lucas.alves@email.com	[null]
6	7	Fernanda Rocha	fernanda.rocha@email.com	1995-06-10
7	8	Gabriel Mendes	gabriel.mendes@email.com	1989-08-08
8	9	Sofia Costa	sofia.costa@email.com	1991-12-24
9	10	Rafael Silva	rafael.silva@email.com	1997-04-18

Total rows: 9 of 9 | Query complete 00:00:00.065

GABARITO EXPLICATIVO

7

Questão 7

A gerência do Hotel Hashtag decidiu iniciar um novo ano fiscal reorganizando o sistema e limpando os registros antigos de estadias. Isso permitirá começar o ano com dados atualizados e espaço no banco de dados para novos registros. O backup dos registros do ano anterior já foi efetuado para que os dados não sejam perdidos.

Agora, você já pode esvaziar todos os registros da tabela estadias, garantindo que a estrutura da tabela seja mantida.

Explicação e Resultado

Para **apagar todos os registros da tabela estadias**, porém **mantendo sua estrutura** para a inserção de novos registros, utilizamos o comando **TRUNCATE TABLE**:

```
TRUNCATE TABLE estadias;
```

Data Output | Messages | Notifications

```
TRUNCATE TABLE

Query returned successfully in 74 msec.
```

Dessa forma, ao **consultarmos novamente a tabela estadias**, vemos que **todos os registros foram excluídos**, porém, **a tabela ainda existe**:

```
SELECT * FROM estadias;
```

	id_estadia [PK] integer	id_cliente integer	id_quarto integer	data_entrada timestamp without time zone	data_saida timestamp without time zone

Total rows: 0 of 0 | Query complete 00:00:00.063

GABARITO EXPLICATIVO

8

Questão 8

Após uma reestruturação no sistema de TI, o Hotel Hashtag contratou um novo software que substituirá o atual banco de dados. Todos os dados do dbHotelHashtag já foram migrados para esse novo software.

Agora, como parte da transição, as tabelas clientes, quartos e estadias precisam ser excluídas completamente, removendo qualquer dependência existente. Realize essa exclusão.

Explicação e Resultado

Para **excluirmos completamente as tabelas clientes, quartos e estadias, removendo qualquer dependência existente**, devemos aplicar um **DROP TABLE** em cada tabela acrescido do comando **CASCADE**:

```
DROP TABLE clientes CASCADE;
DROP TABLE quartos CASCADE;
DROP TABLE estadias CASCADE;
```

Data Output | Messages | Notifications

```
NOTICE:  drop cascades to constraint estadias_id_cliente_fkey on table estadias
NOTICE:  drop cascades to constraint estadias_id_quarto_fkey on table estadias
DROP TABLE

Query returned successfully in 131 msec.
```

MÓDULO 8

FUNÇÕES DE NÚMERO, TEXTO E DATA

FUNÇÕES DE NÚMERO, TEXTO E DATA

FUNÇÕES DE NÚMERO, TEXTO E DATA

PostgreSQL

O QUE VEREMOS NESTE MÓDULO

MÓDULO 8 FUNÇÕES DE NÚMERO, TEXTO E DATA

01	Funções de Número - Ceiling, Floor, Round, Trunc	297
02	Funções de Texto - Upper, Lower, Length, Initcap	301
03	Funções de Texto - Replace	303
04	Funções de Texto - Substring e Strpos	305
05	Funções de Data - Current_Date, Age, Date_Part	310
06	EXERCÍCIOS	314
07	GABARITO EXPLICATIVO	319

Funções de Número: CEILING, FLOOR, ROUND e TRUNC

Vamos supor que queremos descobrir a média do valor unitário dos produtos da tabela products do banco de dados Northwind.

Para isso, podemos executar o seguinte código...

```
SELECT
    AVG(unit_price)
FROM products;
```

... que nos retornará o resultado ao lado:

	avg double precision
1	28.83389609200614

Repare que o resultado retornou com muitas casas decimais. E se quisermos arredondar para que apareçam menos casas decimais, como fazemos?

Para fazer esse arredondamento de valores, existem algumas funções muito úteis:
CEILING, FLOOR, ROUND e TRUNC.

Vejamos cada uma delas.

Funções de Número: CEILING, FLOOR, ROUND e TRUNC

Vamos começar com as funções CEILING e FLOOR:

CEILING: arredonda um valor para cima, ou seja, para um número inteiro logo acima.

FLOOR: arredonda um valor para baixo, ou seja, para um número inteiro logo abaixo.

Portanto, se a função AVG(unit_price) retornou o valor 28.83389609200614, a função CEILING arredondará esse número para 29, enquanto que a função FLOOR o arredondará para 28.

Veja:

```
SELECT
    AVG(unit_price),
    CEILING(AVG(unit_price)),
    FLOOR(AVG(unit_price))
FROM products;
```

	avg double precision	ceiling double precision	floor double precision
1	28.83389609200614	29	28

Funções de Número: CEILING, FLOOR, ROUND e TRUNC

Vejamos agora as funções ROUND e TRUNC:

ROUND: arredonda um valor para a quantidade de casas decimais informada em seu segundo parâmetro.
TRUNC: trunca (corta) um valor na casa decimal informada em seu segundo parâmetro.

Portanto, utilizando novamente o resultado retornado pela função AVG(unit_price), 28.83389609200614, se informarmos para a ROUND que queremos que ela arredonde na 3ª casa decimal, ela retornará o valor 28.834, pois ela arredondará a terceira casa decimal (o número 3) de acordo com o valor que estiver na casa decimal imediatamente posterior: se tiver um número de 0-4, ela retornará o número que estiver na última casa decimal que deverá ser retornada, sem arredondar; se tiver de 5-9, ela arredondará para cima (como fez neste caso, pois na casa imediatamente posterior à 3ª casa decimal tínhamos o número 8).

Já a função TRUNC, se informarmos para ela que queremos que ela trunque na terceira casa decimal, ela simplesmente cortará o número nessa casa, sem arredondar, independentemente do valor que se apresente na casa imediatamente posterior.

Veja:

```
SELECT
    AVG(unit_price),
    ROUND(CAST(AVG(unit_price) AS NUMERIC), 3),
    TRUNC(CAST(AVG(unit_price) AS NUMERIC), 3)
FROM products;
```

	avg double precision	round numeric	trunc numeric
1	28.83389609200614	28.834	28.833

Funções de Número: CEILING, FLOOR, ROUND e TRUNC

Observação importante: no PostgreSQL, é necessário utilizar a função CAST junto com as funções ROUND e TRUNC, para converter o valor no qual queremos aplicar o arredondamento / truncamento para NUMERIC.

Sem isso, o nosso código retornará um erro, pois essa obrigatoriedade faz parte da sintaxe do SGBD:

```
SELECT
    AVG(unit_price),
    ROUND(CAST(AVG(unit_price) AS NUMERIC), 3),
    TRUNC(CAST(AVG(unit_price) AS NUMERIC), 3)
FROM products;
```

Funções de Texto: UPPER, LOWER, LENGTH e INITCAP

Vamos agora trabalhar com funções de textos.

Para isso, utilizaremos a coluna first_name da tabela Employees:

```
SELECT
    first_name
FROM Employees;
```

	first_name character varying (10)
1	Nancy
2	Andrew
3	Janet
4	Margaret
5	Steven
6	Michael
7	Robert
8	Laura
9	Anne

Funções de Texto: UPPER, LOWER, LENGTH e INITCAP

Ao aplicarmos a função UPPER à coluna first_name, ela transforma todas as letras dos nomes em maiúsculas;

Já a função LOWER faz o contrário: transforma todas as letras dos nomes em minúsculas;

A função LENGTH, por sua vez, retorna a quantidade de caracteres de cada nome dessa coluna;

Por fim, a função INITCAP transforma as iniciais de cada palavra em maiúsculas e as demais letras em minúsculas. Como não daria para visualizarmos muito bem o retorno da INITCAP com a coluna first_name, a aplicamos no texto 'sql impressionador'.

Veja:

```
SELECT
    first_name,
    UPPER(first_name),
    LOWER(first_name),
    LENGTH(first_name),
    INITCAP('sql impressionador')
FROM Employees;
```

	first_name character varying (10)	upper text	lower text	length integer	initcap text
1	Nancy	NANCY	nancy	5	Sql Impressionador
2	Andrew	ANDREW	andrew	6	Sql Impressionador
3	Janet	JANET	janet	5	Sql Impressionador
4	Margaret	MARGARET	margaret	8	Sql Impressionador
5	Steven	STEVEN	steven	6	Sql Impressionador
6	Michael	MICHAEL	michael	7	Sql Impressionador
7	Robert	ROBERT	robert	6	Sql Impressionador
8	Laura	LAURA	laura	5	Sql Impressionador
9	Anne	ANNE	anne	4	Sql Impressionador

PostgreSQL

Funções de Texto: REPLACE

Vejamos agora a função de texto REPLACE.

Para isso, utilizaremos a coluna contact_name da tabela customers:

```
SELECT
    contact_title
FROM customers;
```

	contact_title character varying (30)
1	Sales Representative
2	Owner
3	Owner
4	Sales Representative
5	Order Administrator
6	Sales Representative
7	Marketing Manager
8	Owner

Funções de Texto: REPLACE

Nesta coluna, temos as profissões de cada cliente.

Vamos imaginar que, em vez de retornar a palavra “Owner” no nosso resultado, preferimos que seja retornada a sigla “CEO”. Para isso, podemos utilizar a função REPLACE.

Devemos informar três parâmetros para esta função: a coluna na qual queremos aplicá-la (contact_title), o termo que queremos alterar (Owner), e qual o termo que deverá aparecer no lugar (CEO):

```
SELECT
    contact_name,
    contact_title,
    REPLACE(contact_title, 'Owner', 'CEO')
FROM customers;
```

	contact_name character varying (30)	contact_title character varying (30)	replace text
1	Maria Anders	Sales Representative	Sales Representative
2	Ana Trujillo	Owner	CEO
3	Antonio Moreno	Owner	CEO
4	Thomas Hardy	Sales Representative	Sales Representative
5	Christina Berglund	Order Administrator	Order Administrator
6	Hanna Moos	Sales Representative	Sales Representative
7	Frédérique Citeaux	Marketing Manager	Marketing Manager
8	Martín Sommer	Owner	CEO

Funções de Texto: SUBSTRING e STRPOS

Vamos analisar agora as funções de texto SUBSTRING e STRPOS.

Comecemos com a função SUBSTRING. Essa função serve para retornar parte de um texto (string).

Vamos imaginar que temos o número da placa de um carro, 'ABC-9999', e queremos dividir essa placa em duas partes: ABC e 9999.

Com a SUBSTRING, conseguimos fazer isso, se informarmos para ela três parâmetros: o texto do qual queremos extrair uma parte (ABC-9999), a partir de qual caractere queremos iniciar a extração, e quantos caracteres desejamos extrair a partir do caractere inicial.

Nosso código ficaria, portanto assim:

```
SELECT
    'ABC-9999',
    SUBSTRING('ABC-9999', 1, 3),
    SUBSTRING('ABC-9999', 5, 4);
```

	?column? text	substring text	substring text
1	ABC-9999	ABC	9999

PostgreSQL

Funções de Texto: SUBSTRING e STRPOS

Repare que, para extrair o trecho 'ABC', informamos a posição 1 (2º parâmetro), e que gostaríamos de extrair 3 caracteres a partir dessa posição 1 (3º parâmetro);

Já para extrair o trecho '9999', informamos a posição 5 (2º parâmetro), e que gostaríamos de extrair 4 caracteres a partir dessa posição 5 (3º parâmetro):

```
SELECT
    'ABC-9999',
    SUBSTRING('ABC-9999', 1, 3),
    SUBSTRING('ABC-9999', 5, 4);
```

Funções de Texto: SUBSTRING e STRPOS

Agora vamos entender a utilidade da função de texto STRPOS.

Essa função serve para retornar a posição de um determinado caractere dentro de um texto (string).

Voltando ao número da placa do carro, 'ABC-9999': e se quisermos descobrir, por exemplo, a posição do caractere '-'?

Podemos fazer assim:

```
SELECT
    'ABC-9999',
    STRPOS('ABC-9999', '-');
```

	?column? text	strpos integer
1	ABC-9999	4

Veja que, passando o texto como 1º parâmetro e o caractere do qual queremos descobrir a posição como 2º parâmetro, conseguimos descobrir que a posição do caractere '-' é a posição 4.

Funções de Texto: SUBSTRING e STRPOS

Podemos também trabalhar com as funções SUBSTRING e STRPOS em conjunto para automatizar a extração de trechos de strings.

Voltando ao exemplo da placa do carro 'ABC-9999', em vez de informar manualmente para a SUBSTRING a partir de qual posição queremos iniciar ou a quantidade de caracteres que queremos extrair, podemos utilizar a STRPOS para isso.

Veja só:

```
SELECT
    'ABC-9999',
    SUBSTRING('ABC-9999', 1, STRPOS('ABC-9999', '-') -1),
    SUBSTRING('ABC-9999', STRPOS('ABC-9999', '-') +1, 100),
    STRPOS('ABC-9999', '-');
```

	?column? text	substring text	substring text	strpos integer
1	ABC-9999	ABC	9999	4

Funções de Texto: SUBSTRING e STRPOS

Vamos entender melhor esse código:

Primeiro: para extrair o trecho inicial (ABC), informamos que queríamos extrair o texto que se iniciasse na posição 1 (2º parâmetro da SUBSTRING), e que fosse até a posição anterior à posição do caractere '-' (STRPOS('ABC-9999', '-') – 1) (3º parâmetro da SUBSTRING);

Já para extrair o trecho final (9999), informamos que queríamos extrair um texto que se iniciasse na posição imediatamente posterior à posição do caractere '-' (STRPOS('ABC-9999', '-') + 1) (2º parâmetro da SUBSTRING), e que fosse até o final da string (100) (3º parâmetro da SUBSTRING). Aqui, sabendo que queremos retornar apenas 4 caracteres (9999), até poderíamos informar o número 4 como 3º parâmetro, porém, optamos por informar o valor 100 para ilustrar essa possibilidade, pois poderíamos aplicar esse código, por exemplo, à coluna de uma tabela que tivesse inúmeros campos com strings como a placa do carro (com trechos separados por hífen), porém de tamanhos variados. Em um caso assim, se informarmos um valor "alto" como 3º parâmetro da SUBSTRING, conseguimos extrair todo o trecho final dos campos dessa eventual coluna sem incorrer no risco de deixar algum caractere de fora do resultado.

```
SELECT
    'ABC-9999',
    SUBSTRING('ABC-9999', 1, STRPOS('ABC-9999', '-') -1),
    SUBSTRING('ABC-9999', STRPOS('ABC-9999', '-') +1, 100),
    STRPOS('ABC-9999', '-');
```

	?column? text	substring text	substring text	strpos integer
1	ABC-9999	ABC	9999	4

PostgreSQL

Funções de Data: CURRENT_DATE, AGE e DATE_PART

Para fechar este módulo, vejamos algumas funções de data: CURRENT_DATE, AGE e DATE_PART.

Para isso, vamos utilizar a coluna birth_date da tabela Employees:

```
SELECT
    first_name,
    birth_date
FROM Employees;
```

	first_name character varying (10)	birth_date date
1	Nancy	1948-12-08
2	Andrew	1952-02-19
3	Janet	1963-08-30
4	Margaret	1937-09-19
5	Steven	1955-03-04
6	Michael	1963-07-02
7	Robert	1960-05-29
8	Laura	1958-01-09
9	Anne	1966-01-27

Funções de Data: CURRENT_DATE, AGE e DATE_PART

A função CURRENT_DATE retorna a data atual:

```
SELECT
    first_name,
    birth_date,
    CURRENT_DATE,
    AGE(birth_date),
    DATE_PART('DAY', birth_date),
    DATE_PART('MONTH', birth_date),
    DATE_PART('YEAR', birth_date)
FROM Employees;
```

	first_name character varying (10)	birth_date date	current_date date	age interval	date_part double precision	date_part double precision	date_part double precision
1	Nancy	1948-12-08	2023-05-17	74 years 5 mons 9 days	8	12	1948
2	Andrew	1952-02-19	2023-05-17	71 years 2 mons 27 days	19	2	1952
3	Janet	1963-08-30	2023-05-17	59 years 8 mons 18 days	30	8	1963
4	Margaret	1937-09-19	2023-05-17	85 years 7 mons 28 days	19	9	1937
5	Steven	1955-03-04	2023-05-17	68 years 2 mons 13 days	4	3	1955
6	Michael	1963-07-02	2023-05-17	59 years 10 mons 15 days	2	7	1963
7	Robert	1960-05-29	2023-05-17	62 years 11 mons 19 days	29	5	1960
8	Laura	1958-01-09	2023-05-17	65 years 4 mons 8 days	9	1	1958
9	Anne	1966-01-27	2023-05-17	57 years 3 mons 21 days	27	1	1966

Funções de Data: CURRENT_DATE, AGE e DATE_PART

A função AGE retorna o tempo decorrido desde a data informada até a data atual.
No caso do exemplo, retorna a idade dos funcionários, considerando a data constante na coluna birth_date:

```sql
SELECT
    first_name,
    birth_date,
    CURRENT_DATE,
    AGE(birth_date),
    DATE_PART('DAY', birth_date),
    DATE_PART('MONTH', birth_date),
    DATE_PART('YEAR', birth_date)
FROM Employees;
```

	first_name character varying (10)	birth_date date	current_date date	age interval	date_part double precision	date_part double precision	date_part double precision
1	Nancy	1948-12-08	2023-05-17	74 years 5 mons 9 days	8	12	1948
2	Andrew	1952-02-19	2023-05-17	71 years 2 mons 27 days	19	2	1952
3	Janet	1963-08-30	2023-05-17	59 years 8 mons 18 days	30	8	1963
4	Margaret	1937-09-19	2023-05-17	85 years 7 mons 28 days	19	9	1937
5	Steven	1955-03-04	2023-05-17	68 years 2 mons 13 days	4	3	1955
6	Michael	1963-07-02	2023-05-17	59 years 10 mons 15 days	2	7	1963
7	Robert	1960-05-29	2023-05-17	62 years 11 mons 19 days	29	5	1960
8	Laura	1958-01-09	2023-05-17	65 years 4 mons 8 days	9	1	1958
9	Anne	1966-01-27	2023-05-17	57 years 3 mons 21 days	27	1	1966

Funções de Data: CURRENT_DATE, AGE e DATE_PART

Já a função DATE_PART retorna o que informarmos para ela no 1º parâmetro, em relação à data informada no 2º parâmetro.
No exemplo, retornamos o dia (DAY), o mês (MONTH) e o ano (YEAR) da coluna birth_date da tabela Employees:

```sql
SELECT
    first_name,
    birth_date,
    CURRENT_DATE,
    AGE(birth_date),
    DATE_PART('DAY', birth_date),
    DATE_PART('MONTH', birth_date),
    DATE_PART('YEAR', birth_date)
FROM Employees;
```

	first_name character varying (10)	birth_date date	current_date date	age interval	date_part double precision	date_part double precision	date_part double precision
1	Nancy	1948-12-08	2023-05-17	74 years 5 mons 9 days	8	12	1948
2	Andrew	1952-02-19	2023-05-17	71 years 2 mons 27 days	19	2	1952
3	Janet	1963-08-30	2023-05-17	59 years 8 mons 18 days	30	8	1963
4	Margaret	1937-09-19	2023-05-17	85 years 7 mons 28 days	19	9	1937
5	Steven	1955-03-04	2023-05-17	68 years 2 mons 13 days	4	3	1955
6	Michael	1963-07-02	2023-05-17	59 years 10 mons 15 days	2	7	1963
7	Robert	1960-05-29	2023-05-17	62 years 11 mons 19 days	29	5	1960
8	Laura	1958-01-09	2023-05-17	65 years 4 mons 8 days	9	1	1958
9	Anne	1966-01-27	2023-05-17	57 years 3 mons 21 days	27	1	1966

EXERCÍCIOS

1

Questão 1

O setor financeiro está revisando os custos logísticos da empresa para identificar oportunidades de redução de despesas. Como parte dessa análise, é necessário calcular o valor médio dos fretes para compor um relatório de auditoria que será enviado à diretoria.

Calcule o valor médio de frete e arredonde para duas casas decimais. **(Utilize a tabela orders)**

2

Questão 2

A equipe de vendas está preparando a integração com uma nova plataforma de e-commerce que exige que todas as descrições de categorias estejam padronizadas, com apenas a primeira letra de cada palavra em maiúscula. Além disso, há um limite de caracteres por descrição, que precisa ser respeitado para evitar problemas de visualização no site.

Mostre as categorias de produtos, suas descrições com a formatação padronizada (primeira letra de cada palavra em maiúscula) e o número total de caracteres de cada descrição. **(Utilize a tabela categories)**

EXERCÍCIOS

3

Questão 3

O departamento de Recursos Humanos está atualizando os registros para tornar os cargos dos funcionários mais inclusivos e alinhados com as diretrizes globais da empresa. Um dos ajustes inclui substituir o termo "Sales Representative" por "Salesperson" nos registros dos colaboradores.

Liste os IDs, os nomes e sobrenomes, bem como os cargos dos funcionários, efetuando a substituição solicitada. **(Utilize a tabela employees)**

4

Questão 4

A equipe de marketing está planejando uma campanha personalizada de e-mails, onde será necessário utilizar os nomes dos contatos dos clientes em diferentes formatos. Isso permitirá criar mensagens com maior impacto e personalização, aumentando as chances de engajamento.

Liste os contatos dos clientes, uma vez em letras maiúsculas e outra em letras minúsculas. **(Utilize a tabela customers)**

EXERCÍCIOS

5

Questão 5

A equipe de TI foi acionada para implementar um sistema que identifique rapidamente as regiões de entrega dos clientes, baseando-se nos dois primeiros caracteres do código postal. Essa informação ajudará a logística a planejar rotas de forma mais eficiente.

Extraia os dois primeiros caracteres do campo postal_code dos clientes. **(Utilize a tabela customers)**

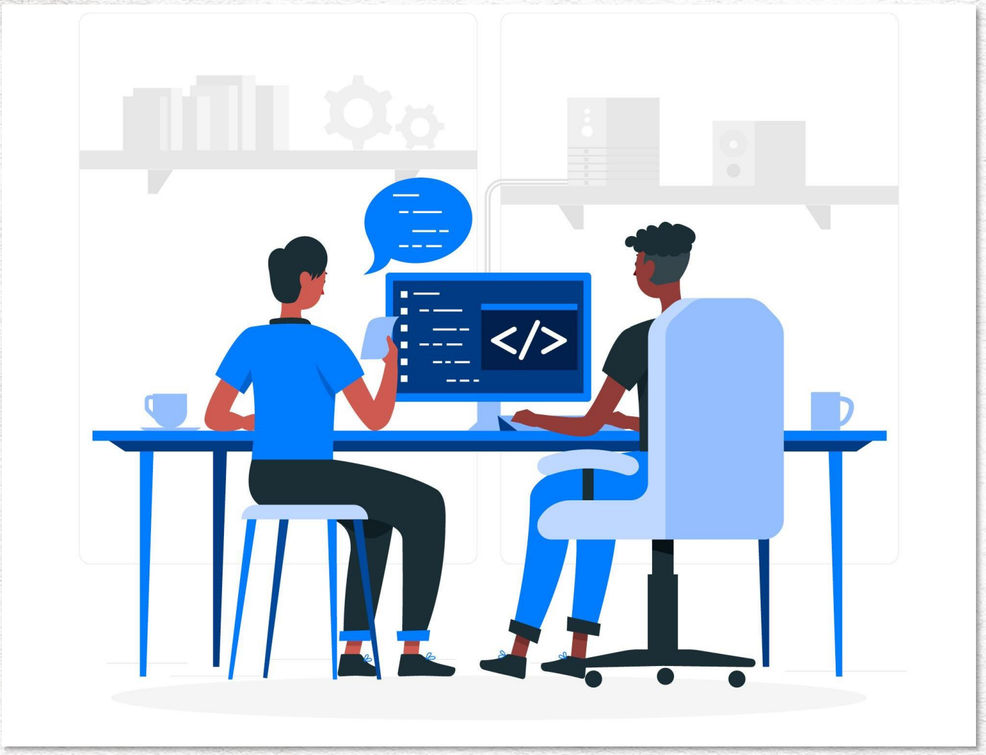
6

Questão 6

O setor financeiro precisa calcular os valores totais das ordens para fins de faturamento. Como parte dessa tarefa, é necessário que os valores sejam sempre arredondados para cima, garantindo que nenhuma fração de centavo seja perdida no processo de cobrança.

Exiba o valor total de cada ordem, calculado pela soma de unit_price * quantity, arredondado para cima. **(Utilize a tabela order_details)**

EXERCÍCIOS



7

Questão 7

O time de marketing está organizando uma campanha especial para promover produtos orgânicos. Para isso, é necessário localizar os produtos que possuem o termo "Organic" no nome, permitindo que a equipe foque seus esforços exclusivamente nesses itens.

Localize a posição do termo "Organic" nos nomes dos produtos, retornando somente os produtos cujo termo seja encontrado. **(Utilize a tabela products)**

8

Questão 8

Para criar um ambiente de trabalho mais acolhedor, o setor de Recursos Humanos está organizando uma campanha para celebrar os aniversariantes do mês. A identificação dos aniversariantes permitirá a criação de mensagens personalizadas e a inclusão desses dados no mural de aniversários da empresa.

Crie uma consulta que exiba os IDs, nomes, sobrenomes, datas de nascimento e a idade dos funcionários que fazem aniversário no mês corrente. Além disso, inclua a data atual na consulta para referência. **(Utilize a tabela employees)**

EXERCÍCIOS

9

Questão 9

O setor de estoque precisa calcular o número máximo de pacotes completos que podem ser formados com os produtos disponíveis. Cada pacote deve conter exatamente 10 unidades de um mesmo produto, e é necessário considerar que não é possível formar pacotes incompletos com as unidades restantes.

Liste os nomes dos produtos, suas quantidades em estoque e o número máximo de pacotes completos que podem ser formados, levando em consideração a quantidade disponível de cada produto. **(Utilize a tabela products)**

10

Questão 10

O setor financeiro está preparando um relatório detalhado sobre os descontos aplicados aos pedidos, que será utilizado em uma reunião estratégica com a diretoria. Para maior precisão, é necessário apresentar a média dos descontos truncada para três casas decimais, sem arredondamentos, garantindo a exatidão dos dados analisados.

Crie uma consulta que exiba a média dos valores de desconto em todos os pedidos, mostrando o valor original e o valor com três casas decimais sem arredondamentos. **(Utilize a tabela order_details)**

GABARITO EXPLICATIVO

1

Questão 1

O setor financeiro está revisando os custos logísticos da empresa para identificar oportunidades de redução de despesas. Como parte dessa análise, é necessário calcular o valor médio dos fretes para compor um relatório de auditoria que será enviado à diretoria.

Calcule o valor médio de frete e arredonde para duas casas decimais. **(Utilize a tabela orders)**

Explicação e Resultado

Para calcular o valor médio de frete e arredondar para duas casas decimais, fazemos o seguinte:

Usamos **AVG(freight)** para calcular a média dos valores na coluna **freight** da tabela **ORDERS**.

Utilizamos a função **CAST(... AS NUMERIC)** para converter o resultado da média para o tipo **NUMERIC**, sendo esta conversão obrigatória pela sintaxe usada no PostgreSQL.

Aplicamos a função **ROUND(..., 2)** ao valor convertido para arredondá-lo para duas casas decimais.

Por fim, atribuímos o alias **media_frete** à coluna calculada para facilitar a leitura do resultado:

```sql
SELECT
    ROUND(CAST(AVG(freight) AS NUMERIC), 2) AS media_frete
FROM orders;
```

Dessa forma, a consulta retorna uma única linha contendo o valor médio dos fretes arredondado:

	media_frete numeric
1	78.24

GABARITO EXPLICATIVO

2

Questão 2

A equipe de vendas está preparando a integração com uma nova plataforma de e-commerce que exige que todas as descrições de categorias estejam padronizadas, com apenas a primeira letra de cada palavra em maiúscula. Além disso, há um limite de caracteres por descrição, que precisa ser respeitado para evitar problemas de visualização no site.

Mostre as categorias de produtos, suas descrições com a formatação padronizada (primeira letra de cada palavra em maiúscula) e o número total de caracteres de cada descrição. **(Utilize a tabela categories)**

Explicação e Resultado

Para atender à necessidade de padronizar descrições e contabilizar o número total de caracteres, seguimos os passos abaixo:

Da tabela **CATEGORIES**, selecionamos a coluna **category_name** para exibir os nomes das categorias.

Usamos a função **INITCAP(description)** para formatar as descrições com a primeira letra de cada palavra em maiúscula, atribuindo o alias **descricao_formatada**.

Aplicamos a função **LENGTH(description)** para calcular a quantidade total de caracteres em cada descrição, atribuindo o alias **qtd_caracteres_descricao**.

```sql
SELECT
    category_name,
    INITCAP(description) AS descricao_formatada,
    LENGTH(description) AS qtd_caracteres_descricao
FROM categories;
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

2

Questão 2

A equipe de vendas está preparando a integração com uma nova plataforma de e-commerce que exige que todas as descrições de categorias estejam padronizadas, com apenas a primeira letra de cada palavra em maiúscula. Além disso, há um limite de caracteres por descrição, que precisa ser respeitado para evitar problemas de visualização no site.

Mostre as categorias de produtos, suas descrições com a formatação padronizada (primeira letra de cada palavra em maiúscula) e o número total de caracteres de cada descrição. **(Utilize a tabela categories)**

Explicação e Resultado

Dessa forma, obtemos o seguinte resultado:

	category_name character varying (15)	descricao_formatada text	qtd_caracteres_descricao integer
1	Beverages	Soft Drinks, Coffees, Teas, Beers, And Ales	43
2	Condiments	Sweet And Savory Sauces, Relishes, Spreads, And Seasonings	58
3	Confections	Desserts, Candies, And Sweet Breads	35
4	Dairy Products	Cheeses	7
5	Grains/Cereals	Breads, Crackers, Pasta, And Cereal	35
6	Meat/Poultry	Prepared Meats	14
7	Produce	Dried Fruit And Bean Curd	25
8	Seafood	Seaweed And Fish	16

Total rows: 8 of 8 Query complete 00:00:00.108

GABARITO EXPLICATIVO

3

Questão 3

O departamento de Recursos Humanos está atualizando os registros para tornar os cargos dos funcionários mais inclusivos e alinhados com as diretrizes globais da empresa. Um dos ajustes inclui substituir o termo "Sales Representative" por "Salesperson" nos registros dos colaboradores.

Liste os IDs, os nomes e sobrenomes, bem como os cargos dos funcionários, efetuando a substituição solicitada. **(Utilize a tabela employees)**

Explicação e Resultado

Para atualizar os registros de cargos dos funcionários, seguimos os passos abaixo:

Selecionamos as colunas **employee_id**, **first_name** e **last_name** da tabela **EMPLOYEES** para exibir os IDs e nomes e sobrenomes dos funcionários.

Incluímos a coluna **title** para mostrar o cargo original.

Usamos a função **REPLACE(title, 'Sales Representative', 'Salesperson')** para substituir "Sales Representative" por "Salesperson", atribuindo o alias **titulo_alterado**:

```sql
SELECT
    employee_id,
    first_name,
    last_name,
    title,
    REPLACE(title, 'Sales Representative', 'Salesperson') AS titulo_alterado
FROM employees;
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

3

Questão 3

O departamento de Recursos Humanos está atualizando os registros para tornar os cargos dos funcionários mais inclusivos e alinhados com as diretrizes globais da empresa. Um dos ajustes inclui substituir o termo "Sales Representative" por "Salesperson" nos registros dos colaboradores.

Liste os IDs, os nomes e sobrenomes, bem como os cargos dos funcionários, efetuando a substituição solicitada. **(Utilize a tabela employees)**

Explicação e Resultado

Dessa forma, garantimos que os registros reflitam a nova nomenclatura conforme as diretrizes globais da empresa:

	employee_id [PK] smallint	first_name character varying (10)	last_name character varying (20)	title character varying (30)	titulo_alterado text
1	1	Nancy	Davolio	Sales Representative	Salesperson
2	2	Andrew	Fuller	Vice President, Sales	Vice President, Sales
3	3	Janet	Leverling	Sales Representative	Salesperson
4	4	Margaret	Peacock	Sales Representative	Salesperson
5	5	Steven	Buchanan	Sales Manager	Sales Manager
6	6	Michael	Suyama	Sales Representative	Salesperson
7	7	Robert	King	Sales Representative	Salesperson
8	8	Laura	Callahan	Inside Sales Coordinator	Inside Sales Coordinator
9	9	Anne	Dodsworth	Sales Representative	Salesperson

Total rows: 9 of 9 Query complete 00:00:00.267

GABARITO EXPLICATIVO

4

Questão 4

A equipe de marketing está planejando uma campanha personalizada de e-mails, onde será necessário utilizar os nomes dos contatos dos clientes em diferentes formatos. Isso permitirá criar mensagens com maior impacto e personalização, aumentando as chances de engajamento.

Liste os contatos dos clientes, uma vez em letras maiúsculas e outra em letras minúsculas. **(Utilize a tabela customers)**

Explicação e Resultado

Para formatar os nomes dos contatos dos clientes em letras maiúsculas e minúsculas, seguimos os passos abaixo:

Selecionamos a coluna **contact_name** da tabela **CUSTOMERS** para exibir o nome original do contato (opcional).

Utilizamos a função **UPPER(contact_name)** para converter o nome para letras maiúsculas, atribuindo o alias **nome_maiusculo**.

Aplicamos a função **LOWER(contact_name)** para exibir o nome em letras minúsculas, com o alias **nome_minusculo**:

```sql
SELECT
    contact_name,
    UPPER(contact_name) AS nome_maiusculo,
    LOWER(contact_name) AS nome_minusculo
FROM customers;
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

4

Questão 4

A equipe de marketing está planejando uma campanha personalizada de e-mails, onde será necessário utilizar os nomes dos contatos dos clientes em diferentes formatos. Isso permitirá criar mensagens com maior impacto e personalização, aumentando as chances de engajamento.

Liste os contatos dos clientes, uma vez em letras maiúsculas e outra em letras minúsculas. **(Utilize a tabela customers)**

Explicação e Resultado

Dessa forma, retornamos os 91 contatos dos clientes em diferentes formatos que podem ser usamos pela equipe de marketing para personalizar os e-mails:

	contact_name character varying (30)	nome_maiusculo text	nome_minusculo text
1	Maria Anders	MARIA ANDERS	maria anders
2	Ana Trujillo	ANA TRUJILLO	ana trujillo
3	Antonio Moreno	ANTONIO MORENO	antonio moreno
4	Thomas Hardy	THOMAS HARDY	thomas hardy
5	Christina Berglund	CHRISTINA BERGLUND	christina berglund
6	Hanna Moos	HANNA MOOS	hanna moos
7	Frédérique Citeaux	FRÉDÉRIQUE CITEAUX	frédérique citeaux
8	Martín Sommer	MARTÍN SOMMER	martín sommer
9	Laurence Lebihan	LAURENCE LEBIHAN	laurence lebihan
10	Elizabeth Lincoln	ELIZABETH LINCOLN	elizabeth lincoln
11	Victoria Ashworth	VICTORIA ASHWORTH	victoria ashworth
12	Patricio Simpson	PATRICIO SIMPSON	patricio simpson
13	Francisco Chang	FRANCISCO CHANG	francisco chang
14	Yang Wang	YANG WANG	yang wang
15	Pedro Afonso	PEDRO AFONSO	pedro afonso

Total rows: 91 of 91 Query complete 00:00:00.119

GABARITO EXPLICATIVO

5

Questão 5

A equipe de TI foi acionada para implementar um sistema que identifique rapidamente as regiões de entrega dos clientes, baseando-se nos dois primeiros caracteres do código postal. Essa informação ajudará a logística a planejar rotas de forma mais eficiente.

Extraia os dois primeiros caracteres do campo postal_code dos clientes. **(Utilize a tabela customers)**

Explicação e Resultado

Para extrair os dois primeiros caracteres do código postal dos clientes, seguimos estes passos:

Selecionamos o **customer_id** e o **postal_code** da tabela **CUSTOMERS**.

Utilizamos a função **SUBSTRING(postal_code, 1, 2)** para obter os dois primeiros caracteres da coluna **postal_code**.

Nomeamos a nova coluna como **regiao** para indicar que se trata da região baseada no código postal:

```
SELECT
    customer_id,
    postal_code,
    SUBSTRING(postal_code, 1, 2) AS regiao
FROM customers;
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

5

Questão 5

A equipe de TI foi acionada para implementar um sistema que identifique rapidamente as regiões de entrega dos clientes, baseando-se nos dois primeiros caracteres do código postal. Essa informação ajudará a logística a planejar rotas de forma mais eficiente.

Extraia os dois primeiros caracteres do campo postal_code dos clientes. **(Utilize a tabela customers)**

Explicação e Resultado

Dessa forma, a consulta retorna os IDs do 91 clientes, seus códigos postais completos e os dois primeiros caracteres de cada código postal, facilitando a identificação das regiões de entrega:

	customer_id [PK] character	postal_code character varying (10)	regiao text
1	ALFKI	12209	12
2	ANATR	05021	05
3	ANTON	05023	05
4	AROUT	WA1 1DP	WA
5	BERGS	S-958 22	S-
6	BLAUS	68306	68
7	BLONP	67000	67
8	BOLID	28023	28
9	BONAP	13008	13
10	BOTTM	T2F 8M4	T2
11	BSBEV	EC2 5NT	EC
12	CACTU	1010	10
13	CENTC	05022	05
14	CHOPS	3012	30
Total rows: 91 of 91		Query complete 00:00:00.083	

GABARITO EXPLICATIVO

6

Questão 6

O setor financeiro precisa calcular os valores totais das ordens para fins de faturamento. Como parte dessa tarefa, é necessário que os valores sejam sempre arredondados para cima, garantindo que nenhuma fração de centavo seja perdida no processo de cobrança.

Exiba o valor total de cada ordem, calculado pela soma de unit_price * quantity, arredondado para cima. **(Utilize a tabela order_details)**

Explicação e Resultado

Para calcular o valor total de cada pedido e garantir que seja sempre arredondado para cima, seguimos estes passos:

Selecionamos o **order_id** da tabela **ORDER_DETAILS**.

Calculamos o valor total de cada pedido com **SUM(unit_price * quantity)**, multiplicando o preço unitário pela quantidade.

Utilizamos a função **CEILING(SUM(unit_price * quantity))** para arredondar o total para cima, garantindo que frações de centavo não sejam perdidas.

Nomeamos a nova coluna como **total_arredondado** para indicar o valor ajustado.

Agrupamos os resultados por **order_id** para calcular o total de cada pedido separadamente:

```sql
SELECT
    order_id,
    SUM(unit_price * quantity) AS total,
    CEILING(SUM(unit_price * quantity)) AS total_arredondado
FROM order_details
GROUP BY order_id;
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

6

Questão 6

O setor financeiro precisa calcular os valores totais das ordens para fins de faturamento. Como parte dessa tarefa, é necessário que os valores sejam sempre arredondados para cima, garantindo que nenhuma fração de centavo seja perdida no processo de cobrança.

Exiba o valor total de cada ordem, calculado pela soma de unit_price * quantity, arredondado para cima. **(Utilize a tabela order_details)**

Explicação e Resultado

Dessa forma, a consulta retorna os IDs dos 830 pedidos, os valores totais originais e os valores arredondados para cima:

	order_id smallint	total double precision	total_arredondado double precision
1	11038	750.9999980926514	751
2	10782	12.5	13
3	10725	287.7999954223633	288
4	10423	1020	1020
5	10518	4150.050006866455	4151
6	10356	1106.3999938964844	1107
7	10963	68	68
8	10596	1476.1000366210938	1477
9	10282	155.40000534057617	156
10	10658	4667.999973297119	4668
11	10283	1414.8000202178955	1415
12	10579	317.75	318
13	10693	2333.999988555908	2334
14	10896	750.5	751

Total rows: 830 of 830 Query complete 00:00:00.092

GABARITO EXPLICATIVO

7

Questão 7

O time de marketing está organizando uma campanha especial para promover produtos orgânicos. Para isso, é necessário localizar os produtos que possuem o termo "Organic" no nome, permitindo que a equipe foque seus esforços exclusivamente nesses itens.

Localize a posição do termo "Organic" nos nomes dos produtos, retornando somente os produtos cujo termo seja encontrado. **(Utilize a tabela products)**

Explicação e Resultado

Para localizar produtos com o termo "Organic" no nome e identificar sua posição, seguimos estes passos:

Selecionamos **product_id** e **product_name** da tabela **PRODUCTS**.

Utilizamos **STRPOS(product_name, 'Organic')** para encontrar a posição da primeira ocorrência do termo **"Organic"** no nome do produto.

Filtramos os resultados com **WHERE STRPOS(product_name, 'Organic') > 0**, garantindo que apenas os produtos que contêm o termo sejam exibidos (pois para os produtos que não tiverem o termo 'Organic' no nome, a posição retornada será 0, uma vez que inexistente. Assim, ao filtrar somente os produtos que retornem uma posição maior que 0, retornamos somente aqueles que possuem 'Organic' em seu nome):

```sql
SELECT
    product_id,
    product_name,
    STRPOS(product_name, 'Organic') AS posicao_organic
FROM products
WHERE STRPOS(product_name, 'Organic') > 0;
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

7

Questão 7

O time de marketing está organizando uma campanha especial para promover produtos orgânicos. Para isso, é necessário localizar os produtos que possuem o termo "Organic" no nome, permitindo que a equipe foque seus esforços exclusivamente nesses itens.

Localize a posição do termo "Organic" nos nomes dos produtos, retornando somente os produtos cujo termo seja encontrado. **(Utilize a tabela products)**

Explicação e Resultado

Dessa forma, a consulta retorna o ID do produto, o nome do produto e a posição do termo "Organic" no nome:

	product_id [PK] smallint	product_name character varying (40)	posicao_organic integer
1	7	Uncle Bob's Organic Dried Pears	13

GABARITO EXPLICATIVO

8

Questão 8

Para criar um ambiente de trabalho mais acolhedor, o setor de Recursos Humanos está organizando uma campanha para celebrar os aniversariantes do mês. A identificação dos aniversariantes permitirá a criação de mensagens personalizadas e a inclusão desses dados no mural de aniversários da empresa.

Crie uma consulta que exiba os IDs, nomes, sobrenomes, datas de nascimento e a idade dos funcionários que fazem aniversário no mês corrente. Além disso, inclua a data atual na consulta para referência. **(Utilize a tabela employees)**

Explicação e Resultado

Para identificar os funcionários que fazem aniversário no mês corrente e calcular suas idades, seguimos estes passos:

Selecionamos os dados relevantes:

As colunas **employee_id**, **first_name**, **last_name** e **birth_date** da tabela **EMPLOYEES** para exibir os detalhes dos funcionários.

CURRENT_DATE AS data_atual para mostrar a data de referência da consulta.

AGE(birth_date) para calcular a idade de cada funcionário.

Filtramos os aniversariantes do mês:

Utilizamos o filtro **WHERE DATE_PART('month', birth_date) = DATE_PART('month', CURRENT_DATE)** para comparar o mês de nascimento com o mês atual:

```sql
SELECT
    employee_id,
    first_name,
    last_name,
    birth_date,
    CURRENT_DATE AS data_atual,
    AGE(birth_date)
FROM employees
WHERE DATE_PART('month', birth_date) = DATE_PART('month', CURRENT_DATE);
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

8

Questão 8

Para criar um ambiente de trabalho mais acolhedor, o setor de Recursos Humanos está organizando uma campanha para celebrar os aniversariantes do mês. A identificação dos aniversariantes permitirá a criação de mensagens personalizadas e a inclusão desses dados no mural de aniversários da empresa.

Crie uma consulta que exiba os IDs, nomes, sobrenomes, datas de nascimento e a idade dos funcionários que fazem aniversário no mês corrente. Além disso, inclua a data atual na consulta para referência. **(Utilize a tabela employees)**

Explicação e Resultado

Dessa forma, a consulta retorna os funcionários aniversariantes do mês, suas idades e a data atual como referência:

	employee_id [PK] smallint	first_name character varying (10)	last_name character varying (20)	birth_date date	data_atual date	age interval
1	8	Laura	Callahan	1958-01-09	2025-01-29	67 years 20 days
2	9	Anne	Dodsworth	1966-01-27	2025-01-29	59 years 2 days

OBS.: Como estamos filtrando pela **data atual** (CURRENT_DATE), **o seu resultado pode ser diferente** do apresentado acima, **dependendo do mês em que você estiver executando esta consulta**.

GABARITO EXPLICATIVO

9

Questão 9

O setor de estoque precisa calcular o número máximo de pacotes completos que podem ser formados com os produtos disponíveis. Cada pacote deve conter exatamente 10 unidades de um mesmo produto, e é necessário considerar que não é possível formar pacotes incompletos com as unidades restantes.

Liste os nomes dos produtos, suas quantidades em estoque e o número máximo de pacotes completos que podem ser formados, levando em consideração a quantidade disponível de cada produto. **(Utilize a tabela products)**

Explicação e Resultado

Para calcular o número máximo de pacotes completos de 10 unidades que podem ser formados com o estoque disponível, seguimos estes passos:

Selecionamos os dados relevantes da tabela **PRODUCTS**:

product_name para exibir o nome do produto.

units_in_stock para mostrar a quantidade disponível em estoque.

Calculamos o número de pacotes completos com **FLOOR(units_in_stock / 10)**:

Dividimos a quantidade em estoque **por 10** (tamanho do pacote).

Usamos **FLOOR()** para arredondar para baixo, garantindo que apenas pacotes completos sejam contabilizados, desconsiderando as unidades restantes:

```
SELECT
    product_name,
    units_in_stock,
    FLOOR(units_in_stock / 10) AS pacotes_completos
FROM products;
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

9

Questão 9

O setor de estoque precisa calcular o número máximo de pacotes completos que podem ser formados com os produtos disponíveis. Cada pacote deve conter exatamente 10 unidades de um mesmo produto, e é necessário considerar que não é possível formar pacotes incompletos com as unidades restantes.

Liste os nomes dos produtos, suas quantidades em estoque e o número máximo de pacotes completos que podem ser formados, levando em consideração a quantidade disponível de cada produto. **(Utilize a tabela products)**

Explicação e Resultado

Dessa forma, a consulta retorna quantos pacotes completos de 10 unidades podem ser montados para cada um dos 77 produtos cadastrados:

	product_name character varying (40)	units_in_stock smallint	pacotes_completos double precision
1	Chai	39	3
2	Chang	17	1
3	Aniseed Syrup	13	1
4	Chef Anton's Cajun Seasoning	53	5
5	Chef Anton's Gumbo Mix	0	0
6	Grandma's Boysenberry Spread	120	12
7	Uncle Bob's Organic Dried Pears	15	1
8	Northwoods Cranberry Sauce	6	0
9	Mishi Kobe Niku	29	2
10	Ikura	31	3
11	Queso Cabrales	22	2

Total rows: 77 of 77 Query complete 00:00:00.082

GABARITO EXPLICATIVO

10

Questão 10

O setor financeiro está preparando um relatório detalhado sobre os descontos aplicados aos pedidos, que será utilizado em uma reunião estratégica com a diretoria. Para maior precisão, é necessário apresentar a média dos descontos truncada para três casas decimais, sem arredondamentos, garantindo a exatidão dos dados analisados.

Crie uma consulta que exiba a média dos valores de desconto em todos os pedidos, mostrando o valor original e o valor com três casas decimais sem arredondamentos. **(Utilize a tabela order_details)**

Explicação e Resultado

Para calcular a média dos descontos aplicados nos pedidos e truncá-la para três casas decimais sem arredondamento, seguimos estes passos:

Calculamos a média dos descontos:

AVG(discount) AS media_desconto: retorna a média dos valores da coluna **discount** da tabela **ORDER_DETAILS**.

Truncamos a média para três casas decimais com **TRUNC(CAST(AVG(discount) AS NUMERIC), 3) AS media_desconto_truncada**:

CAST(AVG(discount) AS NUMERIC): Garante que o valor da média tenha precisão decimal ao convertê-la para NUMERIC (obrigatório pela sintaxe do SQL usado no PostgreSQL).

TRUNC(..., 3): Remove qualquer valor além da terceira casa decimal sem arredondar:

```sql
SELECT
    AVG(discount) AS media_desconto,
    TRUNC(CAST(AVG(discount) AS NUMERIC), 3) AS media_desconto_truncada
FROM order_details;
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

10

Questão 10

O setor financeiro está preparando um relatório detalhado sobre os descontos aplicados aos pedidos, que será utilizado em uma reunião estratégica com a diretoria. Para maior precisão, é necessário apresentar a média dos descontos truncada para três casas decimais, sem arredondamentos, garantindo a exatidão dos dados analisados.

Crie uma consulta que exiba a média dos valores de desconto em todos os pedidos, mostrando o valor original e o valor com três casas decimais sem arredondamentos. **(Utilize a tabela order_details)**

Explicação e Resultado

Dessa forma, a consulta retorna a média original dos descontos e a versão truncada com três casas decimais:

	media_desconto double precision	media_desconto_truncada numeric
1	0.05616705420226066	0.056

MÓDULO 9

SUBQUERIES

SUBQUERIES
SUBQUERIES

O QUE VEREMOS NESTE MÓDULO

MÓDULO 9 SUBQUERIES

01	O que é uma Subquery?	340
02	Subquery: Cláusula WHERE	345
03	Subquery: Cláusula FROM	348
04	Subquery: Cláusula SELECT	353
05	Corrigindo a análise de pedidos acima da média	355
06	EXERCÍCIOS	361
07	GABARITO EXPLICATIVO	366

O que é uma SUBQUERY?

Uma Subconsulta (ou Subquery ou SubSELECT) nada mais é do que uma consulta dentro de outra consulta. Ou seja, com uma Subquery conseguimos utilizar o resultado de uma query (consulta) dentro de outra consulta.

O exemplo abaixo ilustra bem a ideia principal de uma Subquery. Precisávamos descobrir os produtos que têm o unit_price acima da média. Para isso, adicionamos um SELECT dentro de outro SELECT para criar uma consulta dinâmica e otimizada.

Veja:

```
SELECT
    *
FROM products
WHERE unit_price >= (SELECT AVG(unit_price) FROM products);
```

	product_id [PK] smallint	product_name character varying (40)	supplier_id smallint	category_id smallint	quantity_per_unit character varying (20)	unit_price real	units_in_stock smallint	units_on_order smallint	reorder_l smallint
1	7	Uncle Bob's Organic D...	3	7	12 - 1 lb pkgs.	30	15	0	
2	8	Northwoods Cranberr...	3	2	12 - 12 oz jars	40	6	0	
3	9	Mishi Kobe Niku	4	6	18 - 500 g pkgs.	97	29	0	
4	10	Ikura	4	8	12 - 200 ml jars	31	31	0	
5	12	Queso Manchego La ...	5	4	10 - 500 g pkgs.	38	86	0	
6	17	Alice Mutton	7	6	20 - 1 kg tins	39	0	0	
7	18	Carnarvon Tigers	7	8	16 kg pkg.	62.5	42	0	

PostgreSQL

Índice Geral

Índice deste Módulo

O que é uma SUBQUERY?

Vamos entender o que fizemos:

Se executarmos somente a query mais interna (a nossa Subquery), veja que obteremos a média do unit_price dos produtos da tabela products:

```
SELECT
    *
FROM products
WHERE unit_price >= (SELECT AVG(unit_price) FROM products);
```

```
SELECT AVG(unit_price) FROM products;
```

	avg double precision
1	28.8338960920061

O que é uma SUBQUERY?

Sabendo a média, poderíamos simplesmente pegar esse valor e aplicar à cláusula WHERE do nosso SELECT para retornar apenas os produtos acima da média:

```
SELECT
    *
FROM products
WHERE unit_price >= 28.83389609200614
```

	product_id [PK] smallint	product_name character varying (40)	supplier_id smallint	category_id smallint	quantity_per_unit character varying (20)	unit_price real	units_in_stock smallint	units_on_order smallint	reorder_l smallint
1	7	Uncle Bob's Organic D...	3	7	12 - 1 lb pkgs.	30	15	0	
2	8	Northwoods Cranberr...	3	2	12 - 12 oz jars	40	6	0	
3	9	Mishi Kobe Niku	4	6	18 - 500 g pkgs.	97	29	0	
4	10	Ikura	4	8	12 - 200 ml jars	31	31	0	
5	12	Queso Manchego La ...	5	4	10 - 500 g pkgs.	38	86	0	
6	17	Alice Mutton	7	6	20 - 1 kg tins	39	0	0	
7	18	Carnarvon Tigers	7	8	16 kg pkg.	62.5	42	0	

O que é uma SUBQUERY?

Porém, teríamos um problema: e se essa média mudar?

Se os preços forem alterados, ou um se um novo produto for cadastrado ou, ainda, excluído da tabela, essa média será alterada.

Então, em vez de passar o número fixo do resultado da query que retornou a média, nós podemos usar essa própria query dentro da nossa query principal (que retorna os produtos com preço acima da média).

Assim:

```
SELECT
    *
FROM products
WHERE unit_price >= (SELECT AVG(unit_price) FROM products);
```

Repare que, de ambas as formas, obtivemos o mesmo resultado. Entretanto, com a Subquery, nossa consulta ficou dinâmica. Dessa forma, se o valor da média for alterado, a Subquery será capaz de trazer para dentro da nossa query principal o valor atualizado para utilizar como parâmetro de filtragem.

	product_id [PK] smallint	product_name character varying (40)	supplier_id smallint	category_id smallint	quantity_per_unit character varying (20)	unit_price real	units_in_stock smallint	units_on_order smallint	reorder_l smallint
1	7	Uncle Bob's Organic D...	3	7	12 - 1 lb pkgs.	30	15	0	
2	8	Northwoods Cranberr...	3	2	12 - 12 oz jars	40	6	0	
3	9	Mishi Kobe Niku	4	6	18 - 500 g pkgs.	97	29	0	
4	10	Ikura	4	8	12 - 200 ml jars	31	31	0	
5	12	Queso Manchego La ...	5	4	10 - 500 g pkgs.	38	86	0	
6	17	Alice Mutton	7	6	20 - 1 kg tins	39	0	0	
7	18	Carnarvon Tigers	7	8	16 kg pkg.	62.5	42	0	

PostgreSQL

O que é uma SUBQUERY?

Podemos utilizar Subqueries de três formas distintas:

1) Na cláusula **WHERE**, como parâmetro para filtrar uma tabela;

2) Na cláusula **FROM**, como uma tabela;

3) Na cláusula **SELECT**, para retornar uma nova coluna no resultado.

Veremos cada uma dessas possibilidades a seguir.

SUBQUERIES na cláusula WHERE

O exemplo que utilizamos para iniciar este módulo, na verdade, já mostrou como podemos aplicar uma Subquery como parâmetro de filtragem de uma tabela na cláusula WHERE.

Para reforçar, vejamos outro agora:

Vamos imaginar que a gente precise saber quais pedidos têm uma quantidade vendida acima da quantidade vendida média.

Para calcular essa média, nós fazemos assim:

```
SELECT AVG(quantity) FROM order_details;
```

	avg numeric
1	23.8129930394431555

SUBQUERIES na cláusula WHERE

Agora, para saber quais pedidos venderam uma quantidade acima dessa média (23.812993039443155), conforme vimos anteriormente, poderíamos pegar o valor dessa quantidade média e aplicar diretamente na cláusula WHERE da nossa query.

Porém, queremos que nossa consulta seja dinâmica. Por isso, aplicaremos a própria query como parâmetro da cláusula WHERE:

```
SELECT
    *
FROM order_details
WHERE quantity >= (
                    SELECT
                        AVG(quantity)
                    FROM order_details
                  );
```

	order_id [PK] smallint	product_id [PK] smallint	unit_price real	quantity smallint	discount real
1	10249	51	42.4	40	0
2	10250	51	42.4	35	0.15
3	10252	20	64.8	40	0.05
4	10252	33	2	25	0.05
5	10252	60	27.2	40	0
6	10253	39	14.4	42	0
7	10253	49	16	40	0
8	10255	16	13.9	35	0
9	10255	36	15.2	25	0
10	10255	59	44	30	0
11	10257	27	35.1	25	0
12	10258	2	15.2	50	0.2
13	10258	5	17	65	0.2
14	10260	57	15.6	50	0

SUBQUERIES na cláusula WHERE

Porém, temos uma observação a fazer em relação a este código. Se consultarmos a tabela order_details, veremos que um mesmo pedido está dividido em várias linhas, uma para cada produto vendido no mesmo pedido.

```
SELECT * FROM order_details;
```

	order_id [PK] smallint	product_id [PK] smallint	unit_price real	quantity smallint	discount real
1	10248	11	14	12	0
2	10248	42	9.8	10	0
3	10248	72	34.8	5	0
4	10249	14	18.6	9	0
5	10249	51	42.4	40	0
6	10250	41	7.7	10	0
7	10250	51	42.4	35	0.15
8	10250	65	16.8	15	0.15
9	10251	22	16.8	6	0.05
10	10251	57	15.6	15	0.05
11	10251	65	16.8	20	0

Portanto, essa média de quantidade vendida por produto deveria ter sido calculada somando-se a quantidade de cada linha referente ao mesmo pedido.

Por exemplo: no pedido 10248 foram vendidos 3 produtos diferentes, assim, foram geradas 3 linhas para esse pedido, com as quantidades 12, 10 e 5. Portanto, a quantidade vendida deste pedido foi de 27.

Logo abaixo, temos duas linhas para o pedido 10249, com as quantidades 9 e 40, portanto, foram vendidos 49 itens nesse pedido. E assim sucessivamente.

Assim, o correto seria primeiro somarmos essas quantidades por pedido, para depois calcular a média de quantidade vendida por pedido. Faz sentido?

No entanto, para retornar a média dessa forma, nós precisaremos utilizar uma Subquery na cláusula FROM, que será o próximo assunto deste módulo. Por isso, por enquanto, vamos apenas deixar essa observação, pois resolveremos isso ao final do módulo.

PostgreSQL

SUBQUERIES na cláusula FROM

Prosseguindo, vejamos agora a aplicabilidade de uma Subquery na cláusula FROM.

Suponhamos que a gente queira descobrir qual é a média de clientes de acordo com o seu cargo. Como podemos fazer isso?

Se consultarmos a nossa tabela customers, veremos que temos uma coluna chamada contact_title. Nela estão armazenados os cargos de cada um de nossos clientes:

```
SELECT * FROM customers;
```

	customer_id [PK] character varying (5)	company_name character varying (40)	contact_name character varying (30)	contact_title character varying (30)	address character varying (60)	city character varying (15)
1	ALFKI	Alfreds Futterkiste	Maria Anders	Sales Representative	Obere Str. 57	Berlin
2	ANATR	Ana Trujillo Empareda...	Ana Trujillo	Owner	Avda. de la Constituci...	México D.F.
3	ANTON	Antonio Moreno Taqu...	Antonio Moreno	Owner	Mataderos 2312	México D.F.
4	AROUT	Around the Horn	Thomas Hardy	Sales Representative	120 Hanover Sq.	London
5	BERGS	Berglunds snabbköp	Christina Berglund	Order Administrator	Berguvsvägen 8	Luleå
6	BLAUS	Blauer See Delikatess...	Hanna Moos	Sales Representative	Forsterstr. 57	Mannheim
7	BLONP	Blondesddsl père et fils	Frédérique Citeaux	Marketing Manager	24, place Kléber	Strasbourg
8	BOLID	Bólido Comidas prep...	Martín Sommer	Owner	C/ Araquil, 67	Madrid
9	BONAP	Bon app'	Laurence Lebihan	Owner	12, rue des Bouchers	Marseille

SUBQUERIES na cláusula FROM

Para conseguir calcular a média de clientes de acordo com seus cargos, primeiramente precisamos descobrir quantos clientes temos para cada cargo. Para isso, podemos executar uma query que nos retorne uma contagem de clientes agrupados pela coluna contact_title:

```sql
SELECT
    contact_title,
    COUNT(*) total_clientes
FROM customers
GROUP BY contact_title;
```

	contact_title character varying (30)	total_clientes bigint
1	Sales Manager	11
2	Owner	17
3	Owner/Marketing Assistant	1
4	Accounting Manager	10
5	Sales Associate	7
6	Sales Agent	5
7	Assistant Sales Agent	2
8	Assistant Sales Representati...	1
9	Order Administrator	2
10	Marketing Manager	12
11	Sales Representative	17
12	Marketing Assistant	6

SUBQUERIES na cláusula FROM

Vemos que esse código nos trouxe como resultado uma tabela com duas colunas: os cargos distintos (contact_title) e a quantidade de clientes para cada cargo (total_clientes, que foi o alias que demos para essa coluna de contagem):

```
SELECT
    contact_title,
    COUNT(*) total_clientes
FROM customers
GROUP BY contact_title;
```

	contact_title character varying (30)	total_clientes bigint
1	Sales Manager	11
2	Owner	17
3	Owner/Marketing Assistant	1
4	Accounting Manager	10
5	Sales Associate	7
6	Sales Agent	5
7	Assistant Sales Agent	2
8	Assistant Sales Representati...	1
9	Order Administrator	2
10	Marketing Manager	12
11	Sales Representative	17
12	Marketing Assistant	6

SUBQUERIES na cláusula FROM

Sabendo disso, nós podemos utilizar essa tabela retornada no resultado como uma tabela da query em que calcularemos a média de clientes por cargo. Para isso, basta utilizarmos o código (que retorna essa tabela de contagem de clientes por cargo) na cláusula FROM da query que calculará a média de clientes por cargo:

```
SELECT
    AVG(total_clientes)
FROM (
    SELECT
        contact_title,
        COUNT(*) total_clientes
    FROM customers
    GROUP BY contact_title) T;
```

	avg numeric
1	7.5833333333333333

SUBQUERIES na cláusula FROM

Algumas observações importantes, repare que:

1) Utilizamos o alias que demos ao COUNT da Subquery (total_clientes) como nome de coluna que a função AVG deveria considerar para calcular a média;

2) Para que a cláusula FROM reconheça a Subquery como uma tabela, precisamos:

    a) Colocar o código da Subquery entre parênteses;

    b) Dar um alias para a "tabela" (que aqui, chamamos de T).

```
SELECT
    AVG(total_clientes)
FROM (
    SELECT
        contact_title,
        COUNT(*) total_clientes
    FROM customers
    GROUP BY contact_title) T;
```

	avg numeric
1	7.5833333333333333

SUBQUERIES na cláusula SELECT

Agora, veremos que também é possível utilizar Subqueries na cláusula SELECT para retornar uma nova coluna no resultado, que não existe na tabela original.

Vamos supor que, além das colunas da tabela products, nós também precisamos que apareça no resultado uma coluna com a média dos preços dos produtos (unit_price).

Para isso, nós fazemos uma consulta à tabela products, acrescentando uma Subquery na cláusula SELECT que calcule essa média de preços:

```
SELECT
    *,
    (SELECT AVG(unit_price) FROM products) media_preco
FROM products;
```

PostgreSQL

SUBQUERIES na cláusula SELECT

Perceba que, dessa forma, além de todas as colunas existentes na tabela products, à direita, também temos a coluna media_preco que criamos com a Subquery:

```
SELECT
    *,
    (SELECT AVG(unit_price) FROM products) media_preco
FROM products;
```

	product_id [PK] smallint	product_name character varying (40)	supplier_id smallint	category_id smallint	quantity_per_unit character varying (20)	unit_price real	units_in_stock smallint	units_on_order smallint	reorder_level smallint	discontinued integer	media_preco double precision
1	1	Chai	8	1	10 boxes x 30 bags	18	39	0	10	1	28.83389609200614
2	2	Chang	1	1	24 - 12 oz bottles	19	17	40	25	1	28.83389609200614
3	3	Aniseed Syrup	1	2	12 - 550 ml bottles	10	13	70	25	0	28.83389609200614
4	4	Chef Anton's Cajun Seasoning	2	2	48 - 6 oz jars	22	53	0	0	0	28.83389609200614
5	5	Chef Anton's Gumbo Mix	2	2	36 boxes	21.35	0	0	0	1	28.83389609200614
6	6	Grandma's Boysenberry Spread	3	2	12 - 8 oz jars	25	120	0	25	0	28.83389609200614
7	7	Uncle Bob's Organic Dried Pears	3	7	12 - 1 lb pkgs.	30	15	0	10	0	28.83389609200614
8	8	Northwoods Cranberry Sauce	3	2	12 - 12 oz jars	40	6	0	0	0	28.83389609200614
9	9	Mishi Kobe Niku	4	6	18 - 500 g pkgs.	97	29	0	0	1	28.83389609200614
10	10	Ikura	4	8	12 - 200 ml jars	31	31	0	0	0	28.83389609200614
11	11	Queso Cabrales	5	4	1 kg pkg.	21	22	30	30	0	28.83389609200614

Corrigindo a análise de pedidos acima da média

Ao final da nossa explicação sobre o uso de Subqueries na cláusula WHERE, observamos que retornaríamos no exemplo então utilizado para apresentarmos uma solução melhor para o exercício proposto (de retornar no resultado somente os pedidos que tivessem vendido uma quantidade de produtos acima da quantidade vendida média).

Isso porque, na solução apresentada naquele momento, não consideramos o fato de termos vendido vários produtos diferentes em um mesmo pedido, o que gerou vários registros (linhas) para o mesmo pedido.

Assim, o correto seria primeiro somarmos as quantidades vendidas de cada produto por pedido, para depois calcular a média de quantidade vendida por pedido.

Porém, para retornar a média dessa forma, nós precisaríamos utilizar uma Subquery na cláusula FROM, o que, até então, ainda não sabíamos como fazer. Por isso deixamos para mostrar essa solução ao final do módulo.

E esse momento finalmente chegou! 😁

```
SELECT * FROM order_details;
```

	order_id [PK] smallint	product_id [PK] smallint	unit_price real	quantity smallint	discount real
1	10248	11	14	12	0
2	10248	42	9.8	10	0
3	10248	72	34.8	5	0
4	10249	14	18.6	9	0
5	10249	51	42.4	40	0
6	10250	41	7.7	10	0
7	10250	51	42.4	35	0.15
8	10250	65	16.8	15	0.15
9	10251	22	16.8	6	0.05
10	10251	57	15.6	15	0.05
11	10251	65	16.8	20	0

Corrigindo a análise de pedidos acima da média

Conforme dissemos, primeiramente devemos somar as quantidades vendidas de cada produto por pedido. Conseguimos esse resultado se aplicarmos a função SUM à coluna quantity da tabela order_details, agrupando por order_id:

```
SELECT
    order_id,
    SUM(quantity) total_vendido
FROM order_details
GROUP BY order_id;
```

	order_id smallint	total_vendido bigint
1	10248	27
2	10249	49
3	10250	60
4	10251	41
5	10252	105
6	10253	102
7	10254	57
8	10255	110
9	10256	27
10	10257	46
11	10258	121
12	10259	11
13	10260	102
14	10261	40
15	10262	29
16	10263	184

PostgreSQL

Corrigindo a análise de pedidos acima da média

Tendo agora essa “tabela” com as quantidades totais vendidas de cada pedido, conseguimos aplicá-la à cláusula FROM de uma outra consulta que nos retornará a média de quantidade vendida:

```
SELECT
    AVG(total_vendido)
FROM (SELECT
        order_id,
        SUM(quantity) total_vendido
      FROM order_details
      GROUP BY order_id
    ) T;
```

	avg numeric
1	61.8277108433734940

Agora sim temos a média mais correta de quantidade vendida por pedido: 61.8277108433734940

Corrigindo a análise de pedidos acima da média

Por fim, para descobrirmos quais pedidos venderam uma quantidade de produtos acima da quantidade média vendida, nós podemos pegar todo esse código que utilizamos até então para descobrir a média...

```
SELECT
    AVG(total_vendido)
FROM (SELECT
        order_id,
        SUM(quantity) total_vendido
      FROM order_details
      GROUP BY order_id
    ) T;
```

... E utilizá-lo como uma Subquery da cláusula WHERE de outra query que retornará os pedidos que venderam quantidades acima da média.

Corrigindo a análise de pedidos acima da média

Desta forma:

```
SELECT
    order_id,
    SUM(quantity)
FROM order_details
GROUP BY order_id
HAVING SUM(quantity) >= (
        SELECT
          AVG(total_vendido)
        FROM (
            SELECT
                order_id,
                SUM(quantity) total_vendido
            FROM order_details
            GROUP BY order_id) T);
```

	order_id smallint	sum bigint
1	10356	62
2	10658	255
3	10283	76
4	10693	111
5	10253	102
6	10514	233
7	10978	76
8	10340	72
9	10889	80
10	10294	75
11	11064	173
12	10567	103
13	10997	90
14	10403	91
15	10762	134

Corrigindo a análise de pedidos acima da média

Agora, você deve estar se perguntando: mas essa Subquery não foi utilizada na cláusula WHERE, mas na cláusula HAVING. Por quê?

```
SELECT
    order_id,
    SUM(quantity)
FROM order_details
GROUP BY order_id
HAVING SUM(quantity) >= (
        SELECT
          AVG(total_vendido)
        FROM (
            SELECT
                order_id,
                SUM(quantity) total_vendido
            FROM order_details
            GROUP BY order_id) T);
```

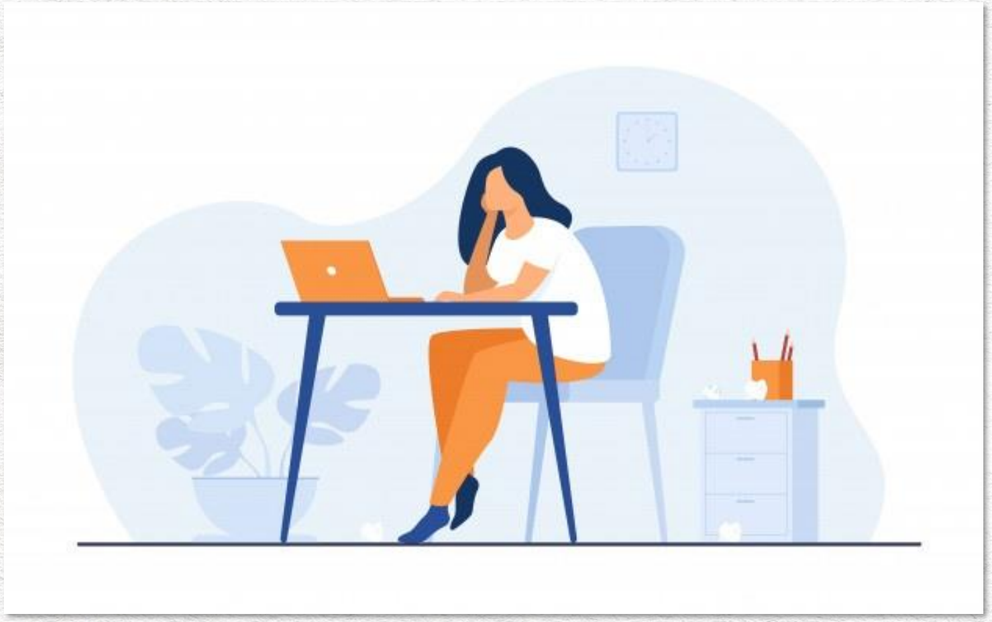
1

2

Porque, em nosso SELECT mais externo (que finalmente retornou os pedidos que venderam quantidades acima da média), primeiro precisamos agrupar a soma de quantidade vendida (SUM(quantity)) por order_id (GROUP BY order_id) 1, para só depois poder comparar essas quantidades com a média de quantidade vendida (retornada pela Subquery passada para a cláusula HAVING 2).

Sendo assim, como primeiro precisamos agrupar para depois comparar o resultado desse agrupamento com a média, tivemos que utilizar a cláusula HAVING em vez da cláusula WHERE.

EXERCÍCIOS



1

Questão 1

A equipe de marketing deseja identificar clientes fiéis, ou seja, aqueles que realizaram mais de uma compra. Esses clientes podem ser alvo de programas de fidelidade ou ofertas exclusivas, aumentando o engajamento e as vendas.

Crie uma consulta que liste os IDs, nomes das empresas, nomes e cargos dos contatos e telefones desses clientes. **(Utilize as tabelas customers e orders)**

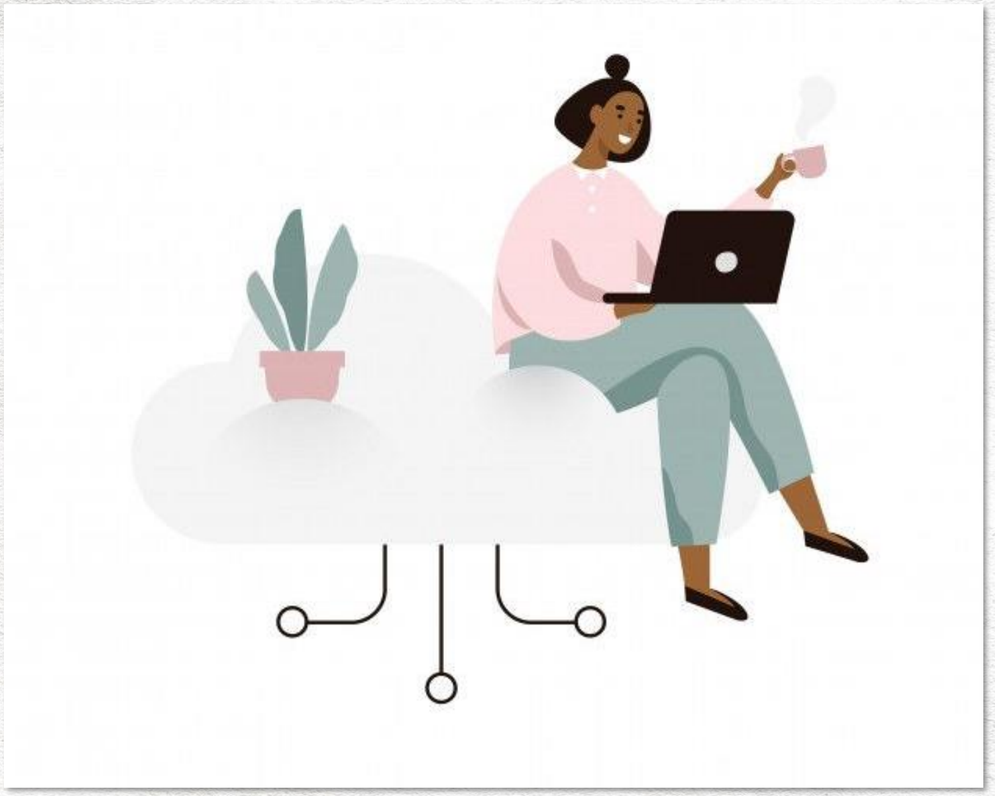
2

Questão 2

A gerência quer analisar o perfil médio de compras dos clientes para ajustar suas estratégias de vendas. Saber quantos pedidos, em média, cada cliente faz ajuda a determinar se é necessário investir em retenção, campanhas promocionais ou incentivos para compras recorrentes.

Crie uma consulta que retorne a média de pedidos feitos por cliente. **(Utilize a tabela orders)**

EXERCÍCIOS



3

Questão 3

O setor de Recursos Humanos deseja homenagear o funcionário mais antigo da empresa em um evento corporativo. Além disso, essa informação pode ser útil para análises sobre retenção de talentos e políticas de valorização profissional.

Crie uma consulta que retorne o ID, nome e sobrenome, bem como a data da contratação desse(a) funcionário(a) que está há mais tempo na empresa. **(Utilize a tabela employees)**

4

Questão 4

A equipe financeira quer entender quais pedidos tiveram um valor total acima do valor total médio de todos os pedidos, pois isso pode indicar compras em grande volume ou de produtos de maior valor agregado. Essas informações podem ser úteis para negociações futuras e para identificar padrões de clientes premium.

Crie uma consulta para obter essa informação, ordenando-a pelo valor total do pedido. Entenda por valor total a soma de unit_price * quantity. **(Utilize a tabela order_details)**

EXERCÍCIOS

5

Questão 5

O setor de vendas deseja segmentar clientes que já compraram produtos da categoria Bebidas (Beverages). Essa informação será usada para direcionar campanhas promocionais específicas, oferecendo novos produtos ou descontos para incentivar compras recorrentes.

Crie uma consulta que liste os IDs, nomes das empresas, nomes e cargos dos contatos e telefones desses clientes. **(Utilize as tabelas customers, orders, order_details, products e categories)**

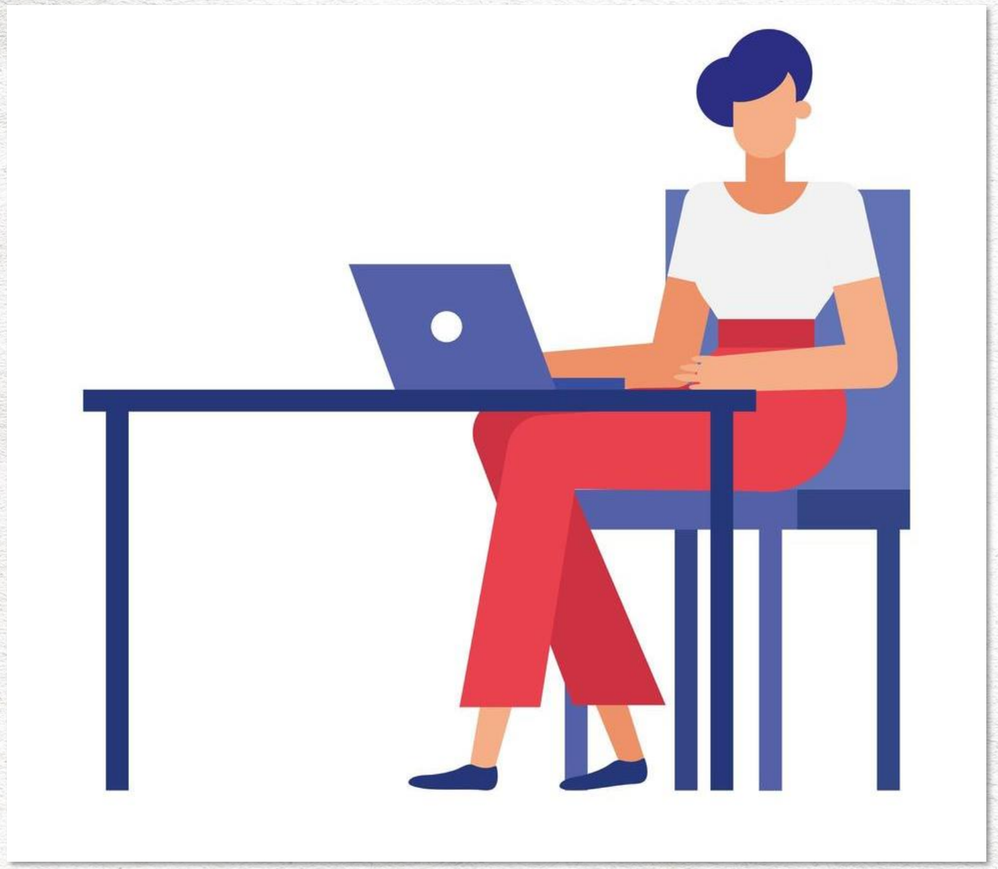
6

Questão 6

A gerência quer identificar produtos premium no catálogo, comparando os preços de itens vendidos com o produto mais caro da categoria Condimentos (Condiments). Essa análise pode ajudar na definição de estratégias de precificação e posicionamento de mercado.

Crie uma consulta que retorne os nomes e os preços desses produtos. **(Utilize as tabelas products e categories)**

EXERCÍCIOS



7

Questão 7

O setor de vendas deseja reconhecer o(a) melhor vendedor(a) da empresa, ou seja, o(a) funcionário(a) que gerou o maior valor total de vendas (considerando como total de vendas de cada funcionário a soma de unit_price * quantity agrupada por employee_id). Essa informação pode ser usada para premiações, bonificações e treinamentos, incentivando um desempenho ainda melhor da equipe comercial.

Crie uma consulta que retorne o ID, o nome e o sobrenome desse(a) funcionário(a). **(Utilize as tabelas employees, orders e order_details)**

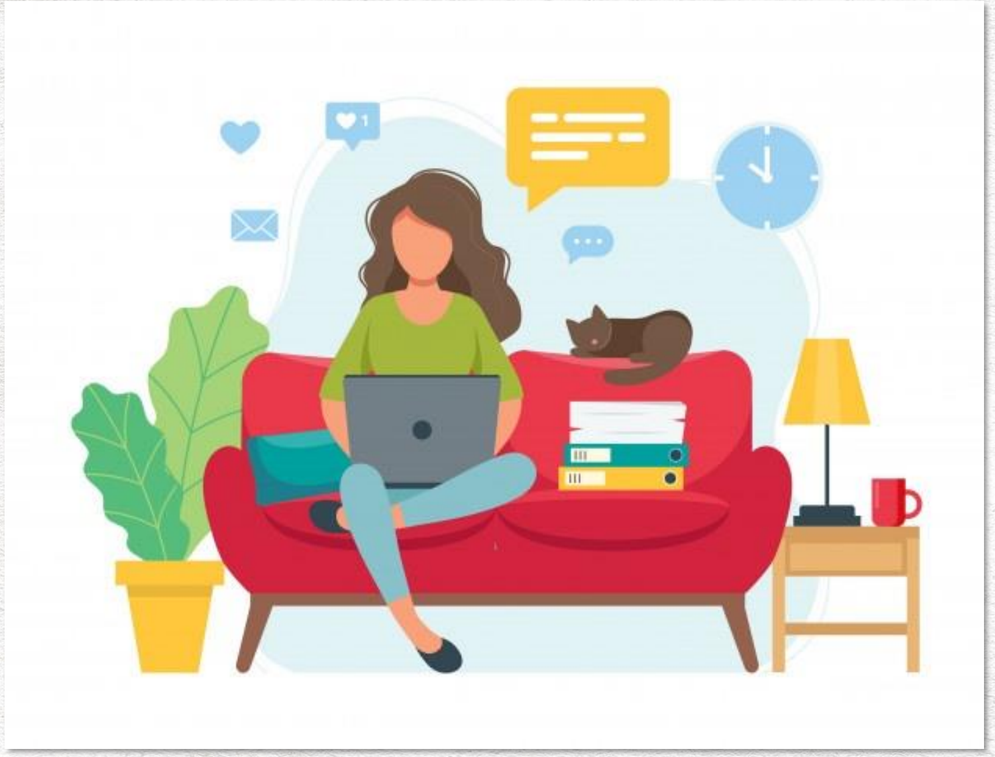
8

Questão 8

O setor comercial deseja identificar clientes que nunca realizaram um pedido, mesmo estando cadastrados no banco de dados. Essas informações são fundamentais para entrar em contato com esses clientes, entender seus motivos e tentar convertê-los em compradores ativos.

Crie uma consulta que retorne os IDs, nomes das empresas, nomes e cargos dos contatos e telefones desses clientes. **(Utilize as tabelas customers e orders)**

EXERCÍCIOS



9

Questão 9

A equipe de compras quer analisar quais fornecedores oferecem produtos premium, ou seja, itens com preços acima da média geral do catálogo. Essas informações podem ser usadas para negociações de contratos e ajustes no portfólio de produtos.

Crie uma consulta que liste os IDs e os nomes dos fornecedores que oferecem produtos com preço acima da média geral de preços cadastrados. **(Utilize as tabelas suppliers e products)**

10

Questão 10

O setor de estoque precisa verificar se existem produtos que nunca foram vendidos, pois esses itens podem representar estoque parado e custos desnecessários. Com essa análise, a empresa pode tomar decisões sobre liquidações, promoções ou até mesmo a retirada desses produtos do catálogo.

Crie uma consulta que retorne os IDs e os nomes desses produtos, caso existam. **(Utilize as tabelas products e order_details)**

GABARITO EXPLICATIVO

1

Questão 1

A equipe de marketing deseja identificar clientes fiéis, ou seja, aqueles que realizaram mais de uma compra. Esses clientes podem ser alvo de programas de fidelidade ou ofertas exclusivas, aumentando o engajamento e as vendas.

Crie uma consulta que liste os IDs, nomes das empresas, nomes e cargos dos contatos e telefones desses clientes. **(Utilize as tabelas customers e orders)**

Explicação e Resultado

Para identificar clientes que realizaram mais de uma compra, começamos verificando na tabela **ORDERS** os agrupamentos de customer_id (**GROUP BY customer_id**) que retornam uma contagem maior que 1 (**HAVING COUNT(*) > 1**). Assim, filtramos os 88 IDs dos clientes que fizeram mais de um pedido:

```
SELECT customer_id
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 1
```

→

	customer_id character
1	TOMSP
2	LONEP
3	OLDWO
4	WARTH
5	MAGAA
6	QUEEN
7	VINET
8	ANTON
9	MORGK
10	GOURL
11	WOLZA
12	GALED

Total rows: 88 of 88

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

1

Questão 1

A equipe de marketing deseja identificar clientes fiéis, ou seja, aqueles que realizaram mais de uma compra. Esses clientes podem ser alvo de programas de fidelidade ou ofertas exclusivas, aumentando o engajamento e as vendas.

Crie uma consulta que liste os IDs, nomes das empresas, nomes e cargos dos contatos e telefones desses clientes. **(Utilize as tabelas customers e orders)**

Explicação e Resultado

Agora, podemos aplicar essa mesma consulta como uma **subquery na cláusula WHERE** de uma consulta principal. Nessa **consulta principal**, selecionamos os campos **customer_id**, **company_name**, **contact_name**, **contact_title** e **phone** da tabela **CUSTOMERS**, aplicando um filtro **WHERE customer_id IN (...)** para restringir os resultados apenas aos IDs dos clientes identificados na subquery:

```sql
SELECT
    customer_id,
    company_name,
    contact_name,
    contact_title,
    phone
FROM customers
WHERE customer_id IN (
                SELECT customer_id
                FROM orders
                GROUP BY customer_id
                HAVING COUNT(*) > 1
            );
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

1

Questão 1

A equipe de marketing deseja identificar clientes fiéis, ou seja, aqueles que realizaram mais de uma compra. Esses clientes podem ser alvo de programas de fidelidade ou ofertas exclusivas, aumentando o engajamento e as vendas.

Crie uma consulta que liste os IDs, nomes das empresas, nomes e cargos dos contatos e telefones desses clientes. **(Utilize as tabelas customers e orders)**

Explicação e Resultado

O resultado dessa consulta é uma lista com apenas os clientes que já realizaram mais de uma compra. No total, 88 clientes atendem a esse critério:

	customer_id [PK] character	company_name character varying (40)	contact_name character varying (30)	contact_title character varying (30)	phone character varying (2
1	ALFKI	Alfreds Futterkiste	Maria Anders	Sales Representative	030-0074321
2	ANATR	Ana Trujillo Emparedados y helados	Ana Trujillo	Owner	(5) 555-4729
3	ANTON	Antonio Moreno Taquería	Antonio Moreno	Owner	(5) 555-3932
4	AROUT	Around the Horn	Thomas Hardy	Sales Representative	(171) 555-7788
5	BERGS	Berglunds snabbköp	Christina Berglund	Order Administrator	0921-12 34 65
6	BLAUS	Blauer See Delikatessen	Hanna Moos	Sales Representative	0621-08460
7	BLONP	Blondesddsl père et fils	Frédérique Citeaux	Marketing Manager	88.60.15.31
8	BOLID	Bólido Comidas preparadas	Martín Sommer	Owner	(91) 555 22 82
9	BONAP	Bon app'	Laurence Lebihan	Owner	91.24.45.40
10	BOTTM	Bottom-Dollar Markets	Elizabeth Lincoln	Accounting Manager	(604) 555-4729
11	BSBEV	B's Beverages	Victoria Ashworth	Sales Representative	(171) 555-1212
12	CACTU	Cactus Comidas para llevar	Patricio Simpson	Sales Agent	(1) 135-5555

Total rows: 88 of 88 | Query complete 00:00:00.081

GABARITO EXPLICATIVO

2

Questão 2

A gerência quer analisar o perfil médio de compras dos clientes para ajustar suas estratégias de vendas. Saber quantos pedidos, em média, cada cliente faz ajuda a determinar se é necessário investir em retenção, campanhas promocionais ou incentivos para compras recorrentes.

Crie uma consulta que retorne a média de pedidos feitos por cliente. **(Utilize a tabela orders)**

Explicação e Resultado

Para calcular a média de pedidos por cliente, primeiro precisamos contar quantos pedidos cada cliente fez. Para isso, agrupamos os registros da tabela **ORDERS** pela coluna **customer_id** e utilizamos **COUNT(order_id)** para contar o total de pedidos por cliente:

```sql
SELECT
    customer_id,
    COUNT(order_id) AS total_pedidos
FROM orders
GROUP BY customer_id
```

	customer_id character	total_pedidos bigint
1	TOMSP	6
2	LONEP	8
3	OLDWO	10
4	WARTH	15
5	MAGAA	10
6	QUEEN	13
7	VINET	5
8	ANTON	7
9	MORGK	5
10	GOURL	9
11	WOLZA	7
12	GALED	5
Total rows: 89 of 89		Query complet

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

2

Questão 2

A gerência quer analisar o perfil médio de compras dos clientes para ajustar suas estratégias de vendas. Saber quantos pedidos, em média, cada cliente faz ajuda a determinar se é necessário investir em retenção, campanhas promocionais ou incentivos para compras recorrentes.

Crie uma consulta que retorne a média de pedidos feitos por cliente. **(Utilize a tabela orders)**

Explicação e Resultado

Agora, podemos calcular a média desses valores retornados na coluna **total_pedidos** da subquery. Para isso, aplicamos a consulta anterior como uma **subquery na cláusula FROM** da consulta principal, calculando a média (**AVG(total_pedidos)**) sobre os resultados da contagem de pedidos:

```sql
SELECT
    AVG(total_pedidos)
FROM (
    SELECT
        customer_id,
        COUNT(order_id) AS total_pedidos
    FROM orders
    GROUP BY customer_id
) t;
```

O resultado dessa consulta retorna a média de pedidos feitos por cliente:

	avg numeric
1	9.3258426966292135

GABARITO EXPLICATIVO

3

Questão 3

O setor de Recursos Humanos deseja homenagear o funcionário mais antigo da empresa em um evento corporativo. Além disso, essa informação pode ser útil para análises sobre retenção de talentos e políticas de valorização profissional.

Crie uma consulta que retorne o ID, nome e sobrenome, bem como a data da contratação desse(a) funcionário(a) que está há mais tempo na empresa. **(Utilize a tabela employees)**

Explicação e Resultado

Para identificar o(a) funcionário(a) mais antigo(a) da empresa, primeiro precisamos determinar a menor data de contratação presente na tabela **EMPLOYEES**. Utilizamos a função **MIN(hire_date)** para encontrar essa data:

```
SELECT
    MIN(hire_date)
FROM employees
```

	min date
1	1992-04-01

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

3

Questão 3

O setor de Recursos Humanos deseja homenagear o funcionário mais antigo da empresa em um evento corporativo. Além disso, essa informação pode ser útil para análises sobre retenção de talentos e políticas de valorização profissional.

Crie uma consulta que retorne o ID, nome e sobrenome, bem como a data da contratação desse(a) funcionário(a) que está há mais tempo na empresa. **(Utilize a tabela employees)**

Explicação e Resultado

Agora, utilizamos essa **subquery na cláusula WHERE** da consulta principal para buscar o(a) funcionário(a) cuja **hire_date** corresponda à menor data encontrada. A consulta principal retorna o ID, nome, sobrenome e data de contratação desse(a) funcionário(a):

```sql
SELECT
    employee_id,
    first_name,
    last_name,
    hire_date
FROM employees
WHERE hire_date = (
                    SELECT
                        MIN(hire_date)
                    FROM employees
                );
```

Dessa forma, conseguimos identificar a colaboradora com maior tempo de empresa:

	employee_id [PK] smallint	first_name character varying (10)	last_name character varying (20)	hire_date date
1	3	Janet	Leverling	1992-04-01

GABARITO EXPLICATIVO

4

Questão 4

A equipe financeira quer entender quais pedidos tiveram um valor total acima do valor total médio de todos os pedidos, pois isso pode indicar compras em grande volume ou de produtos de maior valor agregado. Essas informações podem ser úteis para negociações futuras e para identificar padrões de clientes premium.

Crie uma consulta para obter essa informação, ordenando-a pelo valor total do pedido. Entenda por valor total a soma de unit_price * quantity. **(Utilize a tabela order_details)**

Explicação e Resultado

Para identificar os pedidos que tiveram um valor total acima da média de todos os pedidos, primeiro precisamos calcular a soma de unit_price * quantity para cada order_id. Isso nos dará o valor total de cada pedido:

```
SELECT
    order_id,
    SUM(unit_price * quantity) AS total
FROM order_details
GROUP BY order_id
```

	order_id smallint	total double precision
1	11038	750.9999980926514
2	10782	12.5
3	10725	287.7999954223633
4	10423	1020
5	10518	4150.050006866455
6	10356	1106.3999938964844
7	10963	68
8	10596	1476.1000366210938
9	10282	155.40000534057617
10	10658	4667.999973297119

Total rows: 830 of 830 | Query comple

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

4

Questão 4

A equipe financeira quer entender quais pedidos tiveram um valor total acima do valor total médio de todos os pedidos, pois isso pode indicar compras em grande volume ou de produtos de maior valor agregado. Essas informações podem ser úteis para negociações futuras e para identificar padrões de clientes premium.

Crie uma consulta para obter essa informação, ordenando-a pelo valor total do pedido. Entenda por valor total a soma de unit_price * quantity. **(Utilize a tabela order_details)**

Explicação e Resultado

Agora, utilizamos essa **primeira consulta** como uma **subquery na cláusula FROM** de outra consulta que calcula a média do total de todos os pedidos retornados por esta subquery (**AVG(total)**):

```sql
SELECT
    AVG(total)
FROM (
    SELECT
        order_id,
        SUM(unit_price * quantity) AS total
    FROM order_details
    GROUP BY order_id
) t
```

	avg double precision
1	1631.8778198087073

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

4

Questão 4

A equipe financeira quer entender quais pedidos tiveram um valor total acima do valor total médio de todos os pedidos, pois isso pode indicar compras em grande volume ou de produtos de maior valor agregado. Essas informações podem ser úteis para negociações futuras e para identificar padrões de clientes premium.

Crie uma consulta para obter essa informação, ordenando-a pelo valor total do pedido. Entenda por valor total a soma de unit_price * quantity. **(Utilize a tabela order_details)**

Explicação e Resultado

Por fim, utilizamos essa **segunda consulta** como uma **subquery na cláusula HAVING** da consulta principal (da mesma forma que usaríamos em uma cláusula WHERE) para filtrar apenas os pedidos cujo total (**SUM(unit_price * quantity)**) seja maior do que essa média calculada. Por fim, **ordenamos** o resultado pela coluna calculada na consulta principal **total_pedido**:

```sql
SELECT
    order_id,
    SUM(unit_price * quantity) AS total_pedido
FROM order_details
GROUP BY order_id
HAVING SUM(unit_price * quantity) > (
                    SELECT
                        AVG(total)
                    FROM (
                        SELECT
                            order_id,
                            SUM(unit_price * quantity) AS total
                        FROM order_details
                        GROUP BY order_id
                    ) t
                )
ORDER BY total_pedido;
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

4

Questão 4

A equipe financeira quer entender quais pedidos tiveram um valor total acima do valor total médio de todos os pedidos, pois isso pode indicar compras em grande volume ou de produtos de maior valor agregado. Essas informações podem ser úteis para negociações futuras e para identificar padrões de clientes premium.

Crie uma consulta para obter essa informação, ordenando-a pelo valor total do pedido. Entenda por valor total a soma de unit_price * quantity. **(Utilize a tabela order_details)**

Explicação e Resultado

Com essa consulta, conseguimos identificar os 288 pedidos que tiveram um valor total superior à média do valor total de todos os pedidos:

	order_id smallint	total_pedido double precision
1	10800	1632.149998664856
2	10686	1638.449993133545
3	10758	1644.5999908447266
4	10784	1649.9999771118164
5	10617	1650
6	10808	1660
7	11052	1665
8	10527	1670
9	10404	1674.9999809265137
10	10680	1682.5000381469727
11	10765	1683.9999389648438
12	10459	1687.9999771118164
13	11036	1692
14	10660	1701
15	10751	1701.459976196289

Total rows: 288 of 288 Query comple

GABARITO EXPLICATIVO

5

Questão 5

O setor de vendas deseja segmentar clientes que já compraram produtos da categoria Bebidas (Beverages). Essa informação será usada para direcionar campanhas promocionais específicas, oferecendo novos produtos ou descontos para incentivar compras recorrentes.

Crie uma consulta que liste os IDs, nomes das empresas, nomes e cargos dos contatos e telefones desses clientes. **(Utilize as tabelas customers, orders, order_details, products e categories)**

Explicação e Resultado

Para listar os clientes que compraram produtos da categoria Bebidas (**Beverages**), primeiro precisamos descobrir os **customer_id distintos** desses clientes.

Fazemos isso com uma consulta que começa com a tabela **ORDERS**, associando-a com a tabela **ORDER_DETAILS** (para obter os detalhes dos pedidos), a tabela **PRODUCTS** (para acessar os produtos comprados) e, finalmente, a tabela **CATEGORIES** (para filtrar pela categoria de produto). O filtro **WHERE cat.category_name = 'Beverages'** restringe os resultados apenas aos clientes que compraram produtos dessa categoria:

```
SELECT DISTINCT
    o.customer_id
FROM orders o
INNER JOIN order_details od
    ON o.order_id = od.order_id
    INNER JOIN products p
        ON od.product_id = p.product_id
        INNER JOIN categories cat
            ON p.category_id = cat.category_id
WHERE cat.category_name = 'Beverages'
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

5

Questão 5

O setor de vendas deseja segmentar clientes que já compraram produtos da categoria Bebidas (Beverages). Essa informação será usada para direcionar campanhas promocionais específicas, oferecendo novos produtos ou descontos para incentivar compras recorrentes.

Crie uma consulta que liste os IDs, nomes das empresas, nomes e cargos dos contatos e telefones desses clientes. **(Utilize as tabelas customers, orders, order_details, products e categories)**

Explicação e Resultado

Essa consulta retorna os **customer_id distintos** dos 83 clientes que compraram produtos da categoria **'Beverages'**:

	customer_id character
1	TOMSP
2	LONEP
3	OLDWO
4	WARTH
5	MAGAA
6	QUEEN
7	MORGK
8	ANTON
9	GOURL

Total rows: 83 of 83

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

5

Questão 5

O setor de vendas deseja segmentar clientes que já compraram produtos da categoria Bebidas (Beverages). Essa informação será usada para direcionar campanhas promocionais específicas, oferecendo novos produtos ou descontos para incentivar compras recorrentes.

Crie uma consulta que liste os IDs, nomes das empresas, nomes e cargos dos contatos e telefones desses clientes. **(Utilize as tabelas customers, orders, order_details, products e categories)**

Explicação e Resultado

Agora, podemos aplicar essa consulta como uma **subquery na cláusula WHERE** de uma consulta principal para buscar os dados desses 83 clientes, como **customer_id**, **company_name**, **contact_name**, **contact_title** e **phone**, na tabela **CUSTOMERS**:

```sql
SELECT
    c.customer_id,
    c.company_name,
    c.contact_name,
    c.contact_title,
    c.phone
FROM customers c
WHERE c.customer_id IN (
                SELECT DISTINCT
                    o.customer_id
                FROM orders o
                INNER JOIN order_details od
                    ON o.order_id = od.order_id
                    INNER JOIN products p
                        ON od.product_id = p.product_id
                        INNER JOIN categories cat
                            ON p.category_id = cat.category_id
                WHERE cat.category_name = 'Beverages'
            );
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

5

Questão 5

O setor de vendas deseja segmentar clientes que já compraram produtos da categoria Bebidas (Beverages). Essa informação será usada para direcionar campanhas promocionais específicas, oferecendo novos produtos ou descontos para incentivar compras recorrentes.

Crie uma consulta que liste os IDs, nomes das empresas, nomes e cargos dos contatos e telefones desses clientes. **(Utilize as tabelas customers, orders, order_details, products e categories)**

Explicação e Resultado

A consulta retorna uma lista com as informações dos 83 clientes que compraram produtos da categoria Bebidas:

	customer_id [PK] character	company_name character varying (40)	contact_name character varying (30)	contact_title character varying (30)	phone character varying (2
1	ALFKI	Alfreds Futterkiste	Maria Anders	Sales Representative	030-0074321
2	ANATR	Ana Trujillo Emparedados y helados	Ana Trujillo	Owner	(5) 555-4729
3	ANTON	Antonio Moreno Taquería	Antonio Moreno	Owner	(5) 555-3932
4	AROUT	Around the Horn	Thomas Hardy	Sales Representative	(171) 555-7788
5	BERGS	Berglunds snabbköp	Christina Berglund	Order Administrator	0921-12 34 65
6	BLAUS	Blauer See Delikatessen	Hanna Moos	Sales Representative	0621-08460
7	BLONP	Blondesddsl père et fils	Frédérique Citeaux	Marketing Manager	88.60.15.31
8	BOLID	Bólido Comidas preparadas	Martín Sommer	Owner	(91) 555 22 82
9	BONAP	Bon app'	Laurence Lebihan	Owner	91.24.45.40
10	BOTTM	Bottom-Dollar Markets	Elizabeth Lincoln	Accounting Manager	(604) 555-4729
11	BSBEV	B's Beverages	Victoria Ashworth	Sales Representative	(171) 555-1212
12	CACTU	Cactus Comidas para llevar	Patricio Simpson	Sales Agent	(1) 135-5555
13	CHOPS	Chop-suey Chinese	Yang Wang	Owner	0452-076545
14	CONSH	Consolidated Holdings	Elizabeth Brown	Sales Representative	(171) 555-2282
15	DRACD	Drachenblut Delikatessen	Sven Ottlieb	Order Administrator	0241-039123
Total rows: 83 of 83		Query complete 00:00:00.108			

GABARITO EXPLICATIVO

6

Questão 6

A gerência quer identificar produtos premium no catálogo, comparando os preços de itens vendidos com o produto mais caro da categoria Condimentos (Condiments). Essa análise pode ajudar na definição de estratégias de precificação e posicionamento de mercado.

Crie uma consulta que retorne os nomes e os preços desses produtos. **(Utilize as tabelas products e categories)**

Explicação e Resultado

Para listar os produtos com preço superior ao produto mais caro da categoria Condimentos (**Condiments**), primeiro precisamos descobrir qual é o preço máximo dos produtos dessa categoria.

Fazemos isso com uma consulta que começa com a tabela **PRODUCTS**, associando-a com a tabela **CATEGORIES** para filtrar pela categoria "**Condiments**". O filtro **WHERE c.category_name = 'Condiments'** garante que estamos considerando apenas os produtos dessa categoria, e a função **MAX(unit_price)** retorna o preço máximo entre eles:

```sql
SELECT
    MAX(unit_price)
FROM products p
INNER JOIN categories c
    ON p.category_id = c.category_id
WHERE c.category_name = 'Condiments'
```

	max real
1	43.9

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

6

Questão 6

A gerência quer identificar produtos premium no catálogo, comparando os preços de itens vendidos com o produto mais caro da categoria Condimentos (Condiments). Essa análise pode ajudar na definição de estratégias de precificação e posicionamento de mercado.

Crie uma consulta que retorne os nomes e os preços desses produtos. **(Utilize as tabelas products e categories)**

Explicação e Resultado

Agora, podemos aplicar essa consulta como uma **subquery na cláusula WHERE** de uma consulta principal, para listar os produtos cujo preço (**unit_price**) seja maior que o preço máximo da categoria "**Condiments**".

A consulta principal seleciona os produtos na tabela **PRODUCTS**, retornando o nome do produto e o preço:

```sql
SELECT
    product_name,
    unit_price
FROM products
WHERE unit_price > (
                    SELECT
                        MAX(unit_price)
                    FROM products p
                    INNER JOIN categories c
                        ON p.category_id = c.category_id
                    WHERE c.category_name = 'Condiments'
                );
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

6

Questão 6

A gerência quer identificar produtos premium no catálogo, comparando os preços de itens vendidos com o produto mais caro da categoria Condimentos (Condiments). Essa análise pode ajudar na definição de estratégias de precificação e posicionamento de mercado.

Crie uma consulta que retorne os nomes e os preços desses produtos. **(Utilize as tabelas products e categories)**

Explicação e Resultado

A consulta retorna uma lista com os nomes e preços dos 10 produtos que têm um preço superior ao mais caro da categoria "Condimentos":

	product_name character varying (40)	unit_price real
1	Mishi Kobe Niku	97
2	Carnarvon Tigers	62.5
3	Sir Rodney's Marmalade	81
4	Rössle Sauerkraut	45.6
5	Thüringer Rostbratwurst	123.79
6	Côte de Blaye	263.5
7	Ipoh Coffee	46
8	Manjimup Dried Apples	53
9	Raclette Courdavault	55
10	Tarte au sucre	49.3

Total rows: 10 of 10 Query complete 00

GABARITO EXPLICATIVO

7

Questão 7

O setor de vendas deseja reconhecer o(a) melhor vendedor(a) da empresa, ou seja, o(a) funcionário(a) que gerou o maior valor total de vendas (considerando como total de vendas de cada funcionário a soma de unit_price * quantity agrupada por employee_id). Essa informação pode ser usada para premiações, bonificações e treinamentos, incentivando um desempenho ainda melhor da equipe comercial.

Crie uma consulta que retorne o ID, o nome e o sobrenome desse(a) funcionário(a). **(Utilize as tabelas employees, orders e order_details)**

Explicação e Resultado

Para identificar o(a) melhor vendedor(a) da empresa, ou seja, aquele(a) funcionário(a) que gerou o maior valor total de vendas, precisamos calcular o **total de vendas de cada funcionário** (dado pela **soma de unit_price * quantity, agrupada por employee_id**).

Fazemos isso com uma consulta que começa com a tabela **ORDERS**, associando-a com a tabela **ORDER_DETAILS** para obter os detalhes das vendas. Em seguida, usamos a função **SUM(unit_price * quantity)** para calcular o total de vendas de cada funcionário. O agrupamento é feito por **employee_id**, e a consulta é ordenada de forma decrescente pela soma das vendas (**ORDER BY SUM(unit_price * quantity) DESC**), garantindo que o(a) melhor vendedor(a) apareça no topo da lista.

Para limitar a consulta ao(a) melhor vendedor(a), utilizamos o comando **LIMIT 1** para pegar apenas o primeiro resultado:

```sql
SELECT employee_id
FROM orders o
INNER JOIN order_details od
    ON o.order_id = od.order_id
GROUP BY employee_id
ORDER BY SUM(unit_price * quantity) DESC
LIMIT 1
```

	employee_id smallint
1	4

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

7

Questão 7

O setor de vendas deseja reconhecer o(a) melhor vendedor(a) da empresa, ou seja, o(a) funcionário(a) que gerou o maior valor total de vendas (considerando como total de vendas de cada funcionário a soma de unit_price * quantity agrupada por employee_id). Essa informação pode ser usada para premiações, bonificações e treinamentos, incentivando um desempenho ainda melhor da equipe comercial.

Crie uma consulta que retorne o ID, o nome e o sobrenome desse(a) funcionário(a). **(Utilize as tabelas employees, orders e order_details)**

Explicação e Resultado

Agora, podemos aplicar essa consulta como uma **subquery na cláusula WHERE** da consulta principal, para buscar o ID, nome e sobrenome do(a) melhor vendedor(a). A consulta principal retorna os campos **employee_id**, **first_name** e **last_name** da tabela **EMPLOYEES**, filtrando pelo employee_id retornado pela subquery:

```
SELECT
    employee_id,
    first_name,
    last_name
FROM employees
WHERE employee_id = (
                SELECT employee_id
                FROM orders o
                INNER JOIN order_details od
                    ON o.order_id = od.order_id
                GROUP BY employee_id
                ORDER BY SUM(unit_price * quantity) DESC
                LIMIT 1
            );
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

7

Questão 7

O setor de vendas deseja reconhecer o(a) melhor vendedor(a) da empresa, ou seja, o(a) funcionário(a) que gerou o maior valor total de vendas (considerando como total de vendas de cada funcionário a soma de unit_price * quantity agrupada por employee_id). Essa informação pode ser usada para premiações, bonificações e treinamentos, incentivando um desempenho ainda melhor da equipe comercial.

Crie uma consulta que retorne o ID, o nome e o sobrenome desse(a) funcionário(a). **(Utilize as tabelas employees, orders e order_details)**

Explicação e Resultado

A consulta retorna o ID, o nome e o sobrenome da funcionária que gerou o maior valor total de vendas:

	employee_id [PK] smallint	first_name character varying (10)	last_name character varying (20)
1	4	Margaret	Peacock

GABARITO EXPLICATIVO

8

Questão 8

O setor comercial deseja identificar clientes que nunca realizaram um pedido, mesmo estando cadastrados no banco de dados. Essas informações são fundamentais para entrar em contato com esses clientes, entender seus motivos e tentar convertê-los em compradores ativos.

Crie uma consulta que retorne os IDs, nomes das empresas, nomes e cargos dos contatos e telefones desses clientes. **(Utilize as tabelas customers e orders)**

Explicação e Resultado

Para listar os clientes que nunca realizaram um pedido, começamos identificando os **customer_id** dos clientes que já efetuaram pedidos. Fazemos isso com uma consulta que seleciona os **customer_id distintos** da tabela **ORDERS**. Esses clientes são aqueles que realizaram ao menos um pedido:

```
SELECT DISTINCT
    customer_id
FROM orders
```

	customer_id character
1	TOMSP
2	LONEP
3	OLDWO
4	WARTH
5	MAGAA
6	QUEEN
7	VINET
8	ANTON
9	MORGK
10	GOURL
11	WOLZA
12	GALED
13	RATTC

Total rows: 89 of 89

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

8

Questão 8

O setor comercial deseja identificar clientes que nunca realizaram um pedido, mesmo estando cadastrados no banco de dados. Essas informações são fundamentais para entrar em contato com esses clientes, entender seus motivos e tentar convertê-los em compradores ativos.

Crie uma consulta que retorne os IDs, nomes das empresas, nomes e cargos dos contatos e telefones desses clientes. **(Utilize as tabelas customers e orders)**

Explicação e Resultado

Agora, podemos aplicar essa consulta como uma **subquery na cláusula WHERE** da consulta principal para listar os clientes que nunca realizaram um pedido, usando a cláusula **NOT IN** para filtrar os clientes da tabela **CUSTOMERS** cujo **customer_id** não está na lista retornada pela subquery.

A consulta principal seleciona , filtrando os clientes com customer_id que não aparecem na subquery:

```sql
SELECT
    customer_id,
    company_name,
    contact_name,
    contact_title,
    phone
FROM customers
WHERE customer_id NOT IN (
                    SELECT DISTINCT
                        customer_id
                    FROM orders
                );
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

8

Questão 8

O setor comercial deseja identificar clientes que nunca realizaram um pedido, mesmo estando cadastrados no banco de dados. Essas informações são fundamentais para entrar em contato com esses clientes, entender seus motivos e tentar convertê-los em compradores ativos.

Crie uma consulta que retorne os IDs, nomes das empresas, nomes e cargos dos contatos e telefones desses clientes. **(Utilize as tabelas customers e orders)**

Explicação e Resultado

A consulta retorna uma lista com os campos **customer_id**, **company_name**, **contact_name**, **contact_title** e **phone** da tabela **CUSTOMERS** referentes aos clientes cadastrados nesta tabela, mas que não aparecem na tabela **ORDERS**, ou seja, que nunca realizaram um pedido:

	customer_id [PK] character	company_name character varying (40)	contact_name character varying (30)	contact_title character varying (30)	phone character varying (24)
1	FISSA	FISSA Fabrica Inter. Salchichas S.A.	Diego Roel	Accounting Manager	(91) 555 94 44
2	PARIS	Paris spécialités	Marie Bertrand	Owner	(1) 42.34.22.66

GABARITO EXPLICATIVO

9

Questão 9

A equipe de compras quer analisar quais fornecedores oferecem produtos premium, ou seja, itens com preços acima da média geral do catálogo. Essas informações podem ser usadas para negociações de contratos e ajustes no portfólio de produtos.

Crie uma consulta que liste os IDs e os nomes dos fornecedores que oferecem produtos com preço acima da média geral de preços cadastrados. **(Utilize as tabelas suppliers e products)**

Explicação e Resultado

Para listar os fornecedores que oferecem produtos com preço acima da média geral do catálogo, precisamos primeiro calcular a média dos preços de todos os produtos cadastrados.

Fazemos isso com uma consulta que calcula a **média de unit_price** na tabela **PRODUCTS** utilizando a função **AVG(unit_price)**:

```
SELECT
    AVG(unit_price)
FROM products
```

	avg double precision
1	28.83389609200614

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

9

Questão 9

A equipe de compras quer analisar quais fornecedores oferecem produtos premium, ou seja, itens com preços acima da média geral do catálogo. Essas informações podem ser usadas para negociações de contratos e ajustes no portfólio de produtos.

Crie uma consulta que liste os IDs e os nomes dos fornecedores que oferecem produtos com preço acima da média geral de preços cadastrados. **(Utilize as tabelas suppliers e products)**

Explicação e Resultado

Agora, podemos aplicar essa consulta como uma **subquery na cláusula WHERE** da consulta principal. A consulta principal associa as tabelas **SUPPLIERS** e **PRODUCTS**, selecionando os fornecedores cujos produtos têm um preço (**unit_price**) superior à média calculada pela subquery.

A consulta principal seleciona os campos **distintos de supplier_id e company_name** da tabela **SUPPLIERS**, aplicando o filtro **WHERE p.unit_price > (...)**, para garantir que sejam retornados apenas os fornecedores que oferecem produtos premium:

```sql
SELECT DISTINCT
    s.supplier_id,
    s.company_name
FROM suppliers s
INNER JOIN products p
    ON s.supplier_id = p.supplier_id
WHERE p.unit_price > (
                    SELECT
                        AVG(unit_price)
                    FROM products
                );
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

9

Questão 9

A equipe de compras quer analisar quais fornecedores oferecem produtos premium, ou seja, itens com preços acima da média geral do catálogo. Essas informações podem ser usadas para negociações de contratos e ajustes no portfólio de produtos.

Crie uma consulta que liste os IDs e os nomes dos fornecedores que oferecem produtos com preço acima da média geral de preços cadastrados. **(Utilize as tabelas suppliers e products)**

Explicação e Resultado

A consulta retorna uma lista com os IDs e os nomes distintos dos fornecedores que oferecem produtos com preços acima da média do catálogo:

	supplier_id [PK] smallint	company_name character varying (40)
1	28	Gai pâturage
2	29	Forêts d'érables
3	5	Cooperativa de Quesos 'Las Cabras'
4	11	Heli Süßwaren GmbH & Co. KG
5	18	Aux joyeux ecclésiastiques
6	14	Formaggi Fortini s.r.l.
7	8	Specialty Biscuits, Ltd.
8	4	Tokyo Traders
9	26	Pasta Buttini s.r.l.
10	7	Pavlova, Ltd.
11	24	G'day, Mate
12	12	Plutzer Lebensmittelgroßmärkte AG
13	15	Norske Meierier
14	3	Grandma Kelly's Homestead
15	20	Leka Trading
Total rows: 15 of 15		Query complete 00:00:00.074

GABARITO EXPLICATIVO

10

Questão 10

O setor de estoque precisa verificar se existem produtos que nunca foram vendidos, pois esses itens podem representar estoque parado e custos desnecessários. Com essa análise, a empresa pode tomar decisões sobre liquidações, promoções ou até mesmo a retirada desses produtos do catálogo.

Crie uma consulta que retorne os IDs e os nomes desses produtos, caso existam. **(Utilize as tabelas products e order_details)**

Explicação e Resultado

Para listar os produtos que nunca foram vendidos, primeiro precisamos identificar os **product_id** que aparecem na tabela **ORDER_DETAILS**. Fazemos isso utilizando uma consulta que seleciona os **product_id distintos** da tabela **ORDER_DETAILS**. Esses produtos são aqueles que foram vendidos pelo menos uma vez:

```
SELECT DISTINCT
product_id
FROM order_details
```

→

	product_id smallint
1	74
2	54
3	29
4	71
5	4
6	68
7	34
8	51
9	52
10	70
11	67
12	63
13	10
14	35
15	45

Total rows: 77 of 77

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

10

Questão 10

O setor de estoque precisa verificar se existem produtos que nunca foram vendidos, pois esses itens podem representar estoque parado e custos desnecessários. Com essa análise, a empresa pode tomar decisões sobre liquidações, promoções ou até mesmo a retirada desses produtos do catálogo.

Crie uma consulta que retorne os IDs e os nomes desses produtos, caso existam. **(Utilize as tabelas products e order_details)**

Explicação e Resultado

Agora, podemos aplicar essa consulta como uma **subquery na cláusula WHERE** da consulta principal para listar os produtos que nunca foram vendidos. A consulta principal seleciona os campos **product_id** e **product_name** da tabela **PRODUCTS**, filtrando os produtos cujos product_id não aparecem na subquery (**WHERE product_id NOT IN (...)**):

```sql
SELECT
    product_id,
    product_name
FROM products
WHERE product_id NOT IN (
                    SELECT DISTINCT
                    product_id
                    FROM order_details
                );
```

A consulta retorna o seguinte resultado, em branco, mostrando que **não existem produtos cadastrados na tabela PRODUCTS que nunca foram vendidos**:

product_id [PK] smallint	product_name character varying (40)

Total rows: 0 of 0 | Query complete 00:00:0

MÓDULO 10

VARIÁVEIS E BLOCOS ANÔNIMOS

VARIÁVEIS E BLOCOS ANÔNIMOS

VARIÁVEIS E BLOCOS ANÔNIMOS

O QUE VEREMOS NESTE MÓDULO

MÓDULO 10 VARIÁVEIS E BLOCOS ANÔNIMOS

01	Variáveis, Datatypes e Blocos Anônimos	397
02	Blocos Anônimos – Exemplo 1	401
03	Blocos Anônimos – Exemplo 2	403
04	EXERCÍCIOS	405
05	GABARITO EXPLICATIVO	410

Variáveis, Datatypes e Blocos Anônimos

VARIÁVEIS são pedaços de memória onde armazenamos alguma informação.
Uma variável está sempre associada a um tipo de dados em particular.

Os tipos de dados (Datatypes) mais comuns são:

Numéricos

INT ou DECIMAL

Textos

VARCHAR (n)

Datas

DATE

Variáveis, Datatypes e Blocos Anônimos

No PostgreSQL, é possível criar os chamados BLOCOS ANÔNIMOS.

Os blocos anônimos são blocos de códigos que executam alguma ação dentro do banco de dados: um cálculo, uma função, uma atualização ou inserção de dados em uma tabela, etc.

Eles são a base para entendermos como estruturar Functions e Procedures, que serão os assuntos dos próximos módulos.

Variáveis, Datatypes e Blocos Anônimos

Abaixo, temos a estrutura de um bloco anônimo:

```
<< label >>
declare
   declaracao;
begin
   corpo do codigo;
end label;
```

Cada bloco possui duas sessões:

Declaração (DECLARE) e Corpo (BEGIN).

A sessão de declaração é opcional. É onde declaramos todas as **variáveis** usadas no corpo do código.

A sessão do corpo é obrigatória. É onde criamos os nossos **códigos (instruções)**.

Em ambas as sessões, é obrigatório o uso do ponto-e-vírgula ao final de cada instrução.

Variáveis, Datatypes e Blocos Anônimos

Para iniciar, vejamos um exemplo bem simples de um bloco anônimo:

1. Iniciamos o bloco anônimo com uma label: **do $$**. Essa label serve para delimitar o bloco anônimo. Sem ela, o SQL não saberia onde o bloco inicia e termina. Isso porque, dentro do bloco, cada instrução termina com um ponto-e-vírgula. Sem a label, o SQL iria executar o código até o primeiro ponto-e-vírgula que aparecesse, e entenderia que o código acaba ali. Colocando a label $$ no início e ao final do bloco, a linguagem entende que aquele trecho entre a label inicial e a label final é um único bloco de códigos que devem ser executados sequencialmente;
2. Abrimos o bloco de declaração com o comando DECLARE. Dentro dele, devemos declarar as variáveis que serão utilizadas no corpo do bloco anônimo;
3. Em seguida, iniciamos o corpo do bloco com o comando BEGIN e, dentro dele, inserimos todas as instruções que deverão ser executadas. Neste exemplo, atribuímos valores às variáveis declaradas no bloco de declaração e imprimimos uma mensagem na tela (raise_notice);
4. Por fim, fechamos o bloco anônimo com a label **end $$;**

```
1 do $$
  declare
      nome varchar(100);
2     salario decimal;
      data_contratacao date;
  begin
      nome := 'André';
3     salario := 3500;
      data_contratacao := '25-10-2018';
      raise notice 'O funcionário % foi contratado em % e recebe um salário de R$ %.', nome,
  data_contratacao, salario;

4 end $$;
```

```
Data output | Messages | Notifications

NOTICE:  O funcionário André foi contratado em 2018-10-25 e recebe um salário de R$ 3500.
DO

Query returned successfully in 45 msec.
```

Blocos Anônimos – Exemplo 1

Faremos agora outro exemplo:

Vamos criar uma calculadora simples de valor vendido. Para isso, utilizaremos as variáveis 'quantidade', 'preco', 'valor_vendido' e 'vendedor'.

Veja:

```
do $$
declare
    quantidade int := 50;
    preco decimal := 100;
    valor_vendido int;
    vendedor varchar(100) := 'Bruna';

begin
    valor_vendido := quantidade * preco;

    raise notice 'O vendedor % vendeu o total de R$ %.', vendedor, valor_vendido;

end $$;
```

Data output | Messages | Notifications

```
NOTICE:  O vendedor Bruna vendeu o total de R$ 5000.
DO

Query returned successfully in 106 msec.
```

Blocos Anônimos – Exemplo 1

1. Iniciamos o bloco anônimo com a label **do $$**;

2. Abrimos o bloco de declaração com o comando DECLARE. Dentro dele, declaramos as variáveis que serão utilizadas no corpo do bloco anônimo (quantidade, preco, valor_vendido e vendedor). Repare que, neste exemplo, optamos por já atribuir os valores das variáveis quantidade, preco e vendedor no momento de sua declaração, pois isso também é possível;

3. Em seguida, iniciamos o corpo do bloco anônimo com o comando BEGIN e, dentro dele, inserimos as instruções que deverão ser executadas: a) atribuir o valor da variável valor_vendido, que será calculado multiplicando-se a variável quantidade pela variável preco; b) imprimir uma mensagem na tela (raise_notice);

4. Por fim, fechamos o bloco anônimo com a label **end $$;**

```
1  do $$
   declare
       quantidade int := 50;
2      preco decimal := 100;
       valor_vendido int;
       vendedor varchar(100) := 'Bruna';

   begin
       valor_vendido := quantidade * preco;
3
       raise notice 'O vendedor % vendeu o total de R$ %.', vendedor, valor_vendido;

4  end $$;
```

```
Data output | Messages | Notifications

NOTICE:  O vendedor Bruna vendeu o total de R$ 5000.
DO

Query returned successfully in 106 msec.
```

PostgreSQL

Índice Geral

Índice deste Módulo

Blocos Anônimos – Exemplo 2

Para fechar este módulo, agora veremos outro exemplo utilizando informações da tabela products do banco de dados Northwind.

Queremos saber quantos produtos têm o preço acima da média de preços. Para isso, faremos o seguinte bloco anônimo:

```
do $$
declare
    media_preco decimal;
    qtd_produtos_acima_media int;

begin
    media_preco = (select avg(unit_price) from products);
    qtd_produtos_acima_media = (select count(*) from products where unit_price >= media_preco);

    raise notice 'A quantidade de produtos com preço acima da média é de: % produtos.', qtd_produtos_acima_media;

end $$;
```

Data output | Messages | Notifications

```
NOTICE:  A quantidade de produtos com preço acima da média é de: 25 produtos.
DO

Query returned successfully in 69 msec.
```

Blocos Anônimos – Exemplo 2

1. Iniciamos o bloco anônimo com a label **do $$**;

2. Abrimos o bloco de declaração com o comando DECLARE. Dentro dele, declaramos as variáveis que serão utilizadas no corpo do bloco anônimo (media_preco e qtd_produtos_acima_media);

3. Em seguida, iniciamos o corpo do bloco anônimo com o comando BEGIN e, dentro dele, inserimos as instruções que deverão ser executadas: a) atribuir um valor à variável media_preco, fazendo um SELECT à tabela products para calcular o AVG(unit_price); b) atribuir um valor à variável qtd_produtos_acima_media, por meio de um SELECT à tabela products para contar (COUNT(*)) quantos produtos possuem um unit_price acima do valor atribuído à variável media_preco; c) imprimir uma mensagem na tela (raise_notice);

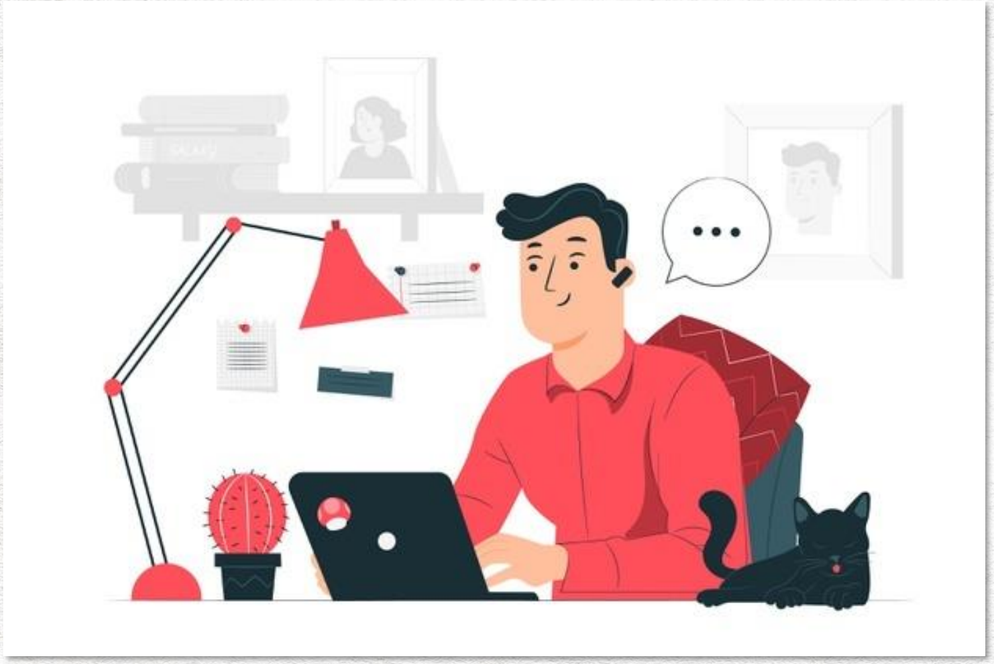
4. Por fim, fechamos o bloco anônimo com a label **end $$;**

```
1 do $$
declare
    media_preco decimal;
2   qtd_produtos_acima_media int;

begin
    media_preco = (select avg(unit_price) from products);
3   qtd_produtos_acima_media = (select count(*) from products where unit_price >= media_preco);

    raise notice 'A quantidade de produtos com preço acima da média é de: % produtos.', qtd_produtos_acima_media;

4 end $$;
```

Data output | Messages | Notifications

```
NOTICE:  A quantidade de produtos com preço acima da média é de: 25 produtos.
DO

Query returned successfully in 69 msec.
```

EXERCÍCIOS



1

Questão 1

A empresa está se preparando para uma apresentação aos investidores e precisa de um indicador geral sobre a capacidade do estoque atual. Esse número será utilizado para demonstrar a estrutura operacional da empresa e sua capacidade de atender à demanda do mercado. Para esse fim, a quantidade total de produtos em estoque é uma métrica relevante para a análise financeira e estratégica do negócio.

Crie uma consulta SQL utilizando uma variável e um bloco anônimo que retorne a quantidade total de unidades de produtos disponíveis no estoque. **(Utilize a tabela products)**

2

Questão 2

A diretoria está analisando o desempenho das vendas da empresa e deseja identificar qual foi o maior valor total já registrado em um único pedido. Essa informação ajudará na definição de estratégias de vendas, como descontos progressivos para compras maiores.

Crie uma consulta SQL utilizando uma variável e um bloco anônimo que retorne o valor total do pedido mais caro já realizado. Utilize a função ROUND no retorno para arredondar o valor total para duas casas decimais. **(Utilize a tabela order_details)**

EXERCÍCIOS

3

Questão 3

O time de precificação quer entender a diferença entre o maior e o menor preço dos produtos vendidos. Essa análise ajudará a definir estratégias como ajuste de preços, criação de pacotes promocionais e avaliação de competitividade no mercado.

Crie uma consulta SQL utilizando variáveis e um bloco anônimo que retorne a diferença entre o maior e o menor preço. Utilize a função ROUND no retorno para arredondar o valor da diferença para duas casas decimais. **(Utilize a tabela products)**

4

Questão 4

O gerente de vendas deseja analisar a média de preços dos produtos da categoria "Bebidas" (Beverages) para definir futuras estratégias, como promoções sazonais ou reajustes de preços. Esse dado também pode ser útil para identificar oportunidades de introdução de novos produtos dentro da mesma faixa de preço.

Crie uma consulta SQL utilizando variáveis e um bloco anônimo que retorne o nome da categoria pesquisada e sua média de preços. Utilize a função ROUND no retorno para arredondar a média para duas casas decimais. **(Utilize as tabelas products e categories)**

EXERCÍCIOS

5

Questão 5

O departamento financeiro precisa calcular a comissão dos vendedores, que é baseada em um percentual fixo de 5% sobre o total de vendas realizadas. Essa informação será usada para o pagamento dos funcionários e para avaliar o desempenho individual dos vendedores.

Crie uma consulta SQL utilizando variáveis e um bloco anônimo que retorne o nome completo do vendedor, seu total de vendas e a comissão que receberá. Utilize a função ROUND no retorno para arredondar o total de vendas e a comissão para duas casas decimais. **(Utilize as tabelas order_details, orders e employees)**

6

Questão 6

O setor de relacionamento com o cliente deseja identificar quem é o cliente mais ativo, ou seja, aquele que mais realizou pedidos na empresa. Com essa informação, a equipe poderá oferecer benefícios exclusivos, como descontos especiais ou atendimento personalizado, para fortalecer a fidelização. Além disso, entender o perfil comportamental desse cliente pode ajudar a empresa a identificar padrões de consumo e ajustar suas estratégias de marketing e atendimento para melhorar a experiência de outros clientes, aumentando a retenção e o engajamento.

Crie uma consulta SQL utilizando variáveis e um bloco anônimo que retorne o nome do cliente e o total de pedidos por ele efetuado. **(Utilize as tabelas customers e orders)**

EXERCÍCIOS

7

Questão 7

O setor de compras quer identificar a categoria de produtos com menor quantidade cadastrada para avaliar a necessidade de expandir o portfólio de produtos. Essa análise é essencial para garantir um mix de produtos diversificado e atender melhor às necessidades dos clientes.

Crie uma consulta SQL utilizando variáveis e um bloco anônimo que retorne o nome da categoria com menos produtos, bem como sua quantidade de produtos. **(Utilize as tabelas categories e products)**

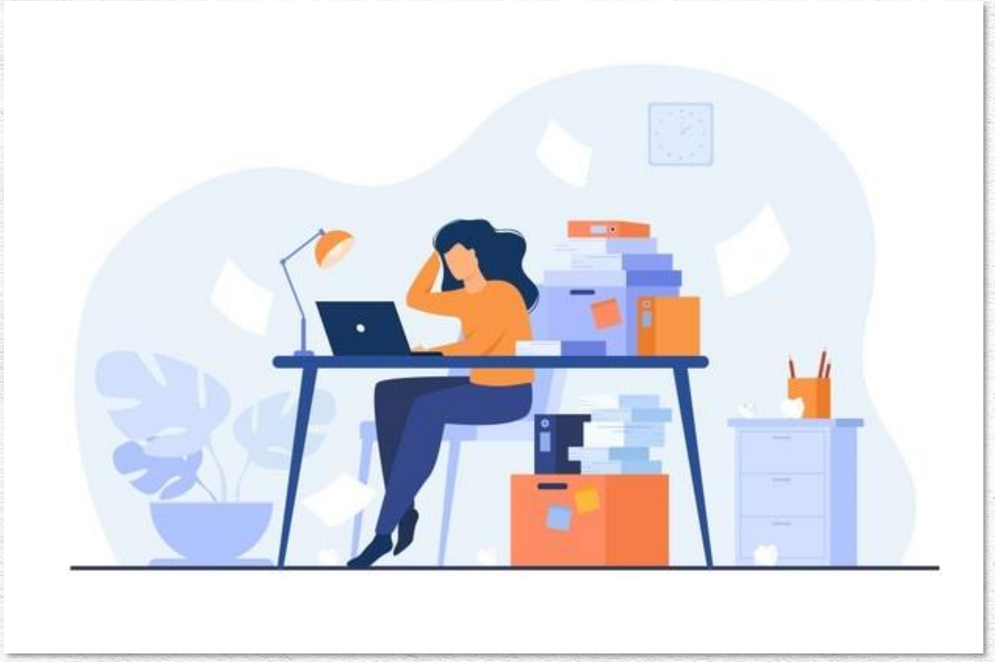
8

Questão 8

O setor de precificação quer identificar os produtos com os preços mais altos e mais baixos para avaliar o posicionamento de mercado da empresa. Essas informações ajudarão na criação de pacotes promocionais, definição de margens de lucro e até mesmo ajustes na precificação para aumentar a competitividade.

Crie uma consulta SQL utilizando variáveis e um bloco anônimo que retorne os nomes dos produtos mais caro e mais barato e seus respectivos preços. Utilize a função ROUND no retorno para arredondar os preços para duas casas decimais. **(Utilize a tabela products)**

EXERCÍCIOS



9

Questão 9

O gerente de vendas deseja saber quantos produtos da categoria "Confecções" (Confections) foram vendidos. Com essa informação, a equipe poderá avaliar a demanda desse segmento e decidir se precisa aumentar a oferta, reduzir preços ou investir em campanhas de marketing para impulsionar as vendas.

Crie uma consulta SQL utilizando variáveis e um bloco anônimo que retorne o nome da categoria e a quantidade de produtos vendidos. **(Utilize as tabelas order_details, products e categories)**

10

Questão 10

O departamento de Recursos Humanos deseja identificar o funcionário mais antigo da empresa para prestar-lhe uma homenagem como forma de reconhecer a dedicação e experiência do colaborador, aumentando a motivação e a valorização da equipe.

Crie uma consulta SQL utilizando variáveis e um bloco anônimo que retorne o nome completo desse funcionário e sua data de contratação. **(Utilize a tabela employees)**

GABARITO EXPLICATIVO

1

Questão 1

A empresa está se preparando para uma apresentação aos investidores e precisa de um indicador geral sobre a capacidade do estoque atual. Esse número será utilizado para demonstrar a estrutura operacional da empresa e sua capacidade de atender à demanda do mercado. Para esse fim, a quantidade total de produtos em estoque é uma métrica relevante para a análise financeira e estratégica do negócio.

Crie uma consulta SQL utilizando uma variável e um bloco anônimo que retorne a quantidade total de unidades de produtos disponíveis no estoque. **(Utilize a tabela products)**

Explicação e Resultado

Para solucionar esta questão, fazemos o seguinte:

Iniciamos o bloco anônimo com a label **do $$**;

Abrimos o bloco de declaração com o comando **DECLARE**. Dentro dele, declaramos a variável que será utilizada no corpo do bloco anônimo (**total_estoque**, do tipo **INT**);

Em seguida, iniciamos o corpo do bloco anônimo com o comando **BEGIN** e, dentro dele, inserimos as instruções que deverão ser executadas:

- ✓ Atribuir um valor à variável **total_estoque**, fazendo um **SELECT** à tabela **PRODUCTS** para calcular a **SUM(units_in_stock)**;
- ✓ Imprimir uma mensagem na tela (**RAISE_NOTICE**);

Por fim, fechamos o bloco anônimo com a label **end $$;**.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

1

Questão 1

A empresa está se preparando para uma apresentação aos investidores e precisa de um indicador geral sobre a capacidade do estoque atual. Esse número será utilizado para demonstrar a estrutura operacional da empresa e sua capacidade de atender à demanda do mercado. Para esse fim, a quantidade total de produtos em estoque é uma métrica relevante para a análise financeira e estratégica do negócio.

Crie uma consulta SQL utilizando uma variável e um bloco anônimo que retorne a quantidade total de unidades de produtos disponíveis no estoque. **(Utilize a tabela products)**

Explicação e Resultado

```
do $$
declare
    total_estoque int;
begin
    total_estoque := (select sum(units_in_stock) from products);

    raise notice 'A quantidade total de unidades de produtos disponíveis no estoque é: %',
                 total_estoque;
end $$;
```

Dessa forma, retornamos uma mensagem com a quantidade total de unidades de produtos disponíveis no estoque:

Data Output | Messages | Notifications

```
NOTICE:  A quantidade total de unidades de produtos disponíveis no estoque é: 3119
DO

Query returned successfully in 68 msec.
```

Total rows: 0 of 0 | Query complete 00:00:00.068

GABARITO EXPLICATIVO

2

Questão 2

A diretoria está analisando o desempenho das vendas da empresa e deseja identificar qual foi o maior valor total já registrado em um único pedido. Essa informação ajudará na definição de estratégias de vendas, como descontos progressivos para compras maiores.

Crie uma consulta SQL utilizando uma variável e um bloco anônimo que retorne o valor total do pedido mais caro já realizado. Utilize a função ROUND no retorno para arredondar o valor total para duas casas decimais. **(Utilize a tabela order_details)**

Explicação e Resultado

Para solucionar esta questão, fazemos o seguinte:

Iniciamos o bloco anônimo com a label **do $$**;

Abrimos o bloco de declaração com o comando **DECLARE**. Dentro dele, declaramos a variável que será utilizada no corpo do bloco anônimo (**pedido_mais_caro**, do tipo **DECIMAL**);

Em seguida, iniciamos o corpo do bloco anônimo com o comando **BEGIN** e, dentro dele, inserimos as instruções que deverão ser executadas:

- ✓ Atribuímos um valor à variável **pedido_mais_caro**, utilizando um **SELECT** para calcular o maior valor total já registrado em um único pedido, **multiplicando as colunas unit_price por quantity**, ambas da tabela **ORDER_DETAILS** e aplicando a função **MAX**;
- ✓ Imprimimos uma mensagem na tela (**RAISE NOTICE**), utilizando a função **ROUND** para arredondar o valor total para duas casas decimais;

Por fim, fechamos o bloco anônimo com a label **end $$;**.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

2

Questão 2

A diretoria está analisando o desempenho das vendas da empresa e deseja identificar qual foi o maior valor total já registrado em um único pedido. Essa informação ajudará na definição de estratégias de vendas, como descontos progressivos para compras maiores.

Crie uma consulta SQL utilizando uma variável e um bloco anônimo que retorne o valor total do pedido mais caro já realizado. Utilize a função ROUND no retorno para arredondar o valor total para duas casas decimais. **(Utilize a tabela order_details)**

Explicação e Resultado

```
do $$
declare
    pedido_mais_caro decimal;
begin
    pedido_mais_caro := (select max(unit_price * quantity) from order_details);

    raise notice 'O pedido mais caro já realizado teve um valor total de R$ %',
                 ROUND(pedido_mais_caro, 2);
end $$;
```

Dessa forma, retornamos uma mensagem informando o maior valor total já registrado em um único pedido:

Data Output | Messages | Notifications

```
NOTICE:  O pedido mais caro já realizado teve um valor total de R$ 15810.00
DO

Query returned successfully in 95 msec.
```

Total rows: 0 of 0 | Query complete 00:00:00.095

GABARITO EXPLICATIVO

3

Questão 3

O time de precificação quer entender a diferença entre o maior e o menor preço dos produtos vendidos. Essa análise ajudará a definir estratégias como ajuste de preços, criação de pacotes promocionais e avaliação de competitividade no mercado.

Crie uma consulta SQL utilizando variáveis e um bloco anônimo que retorne a diferença entre o maior e o menor preço. Utilize a função ROUND no retorno para arredondar o valor da diferença para duas casas decimais. **(Utilize a tabela products)**

Explicação e Resultado

Para solucionar esta questão, fazemos o seguinte:

Iniciamos o bloco anônimo com a label **do $$**;

Abrimos o bloco de declaração com o comando **DECLARE**. Dentro dele, declaramos as variáveis **preco_min**, **preco_max** e **diferenca**, todas do tipo **DECIMAL**, que serão utilizadas no corpo do bloco anônimo;

Em seguida, iniciamos o corpo do bloco anônimo com o comando **BEGIN** e, dentro dele, inserimos as instruções que deverão ser executadas:

Atribuímos um valor à variável **preco_min**, utilizando um **SELECT** para calcular o menor preço de produto, com a função **MIN** aplicada à coluna **unit_price** da tabela **PRODUCTS**;

Atribuímos um valor à variável **preco_max**, utilizando um **SELECT** para calcular o maior preço de produto, com a função **MAX** aplicada à coluna **unit_price** da tabela **PRODUCTS**;

Calculamos a diferença entre o maior e o menor preço e armazenamos o valor na variável **diferenca**;

Imprimimos uma mensagem na tela utilizando o comando **RAISE NOTICE**, exibindo a diferença entre os preços, arredondada para duas casas decimais com a função **ROUND**;

Por fim, fechamos o bloco anônimo com a label **end $$;**.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

3

Questão 3

O time de precificação quer entender a diferença entre o maior e o menor preço dos produtos vendidos. Essa análise ajudará a definir estratégias como ajuste de preços, criação de pacotes promocionais e avaliação de competitividade no mercado.

Crie uma consulta SQL utilizando variáveis e um bloco anônimo que retorne a diferença entre o maior e o menor preço. Utilize a função ROUND no retorno para arredondar o valor da diferença para duas casas decimais. **(Utilize a tabela products)**

Explicação e Resultado

```
do $$
declare
    preco_min decimal;
    preco_max decimal;
    diferenca decimal;
begin
    preco_min := (select min(unit_price) from products);
    preco_max := (select max(unit_price) from products);
    diferenca := preco_max - preco_min;

    raise notice 'A diferença entre o maior e o menor preço de produto é de R$ %',
                 ROUND(diferenca, 2);
end $$;
```

Dessa forma, retornamos uma mensagem com a diferença entre o maior e o menor preço de produto:

Data Output | Messages | Notifications

```
NOTICE:  A diferença entre o maior e o menor preço de produto é de R$ 261.00
DO

Query returned successfully in 43 msec.
```

Total rows: 0 of 0 | Query complete 00:00:00.043

GABARITO EXPLICATIVO

4

Questão 4

O gerente de vendas deseja analisar a média de preços dos produtos da categoria "Bebidas" (Beverages) para definir futuras estratégias, como promoções sazonais ou reajustes de preços. Esse dado também pode ser útil para identificar oportunidades de introdução de novos produtos dentro da mesma faixa de preço.

Crie uma consulta SQL utilizando variáveis e um bloco anônimo que retorne o nome da categoria pesquisada e sua média de preços. Utilize a função ROUND no retorno para arredondar a média para duas casas decimais. **(Utilize as tabelas products e categories)**

Explicação e Resultado

Para solucionar esta questão, fazemos o seguinte:

Iniciamos o bloco anônimo com a label **do $$**;

Abrimos o bloco de declaração com o comando **DECLARE**. Dentro dele, declaramos as variáveis **categoria** (inicializada com o valor **'Beverages'**) e **media_preco**, ambas **do tipo varchar e decimal, respectivamente,** que serão utilizadas no corpo do bloco anônimo;

Em seguida, iniciamos o corpo do bloco anônimo com o comando **BEGIN** e, dentro dele, inserimos as instruções que deverão ser executadas:

- ✓ Atribuímos um valor à variável **media_preco**, utilizando um **SELECT** que calcula a média dos preços (**AVG**) da coluna **unit_price** da tabela **PRODUCTS**, unindo-a (**JOIN**) à tabela **CATEGORIES** pela coluna **category_id** e filtrando pela **category_name** da tabela **CATEGORIES**, que corresponde à categoria desejada;
- ✓ Imprimimos uma mensagem na tela utilizando o comando **RAISE NOTICE**, exibindo o nome da categoria e a média de preços arredondada para duas casas decimais com a função **ROUND**;

Por fim, fechamos o bloco anônimo com a label **end $$;**.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

4

Questão 4

O gerente de vendas deseja analisar a média de preços dos produtos da categoria "Bebidas" (Beverages) para definir futuras estratégias, como promoções sazonais ou reajustes de preços. Esse dado também pode ser útil para identificar oportunidades de introdução de novos produtos dentro da mesma faixa de preço.

Crie uma consulta SQL utilizando variáveis e um bloco anônimo que retorne o nome da categoria pesquisada e sua média de preços. Utilize a função ROUND no retorno para arredondar a média para duas casas decimais. **(Utilize as tabelas products e categories)**

Explicação e Resultado

```
do $$
declare
    categoria varchar(30) = 'Beverages';
    media_preco decimal;
begin
    media_preco := (select avg(unit_price) from products p
                    join categories c on p.category_id = c.category_id
                    where c.category_name = categoria);

    raise notice 'A média de preços dos produtos da categoria % é de R$ %',
                    categoria, ROUND(media_preco, 2);
end $$;
```

Dessa forma, retornamos uma mensagem com o nome da categoria e a média dos preços:

Data Output | Messages | Notifications

```
NOTICE:  A média de preços dos produtos da categoria Beverages é de R$ 37.98
DO

Query returned successfully in 40 msec.
```

Total rows: 0 of 0 | Query complete 00:00:00.040

GABARITO EXPLICATIVO

5

Questão 5

O departamento financeiro precisa calcular a comissão dos vendedores, que é baseada em um percentual fixo de 5% sobre o total de vendas realizadas. Essa informação será usada para o pagamento dos funcionários e para avaliar o desempenho individual dos vendedores.

Crie uma consulta SQL utilizando variáveis e um bloco anônimo que retorne o nome completo do vendedor, seu total de vendas e a comissão que receberá. Utilize a função ROUND no retorno para arredondar o total de vendas e a comissão para duas casas decimais. **(Utilize as tabelas order_details, orders e employees)**

Explicação e Resultado

Para solucionar esta questão, fazemos o seguinte:

Iniciamos o bloco anônimo com a label **do $$**;

Abrimos o bloco de declaração com o comando **DECLARE**. Dentro dele, declaramos as variáveis **nome_vendedor**, **sobrenome_vendedor** (inicializadas com **'Nancy'** e **'Davolio'**), **total_vendas** e **comissao**, dos **tipos varchar e decimal**, que serão utilizadas no corpo do bloco anônimo;

Em seguida, iniciamos o corpo do bloco anônimo com o comando **BEGIN** e, dentro dele, inserimos as instruções que deverão ser executadas:

- ✓ Atribuímos um valor à variável **total_vendas**, utilizando um **SELECT** para calcular o total de vendas, **multiplicando** as colunas **unit_price e quantity** da tabela **ORDER_DETAILS**, unindo-a (**JOIN**) às tabelas **ORDERS** e **EMPLOYEES** pelas colunas **order_id** e **employee_id**, e filtrando pelo **first_name** e **last_name** da tabela **EMPLOYEES**;
- ✓ Calculamos a **comissão** aplicando o **percentual de 5%** sobre o total de vendas;
- ✓ Imprimimos uma mensagem na tela utilizando o comando **RAISE NOTICE**, exibindo o nome completo do vendedor, o total de vendas e a comissão (ambos arredondados para duas casas decimais com a função **ROUND**);

Por fim, fechamos o bloco anônimo com a label **end $$;**.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

5

Questão 5

O departamento financeiro precisa calcular a comissão dos vendedores, que é baseada em um percentual fixo de 5% sobre o total de vendas realizadas. Essa informação será usada para o pagamento dos funcionários e para avaliar o desempenho individual dos vendedores.

Crie uma consulta SQL utilizando variáveis e um bloco anônimo que retorne o nome completo do vendedor, seu total de vendas e a comissão que receberá. Utilize a função ROUND no retorno para arredondar o total de vendas e a comissão para duas casas decimais. **(Utilize as tabelas order_details, orders e employees)**

Explicação e Resultado

```
do $$
declare
    nome_vendedor varchar(30) := 'Nancy';
    sobrenome_vendedor varchar(50) := 'Davolio';
    total_vendas decimal;
    comissao decimal;
begin
    total_vendas := (select sum(unit_price * quantity) from order_details od
                     join orders o on od.order_id = o.order_id
                     join employees e on o.employee_id = e.employee_id
                     where e.first_name = nome_vendedor and e.last_name = sobrenome_vendedor);

    comissao := total_vendas * 0.05;

    raise notice 'O vendedor % % vendeu R$ %. Sua comissão é de R$ %',
                  nome_vendedor, sobrenome_vendedor, ROUND(total_vendas, 2), ROUND(comissao, 2);
end $$;
```

Dessa forma, retornamos uma mensagem com o nome do vendedor, o total de vendas e a comissão:

Data Output | Messages | Notifications

```
NOTICE:  O vendedor Nancy Davolio vendeu R$ 202143.71. Sua comissão é de R$ 10107.19
DO

Query returned successfully in 46 msec.
```

Total rows: 0 of 0 | Query complete 00:00:00.046

GABARITO EXPLICATIVO

6

Questão 6

O setor de relacionamento com o cliente deseja identificar quem é o cliente mais ativo, ou seja, aquele que mais realizou pedidos na empresa. Com essa informação, a equipe poderá oferecer benefícios exclusivos, como descontos especiais ou atendimento personalizado, para fortalecer a fidelização. Além disso, entender o perfil comportamental desse cliente pode ajudar a empresa a identificar padrões de consumo e ajustar suas estratégias de marketing e atendimento para melhorar a experiência de outros clientes, aumentando a retenção e o engajamento.

Crie uma consulta SQL utilizando variáveis e um bloco anônimo que retorne o nome do cliente e o total de pedidos por ele efetuado. **(Utilize as tabelas customers e orders)**

Explicação e Resultado

Para solucionar esta questão, fazemos o seguinte:

Iniciamos o bloco anônimo com a label **do $$**;

Abrimos o bloco de declaração com o comando **DECLARE**. Dentro dele, declaramos as variáveis **melhor_cliente** do **tipo varchar** e **max_pedidos** do **tipo int**, que serão utilizadas no corpo do bloco anônimo;

Em seguida, iniciamos o corpo do bloco anônimo com o comando **BEGIN** e, dentro dele, inserimos as instruções que deverão ser executadas:

- ✓ Atribuímos um valor à variável **melhor_cliente**, utilizando um **SELECT** para identificar o nome do cliente com mais pedidos. A consulta une as tabelas **CUSTOMERS** e **ORDERS** pela coluna **customer_id**, agrupa pelo nome (**company_name**) e **customer_id**, ordena pela contagem de pedidos (**count(o.order_id)**) em ordem **decrescente** e seleciona o primeiro resultado com **LIMIT 1**;
- ✓ Atribuímos um valor à variável **max_pedidos**, utilizando um **SELECT** que conta (**COUNT**) a quantidade de pedidos (**order_id** da tabela **ORDERS**) realizados pelo cliente armazenado na variável **melhor_cliente**;
- ✓ Imprimimos uma mensagem na tela utilizando o comando **RAISE NOTICE**, exibindo o nome do cliente e o total de pedidos efetuados;

Por fim, fechamos o bloco anônimo com a label **end $$;**.

PostgreSQL

Índice Geral

Índice deste Módulo

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

6

Questão 6

O setor de relacionamento com o cliente deseja identificar quem é o cliente mais ativo, ou seja, aquele que mais realizou pedidos na empresa. Com essa informação, a equipe poderá oferecer benefícios exclusivos, como descontos especiais ou atendimento personalizado, para fortalecer a fidelização. Além disso, entender o perfil comportamental desse cliente pode ajudar a empresa a identificar padrões de consumo e ajustar suas estratégias de marketing e atendimento para melhorar a experiência de outros clientes, aumentando a retenção e o engajamento.

Crie uma consulta SQL utilizando variáveis e um bloco anônimo que retorne o nome do cliente e o total de pedidos por ele efetuado. **(Utilize as tabelas customers e orders)**

Explicação e Resultado

```
do $$
declare
    melhor_cliente varchar(100);
    max_pedidos int;
begin
    melhor_cliente := (select company_name
                       from customers c
                       join orders o on c.customer_id = o.customer_id
                       group by c.customer_id, c.company_name
                       order by count(o.order_id) desc
                       limit 1);

    max_pedidos := (select count(o.order_id)
                    from customers c
                    join orders o on c.customer_id = o.customer_id
                    where c.company_name = melhor_cliente);

    raise notice 'O cliente com mais pedidos é % com um total de % pedidos.',
                    melhor_cliente, max_pedidos;
end $$;
```

Dessa forma, retornamos uma mensagem com o nome do cliente mais ativo e a quantidade de pedidos realizados:

Data Output | Messages | Notifications

```
NOTICE:  O cliente com mais pedidos é Save-a-lot Markets com um total de 31 pedidos.
DO

Query returned successfully in 66 msec.
```

Total rows: 0 of 0 | Query complete 00:00:00.066

PostgreSQL

GABARITO EXPLICATIVO

7

Questão 7

O setor de compras quer identificar a categoria de produtos com menor quantidade cadastrada para avaliar a necessidade de expandir o portfólio de produtos. Essa análise é essencial para garantir um mix de produtos diversificado e atender melhor às necessidades dos clientes.

Crie uma consulta SQL utilizando variáveis e um bloco anônimo que retorne o nome da categoria com menos produtos, bem como sua quantidade de produtos. **(Utilize as tabelas categories e products)**

Explicação e Resultado

Para solucionar esta questão, fazemos o seguinte:

Iniciamos o bloco anônimo com a label **do $$**;

Abrimos o bloco de declaração com o comando **DECLARE**. Dentro dele, declaramos as variáveis **categoria_menos_produtos** do **tipo varchar** e **qtd_produtos** do **tipo int**, que serão utilizadas no corpo do bloco anônimo;

Em seguida, iniciamos o corpo do bloco anônimo com o comando **BEGIN** e, dentro dele, inserimos as instruções que deverão ser executadas:

- ✓ Atribuímos um valor à variável **categoria_menos_produtos**, utilizando um **SELECT** que identifica a categoria com menos produtos. A consulta une as tabelas **CATEGORIES** e **PRODUCTS** pela coluna **category_id**, agrupa por **category_id** e **category_name**, ordena pela quantidade de produtos (**count(p.product_id)**) em ordem crescente e seleciona o primeiro resultado com **LIMIT 1**;
- ✓ Atribuímos um valor à variável **qtd_produtos**, utilizando um **SELECT** que conta (**count(*)**) a quantidade de produtos da tabela **PRODUCTS** para a categoria armazenada na variável **categoria_menos_produtos**;
- ✓ Imprimimos uma mensagem na tela utilizando o comando **RAISE NOTICE**, exibindo o nome da categoria e a quantidade de produtos cadastrados;

Por fim, fechamos o bloco anônimo com a label **end $$;**.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

7

Questão 7

O setor de compras quer identificar a categoria de produtos com menor quantidade cadastrada para avaliar a necessidade de expandir o portfólio de produtos. Essa análise é essencial para garantir um mix de produtos diversificado e atender melhor às necessidades dos clientes.

Crie uma consulta SQL utilizando variáveis e um bloco anônimo que retorne o nome da categoria com menos produtos, bem como sua quantidade de produtos. **(Utilize as tabelas categories e products)**

Explicação e Resultado

```
do $$
declare
    categoria_menos_produtos varchar(100);
    qtd_produtos int;
begin
    categoria_menos_produtos := (select category_name
                                 from categories c
                                 join products p on c.category_id = p.category_id
                                 group by c.category_id, c.category_name
                                 order by count(p.product_id)
                                 limit 1);

    qtd_produtos := (select count(*)
                     from products p
                     join categories c on p.category_id = c.category_id
                     where c.category_name = categoria_menos_produtos);

    raise notice 'A categoria com menos produtos é % com um total de % produtos.',
                 categoria_menos_produtos, qtd_produtos;
end $$;
```

Dessa forma, retornamos uma mensagem com a categoria de produtos com menor quantidade e o total de produtos:

Data Output | Messages | Notifications

```
NOTICE:  A categoria com menos produtos é Produce com um total de 5 produtos.
DO

Query returned successfully in 42 msec.
```

Total rows: 0 of 0 | Query complete 00:00:00.042

GABARITO EXPLICATIVO

8

Questão 8

O setor de precificação quer identificar os produtos com os preços mais altos e mais baixos para avaliar o posicionamento de mercado da empresa. Essas informações ajudarão na criação de pacotes promocionais, definição de margens de lucro e até mesmo ajustes na precificação para aumentar a competitividade.

Crie uma consulta SQL utilizando variáveis e um bloco anônimo que retorne os nomes dos produtos mais caro e mais barato e seus respectivos preços. Utilize a função ROUND no retorno para arredondar os preços para duas casas decimais. **(Utilize a tabela products)**

Explicação e Resultado

Para solucionar esta questão, fazemos o seguinte:

Iniciamos o bloco anônimo com a label **do $$**;

Abrimos o bloco de declaração com o comando **DECLARE**. Dentro dele, declaramos as variáveis **produto_mais_caro**, **produto_mais_barato** (ambas do **tipo varchar**), **preco_max** e **preco_min** (ambas do **tipo decimal**), que serão utilizadas no corpo do bloco anônimo;

Em seguida, iniciamos o corpo do bloco anônimo com o comando **BEGIN** e, dentro dele, inserimos as instruções que deverão ser executadas:

- ✓ Atribuímos um valor à variável **preco_max**, utilizando um **SELECT** para calcular o maior preço de produto, aplicando a função **MAX** à coluna **unit_price** da tabela **PRODUCTS**;
- ✓ Atribuímos um valor à variável **produto_mais_caro**, utilizando um **SELECT** para buscar o **product_name** da tabela **PRODUCTS** cujo **unit_price** é igual ao valor de **preco_max**;
- ✓ Atribuímos um valor à variável **preco_min**, utilizando um **SELECT** para calcular o menor preço de produto, aplicando a função **MIN** à coluna **unit_price** da tabela **PRODUCTS**;
- ✓ Atribuímos um valor à variável **produto_mais_barato**, utilizando um **SELECT** para buscar o **product_name** da tabela **PRODUCTS** cujo **unit_price** é igual ao valor de **preco_min**;
- ✓ Imprimimos duas mensagens na tela utilizando o comando **RAISE NOTICE**: uma exibindo o nome do produto mais caro e seu preço arredondado para duas casas decimais com a função **ROUND**, e outra exibindo o nome do produto mais barato e seu preço também arredondado;

Por fim, fechamos o bloco anônimo com a label **end $$;**.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

8

Questão 8

O setor de precificação quer identificar os produtos com os preços mais altos e mais baixos para avaliar o posicionamento de mercado da empresa. Essas informações ajudarão na criação de pacotes promocionais, definição de margens de lucro e até mesmo ajustes na precificação para aumentar a competitividade.

Crie uma consulta SQL utilizando variáveis e um bloco anônimo que retorne os nomes dos produtos mais caro e mais barato e seus respectivos preços. Utilize a função ROUND no retorno para arredondar os preços para duas casas decimais. **(Utilize a tabela products)**

Explicação e Resultado

```
do $$
declare
    produto_mais_caro varchar(100);
    produto_mais_barato varchar(100);
    preco_max decimal;
    preco_min decimal;
begin
    preco_max := (select max(unit_price) from products);
    produto_mais_caro := (select product_name from products where unit_price = preco_max);

    preco_min := (select min(unit_price) from products);
    produto_mais_barato := (select product_name from products where unit_price = preco_min);


    raise notice 'O produto mais caro é % com preço de R$ %.', produto_mais_caro, ROUND(preco_max, 2);
    raise notice 'O produto mais barato é % com preço de R$ %.', produto_mais_barato, ROUND(preco_min, 2);
end $$;
```

Dessa forma, retornamos duas mensagens com os nomes dos produtos mais caro e mais barato, e seus respectivos preços:

Data Output | Messages | Notifications

```
NOTICE:  O produto mais caro é Côte de Blaye com preço de R$ 263.50.
NOTICE:  O produto mais barato é Geitost com preço de R$ 2.50.
DO

Query returned successfully in 49 msec.
```

Total rows: 0 of 0 | Query complete 00:00:00.049

GABARITO EXPLICATIVO

9

Questão 9

O gerente de vendas deseja saber quantos produtos da categoria "Confecções" (Confections) foram vendidos. Com essa informação, a equipe poderá avaliar a demanda desse segmento e decidir se precisa aumentar a oferta, reduzir preços ou investir em campanhas de marketing para impulsionar as vendas.

Crie uma consulta SQL utilizando variáveis e um bloco anônimo que retorne o nome da categoria e a quantidade de produtos vendidos. **(Utilize as tabelas order_details, products e categories)**

Explicação e Resultado

Para solucionar esta questão, fazemos o seguinte:

Iniciamos o bloco anônimo com a label **do $$**;

Abrimos o bloco de declaração com o comando **DECLARE**. Dentro dele, declaramos as variáveis que serão utilizadas no corpo do bloco anônimo (**categoria** do **tipo varchar** e **total_vendidos** do **tipo int**);

Em seguida, iniciamos o corpo do bloco anônimo com o comando **BEGIN** e, dentro dele, inserimos as instruções que deverão ser executadas:

- ✓ Atribuímos um valor à variável **total_vendidos**, utilizando um **SELECT** para calcular a soma de todos os produtos vendidos, somando a coluna **quantity** da tabela **ORDER_DETAILS**, que foi relacionada à tabela **PRODUCTS** por meio da coluna **product_id** e, posteriormente, à tabela **CATEGORIES** por meio da coluna **category_id**, filtrando apenas a categoria **'Confections'**;
- ✓ Imprimimos uma mensagem na tela (**RAISE NOTICE**), exibindo o nome da categoria e o total de produtos vendidos;

Por fim, fechamos o bloco anônimo com a label **end $$;**.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

9

Questão 9

O gerente de vendas deseja saber quantos produtos da categoria "Confecções" (Confections) foram vendidos. Com essa informação, a equipe poderá avaliar a demanda desse segmento e decidir se precisa aumentar a oferta, reduzir preços ou investir em campanhas de marketing para impulsionar as vendas.

Crie uma consulta SQL utilizando variáveis e um bloco anônimo que retorne o nome da categoria e a quantidade de produtos vendidos. **(Utilize as tabelas order_details, products e categories)**

Explicação e Resultado

```
do $$
declare
    categoria varchar(100) := 'Confections';
    total_vendidos int;
begin
    total_vendidos := (select sum(od.quantity)
                       from order_details od
                       join products p on od.product_id = p.product_id
                       join categories c on p.category_id = c.category_id
                       where c.category_name = categoria);

    raise notice 'A categoria % teve um total de % produtos vendidos.',
                 categoria, total_vendidos;
end $$;
```

Dessa forma, retornamos uma mensagem com o nome da categoria e a quantidade total de produtos vendidos:

Data Output | Messages | Notifications

```
NOTICE:  A categoria Confections teve um total de 7906 produtos vendidos.
DO

Query returned successfully in 39 msec.
```

Total rows: 0 of 0 | Query complete 00:00:00.039

GABARITO EXPLICATIVO

10

Questão 10

O departamento de Recursos Humanos deseja identificar o funcionário mais antigo da empresa para prestar-lhe uma homenagem como forma de reconhecer a dedicação e experiência do colaborador, aumentando a motivação e a valorização da equipe.

Crie uma consulta SQL utilizando variáveis e um bloco anônimo que retorne o nome completo desse funcionário e sua data de contratação. **(Utilize a tabela employees)**

Explicação e Resultado

Para solucionar esta questão, fazemos o seguinte:

Iniciamos o bloco anônimo com a label **do $$**;

Abrimos o bloco de declaração com o comando **DECLARE**. Dentro dele, declaramos as variáveis que serão utilizadas no corpo do bloco anônimo (**nome_funcionario_antigo**, **sobrenome_funcionario_antigo**, ambas do **tipo varchar**, e **data_contratacao**, do **tipo date**);

Em seguida, iniciamos o corpo do bloco anônimo com o comando **BEGIN** e, dentro dele, inserimos as instruções que deverão ser executadas:

- ✓ Atribuímos um valor à variável **data_contratacao**, utilizando um **SELECT** para obter a data de contratação mais antiga da tabela **EMPLOYEES**, **ordenando** pela coluna **hire_date** de forma **ascendente** e **limitando** o resultado a **1 registro**;
- ✓ Atribuímos um valor à variável **nome_funcionario_antigo**, utilizando um **SELECT** para obter o **first_name** do funcionário cuja data de contratação corresponde à **data_contratacao** da tabela **EMPLOYEES**;
- ✓ Atribuímos um valor à variável **sobrenome_funcionario_antigo**, utilizando um **SELECT** para obter o **last_name** do funcionário cuja data de contratação corresponde à **data_contratacao** da tabela **EMPLOYEES**;
- ✓ Imprimimos uma mensagem na tela (**RAISE NOTICE**), exibindo o nome completo do funcionário mais antigo e sua data de contratação;

Por fim, fechamos o bloco anônimo com a label **end $$;**.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

10

Questão 10

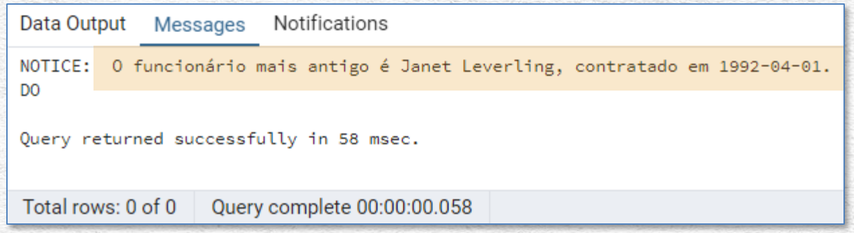
O departamento de Recursos Humanos deseja identificar o funcionário mais antigo da empresa para prestar-lhe uma homenagem como forma de reconhecer a dedicação e experiência do colaborador, aumentando a motivação e a valorização da equipe.

Crie uma consulta SQL utilizando variáveis e um bloco anônimo que retorne o nome completo desse funcionário e sua data de contratação. **(Utilize a tabela employees)**

Explicação e Resultado

```
do $$

declare
    nome_funcionario_antigo varchar(50);
    sobrenome_funcionario_antigo varchar(50);
    data_contratacao date;
begin

    data_contratacao := (select hire_date from employees
                         order by hire_date asc limit 1);

    nome_funcionario_antigo := (select first_name from employees
                                where hire_date = data_contratacao);

    sobrenome_funcionario_antigo := (select last_name from employees
                                where hire_date = data_contratacao);


    raise notice 'O funcionário mais antigo é % %, contratado em %.',
                  nome_funcionario_antigo, sobrenome_funcionario_antigo, data_contratacao;
end $$;
```

Dessa forma, retornamos uma mensagem com o nome completo do funcionário mais antigo e sua data de contratação:



MÓDULO 11

FUNCTIONS

FUNCTIONS
FUNCTIONS

PostgreSQL

Índice Geral
Índice deste Módulo

O QUE VEREMOS NESTE MÓDULO

MÓDULO 11 FUNCTIONS

01	O que é uma Function?	432
02	Como criar uma Function?	433
03	Formas de chamar uma Function	440
04	Como excluir uma Function?	443
05	Functions que retornam uma tabela	445
06	EXERCÍCIOS	451
07	GABARITO EXPLICATIVO	456

O que é uma Function?

Uma FUNCTION é um conjunto de comandos que executam ações e retornam um valor. As functions ajudam a simplificar um código.

Por exemplo, se você tem um cálculo complexo que aparece diversas vezes em seu código, em vez de repetir várias vezes uma série de comandos, você pode simplesmente criar uma função e reaproveitá-la sempre que precisar.

O próprio SQL tem diversas funções do sistema (prontas) e, até agora, já vimos vários exemplos de funções deste tipo, como funções de número, data, texto, agregação, etc.

A essas funções que podemos criar e personalizar damos o nome de User-Defined Functions (UDFs), ou Funções Definidas pelo Usuário.

Falaremos delas neste módulo.

Como criar uma Function?

A sintaxe para criar uma function é a seguinte:

```
create or replace function nome_função(parâmetros)
   returns tipo_dado
   language plpgsql
as
$$
declare
   declaração das variáveis
begin
   códigos
   return dado
end $$;
```

Cada function é composta por:

- ✓ Um "cabeçalho" em que atribuímos: um nome para a função, seus parâmetros, o tipo de dado a ser retornado e a versão do SQL utilizada pelo PostgreSQL (plpgsql);
- ✓ Uma sessão de declaração das variáveis que serão usadas no corpo do código;
- ✓ E uma sessão do corpo do código que, como o próprio nome já diz, é onde criamos os nossos **códigos (instruções)** e definimos o **retorno** do valor (variável).

Como criar uma Function?

Vejamos agora um exemplo para entendermos melhor.

Vamos criar uma function que analisa o estoque de produtos. Essa function retornará o total de produtos que possuem um total de estoque entre um estoque mínimo e um estoque máximo, ambos definidos pelo usuário da function.

Para isso, utilizaremos nossa tabela products do banco de dados Northwind.

Repare que, ao consultar a tabela products, verificamos que existe um total de 77 produtos cadastrados. Perceba também, que temos uma coluna que mostra a quantidade em estoque de cada um desses produtos (**units_in_stock**):

```
select * from products;
```

Data output | Messages | Notifications

	product_id [PK] smallint	product_name character varying (40)	supplier_id smallint	category_id smallint	quantity_per_unit character varying (20)	unit_price real	units_in_stock smallint	units_on_order smallint	reorder_l smallint
1	1	Chai	8	1	10 boxes x 30 bags	18	39	0	
2	2	Chang	1	1	24 - 12 oz bottles	19	17	40	
3	3	Aniseed Syrup	1	2	12 - 550 ml bottles	10	13	70	
4	4	Chef Anton's Cajun Se...	2	2	48 - 6 oz jars	22	53	0	
5	5	Chef Anton's Gumbo ...	2	2	36 boxes	21.35	0	0	

Total rows: 77 of 77 | Query complete 00:00:00.537 | Ln 1156, Col 1

PostgreSQL

Como criar uma Function?

Para descobrirmos quantos desses 77 produtos cadastrados possuem uma quantidade de estoque entre um estoque mínimo e um estoque máximo, criaremos a seguinte function:

```
create or replace function analise_estoque(estoque_min int, estoque_max int)
returns int
language plpgsql
as
$$
declare
    contagem_estoque int;

begin
    contagem_estoque = (select count(*) from products where units_in_stock between estoque_min and estoque_max);

    return contagem_estoque;

end $$;
```

Data output | Messages | Notifications

```
CREATE FUNCTION

Query returned successfully in 140 msec.
```

Como criar uma Function?

1. Iniciamos a função com o comando CREATE OR REPLACE FUNCTION (para criar a função, caso não exista; ou substitui-la, caso já exista), definimos um nome para ela (analise_estoque), e quais serão seus parâmetros (neste caso, dois: estoque_min e estoque_max, ambos do tipo inteiro);

2. Informamos ao comando RETURNS que a função retornará um dado do tipo inteiro;

3. Informamos ao comando LANGUAGE a versão do SQL utilizada pelo PostgreSQL (plpgsql);

4. Prosseguimos com o comando AS e definimos a label a ser utilizada para delimitar a função: $$. Sem essa label, o SQL não saberia onde as instruções da function iniciam e terminam. Isso porque cada instrução termina com um ponto-e-vírgula. Sem a label, o SQL executaria o código até o primeiro ponto-e-vírgula que aparecesse, e entenderia que o código acaba ali. Colocando a label $$ antes de iniciar a declaração das variáveis e ao final da function, a linguagem entende que aquele trecho entre a label inicial e a label final é um único bloco de códigos que devem ser executados sequencialmente;

```
create or replace function analise_estoque(estoque_min int, estoque_max int) 1
returns int 2
language plpgsql 3
as 4
$$
declare
    contagem_estoque int; 5

begin
    contagem_estoque = (select count(*) from products where units_in_stock between estoque_min and estoque_max);
6
    return contagem_estoque;

end $$; 7
```

```
Data output | Messages | Notifications

CREATE FUNCTION

Query returned successfully in 140 msec.
```

PostgreSQL

Índice Geral

Índice deste Módulo

Como criar uma Function?

5 Abrimos o bloco de declaração com o comando DECLARE. Dentro dele, devemos declarar as variáveis que serão utilizadas no corpo da function (no caso, apenas uma: contagem_estoque);

6 Em seguida, iniciamos o corpo da function com o comando BEGIN e, dentro dele, inserimos as instruções que deverão ser executadas. Neste exemplo, fazemos uma contagem da coluna units_in_stock da tabela products de todos os produtos com quantidade em estoque entre o estoque_min e o estoque_max (parâmetros que serão informados pelo usuário da function), atribuindo o valor dessa contagem à variável que criamos no bloco de declaração (contagem_estoque); em seguida, finalizamos o corpo da function com um RETURN contagem_estoque, para que ela retorne o valor da referida variável quando for utilizada pelo usuário;

7 Por fim, encerramos a function com a label end $$;

```
create or replace function analise_estoque(estoque_min int, estoque_max int) 1
returns int 2
language plpgsql 3
as
$$ 4
declare
    contagem_estoque int; 5

begin
    contagem_estoque = (select count(*) from products where units_in_stock between estoque_min and estoque_max);
6
    return contagem_estoque;

end $$; 7
```

```
Data output | Messages | Notifications

CREATE FUNCTION

Query returned successfully in 140 msec.
```

PostgreSQL

Índice Geral

Índice deste Módulo

Como criar uma Function?

Criada a function, podemos utilizá-la como qualquer outra function já existente no sistema. Veja:

```
select analise_estoque(10, 50);
```

	analise_estoque integer
1	42

Aqui, chamamos a function analise_estoque por meio de um SELECT e informamos os valores 10 para o parâmetro estoque_min e 50 para o estoque_max. Assim, descobrimos que 42 produtos cadastrados na tabela products (dos 77) estão com seus estoques dentro do esperado (10 e 50).

Como criar uma Function?

Isso significa que, se fizermos uma contagem na tabela products e subtrairmos o valor retornado pela function analise_estoque, retornaremos o valor 35, que é exatamente a quantidade de produtos que estão abaixo do estoque mínimo esperado (10) ou acima dele (50)...

```
select
    count(*) - analise_estoque(10, 50)
from products;
```

	?column? bigint
1	35

... Pois 77 produtos cadastrados – 42 produtos dentro do estoque = 35 produtos fora dos limites mínimo ou máximo de estoque.

Formas de chamar uma Function

Existem três formas de chamar um função.

A primeira delas é a notação por posição, que foi exatamente o que fizemos nos exemplo anterior. Na notação por posição apenas informamos os valores que queremos que sejam considerados pela function, na ordem em que foram criados:

```
select analise_estoque(20, 50);
```

	analise_estoque integer
1	28

Formas de chamar uma Function

Existem três formas de chamar um função.

A segunda forma é a notação por nome do parâmetro, em que informamos o nome do parâmetro antes do valor que atribuiremos a ele:

```
select analise_estoque(estoque_min := 20, estoque_max := 50);
```

	analise_estoque integer
1	28

A vantagem desta forma em relação à primeira é que podemos inverter a ordem dos parâmetros, se precisarmos:

```
select analise_estoque(estoque_max := 50, estoque_min := 20);
```

	analise_estoque integer
1	28

Formas de chamar uma Function

Existem três formas de chamar um função.

Por fim, temos também a notação mista, em que primeiro informamos o(s) parâmetro(s) por posição, seguido(s) do(s) parâmetro(s) por nome:

```
select analise_estoque(20, estoque_max := 50);
```

	analise_estoque integer
1	28

No entanto, não aconselhamos o seu uso, pois, além de não ser muito organizado, ainda podemos retornar um erro, caso tentemos fazer o contrário, informando primeiro o(s) parâmetro(s) por nome, seguido(s) do(s) parâmetro(s) por posição:

```
select analise_estoque(estoque_max := 50, 20);
```

```
ERROR:  positional argument cannot follow named argument
LINE 1: select analise_estoque(estoque_max := 50, 20);
                                                  ^

SQL state: 42601
Character: 43
```

Como excluir uma Function?

Podemos excluir uma function de duas formas:

```
drop function analise_estoque;
```

OU

```
drop function if exists analise_estoque;
```

Sugerimos que sempre utilize a segunda forma, pois o IF EXISTS é capaz de procurar se a function que queremos excluir de fato existe no sistema. Se existir, o DROP excluirá a referida function:

Data output | Messages | Notifications

```
DROP FUNCTION

Query returned successfully in 48 msec.
```

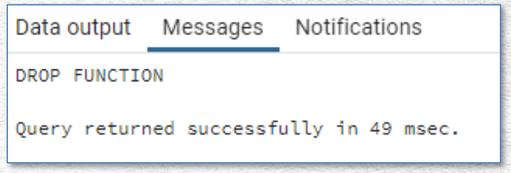
Se não existir, será retornado um aviso na tela (NOTICE), e o sistema prosseguirá para a execução do próximo comando, se houver:

Data output | Messages | Notifications

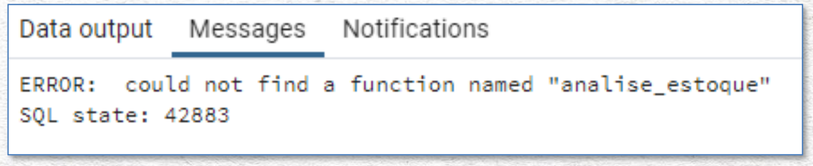
```
NOTICE:  function analise_estoque() does not exist, skipping
DROP FUNCTION

Query returned successfully in 75 msec.
```

Como excluir uma Function?

Já a primeira opção (sem o IF EXISTS), não terá como verificar antes se a função existe ou não. Ele apenas tentará excluir a function informada. Se existir, ele a excluirá:



Porém, se ela não existir, em vez de apenas retornar um aviso na tela e prosseguir para o próximo comando (se houver), ele retornará um erro na tela (ERROR) e interromperá a execução dos próximos comandos:



Functions que retornam uma tabela

É possível criarmos functions que retornam tabelas. A sintaxe para criar uma função deste tipo é a seguinte:

```
create or replace function nome_função(parâmetros)
   returns table (lista_de_colunas)
   language plpgsql
as
$$
declare
   declaração das variáveis
begin
   códigos
   return query
end $$;
```

Cada function que retorna uma tabela é composta por:

- ✓ Um "cabeçalho" em que atribuímos: um nome para a função, seus parâmetros, um RETURN TABLE com a lista de colunas a serem retornadas, e a versão do SQL utilizada pelo PostgreSQL (plpgsql);
- ✓ Uma sessão de declaração das variáveis que serão usadas no corpo do código (**opcional**);
- ✓ E uma sessão do corpo do código que, como o próprio nome já diz, é onde criamos os nossos **códigos (instruções)** e definimos a **query** que será retornada.

Functions que retornam uma tabela

Vejamos agora um exemplo para entendermos melhor.

Vamos criar uma function que retorne uma tabela contendo a lista de clientes que têm o contact_title = 'Owner' (que será fornecido pelo usuário). A tabela retornada pela function conterá as colunas de id, nome, telefone e cargo dos clientes.

Para isso, utilizaremos nossa tabela customers do banco de dados Northwind. Repare que, ao consultar a tabela customers, verificamos que existem as colunas customer_id, contact_name, phone e contact_title, que servirão de fonte de dados para as colunas de id, nome, telefone e cargo, respectivamente, da query que será retornada pela nossa function:

```
select * from customers;
```

Data output | Messages | Notifications

	customer_id [PK] character varying (5)	company_nar character var	contact_name character varying (30)	contact_title character varying (30)	address character var	city character var	region character var	postal_code character var	country character var	phone character varying (24)	fax character var
1	ALFKI	Alfreds F...	Maria Anders	Sales Representative	Obere Str....	Berlin	[null]	12209	Germany	030-0074321	030-0076...
2	ANATR	Ana Trujill...	Ana Trujillo	Owner	Avda. de l...	México D.F.	[null]	05021	Mexico	(5) 555-4729	(5) 555-3...
3	ANTON	Antonio ...	Antonio Moreno	Owner	Matadero...	México D.F.	[null]	05023	Mexico	(5) 555-3932	[null]
4	AROUT	Around th...	Thomas Hardy	Sales Representative	120 Hano...	London	[null]	WA1 1DP	UK	(171) 555-7788	(171) 555...
5	BERGS	Berglunds...	Christina Berglund	Order Administrator	Berguvsv...	Luleå	[null]	S-958 22	Sweden	0921-12 34 65	0921-12 3...

Total rows: 91 of 91 | Query complete 00:00:00.142

PostgreSQL

Functions que retornam uma tabela

Para retornarmos somente os clientes cujo cargo (contact_title) seja igual a 'Owner' (informação esta que será fornecida pelo usuário que utilizar a function), criaremos a seguinte função:

```
create or replace function busca_clientes (title varchar)
returns table
    (
        id customers.customer_id%type,
        nome customers.contact_name%type,
        telefone customers.phone%type,
        cargo customers.contact_title%type
    )
language plpgsql
as
$$
begin

    return query
        select
            customer_id,
            contact_name,
            phone,
            contact_title
        from customers
        where contact_title = title;
end $$;
```

Data output | Messages | Notifications

```
NOTICE:  type reference customers.customer_id%TYPE converted to character varying
NOTICE:  type reference customers.contact_name%TYPE converted to character varying
NOTICE:  type reference customers.phone%TYPE converted to character varying
NOTICE:  type reference customers.contact_title%TYPE converted to character varying
CREATE FUNCTION

Query returned successfully in 106 msec.
```

Functions que retornam uma tabela

1 Iniciamos a função com o comando CREATE OR REPLACE FUNCTION (para criar a função, caso não exista; ou substitui-la, caso já exista), definimos um nome para ela (busca_clientes), e quais serão seus parâmetros (neste caso, apenas um: title do tipo varchar);

2 Informamos ao comando RETURNS que a função retornará uma tabela (TABLE), e listamos, entre parênteses, as colunas que serão retornadas pela nossa query: id, nome, telefone e cargo. Os tipos de dados das colunas da nossa query precisam ser os mesmos das colunas da tabela customers. Repare que, para atribuir o tipo de dados das colunas da nossa query, utilizamos a sintaxe *nome_da_tabela.nome_da_coluna%type*. Esse comando consegue copiar os tipos de dados das colunas das quais extrairemos os dados. Dessa forma, não precisamos nos preocupar em saber quais os tipos de dados das colunas da tabela original para atribuirmos às colunas da query que criaremos no corpo da nossa function;

3 Informamos ao comando LANGUAGE a versão do SQL utilizada pelo PostgreSQL (plpgsql);

```
create or replace function busca_clientes (title varchar) 1
returns table 2
    (
        id customers.customer_id%type,
        nome customers.contact_name%type,
        telefone customers.phone%type,
        cargo customers.contact_title%type
    )
language plpgsql 3
as
$$ 4
begin

    return query 5
        select
            customer_id,
            contact_name,
            phone,
            contact_title
        from customers
        where contact_title = title;
end $$; 6
```

Data output | Messages | Notifications

```
NOTICE:  type reference customers.customer_id%TYPE converted to character varying
NOTICE:  type reference customers.contact_name%TYPE converted to character varying
NOTICE:  type reference customers.phone%TYPE converted to character varying
NOTICE:  type reference customers.contact_title%TYPE converted to character varying
CREATE FUNCTION

Query returned successfully in 106 msec.
```

Functions que retornam uma tabela

4 Prosseguimos com o comando AS e definimos a label a ser utilizada para delimitar a função: $$.

5 Como não utilizaremos variáveis para efetuar algum cálculo, não precisamos da sessão de declaração de variáveis. Portanto, partimos direto para o corpo da function, iniciando-o com o comando BEGIN e, dentro dele, inserimos as instruções que deverão ser executadas. Neste caso, como queremos retornar uma tabela, utilizamos o comando RETURN QUERY e fazemos um SELECT às colunas que queremos da tabela customers (customer_id, contact_name, phone e contact_title). Como desejamos que apenas os clientes de determinado cargo sejam retornados, aplicamos um filtro na nossa query por meio da cláusula WHERE, que compara os valores da coluna contact_title com o valor que for informado pelo usuário ao parâmetro title da function;

6 Por fim, encerramos a function com a label end $$;

```
create or replace function busca_clientes (title varchar) 1
returns table 2
    (
        id customers.customer_id%type,
        nome customers.contact_name%type,
        telefone customers.phone%type,
        cargo customers.contact_title%type
    )
language plpgsql 3
as
$$ 4
begin

    return query 5
        select
            customer_id,
            contact_name,
            phone,
            contact_title
        from customers
        where contact_title = title;
end $$; 6
```

Data output | Messages | Notifications

```
NOTICE:  type reference customers.customer_id%TYPE converted to character varying
NOTICE:  type reference customers.contact_name%TYPE converted to character varying
NOTICE:  type reference customers.phone%TYPE converted to character varying
NOTICE:  type reference customers.contact_title%TYPE converted to character varying
CREATE FUNCTION

Query returned successfully in 106 msec.
```

Functions que retornam uma tabela

Criada a function, podemos utilizá-la da forma representada ao lado.

```
select * from busca_clientes('Owner');
```

Conforme proposto pelo enunciado do nosso exemplo, devemos retornar somente os registros dos clientes cujo cargo (contact_title) seja igual a 'Owner'. Portanto, ao executar a function, passaremos essa informação como parâmetro.

Repare que, como queremos retornar uma tabela, utilizamos o nome da function em um *SELECT * FROM nome_função(parâmetro)*, como se o nome da função fosse o nome de uma tabela.

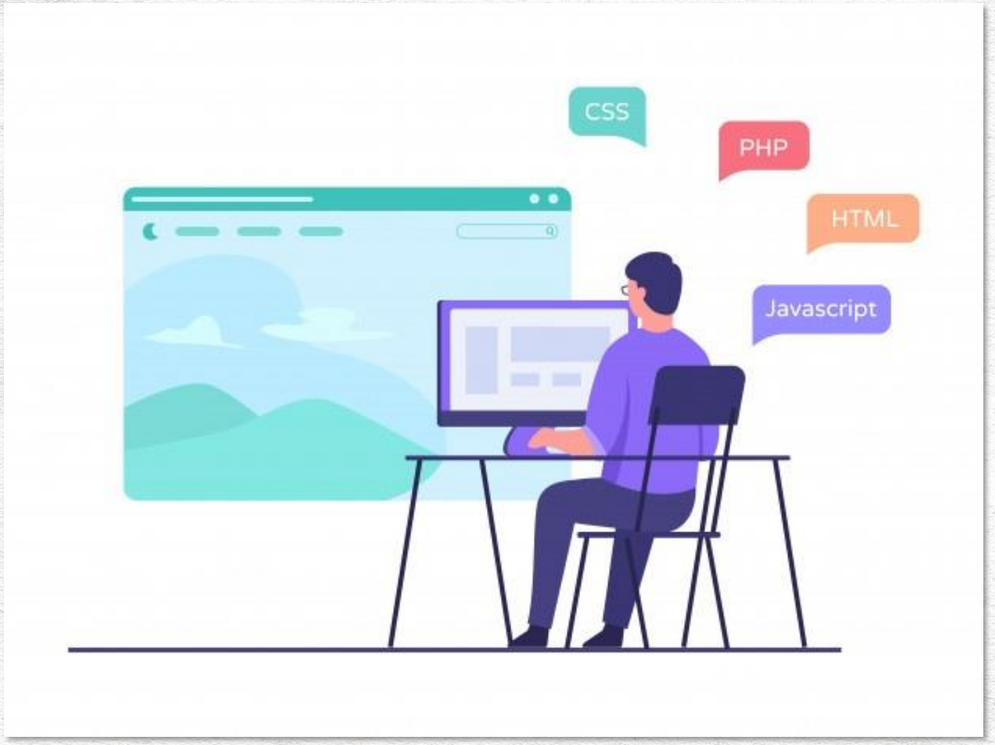
Perceba que a tabela retornou somente os 17 clientes cujo contact_title (cargo) = 'Owner':

Data output | Messages | Notifications

	id character varying	nome character varying	telefone character varying	cargo character varying
1	ANATR	Ana Trujillo	(5) 555-4729	Owner
2	ANTON	Antonio Moreno	(5) 555-3932	Owner
3	BOLID	Martín Sommer	(91) 555 22 82	Owner
4	BONAP	Laurence Lebihan	91.24.45.40	Owner
5	CHOPS	Yang Wang	0452-076545	Owner
6	DUMON	Janine Labrune	40.67.88.88	Owner
7	FOLKO	Maria Larsson	0695-34 67 21	Owner
8	GROSR	Manuel Pereira	(2) 283-2951	Owner
9	LETSS	Jaime Yorres	(415) 555-5938	Owner
10	LINOD	Felipe Izquierdo	(8) 34-56-12	Owner
11	OTTIK	Henriette Pfalzhe...	0221-0644327	Owner
12	PARIS	Marie Bertrand	(1) 42.34.22.66	Owner
13	SANTG	Jonas Bergulfsen	07-98 92 35	Owner
14	SIMOB	Jytte Petersen	31 12 34 56	Owner
15	TORTU	Miguel Angel Pao...	(5) 555-2933	Owner
16	WHITC	Karl Jablonski	(206) 555-4112	Owner
17	WOLZA	Zbyszek Piestrze...	(26) 642-7012	Owner

Total rows: 17 of 17 | Query complete 00:00:00.107

EXERCÍCIOS



1

Questão 1

O time de vendas da empresa está sempre buscando otimizar suas estratégias de marketing e de vendas. Para isso, é essencial entender quais clientes têm o maior volume de compras. Dessa forma, é possível identificar padrões de comportamento e focar em oportunidades de vendas adicionais, como descontos especiais ou ofertas personalizadas para esses clientes.

Crie uma função chamada **total_vendas_cliente** que receba o customer_id como parâmetro e retorne o total de vendas realizadas para aquele cliente. **(Utilize a tabela orders)**

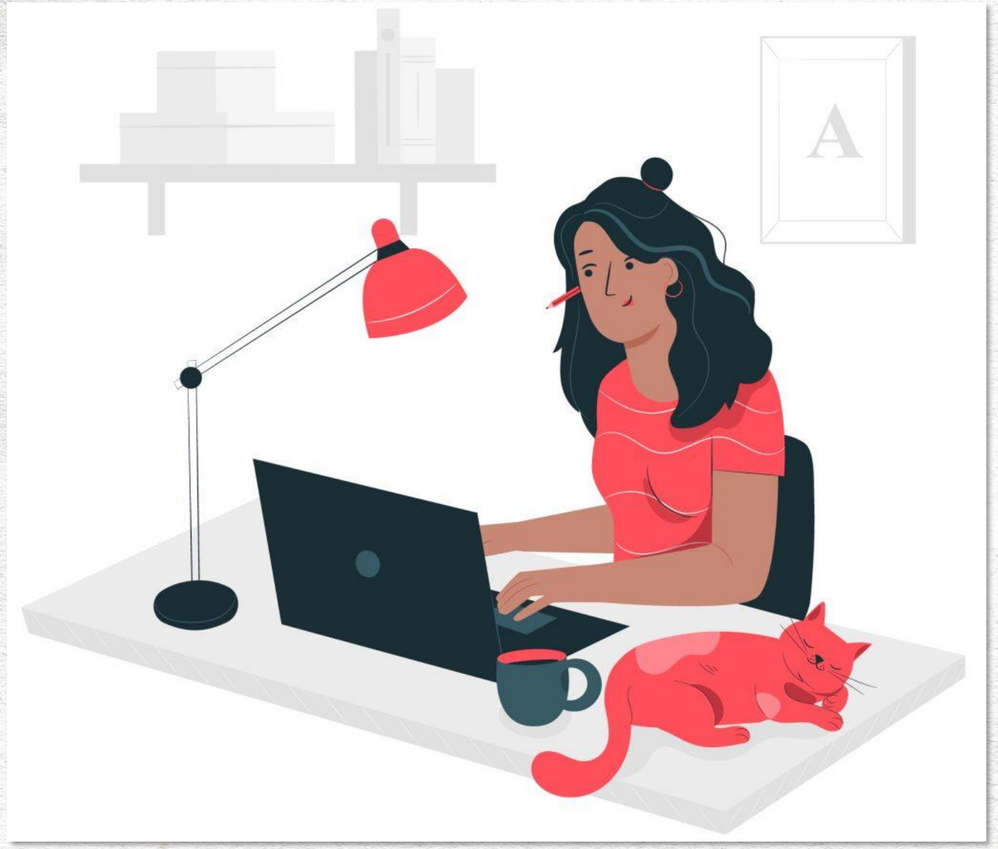
2

Questão 2

A logística da empresa é um dos maiores custos operacionais. A análise detalhada dos valores gastos com cada transportadora ajuda a otimizar os custos de envio e a negociar melhores tarifas. Este tipo de análise também permite que o setor logístico ajuste as estratégias de envio, buscando maior eficiência nos processos de distribuição.

Crie uma função chamada **total_frete_transportadora** que receba o ID da transportadora (ship_via) e retorne o frete total gasto com essa transportadora. **(Utilize a tabela orders)**

EXERCÍCIOS



3

Questão 3

O time de marketing deseja focar seus esforços nas promoções dos produtos que mais atraem a atenção dos clientes. Com isso, é possível aumentar ainda mais as vendas de produtos que já são populares, bem como entender quais categorias de produtos estão apresentando bom desempenho no mercado.

Crie uma função chamada **total_vendas_produto** que receba o product_id e retorne a quantidade total de unidades vendidas desse produto. **(Utilize a tabela order_details)**

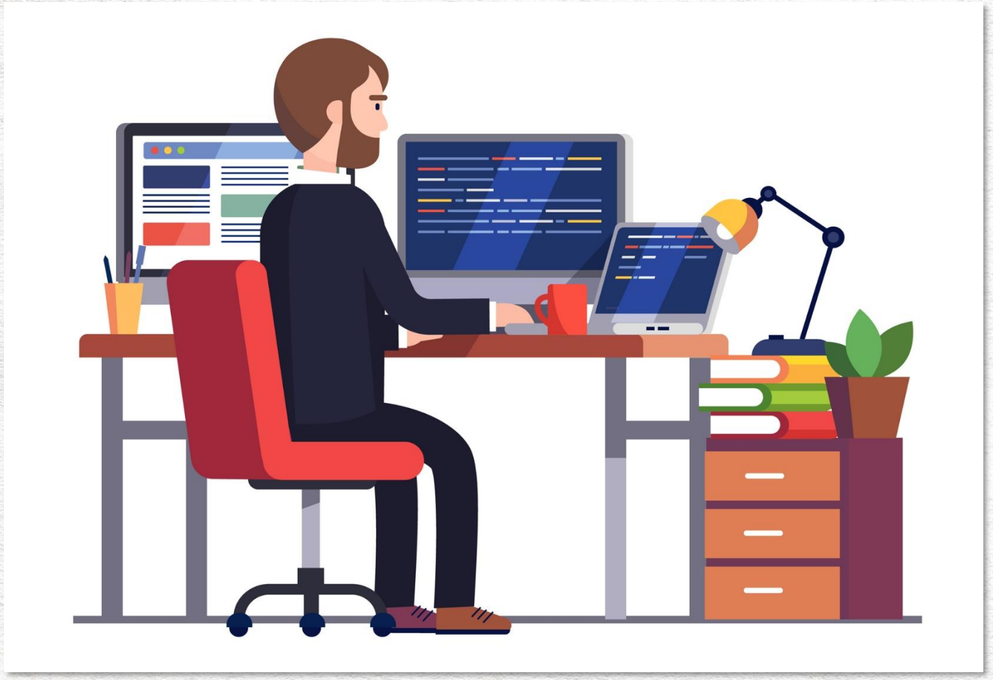
4

Questão 4

O setor financeiro precisa realizar a análise de desempenho dos funcionários de vendas para determinar bônus e outras formas de reconhecimento. Saber quanto cada funcionário vendeu ajuda a otimizar a gestão de recursos humanos e a criar estratégias de incentivo que impactem diretamente no aumento das vendas.

Crie uma função chamada **total_vendas_funcionario** que receba o employee_id e retorne o valor total vendido por ele. **(Utilize as tabelas orders e order_details)**

EXERCÍCIOS



5

Questão 5

O setor de Recursos Humanos precisa de informações sobre a localização geográfica dos funcionários para realizar ações de comunicação interna mais eficazes. Com isso, é possível realizar treinamentos e campanhas de engajamento que atendam às necessidades específicas de cada região.

Crie uma função chamada **funcionarios_por_pais** que receba o nome do país e retorne os IDs dos funcionários, seus nomes, sobrenomes, cargos e cidades. **(Utilize a tabela employees)**

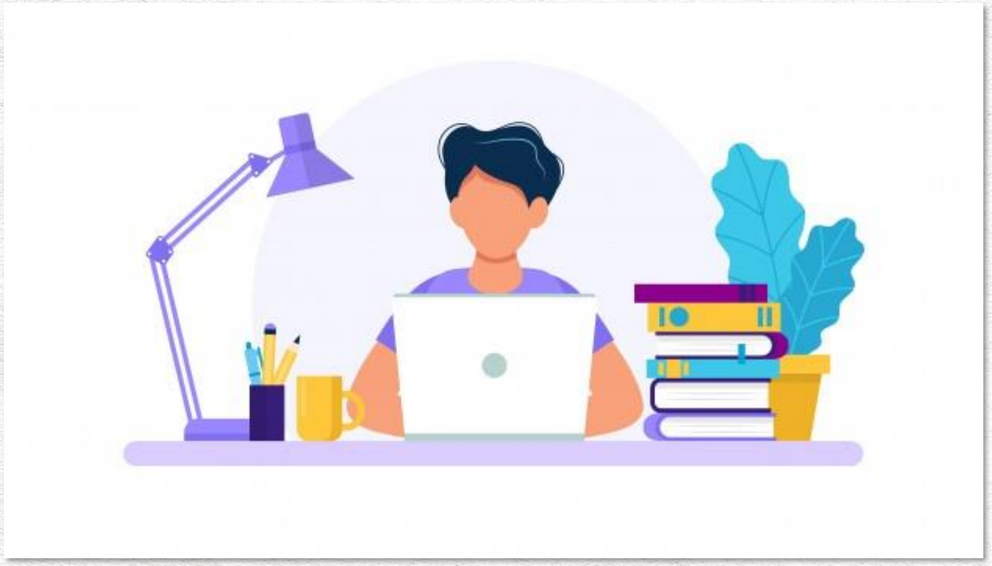
6

Questão 6

O time de produtos da empresa precisa revisar periodicamente o portfólio de produtos disponíveis no estoque. Isso é importante para realizar ações como ajuste de preços, controle de estoque, promoções e até descontinuação de produtos que não estão mais sendo vendidos, garantindo que o portfólio de produtos esteja alinhado com a demanda do mercado.

Crie uma função chamada **produtos_por_categoria** que receba o category_id e retorne os IDs dos produtos, seus nomes, preços unitários e as quantidades em estoque. **(Utilize a tabela products)**

EXERCÍCIOS



7

Questão 7

O setor de RH organiza eventos internos, como festas de aniversário, para promover o engajamento e a integração dos funcionários. Para planejar esses eventos de forma eficiente, o RH precisa de uma lista de aniversariantes de cada mês para enviar mensagens de felicitações e organizar as celebrações de maneira personalizada.

Crie uma função chamada **aniversariantes_mes** que receba o mês como parâmetro e retorne os IDs dos funcionários, seus nomes, sobrenomes, datas de nascimento e cargos. **(Utilize a tabela employees.)**

8

Questão 8

A logística da empresa precisa identificar rapidamente os pedidos que não foram entregues dentro do prazo previsto para evitar insatisfação dos clientes. Identificar e corrigir esses atrasos é crucial para manter a qualidade do serviço de entrega e melhorar a experiência do cliente.

Crie uma função chamada **pedidos_com_atraso** que retorne uma tabela com order_id, customer_id, required_date e shipped_date, filtrando apenas os pedidos onde a data prevista (required_date) for maior ou igual a data estimada informada pelo usuário, assim como a data de envio (shipped_date) foi maior que a data prevista (required_date). **(Utilize a tabela orders)**

EXERCÍCIOS

9

Questão 9

Exclua do banco de dados todas as functions criadas nesta lista de exercícios:

total_vendas_cliente

total_frete_transportadora

total_vendas_produto

total_vendas_funcionario

funcionarios_por_pais

produtos_por_categoria

aniversariantes_mes

pedidos_com_atraso

Obs.: Garanta que, antes de excluir, primeiro a function será procurada no banco. Se existir, será excluída; caso contrário, será retornado um aviso na tela em vez de um erro.

(Não é preciso utilizar tabelas)

GABARITO EXPLICATIVO

1

Questão 1

O time de vendas da empresa está sempre buscando otimizar suas estratégias de marketing e de vendas. Para isso, é essencial entender quais clientes têm o maior volume de compras. Dessa forma, é possível identificar padrões de comportamento e focar em oportunidades de vendas adicionais, como descontos especiais ou ofertas personalizadas para esses clientes.

Crie uma função chamada **total_vendas_cliente** que receba o customer_id como parâmetro e retorne o total de vendas realizadas para aquele cliente. **(Utilize a tabela orders)**

Explicação e Resultado

Para criar uma função que receba o customer_id como parâmetro e retorne o total de vendas realizadas para aquele cliente, fazemos o seguinte:

1. Criação da função:

- ✓ **create or replace function total_vendas_cliente(id_cliente varchar):** Criamos a função total_vendas_cliente que recebe um parâmetro id_cliente do tipo varchar (mesmo tipo da coluna customer_id na tabela orders).
- ✓ **returns int**: O retorno será um valor inteiro.

2. Bloco declare:

- ✓ **total_vendas int**: Declaramos uma variável total_vendas do tipo int para armazenar o valor total das vendas.

3. Bloco begin:

- ✓ **total_vendas = (...)**: Atribuímos o resultado da query entre parênteses à variável total_vendas.
- ✓ **select count(order_id) from orders where customer_id = id_cliente**: Contamos o total de vendas da tabela orders, filtrando somente as vendas efetuadas para o cliente específico cujo customer_id foi passado como parâmetro.
- ✓ **return total_vendas**: Retornamos o valor total calculado.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

1

Questão 1

O time de vendas da empresa está sempre buscando otimizar suas estratégias de marketing e de vendas. Para isso, é essencial entender quais clientes têm o maior volume de compras. Dessa forma, é possível identificar padrões de comportamento e focar em oportunidades de vendas adicionais, como descontos especiais ou ofertas personalizadas para esses clientes.

Crie uma função chamada **total_vendas_cliente** que receba o customer_id como parâmetro e retorne o total de vendas realizadas para aquele cliente. **(Utilize a tabela orders)**

Explicação e Resultado

```
create or replace function total_vendas_cliente(id_cliente varchar)
returns int
language plpgsql
as $$
declare
    total_vendas int;
begin
    total_vendas = (
        select count(order_id)
        from orders
        where customer_id = id_cliente
    );
    return total_vendas;
end $$;
```

Data Output | Messages | Notifications

```
CREATE FUNCTION

Query returned successfully in 63 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

1

Questão 1

O time de vendas da empresa está sempre buscando otimizar suas estratégias de marketing e de vendas. Para isso, é essencial entender quais clientes têm o maior volume de compras. Dessa forma, é possível identificar padrões de comportamento e focar em oportunidades de vendas adicionais, como descontos especiais ou ofertas personalizadas para esses clientes.

Crie uma função chamada **total_vendas_cliente** que receba o customer_id como parâmetro e retorne o total de vendas realizadas para aquele cliente. **(Utilize a tabela orders)**

Explicação e Resultado

Resultado Esperado:

Por exemplo, se chamarmos a função passando como parâmetro o customer_id 'HANAR'...

```
select total_vendas_cliente('HANAR');
```

... o PostgreSQL retornará o valor total de todas as vendas feitas para o cliente com o ID 'HANAR':

	total_vendas_cliente integer
1	14

GABARITO EXPLICATIVO

2

Questão 2

A logística da empresa é um dos maiores custos operacionais. A análise detalhada dos valores gastos com cada transportadora ajuda a otimizar os custos de envio e a negociar melhores tarifas. Este tipo de análise também permite que o setor logístico ajuste as estratégias de envio, buscando maior eficiência nos processos de distribuição.

Crie uma função chamada **total_frete_transportadora** que receba o ID da transportadora (ship_via) e retorne o frete total gasto com essa transportadora. **(Utilize a tabela orders)**

Explicação e Resultado

Para criar uma função que receba o ID da transportadora (ship_via) e retorne o frete total gasto com essa transportadora, fazemos o seguinte:

1. Criação da função:

- ✓ **create or replace function total_frete_transportadora(id_transportadora int)**: Criamos a função total_frete_transportadora que recebe um parâmetro id_transportadora do tipo int (mesmo tipo da coluna ship_via na tabela orders).
- ✓ **returns numeric**: O retorno será um valor numérico, pois estamos lidando com valores monetários (frete).

2. Bloco declare:

- ✓ **total_frete numeric**: Declaramos uma variável total_frete do tipo numeric para armazenar o valor total do frete.

3. Bloco begin:

- ✓ **total_frete = (...)**: Atribuímos o resultado da query entre parênteses à variável total_frete.
- ✓ **select sum(freight) from orders where ship_via = id_transportadora**: Calculamos o total de frete utilizando a função SUM() na coluna freight, filtrando apenas os pedidos realizados com a transportadora cujo ship_via foi passado como parâmetro.
- ✓ **return total_frete**: Retornamos o valor total do frete calculado.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

2

Questão 2

A logística da empresa é um dos maiores custos operacionais. A análise detalhada dos valores gastos com cada transportadora ajuda a otimizar os custos de envio e a negociar melhores tarifas. Este tipo de análise também permite que o setor logístico ajuste as estratégias de envio, buscando maior eficiência nos processos de distribuição.

Crie uma função chamada **total_frete_transportadora** que receba o ID da transportadora (ship_via) e retorne o frete total gasto com essa transportadora. **(Utilize a tabela orders)**

Explicação e Resultado

```sql
create or replace function total_frete_transportadora(id_transportadora int)
returns numeric
language plpgsql
as $$
declare
    total_frete numeric;
begin
    total_frete = (
        select sum(freight)
        from orders
        where ship_via = id_transportadora
    );
    return total_frete;
end $$;
```

Data Output | Messages | Notifications

```
CREATE FUNCTION

Query returned successfully in 64 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

2

Questão 2

A logística da empresa é um dos maiores custos operacionais. A análise detalhada dos valores gastos com cada transportadora ajuda a otimizar os custos de envio e a negociar melhores tarifas. Este tipo de análise também permite que o setor logístico ajuste as estratégias de envio, buscando maior eficiência nos processos de distribuição.

Crie uma função chamada **total_frete_transportadora** que receba o ID da transportadora (ship_via) e retorne o frete total gasto com essa transportadora. **(Utilize a tabela orders)**

Explicação e Resultado

Resultado esperado:

Por exemplo, se chamarmos a função passando como parâmetro o ID da transportadora 2...

```
select total_frete_transportadora(2);
```

... o PostgreSQL retornará o valor total gasto em frete com essa transportadora:

	total_frete_transportadora numeric
1	28244.9

GABARITO EXPLICATIVO

3

Questão 3

O time de marketing deseja focar seus esforços nas promoções dos produtos que mais atraem a atenção dos clientes. Com isso, é possível aumentar ainda mais as vendas de produtos que já são populares, bem como entender quais categorias de produtos estão apresentando bom desempenho no mercado.

Crie uma função chamada **total_vendas_produto** que receba o product_id e retorne a quantidade total de unidades vendidas desse produto. **(Utilize a tabela order_details)**

Explicação e Resultado

Para criar uma função que receba o product_id e retorne a quantidade total de unidades vendidas desse produto, fazemos o seguinte:

1. Criação da função:

- ✓ **create or replace function total_vendas_produto(id_produto int)**: Criamos a função total_vendas_produto que recebe um parâmetro id_produto do tipo int (mesmo tipo da coluna product_id na tabela order_details).
- ✓ **returns int**: O retorno será um valor inteiro, pois estamos somando a quantidade de unidades vendidas.

2. Bloco declare:

- ✓ **total_quantidade int**: Declaramos uma variável total_quantidade do tipo int para armazenar o total de unidades vendidas.

3. Bloco begin:

- ✓ **total_quantidade = (...)**: Atribuímos o resultado da query entre parênteses à variável total_quantidade.
- ✓ **select sum(quantity) from order_details where product_id = id_produto**: Calculamos a soma das unidades vendidas do produto utilizando a função SUM() na coluna quantity, filtrando apenas as linhas em que o product_id corresponde ao valor passado como parâmetro.
- ✓ **return total_quantidade**: Retornamos o valor total calculado.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

3

Questão 3

O time de marketing deseja focar seus esforços nas promoções dos produtos que mais atraem a atenção dos clientes. Com isso, é possível aumentar ainda mais as vendas de produtos que já são populares, bem como entender quais categorias de produtos estão apresentando bom desempenho no mercado.

Crie uma função chamada **total_vendas_produto** que receba o product_id e retorne a quantidade total de unidades vendidas desse produto. **(Utilize a tabela order_details)**

Explicação e Resultado

```sql
create or replace function total_vendas_produto(id_produto int)
returns int
language plpgsql
as $$
declare
    total_quantidade int;
begin
    total_quantidade = (
        select sum(quantity)
        from order_details
        where product_id = id_produto
    );
    return total_quantidade;
end $$;
```

Data Output | Messages | Notifications

```
CREATE FUNCTION

Query returned successfully in 58 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

3

Questão 3

O time de marketing deseja focar seus esforços nas promoções dos produtos que mais atraem a atenção dos clientes. Com isso, é possível aumentar ainda mais as vendas de produtos que já são populares, bem como entender quais categorias de produtos estão apresentando bom desempenho no mercado.

Crie uma função chamada **total_vendas_produto** que receba o product_id e retorne a quantidade total de unidades vendidas desse produto. **(Utilize a tabela order_details)**

Explicação e Resultado

Resultado esperado:

Por exemplo, se chamarmos a função passando como parâmetro o ID do produto 10...

```
select total_vendas_produto(10);
```

... o PostgreSQL retornará a quantidade total de unidades vendidas desse produto:

	total_vendas_produto integer
1	742

GABARITO EXPLICATIVO

4

Questão 4

O setor financeiro precisa realizar a análise de desempenho dos funcionários de vendas para determinar bônus e outras formas de reconhecimento. Saber quanto cada funcionário vendeu ajuda a otimizar a gestão de recursos humanos e a criar estratégias de incentivo que impactem diretamente no aumento das vendas.

Crie uma função chamada **total_vendas_funcionario** que receba o employee_id e retorne o valor total vendido por ele. **(Utilize as tabelas orders e order_details)**

Explicação e Resultado

Para criar uma função que receba o employee_id e retorne o valor total vendido por esse funcionário, fazemos o seguinte:

1. Criação da função:

- ✓ **create or replace function total_vendas_funcionario(id_funcionario int)**: Criamos a função total_vendas_funcionario que recebe um parâmetro id_funcionario do tipo int (mesmo tipo da coluna employee_id na tabela orders).
- ✓ **returns numeric**: O retorno será um valor numérico, pois estamos calculando o valor total das vendas.

2. Bloco declare:

- ✓ **total_vendas numeric**: Declaramos uma variável total_vendas do tipo numeric para armazenar o valor total das vendas realizadas pelo funcionário.

3. Bloco begin:

- ✓ **total_vendas = (...)**: Atribuímos o resultado da query entre parênteses à variável total_vendas.
- ✓ **select sum(od.unit_price * od.quantity)**: Calculamos o valor total das vendas multiplicando o preço unitário (unit_price) pela quantidade (quantity) de cada produto vendido, utilizando a função SUM().

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

4

Questão 4

O setor financeiro precisa realizar a análise de desempenho dos funcionários de vendas para determinar bônus e outras formas de reconhecimento. Saber quanto cada funcionário vendeu ajuda a otimizar a gestão de recursos humanos e a criar estratégias de incentivo que impactem diretamente no aumento das vendas.

Crie uma função chamada **total_vendas_funcionario** que receba o employee_id e retorne o valor total vendido por ele. **(Utilize as tabelas orders e order_details)**

Explicação e Resultado

- ✓ **from orders o inner join order_details od on o.order_id = od.order_id**: Fazemos um INNER JOIN entre as tabelas orders e order_details com base na coluna order_id, para relacionar os pedidos com os detalhes das vendas.
- ✓ **where o.employee_id = id_funcionario**: Filtramos apenas os registros das vendas associadas ao funcionário cujo employee_id foi passado como parâmetro.
- ✓ **return total_vendas**: Retornamos o valor total calculado.

```
create or replace function total_vendas_funcionario(id_funcionario int)
returns numeric
language plpgsql
as $$
declare
    total_vendas numeric;
begin
    total_vendas = (
        select sum(od.unit_price * od.quantity)
        from orders o
        inner join order_details od on o.order_id = od.order_id
        where o.employee_id = id_funcionario
    );
    return total_vendas;
end $$;
```

Data Output | Messages | Notifications

```
CREATE FUNCTION

Query returned successfully in 59 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

4

Questão 4

O setor financeiro precisa realizar a análise de desempenho dos funcionários de vendas para determinar bônus e outras formas de reconhecimento. Saber quanto cada funcionário vendeu ajuda a otimizar a gestão de recursos humanos e a criar estratégias de incentivo que impactem diretamente no aumento das vendas.

Crie uma função chamada **total_vendas_funcionario** que receba o employee_id e retorne o valor total vendido por ele. **(Utilize as tabelas orders e order_details)**

Explicação e Resultado

Resultado esperado:

Por exemplo, se chamarmos a função passando como parâmetro o ID do funcionário 4...

```
select total_vendas_funcionario(4);
```

... o PostgreSQL retornará o valor total vendido pelo funcionário com o ID 4:

	total_vendas_funcionario numeric
1	250187.45025301

GABARITO EXPLICATIVO

5

Questão 5

O setor de Recursos Humanos precisa de informações sobre a localização geográfica dos funcionários para realizar ações de comunicação interna mais eficazes. Com isso, é possível realizar treinamentos e campanhas de engajamento que atendam às necessidades específicas de cada região.

Crie uma função chamada **funcionarios_por_pais** que receba o nome do país e retorne os IDs dos funcionários, seus nomes, sobrenomes, cargos e cidades. **(Utilize a tabela employees)**

Explicação e Resultado

Para criar uma função que receba o nome do país e retorne os IDs, nomes, sobrenomes, cargos e cidades dos funcionários desse país, fazemos o seguinte:

1. Criação da função:

- ✓ **create or replace function funcionarios_por_pais(pais varchar)**: Criamos a função funcionarios_por_pais que recebe um parâmetro pais do tipo varchar (mesmo tipo da coluna country na tabela employees).
- ✓ **returns table (...)**: Definimos o retorno como uma tabela com as seguintes colunas:
  - id: (tipo da coluna employee_id na tabela employees)
  - nome: (tipo da coluna first_name na tabela employees)
  - sobrenome: (tipo da coluna last_name na tabela employees)
  - cargo: (tipo da coluna title na tabela employees)
  - cidade: (tipo da coluna city na tabela employees)

2. Bloco begin:

- ✓ **return query:** Utilizamos o comando return query para retornar o resultado de uma consulta diretamente, sem a necessidade de declarar variáveis intermediárias.
- ✓ **select employee_id, first_name, last_name, title, city from employees where country = pais**: Selecionamos os dados dos funcionários cujo país corresponde ao valor passado como parâmetro.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

5

Questão 5

O setor de Recursos Humanos precisa de informações sobre a localização geográfica dos funcionários para realizar ações de comunicação interna mais eficazes. Com isso, é possível realizar treinamentos e campanhas de engajamento que atendam às necessidades específicas de cada região.

Crie uma função chamada **funcionarios_por_pais** que receba o nome do país e retorne os IDs dos funcionários, seus nomes, sobrenomes, cargos e cidades. **(Utilize a tabela employees)**

Explicação e Resultado

```sql
create or replace function funcionarios_por_pais(pais varchar)
returns table
(
    id employees.employee_id%type,
    nome employees.first_name%type,
    sobrenome employees.last_name%type,
    cargo employees.title%type,
    cidade employees.city%type
)
language plpgsql
as $$
begin
    return query
    select
        employee_id,
        first_name,
        last_name,
        title,
        city
    from employees
    where country = pais;
end $$;
```

Data Output | Messages | Notifications

```
NOTICE:  type reference employees.employee_id%TYPE converted to smallint
NOTICE:  type reference employees.first_name%TYPE converted to character varying
NOTICE:  type reference employees.last_name%TYPE converted to character varying
NOTICE:  type reference employees.title%TYPE converted to character varying
NOTICE:  type reference employees.city%TYPE converted to character varying
CREATE FUNCTION

Query returned successfully in 67 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

5

Questão 5

O setor de Recursos Humanos precisa de informações sobre a localização geográfica dos funcionários para realizar ações de comunicação interna mais eficazes. Com isso, é possível realizar treinamentos e campanhas de engajamento que atendam às necessidades específicas de cada região.

Crie uma função chamada **funcionarios_por_pais** que receba o nome do país e retorne os IDs dos funcionários, seus nomes, sobrenomes, cargos e cidades. **(Utilize a tabela employees)**

Explicação e Resultado

Resultado esperado:

Por exemplo, se chamarmos a função passando o país 'USA'...

```
select * from funcionarios_por_pais('USA');
```

... o PostgreSQL retornará uma tabela com os IDs, nomes, sobrenomes, cargos e cidades de todos os funcionários que estão localizados nos Estados Unidos:

	id smallint	nome character varying	sobrenome character varying	cargo character varying	cidade character varying
1	1	Nancy	Davolio	Sales Representative	Seattle
2	2	Andrew	Fuller	Vice President, Sales	Tacoma
3	3	Janet	Leverling	Sales Representative	Kirkland
4	4	Margaret	Peacock	Sales Representative	Redmond
5	8	Laura	Callahan	Inside Sales Coordinator	Seattle

Total rows: 5 of 5 Query complete 00:00:00.064

GABARITO EXPLICATIVO

6

Questão 6

O time de produtos da empresa precisa revisar periodicamente o portfólio de produtos disponíveis no estoque. Isso é importante para realizar ações como ajuste de preços, controle de estoque, promoções e até descontinuação de produtos que não estão mais sendo vendidos, garantindo que o portfólio de produtos esteja alinhado com a demanda do mercado.

Crie uma função chamada **produtos_por_categoria** que receba o category_id e retorne os IDs dos produtos, seus nomes, preços unitários e as quantidades em estoque. **(Utilize a tabela products)**

Explicação e Resultado

Para criar uma função que receba o category_id e retorne os IDs dos produtos, seus nomes, preços unitários e quantidades em estoque, fazemos o seguinte:

1. Criação da função:

- ✓ **create or replace function produtos_por_categoria(id_categoria int)**: Criamos a função produtos_por_categoria que recebe um parâmetro id_categoria do tipo int (mesmo tipo da coluna category_id na tabela products).
- ✓ **returns table (...)**: Definimos o retorno como uma tabela com as seguintes colunas:
  - id: (tipo da coluna product_id na tabela products)
  - nome_produto: (tipo da coluna product_name na tabela products)
  - preco_unitario: (tipo da coluna unit_price na tabela products)
  - estoque: (tipo da coluna units_in_stock na tabela products)

2. Bloco begin:

- ✓ **return query:** Utilizamos o comando return query para retornar o resultado diretamente da consulta.
- ✓ **select product_id, product_name, unit_price, units_in_stock from products where category_id = id_categoria**: Selecionamos os produtos cuja categoria corresponde ao valor informado no parâmetro.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

6

Questão 6

O time de produtos da empresa precisa revisar periodicamente o portfólio de produtos disponíveis no estoque. Isso é importante para realizar ações como ajuste de preços, controle de estoque, promoções e até descontinuação de produtos que não estão mais sendo vendidos, garantindo que o portfólio de produtos esteja alinhado com a demanda do mercado.

Crie uma função chamada **produtos_por_categoria** que receba o category_id e retorne os IDs dos produtos, seus nomes, preços unitários e as quantidades em estoque. **(Utilize a tabela products)**

Explicação e Resultado

```sql
create or replace function produtos_por_categoria(id_categoria int)
returns table
(
    id products.product_id%type,
    nome_produto products.product_name%type,
    preco_unitario products.unit_price%type,
    estoque products.units_in_stock%type
)
language plpgsql
as $$
begin
    return query
    select
        product_id,
        product_name,
        unit_price,
        units_in_stock
    from products
    where category_id = id_categoria;
end $$;
```

Data Output | Messages | Notifications

```
NOTICE:  type reference products.product_id%TYPE converted to smallint
NOTICE:  type reference products.product_name%TYPE converted to character varying
NOTICE:  type reference products.unit_price%TYPE converted to real
NOTICE:  type reference products.units_in_stock%TYPE converted to smallint
CREATE FUNCTION

Query returned successfully in 71 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

6

Questão 6

O time de produtos da empresa precisa revisar periodicamente o portfólio de produtos disponíveis no estoque. Isso é importante para realizar ações como ajuste de preços, controle de estoque, promoções e até descontinuação de produtos que não estão mais sendo vendidos, garantindo que o portfólio de produtos esteja alinhado com a demanda do mercado.

Crie uma função chamada **produtos_por_categoria** que receba o category_id e retorne os IDs dos produtos, seus nomes, preços unitários e as quantidades em estoque. **(Utilize a tabela products)**

Explicação e Resultado

Resultado esperado:

Por exemplo, se chamarmos a função passando o ID da categoria 2...

```
select * from produtos_por_categoria(2);
```

... o PostgreSQL retornará uma tabela contendo os IDs, nomes, preços unitários e quantidades em estoque de todos os produtos que pertencem à categoria 2:

	id smallint	nome_produto character varying	preco_unitario real	estoque smallint
1	3	Aniseed Syrup	10	13
2	4	Chef Anton's Cajun Seasoning	22	53
3	5	Chef Anton's Gumbo Mix	21.35	0
4	6	Grandma's Boysenberry Spread	25	120
5	8	Northwoods Cranberry Sauce	40	6
6	15	Genen Shouyu	13	39
7	44	Gula Malacca	19.45	27
8	61	Sirop d'érable	28.5	113
9	63	Vegie-spread	43.9	24
10	65	Louisiana Fiery Hot Pepper Sauce	21.05	76
11	66	Louisiana Hot Spiced Okra	17	4
12	77	Original Frankfurter grüne Soße	13	32

Total rows: 12 of 12 | Query complete 00:00:00.059

GABARITO EXPLICATIVO

7

Questão 7

O setor de RH organiza eventos internos, como festas de aniversário, para promover o engajamento e a integração dos funcionários. Para planejar esses eventos de forma eficiente, o RH precisa de uma lista de aniversariantes de cada mês para enviar mensagens de felicitações e organizar as celebrações de maneira personalizada.

Crie uma função chamada **aniversariantes_mes** que receba o mês como parâmetro e retorne os IDs dos funcionários, seus nomes, sobrenomes, datas de nascimento e cargos. **(Utilize a tabela employees.)**

Explicação e Resultado

Para criar uma função que receba o mês e retorne os IDs, nomes, sobrenomes, datas de nascimento e cargos dos funcionários aniversariantes desse mês, fazemos o seguinte:

1. Criação da função:

- ✓ **create or replace function aniversariantes_mes(mes int)**: Criamos a função aniversariantes_mes, que recebe um parâmetro mes do tipo int (mesmo tipo retornado pela função date_part quando extraímos o mês de uma data).
- ✓ **returns table (...)**: Definimos o retorno como uma tabela com as seguintes colunas:
  - id: (tipo da coluna employee_id na tabela employees)
  - nome: (tipo da coluna first_name na tabela employees)
  - sobrenome: (tipo da coluna last_name na tabela employees)
  - nascimento: (tipo da coluna birth_date na tabela employees)
  - cargo: (tipo da coluna title na tabela employees)

2. Bloco begin:

return query: Utilizamos o comando return query para retornar diretamente o resultado da consulta.

select employee_id, first_name, last_name, birth_date, title from employees where date_part('month', birth_date) = mes: Selecionamos os dados dos funcionários cujas datas de nascimento correspondam ao mês informado no parâmetro.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

7

Questão 7

O setor de RH organiza eventos internos, como festas de aniversário, para promover o engajamento e a integração dos funcionários. Para planejar esses eventos de forma eficiente, o RH precisa de uma lista de aniversariantes de cada mês para enviar mensagens de felicitações e organizar as celebrações de maneira personalizada.

Crie uma função chamada **aniversariantes_mes** que receba o mês como parâmetro e retorne os IDs dos funcionários, seus nomes, sobrenomes, datas de nascimento e cargos. **(Utilize a tabela employees.)**

Explicação e Resultado

```sql
create or replace function aniversariantes_mes(mes int)
returns table
(
    id employees.employee_id%type,
    nome employees.first_name%type,
    sobrenome employees.last_name%type,
    nascimento employees.birth_date%type,
    cargo employees.title%type
)
language plpgsql
as $$
begin
    return query
    select
        employee_id,
        first_name,
        last_name,
        birth_date,
        title
    from employees
    where date_part('month', birth_date) = mes;
end $$;
```

Data Output | Messages | Notifications

```
NOTICE:  type reference employees.employee_id%TYPE converted to smallint
NOTICE:  type reference employees.first_name%TYPE converted to character varying
NOTICE:  type reference employees.last_name%TYPE converted to character varying
NOTICE:  type reference employees.birth_date%TYPE converted to date
NOTICE:  type reference employees.title%TYPE converted to character varying
CREATE FUNCTION

Query returned successfully in 40 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

7

Questão 7

O setor de RH organiza eventos internos, como festas de aniversário, para promover o engajamento e a integração dos funcionários. Para planejar esses eventos de forma eficiente, o RH precisa de uma lista de aniversariantes de cada mês para enviar mensagens de felicitações e organizar as celebrações de maneira personalizada.

Crie uma função chamada **aniversariantes_mes** que receba o mês como parâmetro e retorne os IDs dos funcionários, seus nomes, sobrenomes, datas de nascimento e cargos. **(Utilize a tabela employees.)**

Explicação e Resultado

Resultado esperado:

Por exemplo, se chamarmos a função passando o mês 1 (janeiro)...

```
select * from aniversariantes_mes(1);
```

... o PostgreSQL retornará uma tabela contendo os IDs, nomes, sobrenomes, datas de nascimento e cargos de todos os funcionários que fazem aniversário no mês de janeiro:

	id smallint	nome character varying	sobrenome character varying	nascimento date	cargo character varying
1	8	Laura	Callahan	1958-01-09	Inside Sales Coordinator
2	9	Anne	Dodsworth	1966-01-27	Sales Representative

Total rows: 2 of 2 Query complete 00:00:00.073

GABARITO EXPLICATIVO

8

Questão 8

A logística da empresa precisa identificar rapidamente os pedidos que não foram entregues dentro do prazo previsto para evitar insatisfação dos clientes. Identificar e corrigir esses atrasos é crucial para manter a qualidade do serviço de entrega e melhorar a experiência do cliente.

Crie uma função chamada **pedidos_com_atraso** que retorne uma tabela com order_id, customer_id, required_date e shipped_date, filtrando apenas os pedidos onde a data prevista (required_date) for maior ou igual a data estimada informada pelo usuário, assim como a data de envio (shipped_date) foi maior que a data prevista (required_date). **(Utilize a tabela orders)**

Explicação e Resultado

Para criar uma função que receba uma data estimada e retorne os pedidos com atraso, fazemos o seguinte:

1. Criação da função:

- ✓ **create or replace function pedidos_com_atraso(data_estimada date)**: Criamos a função pedidos_com_atraso, que recebe um parâmetro data_estimada do tipo date.
- ✓ **returns table (...)**: Definimos o retorno como uma tabela com as seguintes colunas:
  - id_pedido: (tipo da coluna order_id na tabela orders)
  - id_cliente: (tipo da coluna customer_id na tabela orders)
  - data_prevista: (tipo da coluna required_date na tabela orders)
  - data_envio: (tipo da coluna shipped_date na tabela orders)

2. Bloco begin:

- ✓ **return query**: Utilizamos o comando return query para retornar diretamente o resultado da consulta.
- ✓ **select order_id, customer_id, required_date, shipped_date from orders where required_date >= data_estimada and shipped_date > required_date**: Selecionamos os pedidos em que:
  - A required_date é maior ou igual à data_estimada informada.
  - A shipped_date é posterior à required_date, indicando um atraso na entrega.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

8

Questão 8

A logística da empresa precisa identificar rapidamente os pedidos que não foram entregues dentro do prazo previsto para evitar insatisfação dos clientes. Identificar e corrigir esses atrasos é crucial para manter a qualidade do serviço de entrega e melhorar a experiência do cliente.

Crie uma função chamada **pedidos_com_atraso** que retorne uma tabela com order_id, customer_id, required_date e shipped_date, filtrando apenas os pedidos onde a data prevista (required_date) for maior ou igual a data estimada informada pelo usuário, assim como a data de envio (shipped_date) foi maior que a data prevista (required_date). **(Utilize a tabela orders)**

Explicação e Resultado

```sql
create or replace function pedidos_com_atraso(data_estimada date)
returns table
(
    id_pedido orders.order_id%type,
    id_cliente orders.customer_id%type,
    data_prevista orders.required_date%type,
    data_envio orders.shipped_date%type
)
language plpgsql
as $$
begin
    return query
    select
        order_id,
        customer_id,
        required_date,
        shipped_date
    from orders
    where required_date >= data_estimada and shipped_date > required_date;
end $$;
```

Data Output | Messages | Notifications

```
NOTICE:  type reference orders.order_id%TYPE converted to smallint
NOTICE:  type reference orders.customer_id%TYPE converted to character
NOTICE:  type reference orders.required_date%TYPE converted to date
NOTICE:  type reference orders.shipped_date%TYPE converted to date
CREATE FUNCTION

Query returned successfully in 75 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

8

Questão 8

A logística da empresa precisa identificar rapidamente os pedidos que não foram entregues dentro do prazo previsto para evitar insatisfação dos clientes. Identificar e corrigir esses atrasos é crucial para manter a qualidade do serviço de entrega e melhorar a experiência do cliente.

Crie uma função chamada **pedidos_com_atraso** que retorne uma tabela com order_id, customer_id, required_date e shipped_date, filtrando apenas os pedidos onde a data prevista (required_date) for maior ou igual a data estimada informada pelo usuário, assim como a data de envio (shipped_date) foi maior que a data prevista (required_date). **(Utilize a tabela orders)**

Explicação e Resultado

Resultado esperado:

Por exemplo, se chamarmos a função passando a data 1998-02-02:

```
select * from pedidos_com_atraso('1998-02-02');
```

... o PostgreSQL retornará uma tabela contendo os IDs dos pedidos, IDs dos clientes, datas previstas e datas de envio, apenas para os pedidos com entrega atrasada cuja data prevista seja maior ou igual a 02/02/1998:

	id_pedido smallint	id_cliente character	data_prevista date	data_envio date
1	10816	GREAL	1998-02-03	1998-02-04
2	10847	SAVEA	1998-02-05	1998-02-10
3	10924	BERGS	1998-04-01	1998-04-08
4	10927	LACOR	1998-04-02	1998-04-08
5	10960	HILAA	1998-04-02	1998-04-08
6	10970	BOLID	1998-04-07	1998-04-24

Total rows: 6 of 6 | Query complete 00:00:00.073

GABARITO EXPLICATIVO

9

Questão 9

Exclua do banco de dados todas as functions criadas nesta lista de exercícios:

total_vendas_cliente

total_frete_transportadora

total_vendas_produto

total_vendas_funcionario

funcionarios_por_pais

produtos_por_categoria

aniversariantes_mes

pedidos_com_atraso

Obs.: Garanta que, antes de excluir, primeiro a function será procurada no banco. Se existir, será excluída; caso contrário, será retornado um aviso na tela em vez de um erro. **(Não é preciso utilizar tabelas)**

Explicação e Resultado

Para **excluir todas as functions criadas**, utilizamos o comando **DROP FUNCTION IF EXISTS** para cada uma das functions. Executando cada código, veremos que todas as functions são excluídas com sucesso:

```
DROP FUNCTION IF EXISTS total_vendas_cliente;
```

```
DROP FUNCTION

Query returned successfully in 64 msec.
```

```
DROP FUNCTION IF EXISTS total_frete_transportadora;
```

```
DROP FUNCTION

Query returned successfully in 75 msec.
```

```
DROP FUNCTION IF EXISTS total_vendas_produto;
```

```
DROP FUNCTION

Query returned successfully in 69 msec.
```

```
DROP FUNCTION IF EXISTS total_vendas_funcionario;
```

```
DROP FUNCTION

Query returned successfully in 56 msec.
```

```
DROP FUNCTION IF EXISTS funcionarios_por_pais;
```

```
DROP FUNCTION

Query returned successfully in 75 msec.
```

```
DROP FUNCTION IF EXISTS produtos_por_categoria;
```

```
DROP FUNCTION

Query returned successfully in 71 msec.
```

```
DROP FUNCTION IF EXISTS aniversariantes_mes;
```

```
DROP FUNCTION

Query returned successfully in 46 msec.
```

```
DROP FUNCTION IF EXISTS pedidos_com_atraso;
```

```
DROP FUNCTION

Query returned successfully in 85 msec.
```

MÓDULO 12

TRANSACTIONS E PROCEDURES

TRANSACTIONS E PROCEDURES

TRANSACTIONS E PROCEDURES

O QUE VEREMOS NESTE MÓDULO

MÓDULO 12 TRANSACTIONS E PROCEDURES

01	O que é uma Transaction?	483
02	O que é uma Procedure?	491
03	Como criar uma Procedure?	492
04	Como excluir uma Procedure?	498
05	Exemplo: Procedure (registro de transferência bancária)	500
06	FUNCTIONS vs. PROCEDURES	505
05	EXERCÍCIOS	506
06	GABARITO EXPLICATIVO	512

O que é uma Transaction?

Uma transaction é uma ação realizada dentro do banco de dados, uma unidade de trabalho que consiste em uma ou mais operações. Essa ação pode ser uma inserção (INSERT INTO), uma atualização (UPDATE), ou uma exclusão (DELETE) de dados do banco. Precisamos de transações quando estamos alterando o banco de dados de alguma forma, seja inserindo, atualizando ou excluindo dados.

Um exemplo clássico de uma transação é uma transferência bancária de uma conta para outra: se o remetente transfere um valor de X reais a um destinatário, esse valor deve ser subtraído da conta do remetente e adicionado à conta do destinatário, que deverá receber exatamente esta quantia de X reais, nem mais, nem menos.

Normalmente, não temos muito "controle" sobre transações, a menos que a gente explicite no nosso código que queremos fazer isso.

Assim, a ideia de uma transação é agrupar um conjunto de instruções a serem executadas no banco de dados, e ter a flexibilidade de:

A. Se algo der errado, desfazer essa transação, por meio de um ROLLBACK;
B. Se tudo der certo, salvar essa transação, através de um COMMIT.

O que é uma Transaction?

Vamos fazer um exemplo agora para ficar mais claro.

Para isso, criaremos a seguinte tabela:

```
create table contas(
    id int,
    nome varchar(100),
    saldo decimal);
```

Data output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 79 msec.
```

Vamos inserir o primeiro registro na nossa tabela contas, da forma como já estamos acostumados:

```
insert into contas(id, nome, saldo)
values(1, 'Ana', 5000);
```

Data output | Messages | Notifications

```
INSERT 0 1

Query returned successfully in 64 msec.
```

O que é uma Transaction?

Se consultarmos nossa tabela, veremos que o registro acima foi inserido:

```
select * from contas;
```

Data output | Messages | Notifications

	id integer	nome character varying (100)	saldo numeric
1	1	Ana	5000

Se encerrarmos a nossa sessão no PostgreSQL (fecharmos o sistema), ao iniciarmos uma nova sessão, se consultarmos nossa tabela, veremos que o registro continua inserido na tabela:

```
select * from contas;
```

Data output | Messages | Notifications

	id integer	nome character varying (100)	saldo numeric
1	1	Ana	5000

O que é uma Transaction?

Agora, vamos inserir o segundo registro na nossa tabela contas, desta vez explicitando ao SQL que estamos iniciando uma transação:

```
begin transaction;
insert into contas(id, nome, saldo)
values(2, 'Bruno', 10000);
```

Data output | Messages | Notifications

```
INSERT 0 1

Query returned successfully in 69 msec.
```

Se novamente consultarmos nossa tabela, veremos que o registro acima também encontra-se inserido:

```
select * from contas;
```

Data output | Messages | Notifications

	id integer	nome character varying (100)	saldo numeric
1	1	Ana	5000
2	2	Bruno	10000

O que é uma Transaction?

Porém, se desta vez encerrarmos a nossa sessão no PostgreSQL (fecharmos o sistema), ao iniciarmos uma nova sessão e consultarmos outra vez a nossa tabela, veremos que o segundo registro não se encontra mais inserido na tabela contas:

```
select * from contas;
```

Data output | Messages | Notifications

	id integer	nome character varying (100)	saldo numeric
1	1	Ana	5000

Isso aconteceu porque, quando explicitamente iniciamos uma transação, como fizemos para inserir o segundo registro na tabela, o sistema espera uma finalização dessa transação, seja por meio de um COMMIT ou de um ROLLBACK.

O que é uma Transaction?

Se, ao incluir o segundo registro, tivéssemos efetuado um COMMIT e depois tivéssemos encerrado a sessão, ao retornarmos, se consultássemos novamente nossa tabela, veríamos que o segundo registro fora, de fato, incluído:

O que é uma Transaction?

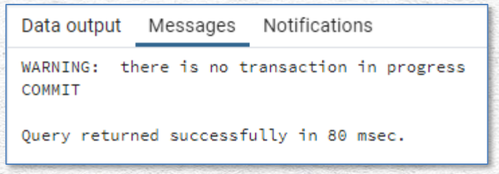
Entretanto, como encerramos a sessão antes de finalizá-la com um COMMIT, o sistema automaticamente deu um ROLLBACK na transação, desfazendo-a. O mesmo teria acontecido se nós mesmos, por algum motivo, em vez de efetuar o COMMIT, tivéssemos executado o comando ROLLBACK: o segundo registro seria retirado da tabela, pois esse comando ROLLBACK teria desfeito a transação:

O que é uma Transaction?

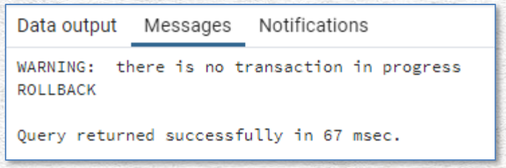
Portanto, sempre que efetuarmos alterações no banco de dados e quisermos ter um maior controle sobre elas, é importante fazer uso das transações (explícitas), caso contrário, o sistema efetuará a transação implicitamente, já salvando-a (dando um COMMIT) automaticamente, ao executarmos um INSERT, um UPDATE ou um DELETE no banco de dados.

Utilizando transações, podemos executar a ação e verificar se de fato ela ocorreu conforme esperávamos. Se sim, efetuamos um COMMIT para salvá-la definitivamente, fazendo com que as mudanças fiquem visíveis em outras sessões (tanto nossas quanto de outros usuários que tenham acesso ao mesmo banco de dados). Caso contrário, se tiver ocorrido algum erro, por exemplo, podemos desfazer a transação por meio de um ROLLBACK.

Mas atenção: após executado um COMMIT ou um ROLLBACK, essa transação é encerrada. Portanto, se, após executado um ROLLBACK, tentarmos efetuar um COMMIT, o sistema retornará um aviso, pois a transação já havia sido encerrada, não sendo mais possível efetuar nenhuma ação em relação a ela:



O contrário também vale: após um COMMIT, a transação é encerrada, não sendo possível efetuar um ROLLBACK:



O que é uma Procedure?

Vamos agora falar das procedures.

Até aqui, aprendemos como criar blocos anônimos e functions. Porém, functions e blocos anônimos não são capazes de executar transações e modificar as informações de um banco de dados.

Ou seja, dentro de uma function, não podemos iniciar uma transação, ou dar um COMMIT/ROLLBACK nela.

Para isso, podemos usar procedures, que também são blocos de código que possuem um nome e podem ser armazenados no banco de dados.

Como criar uma Procedure?

A sintaxe para criação de uma procedure é a seguinte:

```
create or replace procedure nome_procedure(parâmetros)
language plpgsql
as $$
declare
   declaração das variáveis
begin
   corpo do código
end $$;
```

Cada procedure é composta por:

- ✓ Um "cabeçalho" em que atribuímos: um nome para a procedure, seus parâmetros, e a versão do SQL utilizada pelo PostgreSQL (plpgsql). Repare que uma procedure não necessariamente precisa retornar um valor (como uma function precisaria), portanto, não utilizamos o comando returns;
- ✓ Uma sessão de declaração das variáveis que serão usadas no corpo do código (**opcional**);
- ✓ E uma sessão do corpo do código que, como o próprio nome já diz, é onde criamos os nossos **códigos (instruções)**.

Como criar uma Procedure?

Vejamos agora um exemplo para entendermos melhor.

Vamos criar uma procedure que cadastre um novo cliente na tabela contas (tabela esta que criamos na primeira parte deste módulo, sobre transactions):

```
select * from contas;
```

	id integer	nome character varying (100)	saldo numeric
1	1	Ana	5000
2	2	Bruno	10000

Como criar uma Procedure?

Para isso, criaremos o seguinte código:

```
create or replace procedure cadastra_cliente(novo_id int, novo_cliente varchar(100), saldo_inicial decimal)
language plpgsql
as $$
begin
    insert into contas(id, nome, saldo) values
    (novo_id, novo_cliente, saldo_inicial);

    commit;

end $$;
```

Data output | Messages | Notifications

```
CREATE PROCEDURE

Query returned successfully in 65 msec.
```

Como criar uma Procedure?

1. Iniciamos a procedure com o comando CREATE OR REPLACE PROCEDURE (para criar a procedure, caso não exista; ou substitui-la, caso já exista), definimos um nome para ela (cadastra_cliente), e quais serão seus parâmetros (neste caso, três: novo_id do tipo inteiro, novo_cliente do tipo varchar(100) e saldo_inicial do tipo decimal);

2. Informamos ao comando LANGUAGE a versão do SQL utilizada pelo PostgreSQL (plpgsql);

3. Prosseguimos com o comando AS e definimos a label a ser utilizada para delimitar a função: $$. Sem essa label, o SQL não saberia onde as instruções da procedure iniciam e terminam. Isso porque cada instrução termina com um ponto-e-vírgula. Sem a label, o SQL executaria o código até o primeiro ponto-e-vírgula que aparecesse, e entenderia que o código acaba ali. Colocando a label $$ antes de iniciar a declaração das variáveis (sessão opcional) e ao final da procedure, a linguagem entende que aquele trecho entre a label inicial e a label final é um único bloco de códigos que devem ser executados sequencialmente;

```
create or replace procedure cadastra_cliente(novo_id int, novo_cliente varchar(100), saldo_inicial decimal) 1
language plpgsql 2
as $$ 3
begin
    insert into contas(id, nome, saldo) values
    (novo_id, novo_cliente, saldo_inicial);
4
    commit;

end $$; 5
```

Data output | Messages | Notifications

```
CREATE PROCEDURE

Query returned successfully in 65 msec.
```

Como criar uma Procedure?

4 Como não utilizaremos variáveis, não precisamos da sessão de declaração de variáveis. Portanto, partimos direto para o corpo da procedure, iniciando-o com o comando BEGIN e, dentro dele, inserimos as instruções que deverão ser executadas. Neste caso, vamos inserir um novo registro na tabela contas, então, utilizamos um INSERT INTO para incluir nas colunas id, nome e saldo da referida tabela os valores que forem passados aos parâmetros novo_id, novo_cliente e saldo_inicial, respectivamente. Em seguida, aplicamos um COMMIT para garantir que a inserção será efetuada e salva;

5 Por fim, encerramos a procedure com a label end $$;

```
create or replace procedure cadastra_cliente(novo_id int, novo_cliente varchar(100), saldo_inicial decimal) 1
language plpgsql 2
as $$ 3
begin
    insert into contas(id, nome, saldo) values
    (novo_id, novo_cliente, saldo_inicial);
4
    commit;

end $$; 5
```

Data output | Messages | Notifications

```
CREATE PROCEDURE

Query returned successfully in 65 msec.
```

Como criar uma Procedure?

Criada a procedure, podemos utilizá-la da forma apresentada abaixo, por meio do comando CALL.

Vamos inserir o terceiro registro da tabela contas, atribuindo aos parâmetros novo_id o valor 3, novo_cliente a string 'Caio' e saldo_inicial o valor 300, que serão inseridos nas colunas id, nome e saldo da tabela contas, respectivamente:

```
call cadastra_cliente(3, 'Caio', 300);
```

Data output | Messages | Notifications

```
CALL

Query returned successfully in 38 msec.
```

Se consultarmos agora a tabela contas, veremos que o registro acima foi inserido:

```
select * from contas;
```

Data output | Messages | Notifications

	id integer	nome character varying (100)	saldo numeric
1	1	Ana	5000
2	2	Bruno	10000
3	3	Caio	300

Como excluir uma Procedure?

Podemos excluir uma procedure de duas formas:

```
drop procedure cadastra_cliente;
```

OU

```
drop procedure if exists cadastra_cliente;
```

Sugerimos que sempre utilize a segunda forma, pois o IF EXISTS é capaz de procurar se a procedure que queremos excluir de fato existe no sistema. Se existir, o DROP excluirá a referida procedure:

Data output | Messages | Notifications

```
DROP PROCEDURE

Query returned successfully in 35 msec.
```

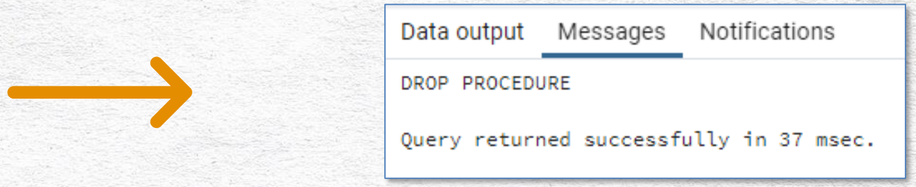
Se não existir, será retornado um aviso na tela (NOTICE), e o sistema prosseguirá para a execução do próximo comando, se houver:

Data output | Messages | Notifications

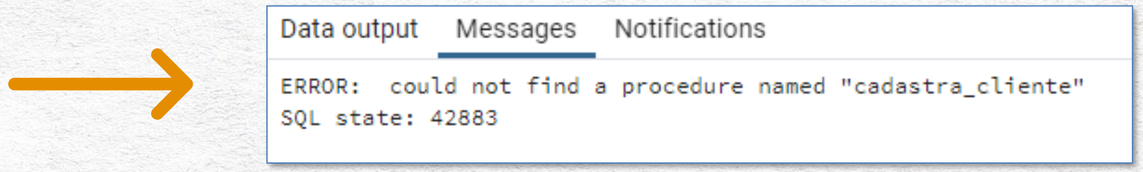
```
NOTICE:  procedure cadastra_cliente() does not exist, skipping
DROP PROCEDURE

Query returned successfully in 31 msec.
```

Como excluir uma Procedure?

Já a primeira opção (sem o IF EXISTS), não terá como verificar antes se a procedure existe ou não. Ele apenas tentará excluir a procedure informada. Se existir, ele a excluirá:



Porém, se ela não existir, em vez de apenas retornar um aviso na tela e prosseguir para o próximo comando (se houver), ele retornará um erro na tela (ERROR) e interromperá a execução dos próximos comandos:



Exemplo: Procedure (registro de transferência bancária)

Vamos ver agora um exemplo um pouquinho mais complexo.

Criaremos uma procedure que controle transferências bancárias entre duas contas. É importante que o valor que sair da conta remetente chegue à conta destinatária, assim como o valor que chegar à conta destinatária saia da conta remetente **de forma simultânea**.

Nesta procedure, novamente utilizaremos a nossa tabela contas, que criamos no início deste módulo.

Apenas para relembrar, vamos consultá-la. Veja que a conta de id = 1 atualmente possui R$ 5.000,00, enquanto que a de id = 2 tem R$ 10.000,00. Guarde esta informação:

```
select * from contas;
```

Data output | Messages | Notifications

	id integer	nome character varying (100)	saldo numeric
1	1	Ana	5000
2	2	Bruno	10000
3	3	Caio	300

Exemplo: Procedure (registro de transferência bancária)

Criaremos a seguinte procedure:

```
create or replace procedure transferencia(remetente int, destinatario int, valor decimal)
language plpgsql
as $$
begin
    -- subtrair o montante transferido pelo remetente
    update contas
    set saldo = saldo - valor
    where id = remetente;

    -- adicionar o montante transferido para o destinatário
    update contas
    set saldo = saldo + valor
    where id = destinatario;

    commit;

end $$;
```

Data output | Messages | Notifications

```
CREATE PROCEDURE

Query returned successfully in 58 msec.
```

Exemplo: Procedure (registro de transferência bancária)

1. Iniciamos a procedure com o comando CREATE OR REPLACE PROCEDURE (para criar a procedure, caso não exista; ou substitui-la, caso já exista), definimos um nome para ela (transferencia), e quais serão seus parâmetros (neste caso, três: remetente do tipo inteiro, destinatario do tipo inteiro e valor do tipo decimal);

2. Informamos ao comando LANGUAGE a versão do SQL utilizada pelo PostgreSQL (plpgsql);

3. Prosseguimos com o comando AS e definimos a label a ser utilizada para delimitar a função: $$.

```
create or replace procedure transferencia(remetente int, destinatario int, valor decimal) 1
language plpgsql 2
as $$ 3
begin
    -- subtrair o montante transferido pelo remetente
    update contas
    set saldo = saldo - valor
    where id = remetente;
4
    -- adicionar o montante transferido para o destinatário
    update contas
    set saldo = saldo + valor
    where id = destinatario;

    commit;

end $$; 5
```

Data output | Messages | Notifications

```
CREATE PROCEDURE

Query returned successfully in 58 msec.
```

Exemplo: Procedure (registro de transferência bancária)

4 Como não utilizaremos variáveis, não precisamos da sessão de declaração de variáveis. Portanto, partimos direto para o corpo da procedure, iniciando-o com o comando BEGIN e, dentro dele, inserimos as instruções que deverão ser executadas. Neste caso, precisamos fazer dois updates na tabela contas:

1º) Subtrair o valor informado ao parâmetro “valor” da conta cujo id seja igual ao número informado para o parâmetro “remetente”;

2º) Adicionar o valor informado ao parâmetro “valor” à conta cujo id seja igual ao número informado para o parâmetro “destinatario”;

Em seguida, aplicamos um COMMIT para garantir que a inserção será efetuada e salva;

5 Por fim, encerramos a procedure com a label end $$;

```
create or replace procedure transferencia(remetente int, destinatario int, valor decimal) 1
language plpgsql 2
as $$ 3
begin
    -- subtrair o montante transferido pelo remetente
    update contas
    set saldo = saldo - valor
    where id = remetente;
4
    -- adicionar o montante transferido para o destinatário
    update contas
    set saldo = saldo + valor
    where id = destinatario;

    commit;

end $$; 5
```

Data output | Messages | Notifications

```
CREATE PROCEDURE

Query returned successfully in 58 msec.
```

Índice Geral

Índice deste Módulo

Exemplo: Procedure (registro de transferência bancária)

Criada a procedure, podemos utilizá-la da forma apresentada abaixo, por meio do comando CALL.

Vamos transferir R$ 500,00 da conta 1 para a conta 2:

```
call transferencia(1, 2, 500);
```

Data output | Messages | Notifications

```
CALL

Query returned successfully in 48 msec.
```

Lembre-se que a conta de id = 1 possuía R$ 5.000,00, enquanto que a de id = 2 tinha R$ 10.000,00. Portanto, efetuada a transferência acima, agora cada uma deverá conter R$ 4.500,00 e R$ 10.500,00, respectivamente.

Se consultarmos a tabela contas, veremos que foi exatamente isso que aconteceu. Veja:

```
select * from contas;
```

Data output | Messages | Notifications

	id integer	nome character varying (100)	saldo numeric
1	3	Caio	300
2	1	Ana	4500
3	2	Bruno	10500

FUNCTIONS vs. PROCEDURES

Muita gente se pergunta quais são as diferenças entre FUNCTIONS e PROCEDURES.

Abaixo, listamos as principais delas:

DIFERENÇAS	PROCEDURES	FUNCTIONS
1	São usadas para executar um processo, uma sequência de comandos e blocos SQL.	São usadas para fazer cálculos.
2	Não podem ser “chamadas” dentro da cláusula SELECT.	Podem ser “chamadas” dentro da cláusula SELECT (desde que não contenham comandos SELECT).
3	Não precisam retornar valor.	Precisam retornar algum valor.

EXERCÍCIOS

Para resolver os exercícios a seguir, recrie o banco de dados dbHotelHashtag (que criamos e excluímos no módulo de CRUD). Você encontrará o passo a passo de como fazer isso no Gabarito Explicativo deste módulo (página 512).

1a

Questão 1-a

A equipe da recepção precisa garantir que, ao cadastrar um novo cliente no sistema, todas as informações sejam registradas corretamente antes de confirmar a operação. Se houver algum erro, a inserção deve poder ser cancelada.

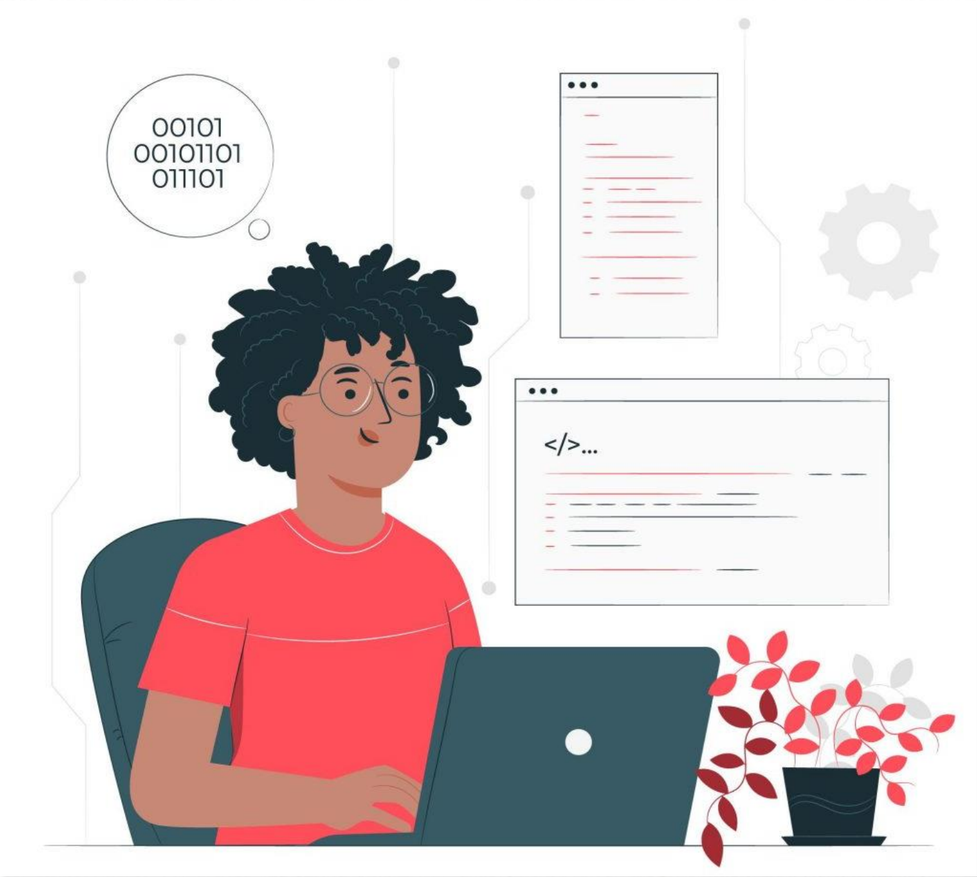
Crie uma transação que insira um novo cliente na tabela clientes e efetue um commit. **(Utilize a tabela clientes)**

1b

Questão 1-b

Adapte o código do item A para criar uma procedure chamada cadastra_novo_cliente que receba os dados do novo cliente como parâmetros e insira um novo registro na tabela clientes. **(Utilize a tabela clientes)**

EXERCÍCIOS



2a

Questão 2-a

O gerente do hotel precisa cancelar a última estadia feita equivocadamente. No entanto, essa operação deve ser realizada dentro de uma transação para garantir que, caso algo dê errado, o cancelamento possa ser revertido.

Exclua a estadia de id_estadia = 15 usando uma transação, garantindo que a exclusão seja confirmada apenas se tudo ocorrer sem erros. **(Utilize a tabela estadias)**

2b

Questão 2-b

Adapte o código do item A para criar uma procedure chamada cancela_estadia que receba como parâmetro o ID da estadia a ser cancelada e exclua esse registro da tabela estadias. **(Utilize a tabela estadias)**

EXERCÍCIOS

3a

Questão 3-a

A administração decidiu ajustar o preço das suítes em 10%. Para evitar erros durante a atualização, a operação deve ser realizada dentro de uma transação.

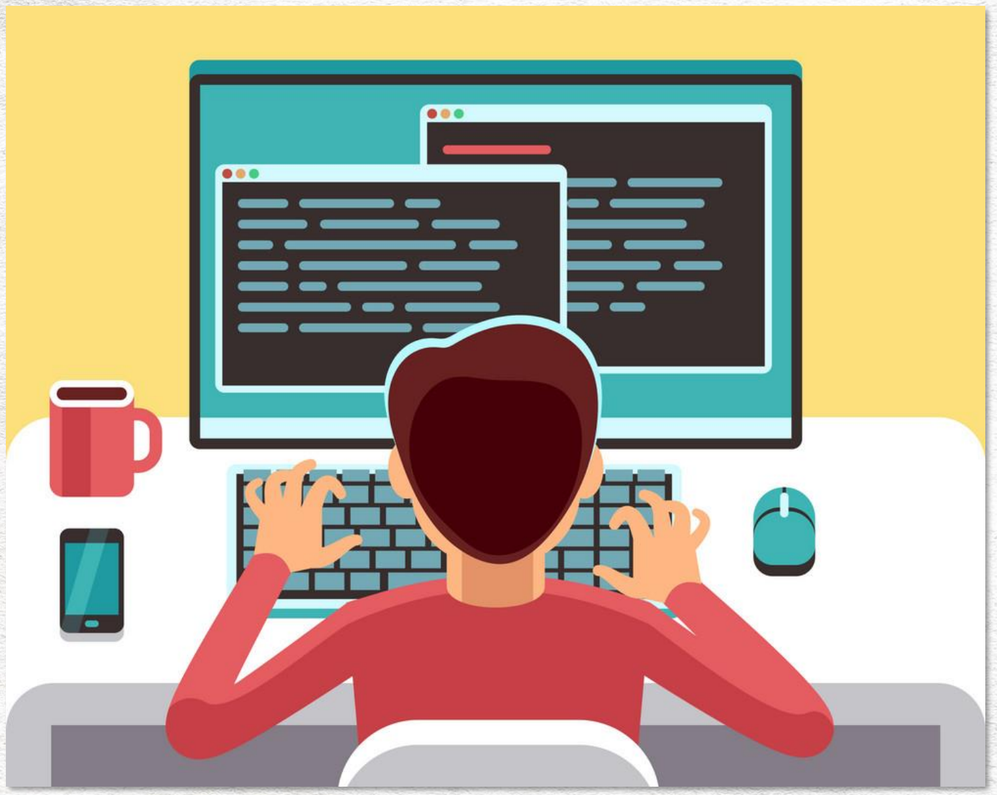
Atualize os preços dos quartos do tipo 'Suite' dentro de uma transação. **(Utilize a tabela quartos)**

3b

Questão 3-b

Adapte o código do item A para criar uma procedure chamada reajusta_precos que receba como parâmetro o tipo de quarto a ter o preço alterado, bem como o percentual do aumento a ser aplicado, para, em seguida, proceder à alteração na tabela quartos. **(Utilize a tabela quartos)**

EXERCÍCIOS



4a

Questão 4-a

O setor de reservas deseja registrar uma nova estadia na tabela. Porém, para evitar erros durante o registro, a operação deve ser realizada dentro de uma transação.

Insira uma nova estadia na tabela estadias usando uma transação. **(Utilize a tabela estadias)**

4b

Questão 4-b

Adapte o código do item A para criar uma procedure chamada registra_nova_estadia que receba como parâmetros os dados da nova estadia (ID da estadia, ID do cliente, ID do quarto e data de entrada), para, em seguida, proceder à alteração na tabela estadias. **(Utilize a tabela estadias)**

EXERCÍCIOS

5a

Questão 5-a

O setor de atendimento ao cliente precisa permitir a atualização segura do email de um cliente. Para evitar erros durante a alteração, a operação deve ser realizada dentro de uma transação.

Altere o e-mail do id_cliente = 5 para 'bia.lima@email.com' usando uma transação. **(Utilize a tabela clientes)**

5b

Questão 5-b

Adapte o código do item A para criar uma procedure chamada atualiza_email_cliente que receba como parâmetros o ID do cliente e seu novo e-mail, para, em seguida, proceder à alteração na tabela clientes. **(Utilize a tabela clientes)**

EXERCÍCIOS

6

Questão 6

Exclua do banco de dados todas as procedures criadas nesta lista de exercícios:

cadastra_novo_cliente

cancela_estadia

reajusta_precos

registra_nova_estadia

atualiza_email_cliente

Obs.: Garanta que, antes de excluir, primeiro a procedure será procurada no banco. Se existir, será excluída; caso contrário, será retornado um aviso na tela em vez de um erro.

(Não é preciso utilizar tabelas)

GABARITO EXPLICATIVO

CRIANDO O BANCO DE DADOS dbHotelHashtag

Para responder os exercícios deste módulo, utilizaremos o banco de dados dbHotelHashtag.

Para criar esse banco de dados e suas tabelas, siga o passo a passo ao lado:

Explicação e Resultado

Em uma janela (Query Tool) do PostgreSQL (pgAdmin 4), execute o seguinte comando:

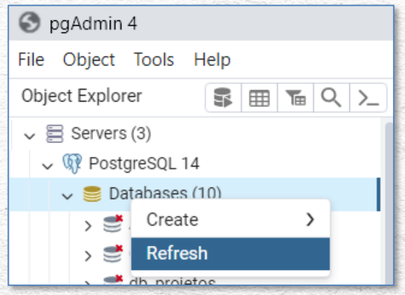
```
CREATE DATABASE dbHotelHashtag;
```

Data Output | Messages | Notifications

```
CREATE DATABASE

Query returned successfully in 635 msec.
```

Para garantir que os próximos comandos serão executados no banco de dados que acabamos de criar, **devemos colocá-lo em uso**.

Para fazer isso, no menu lateral esquerdo do pgAdmin 4, podemos clicar com o botão direito do mouse em **Databases** e selecionar **Refresh**:



CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

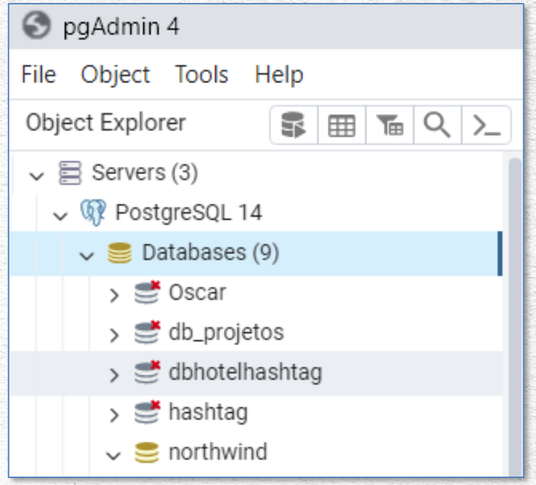
CRIANDO O BANCO DE DADOS dbHotelHashtag

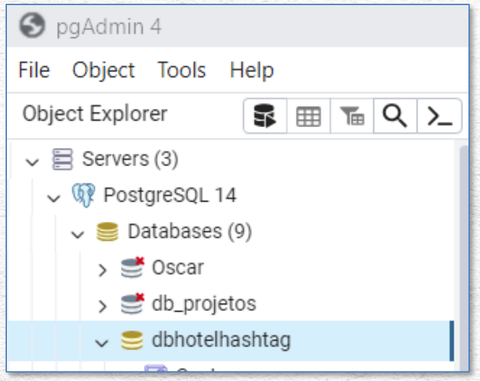
Para responder os exercícios deste módulo, utilizaremos o banco de dados dbHotelHashtag.

Para criar esse banco de dados e suas tabelas, siga o passo a passo ao lado:

Explicação e Resultado

Dessa forma, o banco de dados **dbHotelHashtag aparecerá** no menu, porém **desconectado** (repare que o ícone ao lado de seu nome está cinza com um X vermelho):





Clicamos sobre ele para conectá-lo (repare que agora ele aparece com o ícone em amarelo):

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

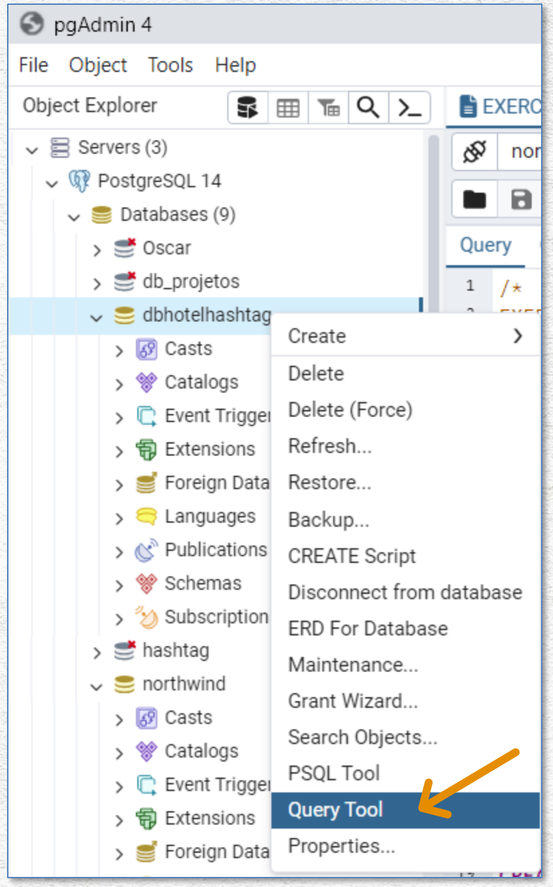
CRIANDO O BANCO DE DADOS dbHotelHashtag

Para responder os exercícios deste módulo, utilizaremos o banco de dados dbHotelHashtag.

Para criar esse banco de dados e suas tabelas, siga o passo a passo ao lado:

Explicação e Resultado

Clicamos com o botão direito do mouse sobre o nome do banco de dados e selecionamos **Query Tool**:



CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

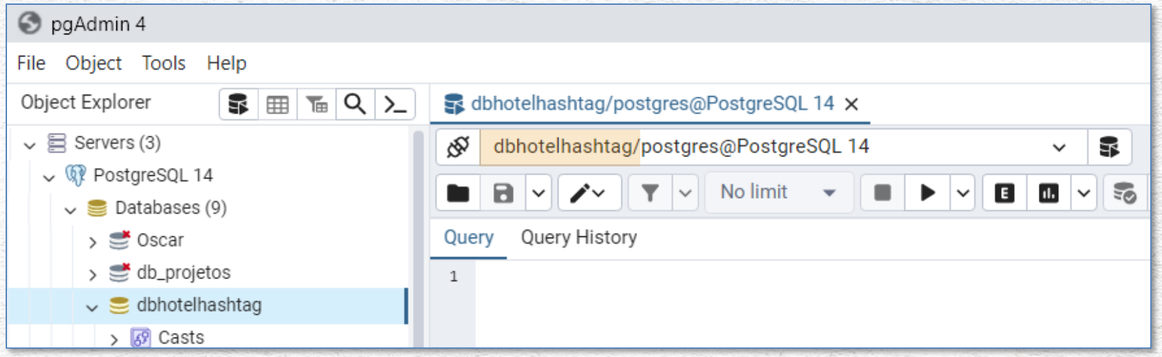
CRIANDO O BANCO DE DADOS dbHotelHashtag

Para responder os exercícios deste módulo, utilizaremos o banco de dados dbHotelHashtag.

Para criar esse banco de dados e suas tabelas, siga o passo a passo ao lado:

Explicação e Resultado

Assim **colocamos o banco de dados dbHotelHashtag em uso, abrindo uma janela** para executarmos os comandos que deverão ser efetuados nele:



Agora estamos prontos para criarmos as tabelas deste banco de dados nesta janela.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

CRIANDO O BANCO DE DADOS dbHotelHashtag

Para responder os exercícios deste módulo, utilizaremos o banco de dados dbHotelHashtag.

Para criar esse banco de dados e suas tabelas, siga o passo a passo ao lado:

Explicação e Resultado

Iniciamos criando a **tabela clientes** executando o **CREATE TABLE** abaixo:

```
-- a) Criando a tabela clientes

CREATE TABLE clientes (
    id_cliente INT,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(50) NOT NULL,
    data_nascimento DATE,
    PRIMARY KEY(id_cliente)
);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 118 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

CRIANDO O BANCO DE DADOS dbHotelHashtag

Para responder os exercícios deste módulo, utilizaremos o banco de dados dbHotelHashtag.

Para criar esse banco de dados e suas tabelas, siga o passo a passo ao lado:

Explicação e Resultado

Em seguida, inserimos as **10 linhas** da **tabela clientes** executando o comando **INSERT INTO** mostrado abaixo:

```
INSERT INTO clientes (id_cliente, nome, email, data_nascimento) VALUES
(1, 'Ana Silva', 'ana.silva@email.com', '1990-03-15'),
(2, 'Carlos Souza', 'carlos.souza@email.com', '1985-07-20'),
(3, 'Maria Oliveira', 'maria.oliveira@email.com', '1993-11-05'),
(4, 'João Santos', 'joao.santos@email.com', '1978-02-25'),
(5, 'Beatriz Lima', 'beatriz.lima@email.com', '2000-01-12'),
(6, 'Lucas Alves', 'lucas.alves@email.com', NULL),
(7, 'Fernanda Rocha', 'fernanda.rocha@email.com', '1995-06-10'),
(8, 'Gabriel Mendes', 'gabriel.mendes@email.com', '1989-08-08'),
(9, 'Sofia Costa', 'sofia.costa@email.com', '1991-12-24'),
(10, 'Rafael Silva', 'rafael.silva@email.com', '1997-04-18');
```

Data Output | Messages | Notifications

```
INSERT 0 10

Query returned successfully in 42 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

CRIANDO O BANCO DE DADOS dbHotelHashtag

Para responder os exercícios deste módulo, utilizaremos o banco de dados dbHotelHashtag.

Para criar esse banco de dados e suas tabelas, siga o passo a passo ao lado:

Explicação e Resultado

Assim, ao consultarmos a **tabela clientes**, vemos que **as 10 linhas foram inseridas**:

```
SELECT * FROM clientes;
```

	id_cliente [PK] integer	nome character varying (100)	email character varying (50)	data_nascimento date
1	1	Ana Silva	ana.silva@email.com	1990-03-15
2	2	Carlos Souza	carlos.souza@email.com	1985-07-20
3	3	Maria Oliveira	maria.oliveira@email.com	1993-11-05
4	4	João Santos	joao.santos@email.com	1978-02-25
5	5	Beatriz Lima	beatriz.lima@email.com	2000-01-12
6	6	Lucas Alves	lucas.alves@email.com	[null]
7	7	Fernanda Rocha	fernanda.rocha@email.com	1995-06-10
8	8	Gabriel Mendes	gabriel.mendes@email.com	1989-08-08
9	9	Sofia Costa	sofia.costa@email.com	1991-12-24
10	10	Rafael Silva	rafael.silva@email.com	1997-04-18

Total rows: 10 of 10 Query complete 00:00:00.443

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

CRIANDO O BANCO DE DADOS dbHotelHashtag

Para responder os exercícios deste módulo, utilizaremos o banco de dados dbHotelHashtag.

Para criar esse banco de dados e suas tabelas, siga o passo a passo ao lado:

Explicação e Resultado

Prosseguimos criando a **tabela quartos** executando o **CREATE TABLE** abaixo:

```
-- b) Criando a tabela quartos

CREATE TABLE quartos (
    id_quarto INT,
    tipo VARCHAR(30) NOT NULL,
    qtd_pessoas INT NOT NULL,
    preco_diaria NUMERIC(10, 2) NOT NULL,
    PRIMARY KEY(id_quarto)
);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 41 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

CRIANDO O BANCO DE DADOS dbHotelHashtag

Para responder os exercícios deste módulo, utilizaremos o banco de dados dbHotelHashtag.

Para criar esse banco de dados e suas tabelas, siga o passo a passo ao lado:

Explicação e Resultado

Em seguida, inserimos as **10 linhas** da **tabela quartos** executando o comando **INSERT INTO** mostrado abaixo:

```
INSERT INTO quartos (id_quarto, tipo, qtd_pessoas, preco_diaria) VALUES
(1, 'Single', 1, 150.00),
(2, 'Double', 2, 250.00),
(3, 'Suite', 4, 500.00),
(4, 'Single', 1, 140.00),
(5, 'Double', 2, 230.00),
(6, 'Suite', 4, 550.00),
(7, 'Single', 1, 160.00),
(8, 'Double', 2, 240.00),
(9, 'Suite', 4, 600.00),
(10, 'Single', 1, 170.00);
```

Data Output | Messages | Notifications

```
INSERT 0 10

Query returned successfully in 106 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

CRIANDO O BANCO DE DADOS dbHotelHashtag

Para responder os exercícios deste módulo, utilizaremos o banco de dados dbHotelHashtag.

Para criar esse banco de dados e suas tabelas, siga o passo a passo ao lado:

Explicação e Resultado

Assim, ao consultarmos a **tabela quartos**, vemos que **as 10 linhas foram inseridas**:

```
SELECT * FROM quartos;
```

	id_quarto [PK] integer	tipo character varying (30)	qtd_pessoas integer	preco_diaria numeric (10,2)
1	1	Single	1	150.00
2	2	Double	2	250.00
3	3	Suite	4	500.00
4	4	Single	1	140.00
5	5	Double	2	230.00
6	6	Suite	4	550.00
7	7	Single	1	160.00
8	8	Double	2	240.00
9	9	Suite	4	600.00
10	10	Single	1	170.00

Total rows: 10 of 10 Query complete 00:00:00.082

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

CRIANDO O BANCO DE DADOS dbHotelHashtag

Para responder os exercícios deste módulo, utilizaremos o banco de dados dbHotelHashtag.

Para criar esse banco de dados e suas tabelas, siga o passo a passo ao lado:

Explicação e Resultado

Finalizamos criando a **tabela estadias** executando o **CREATE TABLE** abaixo:

```
-- c) Criando a tabela estadias

CREATE TABLE estadias (
    id_estadia INT,
    id_cliente INT NOT NULL,
    id_quarto INT NOT NULL,
    data_entrada TIMESTAMP NOT NULL,
    data_saida TIMESTAMP,
    PRIMARY KEY(id_estadia),
    FOREIGN KEY(id_cliente) REFERENCES clientes(id_cliente),
    FOREIGN KEY(id_quarto) REFERENCES quartos(id_quarto)
);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 102 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

CRIANDO O BANCO DE DADOS dbHotelHashtag

Para responder os exercícios deste módulo, utilizaremos o banco de dados dbHotelHashtag.

Para criar esse banco de dados e suas tabelas, siga o passo a passo ao lado:

Explicação e Resultado

Em seguida, inserimos as **15 linhas** da **tabela estadias** executando o comando **INSERT INTO** mostrado abaixo:

```
INSERT INTO estadias (id_estadia, id_cliente, id_quarto, data_entrada, data_saida) VALUES
(1, 1, 2, '2024-01-15 14:00:00', '2024-01-16 12:00:00'),
(2, 3, 5, '2024-02-16 16:00:00', '2024-02-18 11:00:00'),
(3, 4, 1, '2024-03-17 13:00:00', '2024-03-19 11:00:00'),
(4, 2, 3, '2024-04-18 15:00:00', '2024-05-10 10:00:00'),
(5, 5, 7, '2024-05-19 12:30:00', '2024-05-23 12:00:00'),
(6, 6, 9, '2024-06-20 18:00:00', '2024-06-22 14:00:00'),
(7, 8, 4, '2024-07-21 11:00:00', '2024-07-23 09:00:00'),
(8, 9, 10, '2024-08-22 14:00:00', '2024-08-24 12:00:00'),
(9, 7, 6, '2024-09-23 10:00:00', '2024-09-26 10:00:00'),
(10, 10, 8, '2024-10-24 16:00:00', '2024-11-06 11:00:00'),
(11, 1, 5, '2024-11-25 13:00:00', '2024-11-27 10:00:00'),
(12, 3, 2, '2024-12-26 17:00:00', '2024-12-28 13:00:00'),
(13, 4, 1, '2024-12-27 12:00:00', NULL),
(14, 8, 7, '2024-12-28 15:00:00', NULL),
(15, 9, 3, '2024-12-29 14:00:00', NULL);
```

Data Output | Messages | Notifications

```
INSERT 0 15

Query returned successfully in 93 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

CRIANDO O BANCO DE DADOS dbHotelHashtag

Para responder os exercícios deste módulo, utilizaremos o banco de dados dbHotelHashtag.

Para criar esse banco de dados e suas tabelas, siga o passo a passo ao lado:

Agora estamos prontos para resolvermos os exercícios deste módulo. Você pode continuar usando esta mesma janela (Query Tool) do banco de dados dbHotelHashtag, ou abrir uma nova (clicando com o botão direito sobre o nome do banco de dados dbHotelHashtag no menu lateral esquerdo do pgAdmin 4 e selecionando "Query Tool").

Explicação e Resultado

Assim, ao consultarmos a **tabela estadias**, vemos que **as 15 linhas foram inseridas**:

```
SELECT * FROM estadias;
```

	id_estadia [PK] integer	id_cliente integer	id_quarto integer	data_entrada timestamp without time zone	data_saida timestamp without time zone
1	1	1	2	2024-01-15 14:00:00	2024-01-16 12:00:00
2	2	3	5	2024-02-16 16:00:00	2024-02-18 11:00:00
3	3	4	1	2024-03-17 13:00:00	2024-03-19 11:00:00
4	4	2	3	2024-04-18 15:00:00	2024-05-10 10:00:00
5	5	5	7	2024-05-19 12:30:00	2024-05-23 12:00:00
6	6	6	9	2024-06-20 18:00:00	2024-06-22 14:00:00
7	7	8	4	2024-07-21 11:00:00	2024-07-23 09:00:00
8	8	9	10	2024-08-22 14:00:00	2024-08-24 12:00:00
9	9	7	6	2024-09-23 10:00:00	2024-09-26 10:00:00
10	10	10	8	2024-10-24 16:00:00	2024-11-06 11:00:00
11	11	1	5	2024-11-25 13:00:00	2024-11-27 10:00:00
12	12	3	2	2024-12-26 17:00:00	2024-12-28 13:00:00
13	13	4	1	2024-12-27 12:00:00	[null]
14	14	8	7	2024-12-28 15:00:00	[null]
15	15	9	3	2024-12-29 14:00:00	[null]

Total rows: 15 of 15 Query complete 00:00:00.076

GABARITO EXPLICATIVO

1a

Questão 1-a

A equipe da recepção precisa garantir que, ao cadastrar um novo cliente no sistema, todas as informações sejam registradas corretamente antes de confirmar a operação. Se houver algum erro, a inserção deve poder ser cancelada.

Crie uma transação que insira um novo cliente na tabela clientes e efetue um commit. **(Utilize a tabela clientes)**

Explicação e Resultado

Para criar uma transação que insira um novo cliente na tabela clientes e confirme a operação, fazemos o seguinte:

1. Início da transação:

✓ **begin transaction**: Iniciamos uma transação para garantir que a operação de inserção seja executada corretamente antes de ser confirmada.

2. Inserção do cliente:

✓ **insert into clientes (id_cliente, nome, email, data_nascimento) values (...)**: Inserimos um novo cliente na tabela clientes, fornecendo os valores correspondentes às colunas. Por exemplo:

- id_cliente: 11 (identificador do cliente).
- nome: 'Mariana Costa' (nome do cliente).
- email: 'mariana.costa@email.com' (e-mail do cliente).
- data_nascimento: '1992-06-30' (data de nascimento do cliente).

3. Confirmação da transação:

✓ **commit**: Confirma a inserção do novo cliente, tornando a operação permanente no banco de dados.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

1a

Questão 1-a

A equipe da recepção precisa garantir que, ao cadastrar um novo cliente no sistema, todas as informações sejam registradas corretamente antes de confirmar a operação. Se houver algum erro, a inserção deve poder ser cancelada.

Crie uma transação que insira um novo cliente na tabela clientes e efetue um commit. **(Utilize a tabela clientes)**

Explicação e Resultado

```
begin transaction;

insert into clientes (id_cliente, nome, email, data_nascimento) values
(11, 'Mariana Costa', 'mariana.costa@email.com', '1992-06-30');

commit;
```

Data Output | Messages | Notifications

```
COMMIT

Query returned successfully in 97 msec.
```

Resultado esperado:

Se consultarmos a tabela clientes após a transação...

```
select * from clientes;
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

1a

Questão 1-a

A equipe da recepção precisa garantir que, ao cadastrar um novo cliente no sistema, todas as informações sejam registradas corretamente antes de confirmar a operação. Se houver algum erro, a inserção deve poder ser cancelada.

Crie uma transação que insira um novo cliente na tabela clientes e efetue um commit. **(Utilize a tabela clientes)**

Explicação e Resultado

... o PostgreSQL retornará a lista de clientes, incluindo o novo registro inserido para Mariana Costa:

	id_cliente [PK] integer	nome character varying (100)	email character varying (50)	data_nascimento date
1	1	Ana Silva	ana.silva@email.com	1990-03-15
2	2	Carlos Souza	carlos.souza@email.com	1985-07-20
3	3	Maria Oliveira	maria.oliveira@email.com	1993-11-05
4	4	João Santos	joao.santos@email.com	1978-02-25
5	5	Beatriz Lima	beatriz.lima@email.com	2000-01-12
6	6	Lucas Alves	lucas.alves@email.com	[null]
7	7	Fernanda Rocha	fernanda.rocha@email.com	1995-06-10
8	8	Gabriel Mendes	gabriel.mendes@email.com	1989-08-08
9	9	Sofia Costa	sofia.costa@email.com	1991-12-24
10	10	Rafael Silva	rafael.silva@email.com	1997-04-18
11	11	Mariana Costa	mariana.costa@email.com	1992-06-30

Total rows: 11 Query complete 00:00:00.362

GABARITO EXPLICATIVO

1b

Questão 1-b

Adapte o código do item A para criar uma procedure chamada cadastra_novo_cliente que receba os dados do novo cliente como parâmetros e insira um novo registro na tabela clientes. **(Utilize a tabela clientes)**

Explicação e Resultado

Para criar uma procedure que receba os dados de um novo cliente e insira um novo registro na tabela clientes, fazemos o seguinte:

1. Criação da procedure:

✓ **create or replace procedure cadastra_novo_cliente(...)**: Criamos a procedure **cadastra_novo_cliente**, que recebe quatro parâmetros correspondentes às colunas da tabela clientes:

- **novo_id int**: ID do novo cliente.
- **novo_nome varchar**: Nome do cliente.
- **novo_email varchar**: E-mail do cliente.
- **data_nasc date**: Data de nascimento do cliente.

2. Bloco begin ... end:

✓ Dentro do bloco, utilizamos a instrução **insert into** para inserir os dados recebidos pelos parâmetros nas colunas da tabela clientes.

3. Commit da transação:

✓ **commit**: Confirma a inserção do novo cliente, garantindo que os dados sejam gravados permanentemente no banco de dados.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

1b

Questão 1-b

Adapte o código do item A para criar uma procedure chamada cadastra_novo_cliente que receba os dados do novo cliente como parâmetros e insira um novo registro na tabela clientes. **(Utilize a tabela clientes)**

Explicação e Resultado

```
create or replace procedure cadastra_novo_cliente(
                                                    novo_id int,
                                                    novo_nome varchar,
                                                    novo_email varchar,
                                                    data_nasc date
                                                    )
language plpgsql
as $$
begin
    insert into clientes (id_cliente, nome, email, data_nascimento) values
    (novo_id, novo_nome, novo_email, data_nasc);

    commit;
end $$;
```

Data Output | Messages | Notifications

```
CREATE PROCEDURE

Query returned successfully in 99 msec.
```

Chamando a procedure:

Para cadastrar um novo cliente (por exemplo, o Paulo Fernandes), executamos:

```
call cadastra_novo_cliente(12, 'Paulo Fernandes', 'paulo.fernandes@email.com', '1984-09-15');
```

Data Output | Messages | Notifications

```
CALL

Query returned successfully in 66 msec.
```

Índice Geral
Índice deste Módulo
CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

1b

Questão 1-b

Adapte o código do item A para criar uma procedure chamada cadastra_novo_cliente que receba os dados do novo cliente como parâmetros e insira um novo registro na tabela clientes. **(Utilize a tabela clientes)**

Explicação e Resultado

Resultado esperado:

Se consultarmos a tabela clientes após a execução da procedure...

```
select * from clientes;
```

... o PostgreSQL retornará a lista de clientes, incluindo o novo registro inserido para Paulo Fernandes:

	id_cliente [PK] integer	nome character varying (100)	email character varying (50)	data_nascimento date
1	1	Ana Silva	ana.silva@email.com	1990-03-15
2	2	Carlos Souza	carlos.souza@email.com	1985-07-20
3	3	Maria Oliveira	maria.oliveira@email.com	1993-11-05
4	4	João Santos	joao.santos@email.com	1978-02-25
5	5	Beatriz Lima	beatriz.lima@email.com	2000-01-12
6	6	Lucas Alves	lucas.alves@email.com	[null]
7	7	Fernanda Rocha	fernanda.rocha@email.com	1995-06-10
8	8	Gabriel Mendes	gabriel.mendes@email.com	1989-08-08
9	9	Sofia Costa	sofia.costa@email.com	1991-12-24
10	10	Rafael Silva	rafael.silva@email.com	1997-04-18
11	11	Mariana Costa	mariana.costa@email.com	1992-06-30
12	12	Paulo Fernandes	paulo.fernandes@email.com	1984-09-15

Total rows: 12 Query complete 00:00:00.115

GABARITO EXPLICATIVO

2a

Questão 2-a

O gerente do hotel precisa cancelar a última estadia feita equivocadamente. No entanto, essa operação deve ser realizada dentro de uma transação para garantir que, caso algo dê errado, o cancelamento possa ser revertido.

Exclua a estadia de id_estadia = 15 usando uma transação, garantindo que a exclusão seja confirmada apenas se tudo ocorrer sem erros. **(Utilize a tabela estadias)**

Explicação e Resultado

Para excluir uma estadia específica garantindo que a operação só seja confirmada se tudo ocorrer sem erros, fazemos o seguinte:

1. Início da transação:

begin transaction: Inicia a transação, permitindo que as operações sejam executadas dentro de um bloco seguro.

2. Exclusão da estadia:

delete from estadias where id_estadia = 15: Remove da tabela estadias o registro cujo id_estadia seja igual a 15.

3. Confirmação da transação:

commit: Se a exclusão ocorrer sem erros, a transação é confirmada e os dados são removidos permanentemente.

```
begin transaction;

delete from estadias
where id_estadia = 15;

commit;
```

Data Output | Messages | Notifications

```
COMMIT

Query returned successfully in 64 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

2a

Questão 2-a

O gerente do hotel precisa cancelar a última estadia feita equivocadamente. No entanto, essa operação deve ser realizada dentro de uma transação para garantir que, caso algo dê errado, o cancelamento possa ser revertido.

Exclua a estadia de id_estadia = 15 usando uma transação, garantindo que a exclusão seja confirmada apenas se tudo ocorrer sem erros. **(Utilize a tabela estadias)**

Explicação e Resultado

Verificando o resultado:

Após a exclusão, podemos consultar a tabela estadias para confirmar se o registro foi removido:

```
select * from estadias;
```

Resultado esperado:

O PostgreSQL retornará a lista de estadias sem o registro que tinha id_estadia = 15:

	id_estadia [PK] integer	id_cliente integer	id_quarto integer	data_entrada timestamp without time zone	data_saida timestamp without time zone
1	1	1	2	2024-01-15 14:00:00	2024-01-16 12:00:00
2	2	3	5	2024-02-16 16:00:00	2024-02-18 11:00:00
3	3	4	1	2024-03-17 13:00:00	2024-03-19 11:00:00
4	4	2	3	2024-04-18 15:00:00	2024-05-10 10:00:00
5	5	5	7	2024-05-19 12:30:00	2024-05-23 12:00:00
6	6	6	9	2024-06-20 18:00:00	2024-06-22 14:00:00
7	7	8	4	2024-07-21 11:00:00	2024-07-23 09:00:00
8	8	9	10	2024-08-22 14:00:00	2024-08-24 12:00:00
9	9	7	6	2024-09-23 10:00:00	2024-09-26 10:00:00
10	10	10	8	2024-10-24 16:00:00	2024-11-06 11:00:00
11	11	1	5	2024-11-25 13:00:00	2024-11-27 10:00:00
12	12	3	2	2024-12-26 17:00:00	2024-12-28 13:00:00
13	13	4	1	2024-12-27 12:00:00	[null]
14	14	8	7	2024-12-28 15:00:00	[null]

Total rows: 14 Query complete 00:00:00.140

GABARITO EXPLICATIVO

2b

Questão 2-b

Adapte o código do item A para criar uma procedure chamada cancela_estadia que receba como parâmetro o ID da estadia a ser cancelada e exclua esse registro da tabela estadias. **(Utilize a tabela estadias)**

Explicação e Resultado

Para criar uma procedure que receba o ID da estadia e exclua esse registro da tabela estadias, fazemos o seguinte:

1. Criação da procedure:

✓ **create or replace procedure cancela_estadia(estadia_id int)**: Criamos a procedure chamada cancela_estadia, que recebe um parâmetro estadia_id do tipo int.

2. Bloco begin:

✓ Iniciamos o bloco de execução da procedure.

3. Exclusão da estadia:

✓ **delete from estadias where id_estadia = estadia_id**: Remove da tabela estadias o registro cujo id_estadia seja igual ao valor recebido como parâmetro.

4. Confirmação da transação:

✓ **commit**: Garante que a exclusão seja confirmada permanentemente no banco de dados.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

2b

Questão 2-b

Adapte o código do item A para criar uma procedure chamada cancela_estadia que receba como parâmetro o ID da estadia a ser cancelada e exclua esse registro da tabela estadias. **(Utilize a tabela estadias)**

Explicação e Resultado

```
create or replace procedure cancela_estadia(estadia_id int)
language plpgsql
as $$
begin

    delete from estadias
    where id_estadia = estadia_id;

    commit;
end $$;
```

Data Output | Messages | Notifications

```
CREATE PROCEDURE

Query returned successfully in 62 msec.
```

Chamando a procedure:

Para excluir a estadia com ID 14, por exemplo, chamamos a procedure da seguinte forma:

```
call cancela_estadia(14);
```

Data Output | Messages | Notifications

```
CALL

Query returned successfully in 40 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

2b

Questão 2-b

Adapte o código do item A para criar uma procedure chamada cancela_estadia que receba como parâmetro o ID da estadia a ser cancelada e exclua esse registro da tabela estadias. **(Utilize a tabela estadias)**

Explicação e Resultado

Verificando o resultado:

Após a execução da procedure, podemos consultar a tabela estadias para confirmar se o registro foi removido:

```
select * from estadias;
```

Resultado esperado:

O PostgreSQL retornará a lista de estadias sem o registro que tinha id_estadia = 14:

	id_estadia [PK] integer	id_cliente integer	id_quarto integer	data_entrada timestamp without time zone	data_saida timestamp without time zone
1	1	1	2	2024-01-15 14:00:00	2024-01-16 12:00:00
2	2	3	5	2024-02-16 16:00:00	2024-02-18 11:00:00
3	3	4	1	2024-03-17 13:00:00	2024-03-19 11:00:00
4	4	2	3	2024-04-18 15:00:00	2024-05-10 10:00:00
5	5	5	7	2024-05-19 12:30:00	2024-05-23 12:00:00
6	6	6	9	2024-06-20 18:00:00	2024-06-22 14:00:00
7	7	8	4	2024-07-21 11:00:00	2024-07-23 09:00:00
8	8	9	10	2024-08-22 14:00:00	2024-08-24 12:00:00
9	9	7	6	2024-09-23 10:00:00	2024-09-26 10:00:00
10	10	10	8	2024-10-24 16:00:00	2024-11-06 11:00:00
11	11	1	5	2024-11-25 13:00:00	2024-11-27 10:00:00
12	12	3	2	2024-12-26 17:00:00	2024-12-28 13:00:00
13	13	4	1	2024-12-27 12:00:00	[null]

Total rows: 13 Query complete 00:00:00.100

GABARITO EXPLICATIVO

3a

Questão 3-a

A administração decidiu ajustar o preço das suítes em 10%. Para evitar erros durante a atualização, a operação deve ser realizada dentro de uma transação.

Atualize os preços dos quartos do tipo 'Suite' dentro de uma transação. **(Utilize a tabela quartos)**

Explicação e Resultado

Para atualizar os preços das suítes em 10% dentro de uma transação, primeiramente, verificamos os preços atuais das suítes:

```
select * from quartos;
```

	id_quarto [PK] integer	tipo character varying (30)	qtd_pessoas integer	preco_diaria numeric (10,2)
1	1	Single	1	150.00
2	2	Double	2	250.00
3	3	Suite	4	500.00
4	4	Single	1	140.00
5	5	Double	2	230.00
6	6	Suite	4	550.00
7	7	Single	1	160.00
8	8	Double	2	240.00
9	9	Suite	4	600.00
10	10	Single	1	170.00

Total rows: 10 | Query complete 00:00:00.148 | Rows selected: 3

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

3a

Questão 3-a

A administração decidiu ajustar o preço das suítes em 10%. Para evitar erros durante a atualização, a operação deve ser realizada dentro de uma transação.

Atualize os preços dos quartos do tipo 'Suite' dentro de uma transação. **(Utilize a tabela quartos)**

Explicação e Resultado

Agora, atualizamos:

1. Início da transação:

✓ **begin transaction**: Inicia a transação, garantindo que qualquer erro impeça a atualização dos preços.

2. Atualização dos preços:

✓ **update quartos set preco_diaria = preco_diaria * (1 + 0.10) where tipo = 'Suite'**: Atualiza a coluna preco_diaria, aumentando em 10% os valores das suítes.

3. Confirmação da transação:

✓ **commit**: Confirma a alteração, tornando-a permanente no banco de dados.

```
begin transaction;

update quartos
set preco_diaria = preco_diaria * (1 + 0.10)
where tipo = 'Suite';

commit;
```

Data Output | Messages | Notifications

```
COMMIT

Query returned successfully in 80 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

3a

Questão 3-a

A administração decidiu ajustar o preço das suítes em 10%. Para evitar erros durante a atualização, a operação deve ser realizada dentro de uma transação.

Atualize os preços dos quartos do tipo 'Suite' dentro de uma transação. **(Utilize a tabela quartos)**

Explicação e Resultado

Verificando o resultado:

Depois da atualização, podemos consultar a tabela quartos para confirmar se os preços foram ajustados corretamente:

```
select * from quartos;
```

Resultado esperado:

Os valores da coluna preco_diaria foram aumentados em 10% apenas para os registros onde tipo = 'Suite':

	id_quarto [PK] integer	tipo character varying (30)	qtd_pessoas integer	preco_diaria numeric (10,2)
1	1	Single	1	150.00
2	2	Double	2	250.00
3	4	Single	1	140.00
4	5	Double	2	230.00
5	7	Single	1	160.00
6	8	Double	2	240.00
7	10	Single	1	170.00
8	3	Suite	4	550.00
9	6	Suite	4	605.00
10	9	Suite	4	660.00

Total rows: 10 | Query complete 00:00:00.121 | Rows selected: 3

GABARITO EXPLICATIVO

3b

Questão 3-b

Adapte o código do item A para criar uma procedure chamada reajusta_precos que receba como parâmetro o tipo de quarto a ter o preço alterado, bem como o percentual do aumento a ser aplicado, para, em seguida, proceder à alteração na tabela quartos. **(Utilize a tabela quartos)**

Explicação e Resultado

Para criar uma procedure que ajuste os preços dos quartos de um determinado tipo conforme um percentual informado, fazemos o seguinte:

1. Criação da procedure:

✓ A procedure **reajusta_precos** recebe dois parâmetros:

- **tipo_quarto** (tipo do quarto que terá o preço alterado).
- **aumento_percentual** (percentual de aumento a ser aplicado).

2. Atualização dos preços:

✓ O **UPDATE** modifica a coluna **preco_diaria**, aplicando o **percentual** informado apenas nos quartos do **tipo** especificado.

3. Confirmação da transação:

✓ **commit**: confirma a alteração, garantindo que os dados sejam atualizados permanentemente.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

3b

Questão 3-b

Adapte o código do item A para criar uma procedure chamada reajusta_precos que receba como parâmetro o tipo de quarto a ter o preço alterado, bem como o percentual do aumento a ser aplicado, para, em seguida, proceder à alteração na tabela quartos. **(Utilize a tabela quartos)**

Explicação e Resultado

```
create or replace procedure reajusta_precos(
                                                tipo_quarto varchar,
                                                aumento_percentual numeric
                                                )
language plpgsql
as $$
begin

    update quartos
    set preco_diaria = preco_diaria * (1 + aumento_percentual)
    where tipo = tipo_quarto;

    commit;
end $$;
```

Data Output | Messages | Notifications

```
CREATE PROCEDURE

Query returned successfully in 89 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

3b

Questão 3-b

Adapte o código do item A para criar uma procedure chamada reajusta_precos que receba como parâmetro o tipo de quarto a ter o preço alterado, bem como o percentual do aumento a ser aplicado, para, em seguida, proceder à alteração na tabela quartos. **(Utilize a tabela quartos)**

Explicação e Resultado

Para exemplificar, vamos aumentar em 15% os preços dos quartos 'Single'.

Antes de chamar a procedure, consultamos a tabela quartos para verificar o preço desses quartos:

```
select * from quartos;
```

	id_quarto [PK] integer	tipo character varying (30)	qtd_pessoas integer	preco_diaria numeric (10,2)
1	1	Single	1	150.00
2	2	Double	2	250.00
3	4	Single	1	140.00
4	5	Double	2	230.00
5	7	Single	1	160.00
6	8	Double	2	240.00
7	10	Single	1	170.00
8	3	Suite	4	550.00
9	6	Suite	4	605.00
10	9	Suite	4	660.00

Total rows: 10 | Query complete 00:00:00.095 | Rows selected: 4

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

3b

Questão 3-b

Adapte o código do item A para criar uma procedure chamada reajusta_precos que receba como parâmetro o tipo de quarto a ter o preço alterado, bem como o percentual do aumento a ser aplicado, para, em seguida, proceder à alteração na tabela quartos. **(Utilize a tabela quartos)**

Explicação e Resultado

Chamando a procedure:

Agora, chamamos a procedure para aumentar em 15% o preço dos quartos 'Single':

```
call reajusta_precos('Single', 0.15);
```

Data Output | Messages | Notifications

```
CALL

Query returned successfully in 69 msec.
```

Verificando o resultado:

Podemos consultar a tabela quartos depois da execução da procedure para confirmar se a alteração foi efetuada:

```
select * from quartos;
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

3b

Questão 3-b

Adapte o código do item A para criar uma procedure chamada reajusta_precos que receba como parâmetro o tipo de quarto a ter o preço alterado, bem como o percentual do aumento a ser aplicado, para, em seguida, proceder à alteração na tabela quartos. **(Utilize a tabela quartos)**

Explicação e Resultado

Resultado esperado:

Os valores da coluna preco_diaria foram aumentados conforme o percentual informado, apenas para os quartos do tipo especificado:

	id_quarto [PK] integer	tipo character varying (30)	qtd_pessoas integer	preco_diaria numeric (10,2)
1	2	Double	2	250.00
2	5	Double	2	230.00
3	8	Double	2	240.00
4	3	Suite	4	550.00
5	6	Suite	4	605.00
6	9	Suite	4	660.00
7	1	Single	1	172.50
8	4	Single	1	161.00
9	7	Single	1	184.00
10	10	Single	1	195.50

Total rows: 10 | Query complete 00:00:00.142 | Rows selected: 4

GABARITO EXPLICATIVO

4a

Questão 4-a

O setor de reservas deseja registrar uma nova estadia na tabela. Porém, para evitar erros durante o registro, a operação deve ser realizada dentro de uma transação.

Insira uma nova estadia na tabela estadias usando uma transação. **(Utilize a tabela estadias)**

Explicação e Resultado

Para inserir uma nova estadia na tabela estadias garantindo que a operação ocorra dentro de uma transação, fazemos o seguinte:

1. Início da transação:

✓ **begin transaction**: garante que a inserção só seja confirmada caso não ocorra nenhum erro.

2. Inserção dos dados:

✓ A instrução **INSERT INTO** insere os detalhes da nova estadia, incluindo o ID da estadia, o ID do cliente, o ID do quarto e a data de entrada.

3. Confirmação da transação:

✓ **commit**: finaliza a operação e confirma a inserção no banco.

```
begin transaction;

    insert into estadias (id_estadia, id_cliente, id_quarto, data_entrada) values
    (16, 6, 3, '2025-02-13 11:00:00');

commit;
```

Data Output | Messages | Notifications

```
COMMIT

Query returned successfully in 83 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

4a

Questão 4-a

O setor de reservas deseja registrar uma nova estadia na tabela. Porém, para evitar erros durante o registro, a operação deve ser realizada dentro de uma transação.

Insira uma nova estadia na tabela estadias usando uma transação. **(Utilize a tabela estadias)**

Explicação e Resultado

Verificando o resultado:

Podemos conferir se a inserção foi realizada corretamente com a seguinte consulta:

```
select * from estadias;
```

Resultado esperado:

O novo registro aparecerá na tabela estadias com os dados inseridos:

	id_estadia [PK] integer	id_cliente integer	id_quarto integer	data_entrada timestamp without time zone	data_saida timestamp without time zone
1	1	1	2	2024-01-15 14:00:00	2024-01-16 12:00:00
2	2	3	5	2024-02-16 16:00:00	2024-02-18 11:00:00
3	3	4	1	2024-03-17 13:00:00	2024-03-19 11:00:00
4	4	2	3	2024-04-18 15:00:00	2024-05-10 10:00:00
5	5	5	7	2024-05-19 12:30:00	2024-05-23 12:00:00
6	6	6	9	2024-06-20 18:00:00	2024-06-22 14:00:00
7	7	8	4	2024-07-21 11:00:00	2024-07-23 09:00:00
8	8	9	10	2024-08-22 14:00:00	2024-08-24 12:00:00
9	9	7	6	2024-09-23 10:00:00	2024-09-26 10:00:00
10	10	10	8	2024-10-24 16:00:00	2024-11-06 11:00:00
11	11	1	5	2024-11-25 13:00:00	2024-11-27 10:00:00
12	12	3	2	2024-12-26 17:00:00	2024-12-28 13:00:00
13	13	4	1	2024-12-27 12:00:00	[null]
14	16	6	3	2025-02-13 11:00:00	[null]

Total rows: 14 | Query complete 00:00:00.136

GABARITO EXPLICATIVO

4b

Questão 4-b

Adapte o código do item A para criar uma procedure chamada registra_nova_estadia que receba como parâmetros os dados da nova estadia (ID da estadia, ID do cliente, ID do quarto e data de entrada), para, em seguida, proceder à alteração na tabela estadias. **(Utilize a tabela estadias)**

Explicação e Resultado

Para criar uma procedure que insere uma nova estadia na tabela estadias, fazemos o seguinte:

1. Criação da procedure registra_nova_estadia:

✓ A procedure recebe como **parâmetros**:

- novo_id (ID da nova estadia)
- cliente_id (ID do cliente)
- quarto_id (ID do quarto)
- entrada (Data e hora de entrada)

2. Bloco begin ... end:

✓ Dentro do bloco, utilizamos a instrução **insert into** para inserir os dados na tabela estadias.

3. Commit da transação:

✓ **commit**: Confirma a inserção da nova estadia, garantindo que os dados sejam gravados permanentemente no banco de dados.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

4b

Questão 4-b

Adapte o código do item A para criar uma procedure chamada registra_nova_estadia que receba como parâmetros os dados da nova estadia (ID da estadia, ID do cliente, ID do quarto e data de entrada), para, em seguida, proceder à alteração na tabela estadias. **(Utilize a tabela estadias)**

Explicação e Resultado

```
create or replace procedure registra_nova_estadia(
                                                    novo_id int,
                                                    cliente_id int,
                                                    quarto_id int,
                                                    entrada timestamp
                                                    )
language plpgsql
as $$
begin
    insert into estadias (id_estadia, id_cliente, id_quarto, data_entrada) values
    (novo_id, cliente_id, quarto_id, entrada);

    commit;
end $$;
```

Data Output | Messages | Notifications

```
CREATE PROCEDURE

Query returned successfully in 77 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

4b

Questão 4-b

Adapte o código do item A para criar uma procedure chamada registra_nova_estadia que receba como parâmetros os dados da nova estadia (ID da estadia, ID do cliente, ID do quarto e data de entrada), para, em seguida, proceder à alteração na tabela estadias. **(Utilize a tabela estadias)**

Explicação e Resultado

Chamando a procedure:

O comando CALL é utilizado para inserir um novo registro na tabela com os valores informados, como no exemplo abaixo:

```
call registra_nova_estadia(17, 8, 4, '2025-02-17 12:00:00');
```

Data Output | Messages | Notifications

```
CALL

Query returned successfully in 46 msec.
```

Verificando o resultado:

Após a execução da procedure, podemos verificar se o novo registro foi inserido corretamente:

```
select * from estadias;
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

4b

Questão 4-b

Adapte o código do item A para criar uma procedure chamada registra_nova_estadia que receba como parâmetros os dados da nova estadia (ID da estadia, ID do cliente, ID do quarto e data de entrada), para, em seguida, proceder à alteração na tabela estadias. **(Utilize a tabela estadias)**

Explicação e Resultado

Resultado esperado:

O novo registro aparecerá na tabela estadias com os dados informados:

	id_estadia [PK] integer	id_cliente integer	id_quarto integer	data_entrada timestamp without time zone	data_saida timestamp without time zone
1	1	1	2	2024-01-15 14:00:00	2024-01-16 12:00:00
2	2	3	5	2024-02-16 16:00:00	2024-02-18 11:00:00
3	3	4	1	2024-03-17 13:00:00	2024-03-19 11:00:00
4	4	2	3	2024-04-18 15:00:00	2024-05-10 10:00:00
5	5	5	7	2024-05-19 12:30:00	2024-05-23 12:00:00
6	6	6	9	2024-06-20 18:00:00	2024-06-22 14:00:00
7	7	8	4	2024-07-21 11:00:00	2024-07-23 09:00:00
8	8	9	10	2024-08-22 14:00:00	2024-08-24 12:00:00
9	9	7	6	2024-09-23 10:00:00	2024-09-26 10:00:00
10	10	10	8	2024-10-24 16:00:00	2024-11-06 11:00:00
11	11	1	5	2024-11-25 13:00:00	2024-11-27 10:00:00
12	12	3	2	2024-12-26 17:00:00	2024-12-28 13:00:00
13	13	4	1	2024-12-27 12:00:00	[null]
14	16	6	3	2025-02-13 11:00:00	[null]
15	17	8	4	2025-02-17 12:00:00	[null]

Total rows: 15 Query complete 00:00:00.083

GABARITO EXPLICATIVO

5a

Questão 5-a

O setor de atendimento ao cliente precisa permitir a atualização segura do email de um cliente. Para evitar erros durante a alteração, a operação deve ser realizada dentro de uma transação.

Altere o e-mail do id_cliente = 5 para 'bia.lima@email.com' usando uma transação. **(Utilize a tabela clientes)**

Explicação e Resultado

Para atualizar o e-mail de um cliente de forma segura dentro de uma transação, primeiramente, verificamos os e-mails atuais dos clientes, em especial daquele que queremos alterar (id_cliente = 5):

```
select * from clientes;
```

	id_cliente [PK] integer	nome character varying (100)	email character varying (50)	data_nascimento date
1	1	Ana Silva	ana.silva@email.com	1990-03-15
2	2	Carlos Souza	carlos.souza@email.com	1985-07-20
3	3	Maria Oliveira	maria.oliveira@email.com	1993-11-05
4	4	João Santos	joao.santos@email.com	1978-02-25
5	5	Beatriz Lima	beatriz.lima@email.com	2000-01-12
6	6	Lucas Alves	lucas.alves@email.com	[null]
7	7	Fernanda Rocha	fernanda.rocha@email.com	1995-06-10
8	8	Gabriel Mendes	gabriel.mendes@email.com	1989-08-08
9	9	Sofia Costa	sofia.costa@email.com	1991-12-24
10	10	Rafael Silva	rafael.silva@email.com	1997-04-18
11	11	Mariana Costa	mariana.costa@email.com	1992-06-30
12	12	Paulo Fernandes	paulo.fernandes@email.com	1984-09-15

Total rows: 12 | Query complete 00:00:00.106 | Rows selected: 1

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

5a

Questão 5-a

O setor de atendimento ao cliente precisa permitir a atualização segura do email de um cliente. Para evitar erros durante a alteração, a operação deve ser realizada dentro de uma transação.

Altere o e-mail do id_cliente = 5 para 'bia.lima@email.com' usando uma transação. **(Utilize a tabela clientes)**

Explicação e Resultado

Agora, atualizamos:

1. Início da transação:

✓ **begin transaction**: Inicia a transação, garantindo que qualquer erro impeça a atualização do e-mail.

2. Atualização do e-mail:

✓ O **UPDATE** modifica o e-mail do cliente com id_cliente = 5.

3. Confirmação da transação:

✓ **commit**: Confirma a alteração, tornando-a permanente no banco de dados.

```
begin transaction;

    update clientes
    set email = 'bia.lima@email.com'
    where id_cliente = 5;

commit;
```

Data Output | Messages | Notifications

```
COMMIT

Query returned successfully in 78 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

5a

Questão 5-a

O setor de atendimento ao cliente precisa permitir a atualização segura do email de um cliente. Para evitar erros durante a alteração, a operação deve ser realizada dentro de uma transação.

Altere o e-mail do id_cliente = 5 para 'bia.lima@email.com' usando uma transação. **(Utilize a tabela clientes)**

Explicação e Resultado

Verificando o resultado:

Podemos conferir se a atualização foi realizada corretamente com a seguinte consulta:

```
select * from clientes;
```

Resultado esperado:

O e-mail do cliente com id_cliente = 5 foi atualizado para 'bia.lima@email.com':

	id_cliente [PK] integer	nome character varying (100)	email character varying (50)	data_nascimento date
1	1	Ana Silva	ana.silva@email.com	1990-03-15
2	2	Carlos Souza	carlos.souza@email.com	1985-07-20
3	3	Maria Oliveira	maria.oliveira@email.com	1993-11-05
4	4	João Santos	joao.santos@email.com	1978-02-25
5	6	Lucas Alves	lucas.alves@email.com	[null]
6	7	Fernanda Rocha	fernanda.rocha@email.com	1995-06-10
7	8	Gabriel Mendes	gabriel.mendes@email.com	1989-08-08
8	9	Sofia Costa	sofia.costa@email.com	1991-12-24
9	10	Rafael Silva	rafael.silva@email.com	1997-04-18
10	11	Mariana Costa	mariana.costa@email.com	1992-06-30
11	12	Paulo Fernandes	paulo.fernandes@email.com	1984-09-15
12	5	Beatriz Lima	bia.lima@email.com	2000-01-12

Total rows: 12 | Query complete 00:00:00.128 | Rows selected: 1

GABARITO EXPLICATIVO

5b

Questão 5-b

Adapte o código do item A para criar uma procedure chamada atualiza_email_cliente que receba como parâmetros o ID do cliente e seu novo e-mail, para, em seguida, proceder à alteração na tabela clientes. **(Utilize a tabela clientes)**

Explicação e Resultado

Para criar uma procedure que atualize o e-mail de um cliente de forma segura dentro de uma transação, fazemos o seguinte:

1. Criação da procedure:

✓ A procedure **atualiza_email_cliente** recebe dois parâmetros:

- **cliente_id** (ID do cliente cujo e-mail será alterado).
- **novo_email** (novo e-mail a ser cadastrado)

2. Atualização do e-mail:

✓ O **UPDATE** modifica o e-mail do cliente com base no cliente_id fornecido.

3. Confirmação da transação:

✓ **commit**: confirma a alteração, garantindo que os dados sejam atualizados permanentemente.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

5b

Questão 5-b

Adapte o código do item A para criar uma procedure chamada atualiza_email_cliente que receba como parâmetros o ID do cliente e seu novo e-mail, para, em seguida, proceder à alteração na tabela clientes. **(Utilize a tabela clientes)**

Explicação e Resultado

```
create or replace procedure atualiza_email_cliente(
                                                    cliente_id int,
                                                    novo_email varchar
                                                    )

language plpgsql
as $$
begin

    update clientes
    set email = novo_email
    where id_cliente = cliente_id;

    commit;
end $$;
```

Data Output | Messages | Notifications

```
CREATE PROCEDURE

Query returned successfully in 83 msec.
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

5b

Questão 5-b

Adapte o código do item A para criar uma procedure chamada atualiza_email_cliente que receba como parâmetros o ID do cliente e seu novo e-mail, para, em seguida, proceder à alteração na tabela clientes. **(Utilize a tabela clientes)**

Explicação e Resultado

Para exemplificar, vamos alterar o e-mail do cliente de id_cliente = 3 para 'ma.oliveira@email.com'.

Antes de chamar a procedure, consultamos a tabela clientes para verificar o e-mail atual desse cliente:

```
select * from clientes;
```

	id_cliente [PK] integer	nome character varying (100)	email character varying (50)	data_nascimento date
1	1	Ana Silva	ana.silva@email.com	1990-03-15
2	2	Carlos Souza	carlos.souza@email.com	1985-07-20
3	3	Maria Oliveira	maria.oliveira@email.com	1993-11-05
4	4	João Santos	joao.santos@email.com	1978-02-25
5	6	Lucas Alves	lucas.alves@email.com	[null]
6	7	Fernanda Rocha	fernanda.rocha@email.com	1995-06-10
7	8	Gabriel Mendes	gabriel.mendes@email.com	1989-08-08
8	9	Sofia Costa	sofia.costa@email.com	1991-12-24
9	10	Rafael Silva	rafael.silva@email.com	1997-04-18
10	11	Mariana Costa	mariana.costa@email.com	1992-06-30
11	12	Paulo Fernandes	paulo.fernandes@email.com	1984-09-15
12	5	Beatriz Lima	bia.lima@email.com	2000-01-12

Total rows: 12 | Query complete 00:00:00.123 | Rows selected: 1

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

5b

Questão 5-b

Adapte o código do item A para criar uma procedure chamada atualiza_email_cliente que receba como parâmetros o ID do cliente e seu novo e-mail, para, em seguida, proceder à alteração na tabela clientes. **(Utilize a tabela clientes)**

Explicação e Resultado

Chamando a procedure:

Agora, chamamos a procedure para alterar o e-mail:

```
call atualiza_email_cliente(3, 'ma.oliveira@email.com');
```

Data Output | Messages | Notifications

```
CALL

Query returned successfully in 111 msec.
```

Verificando o resultado:

Podemos consultar a tabela clientes depois da execução da procedure para confirmar se a alteração foi efetuada:

```
select * from clientes;
```

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

5b

Questão 5-b

Adapte o código do item A para criar uma procedure chamada atualiza_email_cliente que receba como parâmetros o ID do cliente e seu novo e-mail, para, em seguida, proceder à alteração na tabela clientes. **(Utilize a tabela clientes)**

Explicação e Resultado

Resultado esperado:

O e-mail do cliente com id_cliente = 3 foi atualizado para 'ma.oliveira@email.com':

	id_cliente [PK] integer	nome character varying (100)	email character varying (50)	data_nascimento date
1	1	Ana Silva	ana.silva@email.com	1990-03-15
2	2	Carlos Souza	carlos.souza@email.com	1985-07-20
3	4	João Santos	joao.santos@email.com	1978-02-25
4	6	Lucas Alves	lucas.alves@email.com	[null]
5	7	Fernanda Rocha	fernanda.rocha@email.com	1995-06-10
6	8	Gabriel Mendes	gabriel.mendes@email.com	1989-08-08
7	9	Sofia Costa	sofia.costa@email.com	1991-12-24
8	10	Rafael Silva	rafael.silva@email.com	1997-04-18
9	11	Mariana Costa	mariana.costa@email.com	1992-06-30
10	12	Paulo Fernandes	paulo.fernandes@email.com	1984-09-15
11	5	Beatriz Lima	bia.lima@email.com	2000-01-12
12	3	Maria Oliveira	ma.oliveira@email.com	1993-11-05

Total rows: 12 | Query complete 00:00:00.107 | Rows selected: 1

GABARITO EXPLICATIVO

6

Questão 6

Exclua do banco de dados todas as procedures criadas nesta lista de exercícios:

cadastra_novo_cliente

cancela_estadia

reajusta_precos

registra_nova_estadia

atualiza_email_cliente

Obs.: Garanta que, antes de excluir, primeiro a procedure será procurada no banco. Se existir, será excluída; caso contrário, será retornado um aviso na tela em vez de um erro.

(Não é preciso utilizar tabelas)

Explicação e Resultado

Para **excluir todas as procedures criadas**, utilizamos o comando **DROP PROCEDURE IF EXISTS** para cada uma das procedures. Executando cada código, veremos que todas as procedures são excluídas com sucesso:

```
DROP PROCEDURE IF EXISTS cadastra_novo_cliente;
```

Data Output | Messages | Notifications

```
DROP PROCEDURE

Query returned successfully in 101 msec.
```

```
DROP PROCEDURE IF EXISTS cancela_estadia;
```

Data Output | Messages | Notifications

```
DROP PROCEDURE

Query returned successfully in 43 msec.
```

```
DROP PROCEDURE IF EXISTS reajusta_precos;
```

Data Output | Messages | Notifications

```
DROP PROCEDURE

Query returned successfully in 68 msec.
```

```
DROP PROCEDURE IF EXISTS registra_nova_estadia;
```

Data Output | Messages | Notifications

```
DROP PROCEDURE

Query returned successfully in 41 msec.
```

```
DROP PROCEDURE IF EXISTS atualiza_email_cliente;
```

Data Output | Messages | Notifications

```
DROP PROCEDURE

Query returned successfully in 44 msec.
```

SQL IMPRESSIONADOR

PostgreSQL

Apostila completa